DX Framework: полный учебник разработки космического симулятора

От основ C++ и математики до NPC, Bullet, шейдеров и готового игрового эпизода

Подробные объяснения, расчёты, кодовые маршруты, упражнения и проверка результата

DX_FrameWork

Издание 6.0 · 28 октября 2026 года

Содержание

Учебный курс разработки игры на DX Framework
- Урок 1. От запуска приложения до одного игрового кадра
- Урок 2. ECS как модель игрового мира
- Урок 3. Сцена, редактор и первый игровой уровень
- Урок 4. Математика как инструмент, а не список формул
- Урок 5. Bullet Physics от формы до игрового события
- Урок 6. Рендер DirectX 11 и смысл каждого шейдера
- Урок 7. Корабль как связанная система, а не набор HUD чисел
- Урок 8. Дружественный NPC от аварии до завершённого ремонта
- Урок 9. Вражеский NPC от слуха и зрения до попадания
- Урок 10. NavMesh, двери и движение персонажа
- Урок 11. Как добавлять системы без разрушения архитектуры
- Урок 12. Анимация, оружие и согласование позы
- Урок 13. Сохранение, фоновые задачи и устойчивость сцены
- Урок 14. Интерфейс, звук и понятная обратная связь
- Урок 15. Контент, производительность и граница почти готовой игры
- Урок 16. Выпускная работа на одном корабле

Производственная часть игры
- Урок 17. Конвейер моделей, материалов и анимации
- Урок 18. Миссия и правила игрового эпизода
- Урок 19. Наблюдаемость и диагностика без догадок
- Урок 20. Сборка, упаковка и внешний прогон

Математический задачник для движка
- Задача 1. Движение NPC к рабочей точке
- Задача 2. Угол зрения через dot product
- Задача 3. Выбор правильной стороны поворота
- Задача 4. Порядок матриц на точке
- Задача 5. Нормаль при неравномерном масштабе
- Задача 6. Экранная точка и picking ray
- Задача 7. Порог прибытия и гистерезис
- Задача 8. Полезность ремонта и цена маршрута
- Задача 9. Прицеливание и движение цели
- Задача 10. Физика взрыва и закон сохранения
- Задача 11. Аккумулятор и мощность
- Задача 12. Согласованность симуляции после сохранения

Практикум создания космического симулятора
- Лабораторная 1. Генератор питает дверь и кислородный прибор
- Лабораторная 2. Две пробоины и два инженера
- Лабораторная 3. Враг видит, ищет и стреляет честно

Лабораторная 4. Новый пожар и тушение экипажем

Лабораторная 5. Релизная сборка и проверка вне исходников

Часть I. Паспорт движка

Разбор 1. Что представляет собой проект и как проверить исходное состояние

1. Что это за проект

2. Как пользоваться книгой после долгого перерыва

3. Как движок вырос: история по локальному git

4. Карта репозитория

5. Владение и жизненный цикл приложения

6. Сборка и запуск

7. Один кадр: фактический timeline

8. Время и численная устойчивость

Часть II. C++ и ECS именно в этом проекте

Разбор 2. Entity, Component и System по шагам

Разбор 3. Сцена и редактор как инструменты обучения

9. C++: значения, ссылки, указатели и RAII

10. Как устроена ECS

11. Как добавить компонент и систему

Часть III. Математика игры: от чисел до преобразований

12. Числа, единицы и координатные пространства

13. Векторы: движение, обзор и геометрия

14. Матрицы и преобразование пространства

15. Углы, вращения и интерполяция

16. Камера, лучи, столкновения и границы

17. Численные методы и математика систем корабля

Математическая мастерская. Один игровой эпизод, рассчитанный вручную

Часть IV. DirectX 11 и рендер

Разбор 18. CPU, GPU и жизнь одного draw call

Разбор 19. Пиксельный шейдер и физический смысл материала

Разбор 20. Как добавить новый шейдер безопасно

18. Ресурсы и состояния DirectX 11

19. Фактический render pipeline

20. PBR: математика Cook–Torrance

21. Цвет, HDR и IBL

22. Тени

23. Прозрачность, reflection и refraction

24. Частицы и взрывы

25. Скелетная анимация

Часть V. Bullet и симуляция мира

Разбор 21. Тело, форма и мир физики

Разбор 22. Персонаж, дверь и разрушение

Разбор 23. Физические проверки, которые нужны перед игрой

Разбор 24. Корабельные сети как связанные правила
Разбор 25. Давление, затопление и пространство
Разбор 26. Авария, ремонт и наблюдаемый результат
26. Bullet Physics и синхронизация
27. Character controller
28. Давление, затопление и плавучесть
29. Комнаты и атмосфера
30. Энергетика
31. Damage, cutting и repair
Часть VI. NPC, данные и инструменты
Разбор 27. Собираем NPC как сущность
Разбор 28. Решение NPC — последовательность, а не один enum
Разбор 29. Маршрут, дверь и фактическое движение
32. NPC AI pipeline
33. StateSystem и исполнение
34. Recast/Detour NavMesh
35. JobSystem и асинхронные конвейеры
36. Scene save, runtime save и Entity remap
37. Editor, picking и диагностика
Часть VII. Срез качества и диагностика
Разбор 35. От симптома к нарушенному контракту
Разбор 36. Тестирование без копирования реализации
Разбор 37. Критерий почти готовой игры
38. Сильные стороны движка
39. Реестр слабых мест
40. Производительность без угадывания
41. Как диагностировать симптом
42. Практикум: безопасное добавление engine-функции
43. Карта чтения кода
Часть VIII. Работа с кодом и практикум по NPC
Разбор 30. Что считается ремонтной задачей
Разбор 31. Исполнение работы в `StateSystem`
44. Как изучать собственный движок, а не просто читать файлы
45. Анатомия персонажа в DX Framework
46. Дружественный NPC в текущей архитектуре
47. Проектирование врага на существующем NPC pipeline
48. Практическое задание Первый полноценный дружественный NPC
49. Практическое задание Первый преследующий и атакующий враг
50. План одного рабочего дня и порядок сдачи
Часть IX. Боевой NPC как учебный проект
Разбор 32. Допустимая цель и честное наблюдение
Разбор 33. Бой, укрытие и решение о выстреле

Разбор 34. Пять боевых сцен для проверки
51. Текущее состояние врага и границы следующей версии
52. Практикум Удар направлен вбок
53. Восприятие и память цели
54. Преследование последней известной позиции
55. Укрытия как данные и запрос
56. Атака из укрытия и смена позиции
57. Обход и фланг
58. Несколько врагов и распределение ролей
Часть X. Атлас классов и маршруты данных
Разбор 38. Маршрут от файла сцены до Entity
Разбор 39. Маршрут от ввода до движения персонажа
Разбор 40. Маршрут от аварии до действия экипажа
Разбор 41. Маршрут от вражеского контакта до урона
Разбор 42. Маршрут от команды до картинки на экране
Разбор 43. Как использовать атлас при новой системе
60. Как читать атлас и где проходят границы
61. Ядро, цикл приложения и инфраструктура
62. ECS, сцена, редактор и сериализация
63. Ресурсы и рендеринг
64. Игрок, ввод и перемещение персонажа
65. NPC: решение, команда и исполнение
66. Бой, анимация, инвентарь и взаимодействие
67. Физика подлодки, помещения, энергия и ремонт
68. Сквозные маршруты вызовов
69. Правила зависимостей и кандидаты на разделение
Часть XI. Практика разработки систем и NPC
Разбор 44. Спецификация новой механики до кода
Разбор 45. Вставка в кадр и интеграционные границы
Разбор 46. Сохранение и восстановление состояния игры
Разбор 47. UI и звук как часть правил
Разбор 48. Производство ресурсов
Разбор 49. Миссия и проверяемый вертикальный срез
Разбор 50. Сборка, внешний запуск и критерий завершения
70. Как читать одну систему от входа до результата
71. Дружественный NPC ремонтирует пробоину
72. Вражеский NPC обнаруживает и атакует
73. Добавление новой работы дружественного NPC
74. Добавление нового боевого поведения
75. Bullet Physics как часть симуляции
76. Вершинный и пиксельный шейдеры
77. Рецепт добавления системы в DX Framework

Часть XII. План развития космического симулятора на 14 дней

78. День 1. Воспроизводимая сцена и базовая диагностика
79. День 2. Назначение и освобождение ремонта
80. День 3. Достижимость и двери до выбора цели
81. День 4. Роли экипажа и расход материалов
82. День 5. Пожар как часть симуляции корабля
83. День 6. Экипаж тушит пожар
84. День 7. Восприятие врага без всеведения
85. День 8. Огонь, анимация и перезарядка
86. День 9. Укрытия и выход из тупиков
87. День 10. Первая командная тактика
88. День 11. Авария корабля меняет решения
89. День 12. События, звук и тревога
90. День 13. Сохранение, нагрузка и устойчивость
91. День 14. Вертикальный срез и кандидат к выпуску

Приложение A. Горячие клавиши

Приложение B. Формулы на одной странице

Приложение C. Словарь

Приложение D. Что открыть через год

Приложение E. Первоисточники и границы применимости

Как пользоваться учебником

Книга состоит из последовательных уроков, подробных разборов частей I–XI, математического задачника, лабораторных работ и справочных карт кода. Сначала пройди уроки 1–20 на одной учебной сцене. Затем изучай соответствующий разбор системы, решай задачи вручную и сравнивай числа с debug выводом движка. Короткие карты в конце каждой части служат для поиска классов после подробного объяснения.

Код в блоках курса иногда является учебным псевдокодом; рядом объясняется, что нужно связать с реальными типами проекта. Имена существующих файлов и методов указаны явно. Реализацию проверяй по состоянию репозитория: старые практикумы о групповом бое описывают возможное развитие, а вызов NpcSquadCombatSystem сейчас отключён.

После учебного курса выполни выпускную работу и пройди единственный четырнадцатидневный план в конце книги. План посвящён улучшению NPC и недостающих связей космического симулятора. Для внешнего игрока оценивай не только компиляцию, но и понятность целей, сохранение, работу без редактора и запуск вне исходного каталога.

Учебный курс разработки игры на DX Framework

Цель курса — довести читателя от пустой тестовой сцены до законченного вертикального среза космического симулятора. В нём есть последовательная история: корабль получает повреждение, экипаж замечает аварию и распределяет работу, противник проникает на палубу, игрок принимает решение, а мир продолжает работать за пределами камеры. Этот сценарий станет проверкой каждой системы. Если новая функция не меняет наблюдаемое поведение, её рано считать готовой.

Курс опирается на текущий проект. Путь к классу означает место в исходниках, а не обещание, что все описанные улучшения уже реализованы. Я буду явно отделять действующий код от учебных заданий. Не пропускай расчёты: движок трудно развивать, когда число «0,2» у скорости, давления или полезности не имеет единиц и проверяемого смысла.

Урок 1. От запуска приложения до одного игрового кадра

1.1. Зачем нужен главный цикл

Игра отличается от обычной программы тем, что не завершает работу после одного ответа. Она постоянно принимает ввод, обновляет состояние и рисует результат. В этом проекте путь начинается в Win32, проходит через `SystemClass` и попадает в `ApplicationClass`. Последний является composition root: он знает о SceneManager, ECS, физике, рендере, вводе и игровых системах, а потому обязан задавать порядок их работы. Если система ремонта меняет пробоину после того, как атмосфера уже рассчитана, новый поток воздуха попадёт в расчёт только на следующем кадре. Это может быть допустимо, если задержка осознанная; случайный порядок создаёт трудноуловимые ошибки.

Разбей кадр на четыре вопроса. «Что произошло?» — сообщения окна, нажатия, завершившиеся фоновые задачи. «Что должно произойти в мире?» — команды игрока, решения NPC, двери, питание, атмосфера, физика, урон. «Что теперь видит камера?» — обновлённые Transform, анимация, отбор видимых объектов. «Как это показать?» — render passes, интерфейс и Present. Такую схему удобно держать рядом с `ApplicationClass::Frame`, потому что один большой метод легко воспринимать как случайный список вызовов.

В действующем Play цикле weapon timers обновляются до попыток стрельбы. Затем работают `NpcUtilitySystem`, `CrewTaskCoordinator`, `NpcCombatSystem`, `NpcCoverSystem`, `StateSystem`, поворот персонажа, Bullet controller, анимация и урон. Позже, когда поза известна, `NpcRangedCombatSystem` определяет дуло и вызывает общий `WeaponSystem`. Это не формальная диаграмма: поменяй местами animation и muzzle sampling — пуля выйдет из положения прошлой позы. Поменяй местами решение и движение — персонаж начнёт выполнять приказ на кадр позже. Иногда задержка в один кадр нормальна, но она должна быть названа и проверена.

1.2. Три вида времени

`real delta time` показывает, сколько секунд прошло по часам. `simulation delta time` может быть равен нулю во время паузы или масштабироваться при ускорении мира. `fixed step` дробит накопленное время на равные шаги для физики. Например, кадры пришли через 0,010, 0,020 и 0,020 с. Всего прошло 0,050 с. При шаге 1/60 с физика должна выполнить три шага по ≈0,01667 с, оставив в аккумуляторе небольшой остаток. Если передать Bullet все 0,050 с одним непредсказуемым шагом, контакты и контроллер сильнее зависят от FPS.

Не умножай скорость дважды на dt. Если `movement.desiredVelocity` измеряется в м/с, контроллер сам применяет временной шаг. Запись `desiredVelocity = direction * speed * dt`, после которой контроллер ещё раз умножает на dt, даёт смещение пропорционально dt^2. При удвоении FPS скорость заметно изменится. Простой диагностический опыт: пройди одним NPC одну и ту же прямую при 30 и 120 FPS за 10 секунд реального времени. Разница в положении сверх ожидаемой физической погрешности указывает на ошибку времени или порогов.

1.3. Editor и Play являются разными режимами, но одной сценой

Руководство редактора прямо предупреждает: F5 переключает режим без автоматического восстановления снимка ECS. Если в Play подобрать оружие, сломать дверь и затем вернуться в Editor, runtime изменения могут остаться в Registry. Это не мелкая особенность интерфейса, а правило работы с контентом. Перед опытами сохрани сцену. После опыта загрузи авторскую версию заново, если не хочешь сохранить последствия. В будущей production версии полезно добавить Play snapshot/restore, но до его реализации нельзя описывать F5 как безопасный откат.

Практика. Открой `src/Core/SystemClass.cpp` и `src/Core/ApplicationClass.cpp`. Найди, где вычисляется dt, где проверяется Play, где запускаются NPC и где начинается рендер. Нарисуй на бумаге стрелки от одного результата к следующему. Затем поставь в сцену дверь и NPC и запиши, в каком моменте кадра дверь успевает открыться относительно маршрута NPC. Результат урока — не запомненный список функций, а объяснение, почему каждый потребитель получает уже подготовленные данные.

Урок 2. ECS как модель игрового мира

2.1. Entity не является объектом C++

В `src/ECS/ECSTypes.h` Entity — `uint32_t`, а `INVALID_ENTITY` — максимальное значение этого типа. Идентификатор сам по себе не хранит имя, Transform или здоровье. `ECSRegistry` содержит множество живых ID и отдельные pool по типам компонентов. Компонент — данные. Система — проход по подходящим компонентам и изменение мира. Например, пробоина существует как Entity с HullBreach и Repairable, а `RepairSystem` решает, что значит увеличить её прогресс и завершить ремонт.

Эта модель позволяет сочетать признаки без дерева наследования. Генератор может иметь Transform, Damageable, Power, Interaction и WorkZone. Его не нужно объявлять `class RepairableInteractivePoweredGenerator : ...`. Но композиция требует дисциплины: система должна явно описывать минимальный набор компонентов и поведение при их отсутствии. Если NPC имеет модель, но нет `NpcBrainComponent`, дальний бой не знает, можно ли ему стрелять. Если у стрелка нет FactionComponent, нынешний код возвращает статус `Shooter has no faction`.

2.2. Как работает ForEach

В текущем `ECSRegistry::ForEach<T1,T2>` система перебирает первый pool и проверяет наличие сущности во втором. Поэтому порядок типов влияет на стоимость: если активных NPC десятки, а Transform тысячи, выгоднее начинать с NPC pool. Это не повод менять сигнатуру без измерения, но полезно понимать при профилировании. `GetComponent<T>` возвращает указатель или `nullptr`; проверка обязательна, когда компонент опционален. `HasComponent` отвечает только на наличие, а не на то, что Entity жива и активна. Для цели боя или ремонта проверяй `IsAlive`, `ActiveComponent` и предметные условия.

У текущего Registry есть важное ограничение: ID увеличивается монотонно, generation index в Entity пока нет. Поэтому описание в старых учебных разделах о generation handle относится к предлагаемому усилению, а не к уже действующему API. После уничтожения Entity все её компоненты удаляются, но сохранённый в другом компоненте ID может указывать на несуществующую цель. Перед использованием такого ID нужен `IsAlive`. При загрузке сцены новый ID может отличаться от сохранённого, поэтому SceneManager выполняет remap ссылок.

2.3. Структурные изменения во время обхода

`ForEach` и `GetComponents` перебирают контейнеры pool. Добавление или удаление компонентов того же типа во время обхода может инвалидировать итератор. Даже если один конкретный запуск «случайно работает», это не контракт. Собери команды изменения в отдельный vector и примени после прохода либо используй отложенную очередь. `DamageSystem` и `RepairSystem` особенно чувствительны к этому, потому что создание пробоины или завершение ремонта может менять набор сущностей. Запомни правило: после функции, которая может уничтожить цель, не держи ссылку на её старый компонент и получи её заново, если она ещё существует.

Пример модели данных для будущего пожара:

```
struct FireComponent {
    float intensity = 0.0f;       // 0..1, runtime
    float spreadRatePerSecond = 0.1f;
    Entity room = INVALID_ENTITY;
    bool enabled = true;
};
```

Поля не должны содержать `ID3D11ShaderResourceView*` или `btRigidBody*`, если их задача — описывать пожар. Эти ресурсы принадлежат другим слоям. Компонент хранит то, что необходимо для правила мира; рендер получает снимок интенсивности и показывает огонь.

2.4. Сохранение компонента — отдельный контракт

Добавив C++ структуру, ты ещё не научил сцену её сохранять. Найди в `SceneManager` путь `SerializeObject` и соответствующее чтение. Раздели authoring данные (например, скорость распространения) и runtime данные (текущая интенсивность). Scene save может хранить первое, save game — оба. Сохраняй версию формата и безопасные значения по умолчанию: старые сцены не должны ломаться после добавления поля. Ссылки на другие Entity сериализуй через устойчивые ID файла и remap, а не через адрес памяти. После загрузки проверь, что дверь по-прежнему соединяет те же комнаты, а рабочая зона — нужное оборудование.

Практика. Создай в тестовой ветке один компонент данных, например `TemperatureComponent`, и пройди весь путь: создание через редактор, наличие в Registry, один системный update, отображение в Inspector, scene round trip. Сначала продемонстрируй один объект, затем два независимых. Если второму объекту достаётся состояние первого, вероятно, ты спрятал mutable состояние в static переменную или singleton.

Урок 3. Сцена, редактор и первый игровой уровень

3.1. Уровень начинается с проверяемого сценария

Построй палубу из двух комнат, соединённых одной дверью. В первой находятся игрок и генератор, во второй — пробоина и инженер. Эта простая геометрия заставляет проверить сразу четыре связи: принадлежность объектов комнатам, путь через дверь, питание двери и выбор ремонтной задачи. Если сразу строить большой корабль, ошибка в одной связи затеряется среди десятков помещений. Сначала создай сцену, где каждое изменение видно.

В редакторе используй Scene Hierarchy для выбора сущности, Inspector для данных компонентов и gizmo для Transform. Quick Create удобен как начальный шаблон, но проверяй состав созданной сущности: `Crew NPC (logic)` может быть полноценной логикой без подходящей модели, клипов и физического контура. Визуальная модель не делает NPC способным ремонтировать, а наличие NPCComponent не создаёт автоматически правильно настроенный Bullet controller.

Сохрани авторскую сцену до Play. После входа в Play открой дверь, задай ручную команду, нанеси урон корпусу и выйди. Повтори загрузку сохранённой сцены: авторское состояние должно вернуться. Для игрового save нужен другой вопрос: какие изменения мира должны сохраниться как прогресс игрока? Не смешивай scene authoring и save game только потому, что оба записываются в JSON.

3.2. Набор компонентов для учебного сценария

Для комнаты нужны геометрия, принадлежность и данные среды. Для двери — геометрия, состояние, связь комнат, при необходимости питание и collider. Для пробоины — `HullBreachComponent` и `RepairableComponent`; пробоина может появиться от DamageSystem, поэтому

ручной шаблон должен имитировать те же обязательные данные. Для инженера — NPC, Brain, Movement, CrewMember, Transform, Active, character controller и анимация/модель, если нужна видимая фигура. Для противника дополнительно нужны `FactionComponent`, `NPCCombatComponent`, оружие или melee настройки. Это минимальные контракты, а конкретные фабрики в проекте могут добавлять ещё компоненты.

При настройке объекта задавай смысл числа. `repairDistance = 1,5` — метры от центра NPC до рабочей точки, а не «примерная близость». Размер capsule и расположение WorkZone должны допускать стоять в этой зоне без входа в collider оборудования. `detectionRange` — геометрический фильтр выбора задач, а не доказательство достижимости по NavMesh. `maxPressure` — предел материала корпуса в единицах системы давления. Если Inspector показывает число без единиц, запиши единицы в описание или имя поля.

3.3. Триггеры и игровой поток без нового C++

`TriggerZoneSystem` обнаруживает вход в область, `TriggerActionSystem` переводит событие в действие. Для первой игровой цели можно создать зону возле капитанского терминала: вход показывает сообщение «Восстановите питание двери», а включение генератора переводит сценарий к аварии во втором отсеке. Триггер не должен напрямую менять десяток несвязанных компонентов в обход систем. Он выдаёт событие или команду, а соответствующая gameplay система меняет своё состояние. Такой подход позволяет повторить сценарий после загрузки и понять, кто совершил переход.

Проверка первого уровня: новая игра запускается в правильной комнате; игрок может получить подсказку; дверь закрыта по понятной причине; генератор включает нужную цепь; инженер получает одну аварийную работу; при повторном входе в зону событие не дублируется бесконечно; после сохранения и загрузки текущий этап явно восстанавливается или по правилам начинается заново. Это уже кусочек игры, а не только демонстрация движка.

Урок 4. Математика как инструмент, а не список формул

4.1. Один численный пример проходит через весь движок

Пусть NPC находится в A = (2, 0, 3), а пробоина в B = (8, 0, 11). Разность B − A = (6, 0, 8) — направление и расстояние до цели в ненормированном виде. Горизонтальная длина равна $\sqrt{(6^2+8^2)}=10$ м. При рабочем радиусе 1,5 м NPC должен пройти приблизительно 8,5 м, если между ним и целью нет препятствий. Нормированный вектор (0,6, 0, 0,8) говорит лишь направление; он не содержит скорость. При скорости 2 м/с и dt=0,1 с желаемое смещение (0,12, 0, 0,16) м. Если трасса NavMesh обходит стену и имеет длину 15 м, время без остановок ближе к (15−1,5)/2=6,75 с, а не к 4,25 с по прямой.

Ту же пару точек можно использовать для FOV. Пусть NPC смотрит в +Z: forward=(0,0,1). Dot с нормированным направлением до пробоины равен 0,8; угол arccos(0,8)≈36,87°. При полном

конусе зрения 90° половинный угол 45°, точка внутри. Но для пробоины зрение может вообще не быть условием назначения: экипаж получает данные аварии от корабельной системы. Здесь различаются два типа знания. Геометрический dot проверяет направление взгляда, а игровое правило решает, должен ли конкретный NPC знать о событии.

Пусть после перемещения модель находится в world space, камера смотрит на неё, а shader получает вершину p. В текущем PBR vertex shader путь — `p_world = mul(p, W)`, затем `mul(p_world, V)`, затем `mul(p_view, P)`. Если вместо этого перемножить P и W в обратном порядке, перенос объекта повлияет на проекцию неверно. На CPU `PBRShaderClass.cpp` транспонирует матрицы перед загрузкой в constant buffer. Это не «исправление порядка»; это согласование хранения с HLSL. Проверяй геометрический результат на простом объекте: точка у origin, перенос +5 по X, затем наблюдение камерой.

4.2. Почему численные инварианты сильнее картинки

Картинка может выглядеть правдоподобно при неправильных единицах: корабль тонет слишком быстро, но игрок не знает нормальную скорость. Инвариант можно проверить. Скорость умножается на секунды и даёт метры. Мощность в ваттах умножается на секунды и даёт джоули. Поток воды в м³/с умножается на секунды и даёт м³. При перетоке воздуха между двумя закрытыми комнатами сумма массы остаётся постоянной. При повороте единичного направления чистым вращением длина остаётся 1. При преобразовании нормали и касательной после неравномерного масштаба dot остаётся близким к 0, если применена правильная normal matrix.

Задание. Измени скорость персонажа и частоту кадров, затем вручную предскажи расстояние за 5 секунд. Отдельно вычисли, какой score координатор даст инженеру и медику для одной пробоины, используя коэффициенты из `CrewTaskCoordinator`. Затем сравни с debug полями. Если формула и наблюдение расходятся, не подгоняй коэффициент сразу: проверь clamp, расстояние, текущую роль, уже занятый claim и порядок вызовов.

Урок 5. Bullet Physics от формы до игрового события

5.1. Физический мир и игровой мир отвечают на разные вопросы

Bullet знает тела, collision shapes, контакты, ограничения, массы, скорости и лучи. ECS знает, что конкретное тело является дверью, персонажем, ящиком или частью корпуса. Один слой не заменяет другой. Когда луч пересекает `btCollisionObject`, это геометрический факт. Чтобы решить, можно ли нанести урон, нужно сопоставить объект с Entity, проверить фракцию, здоровье, броню и тип поверхности. Поэтому физика не должна напрямую открывать дверь или выбирать ремонтную задачу.

В проекте `PhysicsWorld` создаёт и обслуживает Bullet world, а `RigidBodyComponent`, character controller и ECS системы связывают тела с сущностями. При чтении класса начни с жизненного цикла: кто создаёт shape, кто добавляет body в world, кто удаляет constraints и actions, кто

освобождает память при смене сцены. Ошибка порядка уничтожения может не проявиться в первом запуске, но даст падение после второй загрузки. Комментарий в `ApplicationClass` указывает важный контракт: ECS владеет Bullet actions, constraints, bodies и shapes; их надо освободить до разрушения PhysicsWorld.

5.2. Выбор тела: static, dynamic, kinematic

Стена палубы — static: она не интегрирует скорость, но участвует в контактах и raycast. Ящик — dynamic: масса больше нуля, столкновения и импульсы меняют скорость. Дверь, которую двигает анимация или логика открытия, может быть kinematic: положение задаёт система, Bullet использует его как препятствие. Для каждой Entity запиши владельца Transform. Если это dynamic, после физического шага мир Bullet является источником положения для рендера. Если kinematic, gameplay задаёт физике новую позу. Если static, положение меняется редко, и физическое представление нужно обновить вместе с Transform.

Типичная ошибка — перед каждой симуляцией ставить dynamic body в координаты Transform, а после шага читать позицию обратно. Получается борьба двух источников: тело отскакивает от стены, но Transform снова переносит его внутрь. Второй симптом — дрожащая модель, когда character controller и код AI одновременно пишут position. Исправление начинается не с коэффициента сглаживания, а с выбора владельца и направления синхронизации.

5.3. Collision shape не равен видимой модели

Для стен часто достаточно коробок или статической вогнутой сетки; для персонажа нужна капсула; для подвижного ящика — box или convex hull. Точная треугольная сетка для каждого динамического предмета повышает стоимость контактов и может вести себя хуже, чем простая форма. При этом слишком грубая форма создаёт игровые противоречия. Если в двери нарисовано отверстие, а collision shape остался сплошным, NPC не пройдёт и луч попадёт в невидимую стену. Если после разрезания корпуса визуальная пробоина появилась, а физическая форма не обновилась, игрок увидит проход, который физика не признаёт.

Отладка начинается с debug draw Bullet shapes одновременно с render mesh. Проверяй положение, масштаб, ориентацию, высоту капсулы, толщину порога и маску столкновения. Особенно опасен non-uniform scale: некоторые формы и алгоритмы не должны получать произвольное масштабирование после создания. Лучше пересоздать shape или использовать явно поддерживаемую операцию. Не делай вывод «NavMesh сломан», пока не увидел реальные физические препятствия.

5.4. Шаг симуляции и импульс

При fixed step 1/60 с контроллер получает одинаковые интервалы даже при неравномерном рендере. Накопитель времени не должен бесконечно догонять после паузы в отладчике: лимит подшагов предотвращает spiral of death. При ограничении нужно знать политику: потерянное

время отбрасывается или симуляция временно отстаёт? Это влияет на таймеры повреждений и NPC, поэтому physics time и gameplay time должны быть согласованы.

Импульс меняет количество движения. Для тела массой 10 кг импульс 30 Н·с по X даёт Δv=3 м/с. Сила 30 Н за один шаг 1/60 с даёт J≈0,5 Н·с и Δv≈0,05 м/с. Если взрыв кажется слабым после замены impulse на force, причина может быть именно в единицах. Приложенный не по центру масс импульс создаёт ещё и момент вращения. Для взрыва вычисли радиальный вектор, обработай нулевую длину, ограничь радиус и не создавай бесконечную силу при расстоянии 0.

5.5. Raycast в трёх разных механиках

Для выбора объекта редактором луч идёт от камеры через курсор. Для зрения NPC он идёт от точки глаза к допустимым точкам цели. Для оружия он начинается у дула после анимации. Все три луча используют геометрию, но правила после попадания разные. Редактору нужна Entity под курсором, NPC — доказательство видимости, оружию — первое препятствие и применимость урона. Если один общий helper сразу возвращает «враг виден» для любого луча, границы ответственности потеряны.

Пример. Глаз NPC находится над низким укрытием, дуло ниже его верхнего края. Луч глаза не встречает стену, а луч дула встречает. Правильное поведение — NPC видит игрока, но не стреляет до выхода в точку выглядывания. В текущем коде зрение проверяет `NpcCombatSystem`, а выстрел и линию огня — `NpcRangedCombatSystem`. Для разбора ошибки нарисуй оба луча разными цветами и подпиши hit Entity и расстояние.

Практика. Поставь дверь между NPC и игроком. Проверь закрытую, открытую и сломанную дверь: видимость, путь и физический контакт должны совпадать с намерением дизайна. Повтори с пробоиной корпуса и ремонтной операцией. Зафиксируй, какая система обновляет collision после изменения геометрии и на каком кадре результат становится доступен raycast.

Урок 6. Рендер DirectX 11 и смысл каждого шейдера

6.1. CPU готовит команду, GPU выполняет её

Renderer не решает, сколько у персонажа здоровья и какую аварию выбрать. Он читает состояние сцены и строит команды рисования. CPU выбирает mesh, material, shader, transform, ресурсы и состояния. GPU читает vertex/index buffers и выполняет стадии графического конвейера. `Renderer::RenderFrame` — точка входа кадра в рендер. Он готовит тени, при необходимости reflection/refraction, затем основную сцену, прозрачность и интерфейс. Наличие условных off-screen passes означает, что изменение материала воды может добавить целый дополнительный проход, а не только несколько операций pixel shader.

Чтобы понять стоимость кадра, раздели её на количество draw calls, количество геометрии, стоимость pixel shader и число проходов. Увеличение размера экрана почти не меняет число вершин, но может резко повысить стоимость пиксельного освещения и прозрачности. Добавление

shadow pass повторно рисует геометрию с точки света. Сначала измерь, где узкое место: CPU подготовка draw list или GPU пиксели. «Оптимизировать рендер» без такого разделения слишком расплывчато.

6.2. Что делает вершинный шейдер

Открой `src/Render/Shader/Vs/pbr.vs`. Vertex input содержит позицию, UV, normal, tangent, binormal и до четырёх весов костей. Для skinned модели сначала применяется bone transform каждого валидного влияния, результаты взвешиваются и нормируются по сумме весов. Статичный mesh не должен использовать оставшуюся в b8 матрицу кости от предыдущего draw: текущий код специально оставляет исходную позицию, когда валидных влияний нет. После этого вершина переводится через world, view и projection. Выход `SV_POSITION` задаёт положение для растеризации; worldPos, UV и базис TBN передаются пиксельному шейдеру.

Почему vertex shader не вычисляет весь PBR цвет? Свет может меняться внутри одного большого треугольника. Если осветить только три вершины и интерполировать готовый цвет, блик внутри треугольника может исчезнуть. Передача worldPos и нормали позволяет PS вычислять освещение для фрагмента. Но чем больше интерполяторов, тем выше стоимость и риск превышения лимитов. В проекте векторы до четырёх источников передаются через явные TEXCOORD слоты; при изменении структуры нужно синхронно менять VS output и PS input.

6.3. Что делает растеризатор и пиксельный шейдер

Растеризатор определяет, какие пиксели покрывает треугольник, и интерполирует данные вершин. Pixel shader в `src/Render/Shader/Ps/pbr.ps` получает UV, normal, tangent, worldPos и параметры освещения. Он читает albedo, normal, metalness, roughness, shadow map, environment и маску повреждений. Результат проходит тест глубины и, возможно, blending. Поэтому прозрачность — не просто `alpha < 1`: порядок draw, depth write и blend state определяют правильность картинки. Стекло, нарисованное раньше непрозрачной стены и записавшее depth, может скрыть стену позади.

PBR требует согласованного цветового пространства. Albedo, записанный в sRGB, надо декодировать в линейное пространство до вычисления света. Значение 0,5 sRGB соответствует примерно 0,214 linear. Normal map — векторные данные, их не следует декодировать как цвет. После линейного освещения HDR результат проходит exposure, tone mapping и кодирование для вывода. Если одновременно применять gamma в шейдере и использовать sRGB render target, сцена окажется слишком светлой или тёмной. Для каждой текстуры и render target веди явную таблицу «что хранит и в каком пространстве».

6.4. Constant buffer — контракт байтов

В HLSL `register(b0)` относится к конкретной стадии. В текущем PBR VS b0 — MatrixBuffer, а в PS b0 — LightColorBuffer. Это не конфликт: VS и PS привязывают разные наборы буферов. C++ структура должна совпадать с HLSL по полям и упаковке; размер D3D11 constant buffer кратен 16 байтам.

При добавлении нового параметра проверь HLSL declaration, C++ struct, заполнение перед draw и правильный bind slot. Иначе эффект может читать старые байты предыдущего объекта, а «исправление» цвета константой лишь замаскирует ошибку.

Матрицы в `PBRShaderClass.cpp` транспонируются перед загрузкой. Не делай двойное транспонирование ради визуально похожего результата в одной сцене. Проверь model point (1,0,0) с переносом +5 по X и поворотом 90° на тестовой модели, а потом проверь skinned mesh и normal. Такой тест разграничивает ошибку матрицы, ошибки в весах костей и ошибку исходной оси модели.

6.5. Добавление шейдерного эффекта через gameplay

Допустим, горячий реактор должен светиться красным. Сначала gameplay считает температуру `T` и предел `Tmax`; render snapshot получает `heat01 = clamp((T–Tstart)/(Tmax–Tstart), 0, 1)`. PS использует heat01 для emissive цвета. Привязка не должна менять температуру: shader не возвращает gameplay результат. Добавь параметр в C++ и HLSL, выставляй его для каждого draw, предусмотрев 0 для объектов без HeatComponent. Если параметр остаётся от предыдущего материала, новый объект неожиданно светится. После изменения проверь основной проход, тени, reflection и прозрачность, потому что разные passes могут использовать разные shaders и состояние контекста.

Практика. Возьми один куб с PBR материалом. Проверь albedo без света, один источник, roughness 0,1 и 0,9, normal map, shadow, emissive и non-uniform scale. Для каждого опыта ожидаемый результат сформулируй заранее. Если свет меняется при перемещении камеры как будто приклеен к ней, worldPos или light direction оказались в разных пространствах.

Урок 7. Корабль как связанная система, а не набор HUD чисел

7.1. Причинная цепь аварии

Космический симулятор становится игрой, когда событие имеет последствия. Снаряд попал в корпус. `DamageSystem` создаёт повреждение или пробоину. Пробоина влияет на комнату: воздух выходит, давление падает; при наличии воды `FloodingSystem` считает приток. Повреждённый генератор уменьшает доступную мощность. Дверь или насос теряют питание. Экипаж получает аварийную задачу, но путь к ней может измениться. Игрок видит HUD, слышит тревогу и выбирает действие. Если каждый шаг соединён только визуальным эффектом, симуляция не существует.

Удобно составить таблицу для каждого ресурса: владелец, единицы, источник, потребитель, период обновления и правило сохранения. Вода: м³ в отсеке, м³/с через отверстие или насос. Энергия: текущая мощность МВт, запас аккумулятора — энергия, а не МВт. Воздух: масса, давление, температура и объём. Повреждение: health и максимальное значение. Смешивание единиц заставляет «балансировать» систему случайными коэффициентами. Сначала добейся правильной размерности, потом выбирай игровые масштабы.

7.2. Давление и затопление на числах

`PressureSystem.cpp` рассчитывает внешнее давление как атмосферное плюс `rho·g·depth`. При rho=1000 кг/м³, g=9,81 м/с² и depth=10 м добавка равна 98 100 Па. Это приблизительно ещё одна атмосфера к атмосферному давлению поверхности. Если maxPressure корпуса ниже этого значения, код начисляет урон пропорционально превышению. Для космического корабля водная модель может быть использована только в специфической сцене; не переноси её автоматически на вакуум. Для разгерметизации в космосе нужен поток газа и разность внутреннего/внешнего давления, а не гидростатическое давление воды.

`FloodingSystem.cpp` суммирует приток пробоин и отток насоса, затем меняет объём на `(inflow−outflow)·dt`. При притоке 0,3 м³/с и насосе 0,1 м³/с за 5 с добавится 1 м³ воды. Условие `hasPower` решает, может ли насос работать; поэтому порядок вычисления сети питания и затопления в кадре важен. Если отток больше текущего объёма, clamp предотвращает отрицательную воду. Если пробоину запечатали, поток должен стать 0, а не продолжать использовать прежний cache.

7.3. Сеть питания и brownout

`ShipPowerGridSystem.h` собирает источники и потребителей, обнуляет результат прошлого кадра, подключает аварийный аккумулятор и распределяет доступную мощность по приоритетам. `PowerConsumerComponent` хранит `requiredMw`, `suppliedMw`, `powerRatio` и `hasPower`. Сейчас `hasPower` означает почти полное обеспечение (`powerRatio >= 0,999`), поэтому устройству с 70% мощности не обязательно разрешено работать. Если будущий насос должен иметь плавную производительность, вычисляй `effectivePumpRate = nominalRate · powerRatio` в его системе и явно определи нижний порог. Не меняй смысл hasPower тайно для одного устройства.

Пример: генератор даёт 5 МВт, жизненно важные потребители просят 3 МВт, остальные 4 МВт. Первые получают 3, остаток 2 делится согласно текущей политике между потребителями следующего приоритета. Это не означает, что каждый из четырёх одинаково нуждающихся объектов получит полный запрос. После включения аккумулятора available может временно вырасти. В HUD показывай generation, demand, battery output, supplied и причину отказа конкретного устройства. Число «энергия 50%» без объяснения недостаточно для решения игрока.

7.4. Связь симуляции и игрового решения

Создай ситуацию: во втором отсеке падает давление, генератор повреждён, дверь требует питание. Инженер получает ремонтную задачу, но не может пройти. Что должно произойти? В текущем коде путь может стать Blocked. Для полной игры нужны правила реакции: запросить другую доступную аварию, идти к ручному переключателю, дождаться восстановления питания или сообщить игроку. Не добавляй все варианты сразу. Сначала обеспечь наблюдаемость: «задача назначена», «путь через дверь N», «дверь без питания», «назначение освобождено». После этого добавляй новую работу.

Практика. В тестовой сцене отключи генератор. Запиши, какие потребители потеряли питание и почему. Добавь одну пробоину и проверь, что скорость утечки, HUD и ремонт связаны. Повтори после загрузки сцены и при другой частоте кадров. Нарушенная размерность, неправильный порядок системы или несохранённая связь Entity должны давать разные диагностические признаки.

Урок 8. Дружественный NPC от аварии до завершённого ремонта

8.1. Раздели обнаружение, назначение, путь и работу

Слова «NPC чинит пробоину» скрывают четыре разных алгоритма. Пробоина сначала должна существовать в ECS и быть незапечатанной. Координатор затем должен назначить работу подходящему члену экипажа. Исполнитель должен привести персонажа к достижимой рабочей точке. Наконец, RepairSystem должен накопить прогресс и изменить мир. Если объединить все четыре операции в `NPCComponent::Update`, сложно будет поддержать ручной приказ, несколько работников, закрытые двери и другой тип задачи.

В текущем коде `DamageSystem` может породить Entity пробоины с `HullBreachComponent` и `RepairableComponent`. `NpcUtilitySystem` вычисляет полезность ремонта. `CrewTaskCoordinator` выдаёт claim. `StateSystem::UpdateRepairTask` занимается перемещением и взаимодействием. `RepairSystem::ApplyRepairProgress` и `CompleteRepair` меняют состояние цели. Такая цепочка важна ещё и для отладки: каждый слой может ответить, почему остановился. «Пробоины нет» отличается от «никто не назначен», «дверь недоступна» и «асинхронное восстановление меша ещё не завершилось».

8.2. Выбор работника на численном примере

Координатор рассматривает доступные цели с Repairable. Для пробоины серьёзность берётся из `leakRate/0,15`, ограниченного интервалом 0,2–1; для оборудования — из доли потерянного здоровья. Близость равна `1–distance/detectionRange` с ограничением 0–1. Дальше используется смесь `0,65·severity + 0,35·distanceScore`, коэффициент роли и `repairSkill`, а итог ограничивается 0–2. Инженер имеет больший коэффициент, чем медик. Это не магический «AI»: каждый множитель имеет игровое объяснение.

Пусть пробоина имеет leakRate=0,12, а инженер находится на половине своего detectionRange. Серьёзность 0,8, близость 0,5. База 0,8·0,65 + 0,5·0,35 = 0,695. Инженер с roleAffinity=1,25 и skill=1,5 получает около 1,303. Медик с 0,62 и skill=1 получает около 0,431. Если score инженера неожиданно нулевой, проверь предел detectionRange, отсутствие Transform, активность NPC и то, не занята ли задача. Если score высок, но назначение не произошло, проверь активную ручную команду и claim другого работника.

У текущего алгоритма геометрическая близость не тождественна длине маршрута. В двух комнатах через запертую дверь цель может быть в 2 м по прямой и недостижимой физически. Для

надёжного экипажа добавь проверку достижимости до окончательного назначения либо быстро освобождай claim после понятного отказа `Blocked`. Иначе один NPC будет навсегда владеть задачей, которую не способен выполнить, а остальные её не возьмут.

8.3. Почему claim находится на цели

`CrewTaskClaimComponent` висит на ремонтируемой Entity и хранит assignee, тип, score и время назначения. Это делает занятость видимой для любой системы и Inspector. `CrewMemberComponent` одновременно хранит assignedTask — удобную обратную ссылку. Две записи требуют согласованного обновления: если одна очищена, а другая нет, мир думает, что задача и занята, и свободна. Координатор начинает проход с очистки недействительных claims, затем сохраняет существующее назначение, если оно ещё корректно, и только после этого ищет новое. Сохранение назначения предотвращает переключение цели от малых колебаний score.

При ручной команде `NpcCommandSystem` система освобождает автоматическое назначение и отдаёт исполнителю ручной приоритет. Когда команда завершается или вызывается `ResumeAutomatic`, маршрут старой команды сбрасывается. Это нужно, даже если конечная точка кажется похожей: старый путь мог идти через другую дверь. При смерти NPC, удалении цели, смене сцены и завершении ремонта claim должен исчезнуть. Для save game обычно сохраняют мировое состояние задачи, но runtime claim безопаснее восстановить заново из живых NPC и доступных задач.

8.4. Исполнение маршрута и рабочая зона

`StateSystem::UpdateRepairTask` перечитывает цель каждый кадр. Для HullBreach берёт `worldPosition` и room. Для оборудования — Transform, RoomMembership и при наличии `WorkZoneComponent` подходящую точку работы. Почему нельзя просто идти к центру модели? У генератора origin может быть в центре большого корпуса, куда капсула физически не войдёт. WorkZone задаёт свободное место перед панелью. Её нужно проверить на достижимость NavMesh и расстояние до самого объекта.

До прибытия NPC находится в `MoveToRepair`. Навигация может переводить его в `WaitingForDoor` и затем обратно, а при невозможности продолжить — в `Blocked`. `StateSystem` задаёт CharacterMovement, но не телепортирует Transform. `CharacterControllerSystem` применяет движение к Bullet capsule. Анимация читает итоговые параметры движения и выбирает Idle/Walk/Run. При достижении горизонтального радиуса `repairDistance` NPC останавливается. Порог должен учитывать капсулу, WorkZone и положение пробоины; слишком маленький порог заставит его вечно толкаться в стену.

8.5. Прогресс не зависит от FPS

В NPC ветке текущего `StateSystem` прирост вычисляется из `requiredProgress`, skill, dt и repairDuration. При requiredProgress=1, duration=8 с, skill=1,5 за dt=0,1 с прирост 0,01875. За 50 таких шагов получится 0,9375; после примерно 54 шагов ремонт завершится. Если за один кадр 0,2

с прогресс возрастёт вдвое относительно 0,1 с, это ожидаемо. Если время ремонта в 120 FPS вдвое меньше, где-то прирост применяется дважды или dt перепутан.

`RepairSystem::ApplyRepairProgress` — общая точка изменения цели. Когда прогресс достигает порога, завершение может удалить Entity пробоины или запустить async восстановление геометрии. Поэтому после вызова запрещено обращаться к ранее полученной ссылке на Repairable как будто она гарантированно жива. Повторно запроси компонент через Registry, если нужна дальнейшая проверка. В журнал мира запиши факт завершения; HUD и звук должны реагировать на событие, а не угадывать завершение по исчезновению mesh.

8.6. Отладочная таблица причин

Если NPC стоит на месте, проверь по порядку: `ActiveComponent`, `CrewMember.onDuty`, `NPCComponent::isHostile` и Faction/Combat, активную ручную команду, доступные Repairable цели, score, claim, repairTarget, состояние NavMesh, room, дверь, WorkZone, distance, `progress`, async job. Порядок важен. Начинать с настройки анимации бессмысленно, если claim вообще не создан. Если State=Repairing, но прогресс не меняется, смотри dt, skill, requiredProgress и вызов ApplyRepairProgress. Если прогресс дошёл до порога, но пробоина видна, смотри завершение runtime mesh operation и состояние collider.

Практика. Создай три варианта одной сцены: открытая дверь, закрытая дверь с питанием, закрытая дверь без питания. Укажи ожидаемый State и status на каждом этапе. Затем вручную прикажи инженеру чинить другое оборудование и вызови ResumeAutomatic. Проверка считается успешной, когда NPC меняет работу без старого маршрута, прежний claim освобождается, а пробоины не получают второго исполнителя без явного правила совместной работы.

Урок 9. Вражеский NPC от слуха и зрения до попадания

9.1. Цель должна быть допустимой

Прежде чем считать расстояние, определи отношения фракций. `FactionComponent` хранит сторону; `AreEnemies` сейчас считает разные ненейтральные стороны враждебными, одинаковые дружественными, Neutral не атакует и не становится целью по этому правилу. Старый `isHostile` ещё встречается для совместимости сцен и разграничения crew utility, но новый боевой NPC должен иметь FactionComponent. Боевая система должна проверять живую, активную и допустимую цель. Иначе мёртвый объект останется целью или NPC начнёт стрелять в союзника, потому что тот ближе.

Для каждой попытки выбора цели записывай причины: не враждебна, слишком далеко, вне FOV, закрыта, неактивна, мертва. Один флаг `targetVisible=false` не объясняет, почему враг не реагирует. Если несколько врагов, порядок обхода контейнера не должен неявно определять цель при одинаковом score: используй явное правило, например расстояние, угрозу и устойчивую развязку по Entity ID.

9.2. Геометрия обзора и реальный raycast

Видимость состоит из быстрого фильтра и дорогой проверки. Сначала расстояние: квадрат длины между NPC и целью сравнивается с квадратом дальности. Затем горизонтальный FOV: dot единичного forward и единичного направления к цели сравнивается с cos половины полного угла. Для FOV 90° порог около 0,707. Цель под 60° к forward даёт dot=0,5 и не проходит. Дешёвые фильтры сокращают число лучей, но не доказывают видимость: стена между NPC и игроком проходит FOV и distance, а raycast должен её обнаружить.

`NpcCombatSystem` проверяет лучи к разным частям цели. Глаз Kevin может браться из кости головы; точка цели может быть выше или ниже центра Entity. Это уменьшает ошибку, когда персонаж стоит за укрытием и видна только голова. Но луч должен стартовать в правильной мировой точке после обновления позы или с понятным fallback, иначе он начнётся внутри собственной капсулы. Для каждой проверки исключай собственное тело и проверяй первое столкновение.

Личное зрение обновляет `targetVisible` и `visibleAimPoint`. При потере контакта NPC хранит lastKnownPosition и время наблюдения. Если когда-нибудь введёшь слух, он должен создавать отдельное свидетельство с меньшей точностью: «звук в области комнаты B», а не «точный Transform игрока». Сообщение товарища может передать последнюю увиденную позицию, но не флаг личной видимости. Это защищает игру от всеведущего AI и делает укрытие игрока осмысленным.

9.3. Состояния и тактика не должны драться за одну запись

После наблюдения `NpcCombatSystem` переводит мозг в Chasing, Attacking или Searching. `NpcCoverSystem` выбирает индивидуальное укрытие и может удерживать уже выбранный маршрут, чтобы новая проверка видимости не отменяла его каждый кадр. `StateSystem` исполняет итоговое состояние: строит путь, выставляет CharacterMovement и останавливает NPC в нужной точке. Тактический компонент хранит память о входящем огне, укрытии, очереди, поиске. Если две системы каждый кадр без приоритета перезаписывают `brain.state`, NPC начнёт дёргаться между укрытием и атакой.

Изучи текущий комментарий в `ApplicationClass.cpp`: `NpcSquadCombatSystem::Update` закомментирован. Поэтому роли прикрывающего и фланкера в старых учебных практикумах являются проектом для будущего, а не гарантированной работой сегодняшней сцены. Индивидуальные короткие очереди, перезарядка, укрытие и поиск не равны групповому подавлению. Не балансируй уровни исходя из того, что отряд уже координируется.

9.4. Выстрел — отдельное разрешение

Наличие состояния Attacking ещё не создаёт пулю. `NpcRangedCombatSystem` проверяет активность, мозг, фракцию, здоровье, ручной приказ, экипированное оружие, патроны/перезарядку, личную видимость, задержку прицеливания, допустимую цель, ориентацию, выход из дула и союзника на центральной линии. Только затем вызывает `WeaponSystem::TryFire`. Последний управляет

оружейным cooldown, разбросом, физическим лучом и событием урона. Разделение сохраняет одно правило оружия для игрока и NPC.

Пример. Враг видит игрока через низкое укрытие, но дуло находится за стенкой. NpcCombatSystem оставляет targetVisible=true, а NpcRangedCombatSystem должен отказать в выстреле из-за первого collision от muzzle ray. Если заменить всё одним флагом «виден», NPC стрелял бы сквозь укрытие. Другой пример: между стрелком и игроком проходит живой союзник. Центральный луч должен удержать огонь. Из-за разброса это не математическая гарантия отсутствия случайного friendly fire, так что game design может дополнительно расширить безопасный коридор.

9.5. Попадание, реакция и смерть

Попадание поступает в DamageSystem. Он изменяет здоровье и может пометить NPC как находящегося под огнём. Тот при следующем решении вправе выбрать укрытие или отступление. Не позволяй pixel shader решать величину урона: он может вообще не выполнить fragment из-за culling или depth, а gameplay должен быть независим от камеры. После смерти проверь прекращение движения и огня, освобождение cover reservation и crew claim, очистку цели у товарищей и корректное повторное создание сцены.

Практика. Построй пять сцен: открытое поле; стена; низкое укрытие; союзник на линии огня; дверь без collider. Для каждой заранее напиши ожидаемые visible, brain.state и rangedStatus. Запусти при одинаковом начальном положении и сравни с наблюдением. Не исправляй AI, пока не исключил ошибку collision shape или muzzle socket.

Урок 10. NavMesh, двери и движение персонажа

10.1. Маршрут не является движением

Recast строит область, по которой может пройти агент с заданными радиусом, высотой и шагом. Detour отвечает на запрос пути. `NpcNavMeshSystem` находит маршрут, а `StateSystem` превращает его в ближайшую цель движения. `CharacterControllerSystem` передаёт желание движения Bullet капсуле. Это три разных результата: «маршрут существует», «NPC хочет идти» и «NPC реально продвинулся». Первое не гарантирует третье. Физический ящик может перекрыть коридор после построения NavMesh, дверь может измениться, а радиус агента может не совпасть с капсулой.

Если NavMesh строится для радиуса 0,3 м, а фактическая капсула имеет радиус 0,5 м, путь вдоль стены может формально существовать, но персонаж будет тереться о неё. Если высота двери на один сантиметр ниже допустимой высоты агента, проектирование target на NavMesh может провалиться. Параметры сборки надо брать из реального character controller и проверять на минимальной сцене с известной шириной прохода.

10.2. Дверь — графовая и физическая связь

Маршрут между комнатами может использовать граф соединений дверей. Само открытие двери — не мгновенное освобождение прохода: створка ещё движется и collider может оставаться на траектории. Поэтому есть `WaitingForDoor`. Если дверь требует питание, `TryOpen` может отказать. В этом случае StateSystem должен записать понятную причину, а задача NPC — получить альтернативный путь или быть освобождённой. Бесконечный цикл «попросить открыть → не удалось → повторить каждый кадр» перегружает и AI, и debug.

Дверь влияет сразу на несколько систем. Collider блокирует движение и raycast; граф NavMesh влияет на маршрут; room connection влияет на атмосферу; power state влияет на возможность открыть; анимация показывает створку. Изменение одной подсистемы без остальных создаст парадокс: NPC проходит через закрытую дверь или воздух течёт через визуально герметичную. При новой двери выпиши все эти контракты и проверь их отдельно.

10.3. Застревание и перепланирование

Сравни фактическую позицию NPC через интервал. Если он хотел идти, но продвинулся меньше порога, увеличивай stuck timer. После тайм-аута сбрось или перестрой маршрут, но ограничь количество попыток и дай причину отказа. Не перестраивай путь каждый кадр: это дорого и может заставить NPC менять ближайший waypoint до того, как он к нему приблизился. Перепланирование требуется при изменении цели, комнаты, двери, проходимости или долгом отсутствии прогресса.

Практика. Поставь NPC за дверью, а целевую точку внутри комнаты; добавь второй путь в обход. Сначала дверь открыта, затем выключи питание, затем заблокируй проход физическим ящиком. Для каждого случая объясни отдельно результат графового поиска, намерение StateSystem и реальное движение Bullet. Отладочные линии маршрута и collision shapes должны показывать разные сущности задачи.

Урок 11. Как добавлять системы без разрушения архитектуры

11.1. Сначала опиши игровое правило

Возьмём будущую систему пожара. Слишком общий запрос «добавить FireSystem» не говорит, когда огонь появляется, что он расходует, кто узнаёт о нём и как он исчезает. Сформулируй поведение: повреждённый реактор создаёт очаг в своей комнате; интенсивность растёт при наличии топлива и кислорода; огонь повышает температуру, повреждает оборудование и создаёт видимый дым; инженер или игрок может тушить его, расходуя ресурс. Если кислорода нет, интенсивность должна падать. Уже здесь появляются проверяемые предельные случаи и единицы.

Запиши инварианты. Интенсивность не отрицательна и ограничена выбранным максимумом. Один источник не создаёт бесконечные дубликаты FireComponent. Урон применяется не чаще и не сильнее, чем задают dt и коэффициент. Выключенная сцена не продолжает фоновые jobs. Завершённый огонь не оставляет claim экипажа. Сохранение и загрузка восстанавливают ровно один очаг с ожидаемой интенсивностью. Инварианты задают границы реализации лучше, чем длинный список красивых эффектов.

11.2. Раздели данные, решение и исполнение

`FireComponent` хранит состояние очага. `FireSystem` обновляет физическую/игровую интенсивность, читая room environment и источник. `CrewTaskCoordinator` решает, кто тушит. `StateSystem` ведёт NPC к точке работы, но сам не вычисляет химическое горение. `ExtinguishSystem` или метод FireSystem принимает действие тушения и уменьшает интенсивность. Renderer рисует огонь по уже вычисленному heat/intensity. Звук и тревога читают WorldEvent. Система, которая одновременно вычисляет температуру, строит NavMesh, создаёт particle texture и записывает Scene JSON, станет хрупкой после первой новой задачи.

Для нового типа работы экипажа расширь `CrewTaskType`, сбор кандидатов, score, claim, StateSystem исполнение, имена состояний, Inspector, сериализацию authoring полей и диагностику. Для ручного приказа расширь `NpcCommandType`, публичный метод `NpcCommandSystem` и обработку StateSystem. Не полагайся на прямую запись `brain.command.type` из каждой кнопки UI: общий метод сбрасывает старый маршрут и увеличивает revision. Для врага новый retreat можно сначала выразить существующим состоянием TakingCover и тактической точкой; отдельный enum нужен лишь когда исполнение действительно другое.

11.3. Вставка в кадр и зависимость по данным

Назови производителя и потребителя каждого поля. Если PowerGrid выставляет `hasPower`, а насос в FloodingSystem читает его, PowerGrid должен отработать раньше насоса для эффекта в том же тике. Если FireSystem меняет health оборудования через DamageSystem, порядок событий определяет, когда координатор увидит новую работу. Избегай скрытых циклов: «Power зависит от Damage, Damage зависит от Heat, Heat зависит от Power». Для такой модели выбери явные фазы и состояние прошлого тика. Тогда задержка в один тик становится договором системы, а не ошибкой порядка.

Псевдокод шага новой системы показывает только распределение ответственности:

```
void FireSystem::Update(ECSRegistry& ecs, float dt, EventQueue& out) {
    for (auto& [entity, fire] : ecs.GetComponents<FireComponent>()) {
        if (!ecs.IsAlive(entity) || !fire.enabled) continue;
        const float oxygen = ReadRoomOxygen(ecs, fire.room);
        const float growth = ComputeGrowth(fire, oxygen) * dt;
        fire.intensity = std::clamp(fire.intensity + growth, 0.0f, 1.0f);
        if (CrossedAlarmThreshold(fire)) out.Push(FireAlarm{entity});
    }
}
```

Это схема, не готовый код проекта: `EventQueue`, `ReadRoomOxygen` и тип FireAlarm надо связать с реальными `WorldEvent` и данными комнаты. Важны проверка Entity, dt в секундах, clamp и событие порога, которое не должно отправляться заново каждый кадр. После реализации сравни 10 секунд симуляции при 30 и 120 FPS.

11.4. Узкий вертикальный срез

Первый рабочий вариант включает один очаг, одну комнату, одно устройство, одного NPC и одну кнопку ручного тушения. Не добавляй распространение по всем отсекам, частицы, звук, обугливание меша и сложную тактику в один коммит. Когда минимальный путь работает, добавляй второй очаг и второго NPC: именно тогда проявятся конфликты claim и приоритетов. Потом — закрытую дверь, отсутствие ресурса, паузу, save/load и объект вне камеры. Каждый следующий случай проверяет новый контракт, а не просто делает сцену красивее.

Урок 12. Анимация, оружие и согласование позы

12.1. Кто двигает персонажа

У игрового персонажа есть несколько источников движения: намерение AI или input, физическая капсула, root motion клипа, поворот корпуса и визуальная поза костей. Нужно выбрать владельца перемещения в мире. В текущем NPC пути `StateSystem` выставляет CharacterMovement, `CharacterControllerSystem` двигает капсулу, `AnimationController` выбирает визуальный клип. Если импортированная анимация Walk одновременно переносит root bone по миру, NPC может двигаться вдвое быстрее или шагать назад относительно collider. Для этого проекта root motion надо либо исключить из локомоции, либо явно интегрировать в контроллер как единственный источник смещения.

Скорость клипа и скорость капсулы должны согласовываться. Если шаг визуально соответствует 1,5 м/с, а контроллер движется 4 м/с, ноги будут скользить. Можно менять playback speed пропорционально реальной горизонтальной скорости, но сначала измерь длину шага и длительность цикла клипа. При резком переходе из Idle в Run полезен blend, однако blend не исправит неверный facing. Модель может иметь собственную «ось вперёд», отличную от +Z мира; это должно быть одной настройкой модели, а не серией дополнительных π в разных системах.

12.2. Поза до дула

Вооружённый NPC должен стрелять из muzzle socket актуальной позы. Порядок: AI выбирает огневое состояние, facing разворачивает персонажа, animation обновляет кости, weapon IK или socket расчёт получает мировой transform дула, затем `NpcRangedCombatSystem` формирует луч. Если muzzle рассчитывается до анимации, он на кадр отстаёт при развороте. Если кость в локальном пространстве умножена не на те родительские transforms, пуля стартует у ног или в центре сцены.

Проверка: нарисуй в debug мировую точку дула, его forward и луч на 2 м. Сравни Idle, Walk, Run, Aim и Reload, а также first-person и third-person модель. Сначала проверь отсутствие клипа: fallback socket должен иметь объяснимое положение. Потом включай анимацию и IK. Если прицел визуально наведён, а луч направлен иначе, изолируй model forward, socket rotation, bone global transform и направление камеры/цели.

12.3. Оружейные числа имеют физический и игровой смысл

`range` — длина физического луча. `fireRate` или cooldown определяет интервал попыток выстрела. `spread` определяет распределение направления, а не гарантию попадания. `damage` применяется после геометрического hit с учётом допустимой цели. `rangedAimDelay` описывает время подготовки NPC. Для коротких очередей тактический компонент хранит число оставшихся выстрелов и паузу. Если оружие пусто, `WeaponSystem::RequestReload` запускает перезарядку; визуальная анимация должна соответствовать этому runtime состоянию, но не быть единственным владельцем патронов.

Для баланса сравни expected time to kill на бумаге с наблюдением. Если у цели 100 health, выстрел наносит 20, фактический интервал между попаданиями 0,5 с, то для пяти точных попаданий от первого до пятого проходит минимум 2 с плюс aim delay. Разброс, укрытие и перезарядка увеличивают время. Если игра ощущается нечестной, не обязательно уменьшать damage: возможно, враг узнаёт скрытую позицию или стреляет без видимого окна прицеливания. Сначала покажи игроку причину угрозы.

Урок 13. Сохранение, фоновые задачи и устойчивость сцены

13.1. Два типа сохранения

Scene JSON хранит авторское устройство уровня: объекты, Transform, настройки компонентов, связи и редакторские значения. Save game хранит последствия игры: повреждения, состояние дверей, запасы, положение персонажей, текущий этап сценария и мировое время. Эти форматы могут использовать общую сериализацию полей, но их назначение различно. Если scene save записывает текущие runtime повреждения после Play, автор может случайно перезаписать исходную палубу аварией. Если save game хранит только исходную сцену, игрок потеряет прогресс.

При загрузке не доверяй сохранённому числовому Entity ID как runtime ID. SceneManager создаёт новые Entity и remap ссылок. Сначала загрузи сущности и их данные, затем восстанови связи, затем пересоздай runtime ресурсы: Bullet bodies, GPU buffers, path caches, claims и временные указатели. Для сохранения на диск используй временный файл и атомарную замену, чтобы сбой питания не оставил обрезанный основной save. Перед принятием новой версии формата проверь старый save и значения по умолчанию новых полей.

13.2. Что не должно попадать в JSON

Адрес `btRigidBody*`, COM interface pointer, указатель на компонент в unordered_map, ID GPU buffer и `std::future` не являются долгосрочным состоянием игры. После перезапуска они недействительны. Сохраняй данные, из которых их можно восстановить: transform, shape type/параметры, путь asset, настройки материала, логическую связь. Runtime claim экипажа лучше пересчитать из существующих задач, чем восстанавливать старый указатель на assignee. Для асинхронной операции сохрани устойчивое игровое состояние, а работу пересоздай или заверши по правилам загрузки.

13.3. JobSystem и граница главного потока

Фоновый worker подходит для чтения/разбора asset, расчёта геометрии или чистого path query на неизменяемом снимке. Он не должен напрямую создавать D3D11 ресурсы на устройстве без установленного контракта потоков, менять ECS Registry во время обхода главного потока или обращаться к сцене после её уничтожения. Для операции mesh repair типичный путь: main thread делает снимок входных данных и revision; worker вычисляет результат; main thread проверяет, что сцена и Entity ещё актуальны; только затем применяет ECS изменение и GPU upload.

Представь: игрок запускает восстановление меша, сразу загружает другую сцену, а job заканчивает на старом указателе SceneManager. Если completion не проверяет generation сцены или cancel token, получится use-after-free. На shutdown сначала перестань принимать новые задания, дождись или отмени существующие согласно контракту, обработай оставшиеся main-thread completions безопасно, затем освободи ECS, Bullet и D3D. Порядок важнее одного дополнительного mutex.

Практика. Сохрани и загрузи сцену с дверью, пробоиной, инженером и оружием. Сравни числа объектов и связи до/после. Затем начни асинхронный импорт модели и смени сцену до завершения. Ожидаемый результат — отсутствие падения, применения к новой сцене и утечки. Если такой сценарий пока не проходит, оставь его явным release blocker, а не скрытой заметкой.

Урок 14. Интерфейс, звук и понятная обратная связь

14.1. HUD должен объяснять состояние мира

Игроку не нужна отладочная таблица всех ECS компонентов, но ему необходима информация для решения. Если дверь не открылась, покажи «нет питания» или «заблокирована», а не молча оставь кнопку без эффекта. Если экипаж пошёл к аварии, покажи назначенного работника и примерное местоположение. Если давление падает, показывай отсек, направление изменения и срочность. Debug UI может быть гораздо подробнее: точные численные значения, Entity ID, score, last transition и текст отказа. Раздели presentation layer для игрока и diagnostics для разработчика, оставив общим источник истины.

Интерфейс должен читать события и snapshot состояния после симуляции. Не делай кнопку HUD единственной точкой, которая физически чинит корпус: ту же команду могут вызвать NPC, скрипт и контроллер игрока. UI инициирует действие, игровая система валидирует его и публикует результат. Так ошибка «нажали кнопку, но ремонт не произошёл» имеет проверяемое место: input, command, validation, execution или отображение.

14.2. Звук как информация, а не только украшение

В текущем дереве исходников не видно полноценного отдельного аудиомодуля, поэтому этот подраздел описывает проектируемый контракт. Звук пробоины, выстрела и тревоги должен получать позицию, категорию, радиус и время события. Микшер решает громкость, пространственное ослабление и приоритет. AI слышит не PCM данные и не текущую позицию скрытого игрока, а gameplay событие «шум в такой области» с вероятной точностью и давностью. Это позволяет реализовать поиск по звуку без телепатии.

Не создавай новые звуковые голоса каждый кадр, пока горит пожар. Событие начала воспроизводит loop, изменение интенсивности обновляет параметр, завершение останавливает loop. При выгрузке сцены все принадлежащие ей голоса останавливаются. В меню и при паузе явно реши, какие звуки продолжаются. Для игрока звук должен подтверждать решение: щелчок двери, неудача из-за питания, начало ремонта, завершение, попадание, аварийный сигнал.

14.3. Доступность и читаемость

Не кодируй важное состояние только цветом: добавь иконку, текст или форму. Сообщение о пробоине должно оставаться достаточно долго, чтобы его прочитать, и иметь историю в журнале событий. Раздели настройку громкости по категориям, сделай читаемый размер интерфейса на разных разрешениях, поддержи переназначение клавиш или хотя бы полный список управления. Это не финальная косметика: если игрок не понимает, почему NPC бросил ремонт или почему оружие не стреляет, симуляция кажется сломанной даже при правильном коде.

Урок 15. Контент, производительность и граница почти готовой игры

15.1. Вертикальный срез как проверка всего проекта

Минимальный цельный эпизод: игрок приходит в машинный отсек, получает тревогу о пробоине, видит снижение давления, отдаёт инженеру приказ или берёт ремонт на себя, сталкивается с врагом, восстанавливает питание, закрывает дверь, побеждает угрозу и сохраняет игру. После загрузки все важные последствия остаются согласованными. В эпизоде должны быть начало, понятная цель, несколько путей решения, риск, обратная связь и окончание. Редакторские окна и debug hotkeys не должны быть обязательными для прохождения.

Для каждого объекта уровня составь контракт. Дверь: геометрия, collider, graph link, атмосфера, power, взаимодействие, звук. Генератор: внешний вид, health, power output, fuel или правило бесконечной работы, рабочая зона, HUD, save. NPC: фракция, controller, brain, навигация, анимация, оружие/инструмент, реакция на смерть и загрузку. Такой список предотвращает «красивые» объекты, которые не участвуют в мире. Контент производи повторяемыми шаблонами, чтобы вторая палуба не требовала ручного исправления десятков компонентных связей.

15.2. Бюджет кадра

При 60 FPS один кадр занимает около 16,67 мс. Если CPU simulation тратит 8 мс, render submission 5 мс и GPU 20 мс, итог ограничен GPU, а сокращение AI на 1 мс не даст 60 FPS. Измеряй CPU и GPU отдельно; записывай сцену, число NPC, комнат, источников света, particles и draw calls. Не отключай AI вне камеры ради FPS: экипаж должен продолжать чинить аварию, а пожар распространяться. Используй более редкое принятие решений для далёких NPC, но движение и мировые события должны оставаться согласованными по игровому времени.

Ищите дорогие места по профилю. Для AI это могут быть массовые raycasts и повторные NavMesh queries. Для рендера — дополнительные reflection/refraction passes, число теневых объектов, overdraw прозрачности и тяжёлый PBR PS. Для физики — слишком сложные collision meshes, избыток динамических тел и частые пересоздания shape. Меняй один фактор за раз и сравни результат на той же сцене. Оптимизация, нарушившая правильность ремонта или save, не является улучшением.

15.3. Проверки качества перед выпуском

Создай набор проверяемых сценариев. Пустая сцена и отсутствующий asset не должны ронять приложение. Новый и старый save открываются согласно версии формата. После многократной смены сцен число Bullet objects и D3D resources возвращается к ожидаемому уровню. NPC не стреляет без фракции и оружия, не чинит исчезнувшую цель, освобождает claim после смерти и не видит игрока сквозь стену. Пауза останавливает simulation time, а UI остаётся управляемым. Долгий кадр не заставляет physics бесконечно догонять. Все эти проверки нужны потому, что каждая охватывает границу между системами.

Проект содержит `scripts/verify_engine.ps1`, который проверяет shader entry points, собирает x64 Debug/Release и запускает core contract tests. Это полезный технический gate, но он не заменяет игровой прогон. Сборка показывает, что код компилируется; ручной сценарий показывает, что игрок понимает происходящее. Перед кандидатом к выпуску создай отдельный build package и запусти его из каталога, отличного от каталога исходников. Иначе относительные пути к assets и shaders могут случайно работать только у разработчика.

15.4. Конкретное определение почти готовой игры

Игровой цикл проходится без редактора. Контент выдерживает несколько часов прохождения без обязательного debug вмешательства. Вход, сохранение, загрузка, смерть, перезапуск и выход из игры определены. У NPC есть объяснимые причины действий; у корабля — последовательные последствия аварий; у игрока — достаточная информация для решения. Производительность измерена на целевой машине, а критические утечки и падения устранены. Для каждого используемого asset понятны источник, условия использования и способ повторной сборки. Это не список «идеальных возможностей», а практическая граница между демонстрацией механик и игрой, которую можно дать внешнему человеку.

Урок 16. Выпускная работа на одном корабле

16.1. Сценарий и карта ответственности

Собери уровень из трёх отсеков: мостик, машинный, ремонтный. Между ними две двери, один генератор, один аккумулятор, одна пробоина, один повреждаемый кислородный прибор, два дружественных NPC и два врага. Игрок начинает с доступом к мосту, но без полного знания аварии. Тревога сообщает комнату, инженер получает работу, оператор может восстановить устройство, враги реагируют на видимый контакт и звук. Структура достаточно мала, чтобы её проверить вручную, и достаточно сложна, чтобы выявить ошибки порядка систем.

Перед реализацией нарисуй граф зависимостей: Damage → Breach → Atmosphere; Generator/Battery → PowerGrid → Door/Oxygen; авария → CrewTaskCoordinator → StateSystem → Bullet controller → RepairSystem; контакт врага → Perception → State/Cover → Animation/Weapon → Damage. На каждой стрелке подпиши тип данных и время обновления. Если стрелка пересекает worker thread, добавь снимок входов и проверку актуальности сцены на выходе. Если пересекает save/load, добавь сериализуемый ключ и remap.

16.2. Последовательность реализации без второй дорожной карты

Сначала добейся сценария без красивых моделей: примитивные комнаты, debug линии, текстовые статусы. Затем подключай визуальные модели, анимации, частицы и звук к уже работающим событиям. Это принцип построения одного вертикального среза, а не дополнительный план по дням. После каждого подключения сохраняй способность объяснить, где истинное состояние: в ECS, Bullet, WorldEvent или render snapshot. Никогда не заставляй визуальный эффект быть единственным доказательством геймплейного события.

Наконец дай сборку человеку, который не читал код. Наблюдай, где он не понимает следующую цель, почему дверь не открывается, куда пошёл NPC и что означает тревога. Не подсказывай через debug UI. Запиши каждый момент непонимания и исправь интерфейс, звук, анимацию или правило мира. Именно этот прогон покажет, стала ли система игрой.

16.3. Критерий сдачи

Сценарий можно пройти от начала до конца, проиграть и повторить. Вражеский NPC не получает скрытые координаты, дружественный NPC не удерживает недостижимую работу навсегда, разрушение и ремонт согласованы с коллизией и атмосферой, сохранение восстанавливает состояние, а релизная сборка запускается вне рабочего каталога. У каждой аварии есть причина, численный эффект и ясная обратная связь. У каждого NPC решения есть объяснение в debug monitor. Когда эти условия выполнены, справочные главы дальше по книге помогут расширять проект без потери архитектуры.

Производственная часть игры

Эта часть отвечает на вопрос, который не решается одной правильной системой: как превратить инженерный прототип в повторяемый игровой опыт. Она не вводит второй календарный план. Разделы ниже можно применять к каждому уровню и каждому кандидату сборки. Пометки «добавить» означают работу над проектом, а не уже готовый функционал.

Урок 17. Конвейер моделей, материалов и анимации

17.1. Один asset проходит несколько пространств и владельцев

Модель создаётся во внешнем редакторе, экспортируется в glTF/GLB, читается загрузчиком, получает mesh/material, появляется как Entity в сцене, рисуется рендером и при необходимости имеет collision shape и skeleton. Ошибка может возникнуть на любом шаге. Например, корабельная панель видна в Blender, но невидима в игре из-за неверного winding; видна, но не выбирается лучом из-за отсутствующего collider; выбирается, но недоступна NPC, потому что её WorkZone находится внутри корпуса.

Для каждого игрового asset веди мини-паспорт: имя файла, источник и условия использования, масштаб в метрах, направление «вперёд», pivot, material slots, текстуры с цветовым пространством, collision strategy, skeleton, анимационные клипы, socket имена и ожидаемые размеры bounding box. Этот паспорт дешевле повторной диагностики после каждого переэкспорта. Если модель не влезает в дверь, сначала сравни её scale и capsule, а не меняй NavMesh параметры на всей карте.

17.2. Геометрия и физика имеют разные бюджеты

Видимый mesh может содержать множество полигонов ради нормалей, силуэта и детализации. Collision shape должен описывать нужные контакты с меньшей стоимостью. Для большого неподвижного интерьера годится статическая геометрия; для динамического ящика — выпуклая форма; для персонажа — капсула. Раздели LOD рендера и физическую точность. Замена далёкой модели на низкий LOD не должна менять collision, если игрок может взаимодействовать с ней с близкого расстояния. Если runtime cutting создаёт реальную пробоину, обновление физической формы должно быть частью операции или явно ограниченной возможностью.

Границы также важны для culling. Ошибочный AABB может скрыть объект, хотя mesh правильный. После импорта проверь min/max координаты модели, world bounds после Transform и debug фрустум. Для анимированного персонажа bounds должны охватывать движение костей и оружие либо обновляться; иначе рука или вся фигура могут исчезать у края камеры.

17.3. Материалы и текстуры

PBR материал состоит не из одной картинки. Albedo задаёт базовый цвет, normal — направление микроповерхности, roughness — ширину блика, metallic — характер отражения металла, emissive — собственное свечение. Не подставляй roughness map в albedo slot: даже если объект виден, свет станет физически бессмысленным. Проверь, как импортёр и MaterialClass задают defaults при отсутствии карты. Ошибка «всё чёрное» может быть следствием отсутствующего SRV, нулевой normal, неверного sRGB или NaN в BRDF.

Для каждой текстуры запиши: является ли она цветом или данными; нужны ли mipmaps; допустим ли alpha; какой sampler нужен; можно ли её загружать асинхронно. Копирование данных на GPU остаётся на главном потоке по контракту проекта. Фоновый worker может декодировать файл, но результат должен применяться только если сцена и материал ещё живы.

17.4. Skeleton и клипы NPC

Перед подключением AI убедись, что rig и анимации самостоятельны. Один и тот же rest pose, названия костей, направление модели и scale должны сохраняться при экспорте Idle, Walk, Run, Aim, Fire, Reload, Repair и Death. У клипа есть длительность, loop policy и момент события. Для выстрела gameplay решение остаётся у WeaponSystem; событие анимации помогает синхронизировать вспышку, звук и recoil, но не должно бесконтрольно дублировать пулю. Для ремонта progress обычно идёт по игровому времени, а клип является визуальным циклом; при прерывании работы анимация прекращается без потери или с сохранением прогресса согласно правилу.

Проверяй sockets после blend: оружие в руке, дуло, точка инструмента и камера first-person. Скриншот одной позы недостаточен. Просмотри переходы и крайние моменты: начало Reload, конец Aim, поворот на месте, бег с оружием, смерть во время ремонта. Если socket меняет сторону или проваливается в тело, ищи порядок bone transforms и разницу локального/мирового пространства.

17.5. Проверка нового asset до включения в уровень

Открой его в маленькой тестовой сцене с белым светом, сеткой метров и debug axes. Проверь размер, pivot, нормали, текстуры, alpha, shadow, collision, picking, LOD и сохранение ссылки на файл. Для NPC добавь capsule и один клип, затем все остальные. Только после этого помещай asset в большой корабль. Такой gate не запрещает экспериментировать; он делает ошибку дешёвой для локализации.

Урок 18. Миссия и правила игрового эпизода

18.1. Цель игрока задаёт смысл системам

В космическом симуляторе можно бесконечно чинить панели, но игра требует причины. Пример миссии: удержать корабль в работоспособном состоянии, пока вычисляется маршрут эвакуации. Игроку нужны три понятных показателя: время до расчёта, критические системы и угроза противника. Аварии делают выбор сложнее: пробоина требует инженера, повреждённый генератор отключает дверь, враг контролирует коридор. Механики связаны, потому что действие с одной меняет возможности других.

Опиши миссию состояниями, а не набором произвольных bool. `NotStarted`, `AlarmRaised`, `PowerRestoration`, `EvacuationReady`, `Completed`, `Failed` — учебный пример. Для каждого перехода назови событие, условие и действие. Например, `PowerRestoration → EvacuationReady`, когда двигатель/связь получили питание и расчёт времени завершён. Не делай переход в UI рендере; игровая система должна принять событие, записать состояние и сохранить его. Триггерная зона может выдать «игрок вошёл в мостик», но не должна напрямую считать все ресурсы и переключать пять экранов.

18.2. Победа, поражение и повторение

Победа должна иметь наблюдаемый результат: короткое завершение эпизода, запись в save и понятный переход дальше. Поражение бывает несколькими путями: игрок погиб, критическое давление или кислород упали ниже порога, экипаж потерял возможность восстановить важную систему. Реши, является ли необратимая потеря возможной: если игра допускает «мёртвый корабль», покажи предупреждение до того, как ситуация станет безнадёжной. Если игрок может всегда восстановиться, дай логичный путь возврата, а не скрытую debug кнопку.

Повторение сценария проверяет детерминизм там, где он нужен. Одинаковые условия не обязаны приводить к идентичной перестрелке, если есть случайный spread, но событие «дверь без питания» должно давать одинаковый отказ. Для случайных событий логируй seed и причины выбора, чтобы ошибку можно было воспроизвести. Не подменяй сложность случайным наказанием игрока вне его наблюдения.

18.3. Уровень учит механикам через последствия

Первый отсек знакомит с терминалом и питанием без врага. Второй показывает пробоину и инженера. Третий заставляет использовать укрытие и услышать врага до контакта. Это порядок подачи информации внутри одной миссии, а не отдельная дорожная карта разработки. На каждом этапе игрок должен понимать, чему его научил опыт. Если после первой пробоины появляется только число «pressure 0,4», он может не понять, что открытая дверь распространила опасность на другую комнату. Дай звук, движение воздуха, подсказку терминала и запись события.

Не рассчитывай на знание читателя учебника у будущего игрока. Проведи испытание на человеке, который не видел код. Отмечай, что он пробует, где останавливается и какие слова интерфейса неверно трактует. Исправление может быть в цели, маркере, звуке, timing события или правилах NPC, а не в дополнительном текстовом окне.

Урок 19. Наблюдаемость и диагностика без догадок

19.1. Для каждой системы нужна причина отказа

Успешное действие обычно легко увидеть. Отказ без причины выглядит как баг. Поэтому у двери — причина отказа открытия; у NPC — причина отсутствия маршрута; у оружия — `rangedStatus`; у электросети — requested/supplied; у ремонта — состояние цели и progress; у загрузки asset — путь файла и этап. Сообщение должно называть конкретное условие, а не универсальное «failed». В release UI эти причины переводятся в понятный игроку язык; в debug выводятся точные поля и Entity ID.

Пример трассы ремонтной задачи: «Breach 84 created in Room 5», «Engineer 12 score 1,30», «claim 84→12», «path via Door 9», «Door 9 requires power», «route blocked», «claim released», «Engineer 12 considers next task». Если есть только последняя строка «NPC idle», восстановить причину невозможно. Не обязательно хранить весь журнал вечно; кольцевой буфер на последние события и фильтр по Entity часто достаточно.

19.2. Метод локализации ошибки

Начни с первого неправильного значения в цепочке. Если экран пуст, проверь: Entity есть в Registry? Transform верный? asset загружен? render candidate создан? culling оставил объект? shader получил правильную матрицу? depth не скрывает? Не меняй exposure, пока кандидат вообще не дошёл до draw. Если NPC не стреляет, проверь: target допустим? visible? brain в Attacking? анимация и muzzle готовы? союзник на линии? cooldown? WeaponSystem вернул true? Такой порядок переводит огромный проект в последовательность проверок.

После нахождения причины создай минимальную сцену, где симптом воспроизводится. Исправь там, затем верни сложную сцену. Минимальная сцена нужна потому, что в большом корабле одновременно меняются двери, давление, power и AI. Для багов от FPS повтори при фиксированной simulation speed и разных render FPS. Для багов загрузки повтори clean start, а не только hot reload после старого состояния.

19.3. Диагностика утечки

Считай сущности, pool sizes, Bullet bodies/actions/constraints/shapes, COM resources, jobs в очереди и активные audio voices. При переходе сцены ожидаемые значения могут временно подняться во время импорта, но после завершения должны вернуться к устойчивому уровню. Если каждое переключение добавляет один body или shader resource, ищи владельца по типу. Не полагайся

только на общий расход памяти Windows: allocator может держать освобождённые блоки, а маленькая утечка может долго не расти заметно. Счётчики объектов точнее связывают проблему с конкретным lifecycle.

Урок 20. Сборка, упаковка и внешний прогон

20.1. Что уже проверяет проект

`scripts/verify_engine.ps1` ищет MSBuild, проверяет shader entry points, собирает x64 Debug и Release, запускает core contract tests и проверяет наличие `assimp-vc143-mt.dll` рядом с exe. Это хороший технический входной gate. Но `scripts/run_engine.ps1` выполняет `Push-Location` в корень проекта и запускает exe оттуда. Поэтому его успех не доказывает, что отдельная папка с игрой содержит все assets и shaders. Не выдавай build из Visual Studio за релизный package.

20.2. Манифест runtime файлов

Составь список файлов, которые нужны игре: exe; runtime DLL; shader source/compiled assets в зависимости от текущей загрузки; модели, текстуры, анимации, сцены, конфигурации; лицензии сторонних компонентов и контента, если они требуются распространением. Для каждого файла запиши, где код его ищет: относительно process working directory, exe directory или настроенного asset root. Один согласованный asset root лучше набора случайных относительных путей. Если FileSystem создаёт несуществующий каталог, это не должно молча скрыть отсутствие обязательных игровых ресурсов.

Упаковка должна быть повторяемой. Скрипт копирует ровно перечисленные файлы в чистый каталог, проверяет хэши или наличие, запускает exe с рабочей директорией, отличной от проекта, и пишет лог ошибок загрузки. Не копируй весь исходный репозиторий: это скрывает недостающий манифест и увеличивает размер. Не удаляй важные файлы из package на глаз, если сначала не измерил обращения при загрузке миссии и всех материалов.

20.3. Чистая машина как другой мир

На машине разработчика уже установлены Visual Studio, SDK, DLL, кэш шейдеров и модели в исходных папках. У игрока этого нет. Поэтому полезна чистая Windows VM или другой компьютер без среды разработки. Проверь старт, меню, новую игру, загрузку save, смену разрешения, звук, ввод, игровой цикл и выход. Если игра падает, лог должен показать путь последней операции, а не только exit code. Для графических проблем запиши GPU, драйвер, режим окна и разрешение. Не делай высокоуровневых выводов по одному компьютеру.

20.4. Кандидат к выпуску

Кандидат имеет зафиксированный исходный commit, список assets, версию save и записи известных ограничений. После исправления критической ошибки собирай новый кандидат

целиком и повторяй обязательные сценарии: новая игра, загрузка старого save, ремонт, бой, гибель NPC, пауза, смена сцены, завершение миссии. Не копируй один обновлённый exe поверх старого package без пересборки манифеста: shader или config мог измениться вместе с кодом. Отдельно храни символы отладки для диагностики падений, не обязательно включая их в пакет игрока.

20.5. Что ещё отличает почти релиз от технической демонстрации

Игрок понимает управление и цели без разработчика рядом. Сохранение и повторное открытие работают. Критические действия имеют звук и визуальное подтверждение. Камера, анимация, столкновения и UI не спорят друг с другом. Производительность измерена на целевой машине. Сцена не требует debug команд для выхода из тупика. Все используемые модели и звуки имеют понятные права использования. Известные ошибки перечислены честно, а блокирующие падения и потеря сохранений исправлены. Это практическое определение готовности, которое можно проверить, а не абстрактное «полировка».

Математический задачник для движка

Задачник нужен как рабочая тетрадь. Сначала реши пример вручную, затем найди соответствующий метод в коде и сравни числа с debug выводом. Запись формулы без единиц и пространства координат считается неполным решением. В каждом примере ниже объясняется не только вычисление, но и причина выбора метода.

Задача 1. Движение NPC к рабочей точке

Дано: позиция NPC A=(2; 0; 3) м, точка ремонта B=(8; 0; 11) м, скорость 2 м/с, шаг симуляции 0,1 с, допустимый радиус ремонта 1,5 м. Сначала вычти точки: B−A=(6; 0; 8) м. Длина 10 м. Единичное направление n=(0,6; 0; 0,8). Скорость как вектор — n·2=(1,2; 0; 1,6) м/с. Желаемое смещение за шаг — v·0,1=(0,12; 0; 0,16) м. Если нет препятствий, до границы рабочего радиуса осталось 8,5 м, то есть примерно 4,25 с движения.

Почему нельзя взять только `B.x−A.x` и `B.z−A.z` как готовую скорость? Тогда объект на диагонали двигался бы быстрее: длина (6,8) равна 10, а не 1. Нормализация отделяет направление от скорости. Почему для ремонта игнорируем Y? `StateSystem::UpdateRepairTask` оценивает рабочую дистанцию по палубе. Для трёхмерного выстрела Y игнорировать нельзя: цель этажом выше может иметь нулевую горизонтальную дистанцию, но быть далеко по прямой.

Проверка в игре: временно отобрази A, B, длину d, текущий waypoint и горизонтальную скорость. Если персонаж не дошёл за 4,25 с, это не обязательно ошибка формулы: реальный путь может быть длиннее, дверь может ждать открытия, а Bullet может остановить капсулу у препятствия. Раздели «время по прямой» и «время по маршруту».

Задача 2. Угол зрения через dot product

Дано: forward=(0; 0; 1), цель в направлении (3; 0; 4). Длина направления равна 5, поэтому unitToTarget=(0,6; 0; 0,8). Dot(forward,unitToTarget)=0,8. Угол arccos(0,8)≈36,87°. При полном FOV 70° половина равна 35°, cos35°≈0,819. Условие `dot >= cos(halfFov)` не выполняется: цель немного за пределом. При полном FOV 90° порог cos45°≈0,707 и цель проходит.

Здесь особенно легко перепутать полный угол с половиной. Если поставить порог cos70°≈0,342 для поля зрения 70°, фактически разрешишь цели примерно до 70° в каждую сторону, то есть конус 140°. Другая ошибка — сравнить dot с 35 без cos. Dot всегда в диапазоне −1…1 для единичных направлений, поэтому сравнение с 35 никогда не выполнится. Третья ошибка — забыть нормализацию: dot с исходным (3,0,4) равен 4 и пройдёт почти любой порог.

Проверка: поставь цель строго впереди, на половинном угле и за спиной. Значения dot должны быть примерно 1, cos(halfFov) и −1. После FOV raycast всё равно нужен: цель за стеной внутри конуса не видна.

Задача 3. Выбор правильной стороны поворота

Дано: NPC смотрит вдоль +Z, цель находится справа по +X. Геометрически требуется повернуть корпус на 90° к цели. В какой знак yaw это превратится, зависит от соглашения вращения конкретного Transform и исходной оси модели. Поэтому не выводи знак из слова «справа» по памяти. Возьми тестовый basis vector +Z, примени к нему матрицу поворота на +π/2, посмотри, куда он попал в world space. Если в +X, знак найден; если в −X, нужен обратный. После этого проверь модель: её импортированный «forward» может быть −Z, и визуально персонаж окажется спиной к цели даже при верной мировой матрице.

Сравни два способа. `atan2(dx,dz)` часто используют для направления на плоскости XZ, но порядок аргументов и нулевое направление важны. Альтернатива — строить целевое вращение из forward и desired direction, затем интерполировать кватернион. В обоих случаях при A=B угол не определён: сохраняй прежний поворот. При интерполяции через границу −π/π от 179° к −179° выбирай короткий ход 2°, а не полный оборот 358°.

Проверка: четыре цели по +Z,+X,−Z,−X и диагонали. Debug стрелка логического forward должна совпадать с направлением модели и ствола. Если стрелка верна, а визуальная модель неверна, исправляй model offset, а не AI.

Задача 4. Порядок матриц на точке

Дано: локальная точка p=(1,0,0,1). Масштаб по X равен 2; потом объект переносится на +5 по X. При контракте вектор-строка `p·S·T` сначала даёт (2,0,0,1), затем (7,0,0,1). Если применить `p·T·S`, сначала будет (6,0,0,1), затем (12,0,0,1). Оба результата математически правильны для разных последовательностей действий; игровой объект должен соответствовать задуманному порядку.

Теперь добавь поворот. Для локальной точки (1,0,0) после масштаба 2 и поворота на 90° вокруг Y её расстояние от pivot остаётся 2, только направление меняется. После переноса на (5,0,0) расстояние от новой точки до (5,0,0) равно 2. Этот инвариант работает даже если конкретный знак Z зависит от соглашения осей. Если расстояние 7 или 12, перенос попал не туда в цепочке.

В проекте HLSL `pbr.vs` использует `mul(finalPos, worldMatrix)`, потом view, потом projection. CPU транспонирует матрицы перед записью в shader buffer. Не меняй геометрический порядок из-за слова transpose. Для диагностики запиши p после model, world и view, а не только экранный пиксель.

Задача 5. Нормаль при неравномерном масштабе

Возьми поверхность с касательной t=(1,1,0) и нормалью n=(1,−1,0), нормализованными по желанию. Dot(t,n)=0. Применим к точкам масштаб S=(2,1,1). Касательная станет t'=(2,1,0). Если нормаль тоже просто умножить на S, получим n_bad=(2,−1,0), и dot(t',n_bad)=4−1=3: нормаль

перестала быть перпендикулярной. Применение inverse transpose к нормали даёт n_correct=(1/2,−1,0), и dot(t',n_correct)=1−1=0. После этого нормаль нужно нормализовать.

Причина обратной матрицы не в «особом шейдерном трюке»: нормаль задаёт плоскость, а её уравнение должно оставаться согласованным после преобразования поверхности. Для rigid rotation inverse transpose совпадает с самим вращением, поэтому ошибка может годами не проявляться. Она становится видна на растянутых мешах. Проверь tangent и binormal: TBN должен быть согласован с новой нормалью, иначе normal map освещается в неверном направлении.

Задача 6. Экранная точка и picking ray

Дано: viewport 1920×1080, курсор (1440,270). Нормализованные координаты: `x_ndc=2·1440/1920−1=0,5`, `y_ndc=1−2·270/1080=0,5`. В NDC Y растёт вверх, хотя экранная Y растёт вниз. Для центра (960,540) результат (0,0). Это только координаты экрана: их нельзя напрямую передать Bullet как мировое направление.

Построй две точки в clip/NDC на ближней и дальней плоскости, восстанови world position через обратные projection и view, раздели на w. Затем `direction=normalize(farWorld−nearWorld)`. Если используешь перспективную камеру, origin луча обычно берут из положения камеры или nearWorld в зависимости от задачи, но направление должно соответствовать выбранной экранной точке. Для ортографической камеры направления всех экранных лучей параллельны, а origin различается; один алгоритм без проверки типа камеры даст ошибку.

Проверка: щёлкни по четырём углам и центру, нарисуй лучи. Центральный луч совпадает с camera forward; верхний левый идёт через верхний левый угол фрустума. Если по вертикали всё отражено, потерян знак в формуле y_ndc.

Задача 7. Порог прибытия и гистерезис

Пусть NPC стоит в 1,49 м от WorkZone, а порог Repairing=1,5 м. При коррекции физики следующего тика расстояние стало 1,51 м; простое правило переключит его обратно в MoveToRepair. Затем движение уменьшит расстояние до 1,49, и цикл повторится. Визуально персонаж дёргается, а ремонт прогрессирует рывками. Сделай разные пороги входа и выхода: входить при ≤1,5 м, выходить при >1,8 м. Промежуток 0,3 м — зона устойчивого состояния.

Почему нельзя просто добавить большой `epsilon` ко всем расстояниям? Он может разрешить ремонт через стену. Гистерезис решает только переключение состояния; LOS, room, достижимость и рабочая точка остаются отдельными условиями. Подбирай пороги относительно радиуса капсулы и расположения WorkZone. Тот же приём полезен для attack/chase границы и выключения/включения питания при почти равном supply.

Задача 8. Полезность ремонта и цена маршрута

Две пробоины имеют одинаковую severity=0,8. До A по прямой 3 м, но путь через дверь 20 м; до B по прямой 8 м и путь 9 м. При detectionRange=20 м текущая близость A равна 1−3/20=0,85, B равна 0,6. Текущая база score A: 0,8·0,65+0,85·0,35=0,8175. B: 0,8·0,65+0,6·0,35=0,73. С одинаковыми ролью и skill A выигрывает. Но если заменить геометрическую дистанцию оценкой пути, близость A станет 0, B — 0,55, и базы будут 0,52 против 0,7125. B станет разумнее.

Не обязательно выполнять дорогой path query для каждой пары NPC–задача каждый кадр. Используй грубый геометрический фильтр, кэш длины маршрута с revision NavMesh/дверей и обновляй выбор при изменении мира. Недостижимая цель получает не просто «очень длинный путь», а явный статус. Проверь, что сохранённое назначение не удерживается бесконечно после закрытия двери.

Задача 9. Прицеливание и движение цели

Базовый hitscan в текущем оружии проверяет луч в момент выстрела; поэтому для него не нужно рассчитывать баллистическое упреждение пули с временем полёта. Если позже добавишь физический снаряд со скоростью s, цель движется со скоростью v, стрелок в p, цель в q. Нужно найти t≥0, для которого `|q+v·t−p|=s·t`. Квадрат обеих сторон даёт `(v·v−s²)t² + 2(r·v)t + r·r = 0`, где r=q−p. Выбирай наименьший неотрицательный корень. Если действительного корня нет, снаряд не может догнать цель при текущих скоростях.

Простой случай: цель в 10 м и движется прямо от стрелка со скоростью 2 м/с, снаряд летит 12 м/с. Время перехвата t=10/(12−2)=1 с, точка прицеливания на 12 м от стрелка. Прицеливание в текущие 10 м даст промах позади цели. Но использовать эту формулу для hitscan означало бы искусственно вести прицел впереди без необходимости. Сначала определи физическую модель оружия, потом выбирай математику.

Задача 10. Физика взрыва и закон сохранения

Взрыв даёт телу массой 20 кг импульс J=40 Н·с. Из `J=m·Δv` следует Δv=2 м/с в направлении импульса. Если тело в 5 м от центра, а второе в 10 м, выбери закон ослабления, например `factor=max(0,1−r/R)` для радиуса R=15 м. Тогда факторы 2/3 и 1/3. Это игровая модель, а не физическое следствие самого слова «взрыв»; её надо явно настроить. Не используй `1/r²` без epsilon и верхнего ограничения: при r≈0 импульс станет огромным.

Если взрыв вызван разгерметизацией, количество движения воздуха и корпуса должно иметь понятный источник, но движок может применять упрощённый импульс ради игры. Документируй допущение, чтобы позже не перепутать его с точной симуляцией. Проверяй массы: одинаковый импульс меньше меняет скорость тяжёлого тела. Проверь точку приложения: импульс не через центр масс добавляет вращение.

Задача 11. Аккумулятор и мощность

Источник выдаёт 3 МВт в течение 10 с. Отданная энергия $E=P\cdot t=30$ МДж. Если аккумулятор хранит 0,01 МВт·ч, это $0{,}01\cdot 3{,}6\cdot 10^9=36$ МДж; его достаточно примерно на 12 с при постоянных 3 МВт без потерь. Число 0,01 может выглядеть маленьким, но единица МВт·ч крупная. В коде не смешивай МВт, МДж и Вт·ч в одном float без названия единицы. Разряд должен зависеть от dt и не уходить ниже нуля.

Если потребитель требует 4 МВт, а получает 3, powerRatio=0,75. В текущей сети `hasPower` истинно лишь около полного обеспечения. Поэтому устройство с частичной выдачей может считаться выключенным; это предметное правило. Для нового устройства реши: оно останавливается ниже 100%, работает с 75% производительности или имеет порог 50%. Правило должно быть одинаковым в gameplay, HUD и save.

Задача 12. Согласованность симуляции после сохранения

Перед сохранением пробоина выпускает 0,2 единицы массы воздуха в секунду, давление падает, инженер имеет claim, дверь закрыта. После загрузки сохранённые authoring и runtime данные восстанавливаются, но временные claims и Bullet pointers создаются заново. Если прогресс ремонта 0,4 сохранился, новая работа должна продолжиться с 0,4 либо по явно выбранному правилу начаться с нуля. Оба поведения возможны, но смешение — ошибка: UI может показывать 40%, а ApplyRepairProgress читать 0.

Составь контрольную таблицу до/после: число живых Entity; room links; health генератора; water/air mass; дверь; repair progress; текущая цель NPC; число Bullet bodies и GPU resources. Для физических величин сравни с погрешностью, например $|a-b|\le 1e{-}4\cdot\max(1,|a|)$, а не битовое равенство float. Для идентификаторов после remap сравни логические связи, а не прежние runtime числа Entity. Этот приём превращает «после загрузки всё странно» в список проверяемых расхождений.

Практикум создания космического симулятора

Следующие лабораторные работы не заменяют четырнадцатидневный план в конце книги. Они показывают, как выполнить законченный результат по любому удобному графику. Каждая работа начинается с наблюдаемого поведения, затем проходит по данным и коду, а заканчивается проверкой отказов. Если базовая сцена не воспроизводится, вернись к Уроку 3 и сократи число объектов.

Лабораторная 1. Генератор питает дверь и кислородный прибор

Результат для игрока

На терминале видно, что генератор выдаёт мощность; дверь и прибор работают. После повреждения генератора мощность падает, дверь не открывается, а прибор перестаёт поддерживать атмосферу. После ремонта состояние возвращается. Игрок должен понять причину через интерфейс, а не по случайному изменению цвета лампы.

Данные и подготовка

Создай один источник мощности и два потребителя. Для источника проверь GeneratorComponent и его включённость/health; для потребителей — PowerConsumerComponent с требуемой мощностью и приоритетом. Связь с конкретной сетью должна быть определена данными сцены, иначе два независимых корабля начнут делить одну мощность. Поставь `DamageableComponent` и `RepairableComponent` на генератор, а WorkZone перед панелью, куда может подойти инженер. Сделай дверь частью графа двух комнат и добавь требование питания.

Пусть генератор производит 5 МВт, дверь требует 1 МВт, кислородный прибор 3 МВт. Запас 1 МВт остаётся для будущих систем. При повреждении генератор выдаёт 2 МВт: согласно приоритетам одним устройствам может достаться полное питание, другим частичное. Не ожидай, что оба просто переключатся одинаково; читай `ShipPowerGridSystem::SupplyEvenly` и реальные приоритеты. Для каждого потребителя показывай `requiredMw`, `suppliedMw`, `powerRatio`, `hasPower`. Если у прибора есть `hasPower=false` при powerRatio 0,66, это соответствует текущему почти полному порогу, а не ошибке HUD.

Путь реализации

Сначала создай предметы в сцене и проверь PowerGrid без урона. Затем вызови повреждение через существующий DamageSystem, а не прямую запись health в панели UI. После изменения health проверь, что GeneratorSystem действительно меняет production, а PowerGrid пересчитывает выдачу в следующем нужном тике. Теперь привяжи DoorSystem и OxygenGeneratorSystem к результату питания. Последним добавь ремонт: координатор назначает инженера, StateSystem

доводит до WorkZone, RepairSystem восстанавливает оборудование. В этом порядке каждое звено можно диагностировать отдельно.

Проверка и признаки ошибки

Выключи генератор: дверь должна объяснить отказ, а прибор — показать отсутствие питания. Включи аккумулятор: результат должен соответствовать мощности и времени разряда, а не постоянному бесплатному питанию. Удали генератор: система не должна читать указатель на уничтоженный компонент. Сохрани и загрузи сцену: связи потребителей с сетью и рабочей зоны с генератором сохраняются через remap. Поставь второй корабль: энергосети не смешиваются. Если прибор работает при нулевой выдаче, ищи кэш прошлого кадра или обход `hasPower`.

Лабораторная 2. Две пробоины и два инженера

Результат для игрока

После двух повреждений корпуса на разных палубах каждый инженер выбирает свою аварийную работу. Один путь может быть длиннее, но доступен, другой может быть закрыт дверью без питания. Если инженер не может дойти, задача возвращается в общий список с ясной причиной. После ремонта утечка прекращается, мировое событие сообщает об этом, а работник переходит к другой полезной задаче.

Постройка сцены

Сделай три комнаты, две двери и две пробоины с различными leakRate. Один инженер стоит рядом с серьёзной пробоиной, второй — далеко. Задай обоим Active, CrewMember onDuty, NpcBrain, Movement, Transform и корректный controller. Начни с открытых дверей. Прежде чем запускать AI, проверь NavMesh debug: точки старта, обе рабочие зоны и маршруты должны проецироваться на навигацию. Пробоина, расположенная в воздухе за стеной, не является пригодной целью только потому, что её мировая позиция имеет правильный XZ.

Прогон назначения

На первом кадре `NpcUtilitySystem` вычисляет repairScore; затем `CrewTaskCoordinator` собирает кандидатов и назначает их в устойчивом порядке Entity ID. Запиши score каждой пары работник–задача, занятые claims и причину выбора. При two tasks и two workers второй не должен выбрать уже заявленную цель. Не меняй коэффициенты наугад, если распределение кажется странным: в score входит геометрическое расстояние и роль, но пока не гарантирована длина маршрута. Для обходного пути нужен отдельный улучшенный критерий.

Когда один инженер получает ручную команду Stop, его automatic claim освобождается. Другой работник сможет взять работу, если она подходит. После ResumeAutomatic первый должен получить новое решение без использования старого `doorRoute`. Во время ремонта удаление

пробоины заставляет StateSystem и Coordinator безопасно перечитать Entity. Не держи в debug UI указатель на старый компонент после завершения.

Сложные случаи

Закрой дверь и выключи питание. Ожидай `WaitingForDoor`, если створка ещё открывается, либо `Blocked` с причиной, если открыть нельзя. Убей назначенного инженера: claim должен освободиться. Перемести пробоину во время движения: route revision должен заставить переоценить путь. Сделай `repairDuration` очень малой: завершение не должно дважды удалить Entity. Поставь паузу: progress остаётся постоянным. Повтори при 30 и 120 FPS: длительность ремонта в игровых секундах примерно совпадает.

Лабораторная 3. Враг видит, ищет и стреляет честно

Результат для игрока

Враг замечает игрока на открытой палубе, наводится, делает короткую очередь, теряет контакт за укрытием, идёт к последней известной точке и ищет. Он не поворачивается мгновенно к скрытому игроку. Если союзник пересекает линию огня, стрелок удерживает выстрел. Если дуло упирается в стенку, пуля не выходит сквозь неё.

Состав сущности

Враг должен быть Active, иметь NPCComponent, NpcBrainComponent, NPCMovementComponent, Transform, FactionComponent, NPCCombatComponent, физический controller и визуальную модель с корректной анимацией. Для дальнего боя нужно экипированное оружие с WeaponComponent и socket дула. Отсутствие любого из ключевых компонентов имеет наблюдаемый отказ. Не используй старый `isHostile=true` как единственное разрешение боя: отношение конкретной пары задаёт FactionComponent.

План диагностики одного контакта

Сначала проверяй фракцию и существование цели. Затем distance² и FOV. Затем нарисуй луч зрения от головы к выбранной части цели. Убедись, что первое пересечение — сама цель или допустимая видимая поверхность. Смотри `targetVisible`, `visibleAimPoint`, `lastKnownPosition`. Далее смотри `brain.state` и приоритет укрытия. После анимации нарисуй muzzle origin и направление. Наконец смотри `rangedStatus` и WeaponSystem cooldown. Если враг не стреляет, «оружие сломано» — последняя гипотеза, когда предыдущие проверки пройдены.

При потере LOS запиши старую lastKnownPosition. Перемести игрока за стеной в другое место. Враг не должен изменить маршрут на новую позицию без нового наблюдения или честного звукового события. Если изменил, найди чтение живого Transform скрытой цели в поиске/cover. При

появлении игрока снова память должна обновиться; старые search sectors сбрасываются, иначе враг после второго контакта может считать, что уже осмотрел все точки.

Проверка огневой дисциплины

Поставь второго NPC той же фракции между стрелком и игроком. Центральный muzzle ray должен вернуть союзника и удержать огонь. В результате нет вызова `WeaponSystem::TryFire` для этого кадра. После ухода союзника оружие должно ждать только реальных ограничений: aim delay, cooldown, reload. Проверь также наклон модели и высоту укрытия: глаз может видеть цель, а дуло быть закрыто. Для полной защиты от friendly fire можно проектировать более широкий коридор или учитывать разброс, но пометь такую доработку как отдельное правило.

Лабораторная 4. Новый пожар и тушение экипажем

Правила мира до кода

Очаг начинается с intensity 0,2. При достаточном кислороде растёт на 0,05/с до 1; при недостатке кислорода убывает на 0,08/с; тушение инженером уменьшает на `0,15·skill` в секунду. Эти числа — учебные игровые параметры, а не физическая модель горения. В сценарии огонь нагревает устройство и может снизить его health, но за один тик не должен создавать новый пожар на самом себе. Прежде чем писать визуализацию, проверь за 10 секунд на бумаге: без тушения intensity вырастет примерно до 0,7; при skill 1 и тушении в течение 2 секунд уменьшится на 0,3 без учёта одновременного роста. Явно выбери, суммируются ли рост и тушение.

Компоненты и порядок кадра

Добавь FireComponent с интенсивностью, комнатой, коэффициентами и ссылкой на источник. `FireSystem` читает текущий кислород/температуру, рассчитывает следующий шаг и публикует пожарное событие при переходе порога. Не отправляй тревогу каждый кадр, иначе журнал и звук заполнятся дублями. Оборудование получает damage через единый DamageSystem. CrewTaskCoordinator видит новый тип задачи после появления очага; StateSystem доводит назначенного работника до WorkZone; действие тушения уменьшает intensity. Renderer и particle system читают intensity после симуляции.

Определи порядок относительно атмосферы. Если FireSystem на шаге k читает кислород после расчёта комнаты, а расход записывает на шаг k+1, это одна явная задержка. Если нужно одновременное решение, разбей обновление на фазы «собрать потоки» и «применить». Не заставляй FireSystem напрямую вызывать RoomSystem посреди его обхода. В debug выводи кислород до шага, после шага, рост, тушение и итог intensity, иначе отрицательный огонь придётся искать визуально.

Расширение AI и отказов

В `CrewTaskType` добавь ExtinguishFire. Score может зависеть от intensity, близости, ценности соседнего оборудования и доступного ресурса. Claim хранится на очаге, как ремонтный claim на цели. Для ручного приказа добавь метод `NpcCommandSystem::ExtinguishEntity`, который вызывает общий Begin и сбрасывает маршрут; StateSystem выполняет новую команду. NPC с пустым огнетушителем не должен навсегда занимать очаг. При исчезновении огня маршрут и claim освобождаются, а анимация тушения прекращается. При закрытой двери с отсутствием питания задача получает ясный Blocked статус.

Контроль результата

Проверь: один очаг и один инженер; два очага и два инженера; очаг без кислорода; нет ресурса; смерть работника; удаление источника; save/load; пауза; очаг вне камеры. Только после этого добавляй красивый дым и свет. Если частицы есть, а FireComponent не существует, это визуальный декор, а не системная авария. Если FireComponent существует, но игрок не понимает, что делать, нужен интерфейс и сценарная подача.

Лабораторная 5. Релизная сборка и проверка вне исходников

Технический gate проекта

Скрипт `scripts/verify_engine.ps1` собирает x64 Debug и Release по MSBuild, вызывает проверку shader entry points и core contracts; параметр `-Configuration` позволяет выбрать Debug, Release или All. `scripts/run_engine.ps1` проверяет exe, `assimp-vc143-mt.dll` и assets, но запускает игру из корня проекта через `Push-Location`. Поэтому успешный `run_engine.ps1` доказывает работоспособность рабочего дерева, но не доказывает переносимость отдельно упакованной игры.

Для кандидата к выпуску создай отдельный каталог с exe, нужными DLL, shaders, assets, сценами и конфигурацией. Путь к нему должен быть определён манифестом сборки, а не ручным копированием «что вспомнил». Запусти exe из другого current working directory и проверь, что FileSystem корректно находит runtime root. Если ресурс отсутствует, приложение должно показать точную ошибку и имя файла. Обучающие временные assets и огромные исходные blend файлы не должны автоматически попадать в package, если игра их не использует.

Игровой gate

Пройди выпускную работу Урока 16 с чистым профилем и без редактора. Сохранись перед боем, выйди из игры, снова загрузи save. Убедись, что инженер не получает вторую копию своей задачи, дверь остаётся в правильном состоянии, пробоина не появляется снова, патроны и health согласованы. Поставь симуляцию на паузу и вернись. Убей NPC во время навигации. Сменяй сцену при фоновой загрузке модели. Несколько раз перезапусти game loop. Каждая из этих проверок пересекает несколько слоёв и потому лучше простого запуска главного меню.

Полезный отчёт об ошибке

Записывай build configuration, commit, имя сцены, seed при наличии случайности, действия игрока, ожидаемый и фактический результат, значения debug статусов, скриншот или короткое видео. «NPC тупит» нельзя воспроизвести; «после команды Stop claim на пробоине Entity 84 остаётся у NPC Entity 12, второй инженер не берёт задачу» уже является технической задачей. Исправление готово, когда сценарий повторно проходит и не ломает соседние случаи.

Часть I. Паспорт движка

Эта часть рассчитана на читателя, который впервые открыл проект. Её результат — не знание списка файлов, а способность объяснить, почему объект появился в сцене, почему система обновила его именно в этот момент и где искать отказ. Двигатель соединяет Editor, ECS, Bullet, DirectX 11 и игровую симуляцию. Каждый слой имеет собственный набор данных и время жизни. Чтобы построить игру почти до выпуска, сначала научись запускать и останавливать весь цикл без случайных зависимостей от исходного каталога.

Разбор 1. Что представляет собой проект и как проверить исходное состояние

1.1. Движок, редактор и игра — один процесс с разными режимами

DX Framework — Windows/C++ проект на DirectX 11, в котором редактор и игровой режим делят значительную часть инфраструктуры. Редактор показывает сцену, Inspector, Hierarchy, gizmo и инструменты создания объектов; Play разрешает игровым системам менять мир. Это значит, что кнопка Play не запускает магически отдельную «правильную игру». Она меняет набор выполняемых действий, а исходные данные сцены могут быть теми же. Поэтому первая ошибка новичка — считать, что объект, видимый в Editor, уже имеет все данные для поведения в Play. Mesh и Transform дают картинку, но не назначают NPC роль, не строят путь и не создают Bullet collider.

Составь паспорт сцены. Для каждого важного объекта выпиши: чем он видим, какие данные хранятся в ECS, есть ли collider, кто меняет Transform, как объект сохраняется и какой режим обновляет его. Пример для двери: mesh рисует створку; компонент двери хранит состояние и связь комнат; collider физически перекрывает проход; power grid определяет доступность привода; NavMesh/room graph задаёт маршрут; система анимации показывает открытие. Если в паспорте отсутствует одна строка, дверь может выглядеть открытой, но продолжать блокировать NPC. В дальнейшем каждый новый объект проверяй таким же паспортом.

1.2. Первый воспроизводимый запуск

Сначала собери проект штатным способом из README и `scripts/verify_engine.ps1`. Скрипт проверяет сборки и основные тесты, но не доказывает, что Release можно перенести на другой компьютер. Отдельно запускай Editor, создавай простую сцену, сохраняй её, входи в Play, возвращайся в Editor и загружай исходное состояние. Запиши точную команду, рабочий каталог, выбранную сцену и сообщения об ошибках. Отсутствующий shader файл и неверная относительная папка assets могут выглядеть как ошибка рендера, хотя код шейдера исправен.

При первом сбое не изменяй десять настроек подряд. Зафиксируй один наблюдаемый симптом: окно не создаётся, сцена не грузится, GPU возвращает ошибку, модель не появляется или NPC стоит на месте. Затем найди первую границу, на которой ожидаемые данные исчезли. Если объект есть в JSON, но отсутствует в Registry, ищи загрузку сцены; если в Registry есть Transform и RenderComponent, но нет draw call, переходи к renderer; если draw call есть, но объект невидим, проверяй камеру, материал, culling и depth. Так диагностика превращается в последовательность проверок, а не в угадывание.

1.3. Один кадр как книга учёта данных

`ApplicationClass::Frame(float dt)` — место, где сходятся ввод, editor/play логика, симуляция, физика и подготовка кадра. Не нужно запоминать каждый вызов; нужно понимать зависимость. Система чтения ввода должна закончиться прежде, чем команда игрока используется NPC. Назначение ремонтной работы должно произойти прежде, чем `StateSystem` исполнит выбранное состояние. Анимационная поза должна обновиться до расчёта дульного socket для стрельбы. Physics step должен иметь понятного владельца Transform: либо он читает намерение движения и записывает результат, либо рендер читает интерполированную позу. Если две системы пишут одно поле без договора, «последняя победила» становится скрытым игровым правилом.

Проведи эксперимент с журналом кадра. Для одного NPC запиши до и после каждого ключевого вызова: entity, state, target, desired velocity, Bullet position и animation state. На первом кадре он узнаёт о пробоине; на следующем получает путь; затем движется. Запаздывание на один кадр не обязательно ошибка, если оно следует из порядка обновления. Ошибка — когда один слой систематически использует данные предыдущего кадра, а UI обещает мгновенную реакцию. Документируй в виде таблицы «кто производит → кто потребляет → на каком шаге».

1.4. Упражнение с двумя комнатами

Создай комнату A с генератором и комнату B с пробоиной. Между ними дверь, рядом инженер. Сначала при открытой двери и полном питании убедись, что инженер видит задачу и доходит до WorkZone. Затем отключи питание и повтори. Сравни четыре слоя: power state двери, физическое положение створки и collider, route result, состояние NPC. Если инженер не движется, назови конкретный слой отказа. Это первая полноценная лаборатория архитектуры: ни один красивый скриншот не заменяет объяснимую цепочку данных.

1. Что это за проект

DX Framework — учебный игровой движок и одновременно основа космического симулятора. Он написан на C++20 для Windows и использует DirectX 11, DirectXMath, Bullet Physics, Assimp, TinyGLTF, Recast/Detour, ImGui, DirectWrite и JSON. Это уже не набор отдельных уроков: рендер, ECS, физика, редактор, импорт корабля, персонажи, NPC, атмосфера, энергетика, повреждения и сохранение сцены соединены одним жизненным циклом.

Главная ценность проекта — вертикальная интеграция. Например, пробоина — это не только картинка:

1. попадание создаёт событие повреждения;
2. повреждение меняет состояние панели или runtime-меша;
3. пробоина создаёт приток воды или утечку атмосферы;
4. помещение меняет параметры среды;
5. HUD и отладчик показывают последствия;
6. Utility AI может выбрать ремонт;
7. StateSystem строит исполнение задачи;
8. персонаж идёт по NavMesh и завершает ремонт;
9. состояние попадает в scene save.

Именно такие цепочки превращают отдельные технологии в движок.

1.1. Что реализовано сейчас

Область	Реальное состояние на срезе
Платформа	Windows x64, Visual Studio 2022, toolset v143
Язык	C++20 в Debug x64; конфигурация Release x64 настроена неполно
Графика	DirectX 11 forward renderer, PBR, тени, HDR sky, reflection/refraction, частицы, debug draw
Мир	ECS на `Entity = uint32_t` и типизированных component pools
Физика	Bullet rigid bodies, raycast, fixed step 60 Гц, capsule character controller
Навигация	Recast build и Detour pathfinding, локальное избегание динамических препятствий
Редактор	ImGui, picking, gizmo, инспектор компонентов, импорт и save/load
Контент	Assimp/TinyGLTF, GLB-корабль, материалы, skinned animation
Симуляция	комнаты, двери, кислород, давление, затопление, плавучесть, power grid, damage/repair
Параллельность	JobSystem, CPU-фаза фоновой загрузки, main-thread GPU upload
Тесты	есть `DXFrameworkCoreTests` и локальный build/shader gate; внешний CI пока не настроен

1.2. Уровень зрелости

Точный диагноз: **функционально богатый alpha-прототип собственного движка**.

Это выше «учебной демки», потому что подсистемы реально связаны. Это ещё не production engine, потому что:

- Debug x64 собирается, а Release x64 не воспроизводится из текущего project file;
- есть начальные core-contract tests, но coverage ещё узкое и внешний CI не настроен;
- есть несколько разных соглашений об углах и времени;
- крупные координаторы выросли до нескольких тысяч строк;
- часть физической и графической математики содержит подтверждённые расхождения;
- save/load не является транзакционным runtime-save;
- жизненный цикл ряда raw-ресурсов зависит от ручного порядка очистки.

Это нормальное состояние для быстро выросшего авторского движка. Следующий этап — не добавлять ещё десять подсистем, а сделать существующие правила явными, проверяемыми и повторяемыми.

2. Как пользоваться книгой после долгого перерыва

Не нужно читать её подряд. Сначала установи тип задачи.

Если нужно…	**Открыть**
вспомнить общий поток кадра	главы 6–7
добавить клавишу или кнопку мыши для игры	раздел 7.4
найти владельца объекта или GPU/Bullet-ресурса	главы 5 и 9
понять Transform, камеру или picking	главы 10–13
разобраться с PBR, HDR или тенями	главы 14–17
исправить дрожание и зависимость от FPS	главы 7, 18–20
понять затопление, давление и плавучесть	главы 21–23
добавить компонент или систему	главы 8 и 34
добавить задачу NPC	главы 27–29
понять scene save и Entity remap	глава 31
найти известный риск	глава 36

После перерыва ответь на семь вопросов:

1. Какие данные являются источником истины?
2. Какая система имеет право менять эти данные?
3. На каком потоке она работает?
4. В какой фазе кадра она вызывается?
5. В каких единицах хранятся значения?
6. Кто владеет внешним ресурсом и кто его освобождает?
7. Какая проверка доказывает корректность?

Если хотя бы один ответ неизвестен, изменение пока опасно.

3. Как движок вырос: история по локальному git

История коммитов показывает естественную эволюцию проекта.

Период	Этап
4–7 марта 2026	DirectX 11 каркас, multitexture, normal map, render-to-texture, ShaderManager, fog
20 марта	добавлены ECS и набор шейдеров
1 апреля	появились collision components
19–25 апреля	первый цикл скелетной анимации и animated mesh
12 мая	отдельный проход по памяти и владению ресурсами
13–17 мая	skybox cache, камера, Editor/Play, player controller, PBR GLB, reflection, animation state, debug UI
26 мая – 14 июня	интеграционные исправления; последний коммит основной ветки `f31cfbb`
после 14 июня	большой незакоммиченный этап: перенос кода в `src`, NPC/NavMesh, корабль, power, atmosphere, particles, repair, асинхронные системы и новые документы

История объясняет нынешнюю форму архитектуры. `ApplicationClass` и `SceneManager` стали большими не из-за одной ошибки, а потому что каждый новый вертикальный срез естественно подключался к уже существующим координаторам. Теперь граница роста достигнута: дальнейшее качество зависит от выделения стабильных интерфейсов и тестов.

4. Карта репозитория

Рабочий проект находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork
```

Решение Visual Studio находится уровнем выше:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln
```

Основная документация находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork\docs
```

4.1. Ответственность каталогов

Каталог	Что является его задачей
`src/Core`	WinMain, окно, приложение, время, DirectX/Bullet bootstrap, JobSystem, файлы
`src/ECS`	registry, pools, базовые компоненты и физико-сценовые системы
`src/Scene`	создание сущностей, импорт, model cache, render traversal, JSON save/load

Каталог	Что является его задачей
src/Render	D3D11, camera, renderer, models, materials, shaders, textures, culling, text
src/Gameplay/Player	input intent, movement, facing, capsule controller, vitals, first person
src/Gameplay/AI	Recast/Detour NavMesh и path queries
src/Gameplay/Ship	rooms, atmosphere, alerts, room culling, decompression, submarine data
src/Gameplay/Power	generators, consumers, battery и распределение мощности
src/Gameplay/Combat	damage events, weapons, explosions
src/Gameplay/Repair	repair state и runtime mesh restoration
src/Gameplay/Effects	CPU particles и particle renderer
src/Editor	ImGui panels, gizmo, drag-and-drop, object debug
assets	модели, текстуры, HLSL и данные мира
docs	руководства, теория и планы

Некоторые новые файлы AI и world time пока находятся в корне проекта: NPCComponents.h, NpcCommandSystem.h, NpcUtilitySystem.h, StateSystem.h, GameTimer.h, WorldEvent.h, WorldEventLog.h. Это факт текущего среза, а не рекомендуемое окончательное размещение.

4.2. Горячие точки

Самые крупные авторские файлы на момент аудита:

Файл	Примерный размер	Почему важен
src/Editor/EditorUI.cpp	4,8 тыс. строк	почти весь редактор и множество component panels
src/Scene/SceneManager.cpp	4,4–4,9 тыс. строк	импорт, сцена, save/load и часть render traversal
src/Core/ApplicationClass.cpp	2,3 тыс. строк	bootstrap и точный порядок кадра
src/ECS/System/MeshCuttingTest.cpp	2,2 тыс. строк	cutting, damage history и асинхронные операции
StateSystem.h	1,8 тыс. строк	исполнение NPC-состояний и маршрутов
src/Render/Model/assimpmodelclass.cpp	1,8 тыс. строк	импорт, GPU buffers, animation, bone matrices
src/Gameplay/AI/NpcNavMeshSystem.cpp	1,4 тыс. строк	build/query/debug Recast/Detour
src/ECS/System/PhysicsSystem.cpp	1,2 тыс. строк	shapes, Bullet sync, forces, buoyancy

Большой файл не обязательно плох. Риск появляется, когда он одновременно владеет ресурсами, принимает архитектурные решения и выполняет детали нескольких подсистем.

5. Владение и жизненный цикл приложения

5.1. Верхний уровень

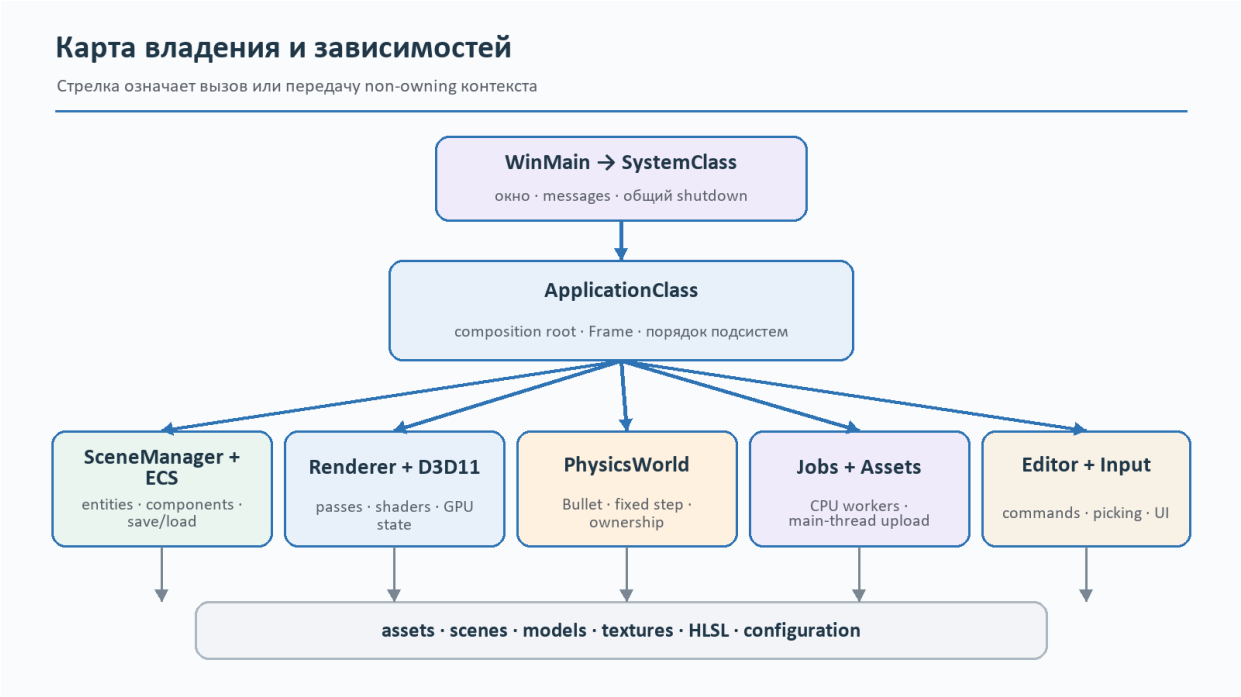


Рисунок 1. Владение и главные зависимости движка.

```
WinMain
  └─ SystemClass (stack)
      ├─ InputClass (unique_ptr)
      └─ ApplicationClass (raw owning pointer)
          ├─ D3DClass (unique_ptr)
          ├─ CameraClass (unique_ptr)
          ├─ SceneManager (value member)
          │   └─ ECSRegistry (value member)
          ├─ Renderer (raw owning pointer)
          ├─ JobSystem (value member)
          ├─ AssetLoadSystem (value member, depends on JobSystem)
          ├─ EditorUI (value member)
          └─ gameplay/render helper systems
```

`WinMain` создаёт `SystemClass` на стеке, вызывает `Initialize`, затем `Run`, затем `Shutdown`. Это хорошая корневая форма: даже при неуспешной инициализации вызывается общий shutdown.

5.2. Почему порядок shutdown важен

Внешние API хранят невладеющие ссылки:

- Bullet world помнит rigid bodies, ghost objects, actions и constraints;
- D3D context помнит связанные SRV, buffers, shaders и render targets;
- ECS-компоненты могут содержать указатели на Bullet и render resources;
- фоновые jobs могут держать `this`, Entity или CPU-результат для main-thread upload;
- материалы ссылаются на шейдеры ShaderManager.

Поэтому безопасный порядок выглядит так:

1. запретить новые фоновые задачи;
2. дождаться workers и выполнить разрешённые main-thread completions;
3. отвязать D3D state;
4. удалить Bullet actions/constraints/bodies;
5. очистить ECS и scene-owned materials/models;
6. удалить renderer helpers и render textures;
7. закрыть Editor/DirectWrite;
8. удалить ShaderManager;
9. освободить D3D device последним.

> **ОГРАНИЧЕНИЕ.** Текущий `SceneManager::LoadScene` напрямую заменяет `ECSRegistry`, тогда как character controllers и constraints требуют явного удаления из Bullet world. Полный application shutdown учитывает world лучше, чем горячая загрузка новой сцены.

5.3. Правила raw pointer

В проекте raw pointer имеет три разных смысла:

Вид	Пример	Правило
owning	`Renderer* m_renderer`	ровно один `delete`, поле сразу `nullptr`
borrowed	`ShaderBaseClass*` из ShaderManager	никогда не удалять в потребителе
API handle	`ID3D11Buffer*`, `btRigidBody*`	освобождать по контракту API и сначала отвязать от владельца

Если из объявления нельзя понять вид указателя, это будущая ошибка. Целевая модель: owning C++-объекты — `unique_ptr`; COM — `Microsoft::WRL::ComPtr`; невладеющие ссылки — pointer/reference с комментарием и ограниченным временем жизни.

6. Сборка и запуск

6.1. Подтверждённый baseline

`Debug|x64` собирается локальным MSBuild из Visual Studio 2022. Результат находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\x64\Debug\DX_FrameWork.exe
```

Использованная команда:

```
& "C:\Program Files\Microsoft Visual Studio\2022\Community\MSBuild\Current\Bin\MSBuild.exe" `
  "C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln" `
  /m /p:Configuration=Debug /p:Platform=x64
```

6.2. Почему Release x64 сейчас падает

В `DX_FrameWork.vcxproj` Debug x64 получает:

- `EngineIncludeDirectories`;
- Bullet include paths;
- C++20;
- Assimp, Bullet Debug, Recast/Detour libraries;
- Windows subsystem.

Release x64 содержит Recast/Detour, но не повторяет include directories, C++20 и полный набор Assimp/Bullet libraries. Поэтому это конфигурационный дефект, а не доказательство ошибок каждой исходной подсистемы.

6.3. Runtime-зависимость от working directory

Многие пути записаны как `assets/...`. `ApplicationClass` создаёт файловую систему от `current_path()/assets`. Следовательно, запуск из Visual Studio работает, а запуск EXE из произвольной папки может не найти shaders/models/textures. Готовый движок должен вычислять runtime root от пути EXE или получать его явно.

6.4. Минимальный smoke-test

После любой архитектурной правки проверить:

1. запуск без shader compiler error;
2. загрузку сцены и HDR sky;
3. `F5` Editor/Play;
4. ходьбу, бег, прыжок и спокойный idle на полу;
5. NPC path через дверь;
6. пробоину и изменение среды;
7. ремонт;
8. save → очистка → load;
9. возврат в Editor;
10. чистый shutdown без live D3D objects и dangling Bullet objects.

7. Один кадр: фактический timeline

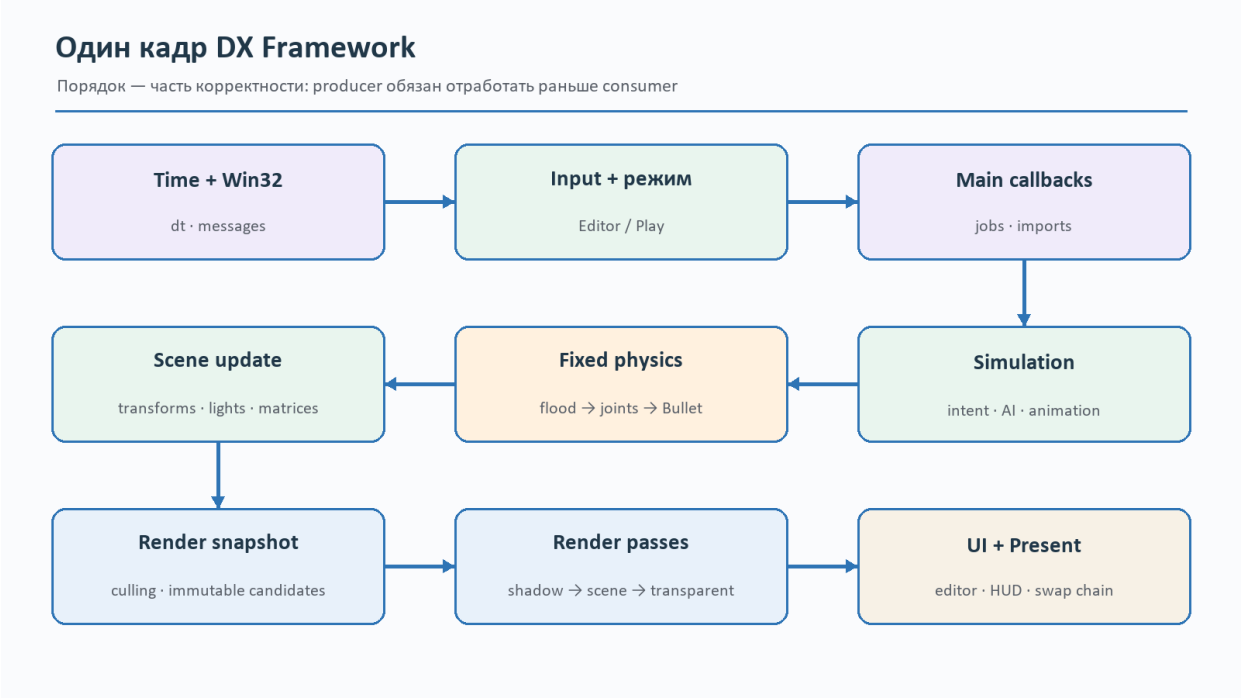


Рисунок 2. Фактическая последовательность фаз кадра.

Главная точка входа — `ApplicationClass::Frame(float dt)`. Порядок систем является частью поведения, а не косметикой.

```
Windows messages / TimeManager
        ↓
InputManager.Update + EventQueue.Clear
        ↓
Engine hotkeys (F1/F2/F5/F6)
        ↓
GameplayInputSystem.Sample (только действия Play)
        ↓
Main-thread jobs + asset upload + async ship import budget
        ↓
Editor camera  [Editor]
        ↓
Interaction → doors → weapon/repair  [Play]
        ↓
Player intent
        ↓
NPC Utility → StateSystem → Facing → CharacterController → Animation
        ↓
Damage → particles
        ↓
Sync compartments
        ↓
Flooding → joints → submarine → PhysicsSystem → pressure
        ↓
Post-physics camera
        ↓
SceneManager.UpdateAll
  animation / attachments / rooms / generator / grid / lights /
  oxygen / room environment / player vitals / ship alerts
        ↓
Camera matrices + debug/navmesh
        ↓
Renderer.RenderFrame
        ↓
```

```
Editor UI or Play UI / HUD
        ↓
Present
```

7.1. Зависимости порядка

- Utility должна принять решение до `StateSystem`, иначе команда исполнится на кадр позже.
- Facing читает `walkDirection`, поэтому идёт после AI, но до controller/animation.
- Камера Play следует после физики, иначе будет показывать предыдущую позицию.
- Power grid должен обновиться до потребителей и oxygen generator.
- Render видит только уже завершённое состояние кадра.

7.2. Editor и Play

`F5` переключает режим. При выходе из Play код явно сбрасывает movement intent и Bullet controller velocity, иначе кэшированное направление продолжило бы двигать персонажа в Editor. Это хороший пример архитектурного инварианта: **отключить producer недостаточно; нужно сбросить состояние consumer**.

7.3. Очередь Windows

`SystemClass::Run` использует `PeekMessage`, но на срезе обрабатывает одно сообщение за итерацию. При потоке mouse/window messages очередь может отставать. Целевая форма — внутренний `while (PeekMessage(...))`, после которого выполняется ровно один simulation/render frame.

7.4. Игровой ввод: от кнопки к действию

ФАКТ на 5 октября 2026 года. `SystemClass` получает `WM_KEYDOWN/UP` и сообщения левой, правой и средней кнопок мыши. `InputClass` хранит их физическое состояние. `InputManager` обслуживает редакторский ввод и события интерфейса. В `ApplicationClass::Frame` после решения о режимах F5 и F2 вызывается `GameplayInputSystem::Sample` ровно один раз. Он преобразует физические кнопки в `GameInputFrame`. Игровые системы читают этот снимок, а не коды клавиш.

```
WM_LBUTTONDOWN → InputClass::MouseLeftDown
               → GameplayInputSystem::Sample
               → GameInputFrame::Down(GameAction::Fire)
               → WeaponSystem::Update
```

`GameInputFrame` хранит три состояния для каждого `GameAction`:

Запрос	Смысл	Пример
`Down(action)`	кнопка удерживается в данном кадре	движение, ремонт, автоматический огонь
`Pressed(action)`	переход из отпущенного состояния в нажатое	взаимодействие, смена слота, пауза
`Released(action)`	переход из нажатого состояния в отпущенное	отпускание прицела или зарядки

Одна игровая команда может иметь несколько физических привязок: `Fire` сейчас принимает ЛКМ и `H`. Действия `Aim` (ПКМ) и `PrecisionAim` (средняя кнопка) уже описаны в слое ввода, но их игровую механику ещё нужно подключить; само наличие действия не переключает прицел. Дельта движения мыши для поворота камеры пока обрабатывается `CameraController::UpdateFollowLook` отдельно от дискретных действий. Клавиши редактора F1, F2, F5, F6 и свободная камера тоже остаются в контуре редактора.

Как добавить игровое действие, например перезарядку по `R`:

1. Добавь `Reload` в `GameAction` в `src/Gameplay/Player/GameplayInputSystem.h`, перед `Count`.
2. В конструкторе `GameplayInputSystem` добавь `SetBindings(GameAction::Reload, { GameInputBinding::Key('R') });`. Для мыши используй `GameInputBinding::Mouse(GameMouseButton::Right)` или другую кнопку. Чтобы сохранить несколько способов ввода, передай их в одном списке или вызови `AddBinding`.
3. В нужной системе прими `const GameInputFrame& input` и читай `input.Pressed(GameAction::Reload)`. Для действия, которое длится всё время удержания, используй `Down`. Если показываешь игроку подсказку, возьми подпись первой привязки через `GetBindingLabel`; так уже устроена подсказка `Interact` на HUD.
4. Передай снимок из `ApplicationClass::Frame` в эту систему. Не опрашивай там `InputClass::IsKeyDown('R')`: иначе привязка снова разойдётся по коду.
5. Проверь один короткий клик, удержание, отпускание, паузу, переключение Editor/Play и F2 с открытым UI. Проверка подтверждает, что событие приходит один раз и не попадает в игру, пока интерфейс владеет вводом.

`Sample` обнуляет игровые действия в Editor и при включённом F2 курсоре. Переключение вида, пауза, масштаб времени и HUD продолжают получать ввод в Play при F2. При возвращении курсора удерживаемая кнопка считается новым нажатием; это следствие вычисления `Pressed` по состоянию действий, поэтому для опасных действий после UI можно отдельно потребовать отпускания. Когда симуляция стоит на паузе, `Sample` всё равно обновляет границы нажатий, а игровой код, зависящий от `simulationDeltaTime`, не выполняется. Старые горячие клавиши `Space` и `K` из `SystemClass` управляют отладочным `TimeManager` только в Editor; в Play `Space` прыгает, а `Esc` закрывает открытую панель.

Где искать код: `src/Core/SystemClass.cpp` → `src/Input/InputClass.h` → `src/Gameplay/Player/GameplayInputSystem.h/.cpp` → `src/Core/ApplicationClass.cpp` → `CharacterMovementSystem`, `WeaponSystem`, `InteractionSystem`, `RepairSystem` или `GameHud`. Начинай чтение с конструктора `GameplayInputSystem`: там собраны текущие привязки.

8. Время и численная устойчивость

В проекте существуют два понятия времени:

- `TimeManager` — QPC, pause/step и delta игрового цикла;
- `GameTimer` — новый world/game total time для событий.

Большинство систем получает `dt` напрямую. Такое дублирование допустимо временно, но без контракта непонятно, что останавливается при pause и что масштабируется.

8.1. Variable step

Обычное интегрирование:

```
x_next = x + v · dt
v_next = v + a · dt
```

Оно подходит для камеры, UI и эффектов, если `dt` ограничен. После breakpoint или подвисания `dt` может стать огромным, поэтому безопасно:

```
dt_sim = clamp(dt_real, 0, dt_max)
```

8.2. Fixed step Bullet

`PhysicsWorld` использует:

```
h = 1 / 60 s
accumulator = min(accumulator + dt, 0.25 s)
while accumulator >= h:
    stepSimulation(h)
    accumulator -= h
alpha = accumulator / h
```

Ограничение 0,25 секунды предотвращает «спираль смерти»: движок не пытается бесконечно догнать потерянное время.

8.3. Интерполяция

После fixed tick визуальная позиция может быть:

```
x_render = lerp(x_previous, x_current, alpha)
```

В проекте character controller имеет отдельную стабилизацию/интерполяцию, а обычные rigid bodies в основном копируют текущий Bullet transform. Поэтому на высоком FPS часть тел может визуально обновляться ступенями 60 Гц.

8.4. FPS-независимое сглаживание

Формула `lerp(old, target, k · dt)` лишь приближённо независима от FPS и должна ограничивать `t` в `[0,1]`. Более устойчивый коэффициент экспоненциального приближения:

```
t = 1 - exp(-lambda · dt)
value = lerp(value, target, t)
```

Здесь `lambda` — скорость сходимости в 1/с. Период полураспада связан с ней:

```
lambda = ln(2) / halfLife
```

> **ОГРАНИЧЕНИЕ.** Сглаживание submersion в `PhysicsSystem` использует постоянные `0.3` на render frame. Это означает разную плавучесть при 30, 60 и 144 FPS.

8.5. Силы должны жить внутри fixed tick

На срезе buoyancy/drag/torque применяются один раз на render frame, после чего Bullet может выполнить ноль, один или несколько fixed steps. При низком FPS Bullet очистит силы после первого шага, и следующие шаги останутся без них; при высоком FPS силы могут накапливаться до следующего шага. Целевая модель: вычислять continuous forces для каждого fixed tick.

Часть II. C++ и ECS именно в этом проекте

ECS в проекте — способ собирать игровые объекты из данных и обновлять их системами. Он не отменяет обычные правила C++: ссылки имеют время жизни, указатели могут стать недействительными, владение ресурсом должно быть определено, а порядок записи имеет значение. Изучай ECS на маленькой сущности, затем переходи к NPC, где одновременно работают десятки компонентов.

Разбор 2. Entity, Component и System по шагам

2.1. Entity ID — имя, а не объект

`Entity` в текущем Registry является числовым ID. `CreateEntity` выдаёт следующий идентификатор, заносит его в набор живых и возвращает число. `IsAlive` проверяет наличие в наборе. У ID пока нет generation counter: если когда-нибудь изменится стратегия повторного использования чисел, старые сохранённые ссылки могут стать опасными. Даже без повторного использования Destroy удаляет компоненты; сохранённые указатели на них больше использовать нельзя. Поэтому в игровых системах храни Entity как ссылку на цель, а перед каждым важным действием проверяй живость и необходимые компоненты.

Представь инженера с `repairTarget=42`. Через кадр ремонт завершился и цель 42 удалена. Само число 42 не становится «нулём» и не сообщает об исчезновении. Система, которая продолжает читать старый указатель на `RepairableComponent`, рискует получить неопределённое поведение. Корректный контракт: после события, которое может уничтожить Entity, вновь запросить компонент через Registry; при отсутствии цели освободить claim, очистить состояние и выбрать следующую задачу. В debug UI показывай отдельно `targetId` и `targetAlive`.

2.2. Компонент хранит данные, система меняет мир

Компонент не обязан быть пассивным POD, но он должен иметь ясный смысл и границы ответственности. `TransformComponent` хранит положение/ориентацию, `RepairableComponent` описывает цель ремонта, `FactionComponent` определяет принадлежность, `NPCBrainComponent` хранит текущее решение и память. Система берёт множество подходящих сущностей и применяет правило. Например, coordinator назначает работу, utility AI выбирает намерение, `StateSystem` исполняет переходы, `RepairSystem` меняет ремонтируемое состояние. Если ты положил в один компонент и данные двери, и Direct3D texture, и pointer на Bullet body, сохранение и копирование этого компонента станут проблемой.

Существующий `ECSRegistry::ForEach<T1,T2>` итерирует pool первого типа и проверяет наличие второго. Порядок шаблонных типов поэтому может влиять на стоимость: если `T1` есть у тысячи объектов, а `T2` у десяти, будут тысячи проверок. Это не повод преждевременно переписать Registry; сначала измерь. Более важное правило корректности — не удалять текущую сущность и не менять структуру итерируемого pool прямо внутри обхода, пока контейнер не гарантирует такую операцию. Собирай команды на удаление, исполняй их после прохода. Компонент, полученный как ссылка, нельзя переносить через операцию, которая может перераспределить storage pool.

2.3. Время жизни C++ ресурсов

`unique_ptr` выражает единоличное владение объектом; raw pointer может быть не владеющим наблюдателем, но этот договор нужно указать. Bullet body, motion state, collision shape и constraint имеют разные правила регистрации и освобождения. Direct3D ресурсы живут через COM/RAII. ECS Entity ID не владеет автоматически GPU ресурсом или Bullet body; при уничтожении сущности необходимо вызвать соответствующую систему очистки до освобождения памяти. Если scene reload оставил физическое тело, игрок столкнётся с «невидимой стеной», хотя Entity уже отсутствует.

Построй таблицу владельцев на примере одной двери. Кто создаёт collider? Кто удаляет его из dynamics world? Кто освобождает память? Кто уведомляет NavMesh и room graph об изменении проходимости? Кто сбрасывает указатель в компоненте? Нельзя считать задачу законченной, если дверь исчезла из Hierarchy, но осталась в Bullet. В практическом коде предпочитай маленькие функции `Spawn`, `Despawn`, `Rebuild`, которые выполняют весь жизненный цикл, вместо отдельных вызовов в разных местах UI.

2.4. Пишем минимальную систему правильно

Допустим, нужен датчик температуры. Начни с правила: устройство в комнате получает температуру комнаты; при превышении порога оно выходит из строя. Данные: `TemperatureSensorComponent` с room ID, текущей температурой, порогом и состоянием. Отдельная система читает комнату, обновляет sensor и публикует событие только при переходе через порог.

Не прячь изменение в `Render()` и не вызывай его из Inspector: тогда симуляция зависела бы от того, видит ли редактор объект.

Псевдокод показывает структуру, а реальные имена Registry и room компонентов подставляются по проекту:

```
void TemperatureSensorSystem::Update(ECSRegistry& ecs, float dt) {
    std::vector<Entity> failed;
    ecs.ForEach<TemperatureSensorComponent>([&](Entity e, auto& sensor) {
        const RoomState* room = FindRoom(sensor.roomId);
        if (!room) { sensor.status = SensorStatus::MissingRoom; return; }
        sensor.temperatureC = room->temperatureC;
        const bool crossed = !sensor.failed &&
                             sensor.temperatureC >= sensor.failureThresholdC;
        if (crossed) failed.push_back(e);
    });
    for (Entity e : failed) FailSensorAndEmitEvent(ecs, e);
}
```

`dt` здесь не используется, потому что датчик копирует состояние комнаты, а не интегрирует скорость. Не добавляй `*dt` механически ко всем формулам. Если датчик должен реагировать с задержкой, введи состояние фильтра и конкретную постоянную времени. Проверь два независимых датчика в разных комнатах: значение одного не должно оказаться у другого из-за `static` переменной внутри Update.

2.5. Подключение системы к редактору и сцене

Новый компонент должен существовать не только в C++. Пользователь создаёт его через Inspector/Quick Create, сохраняет сцену, загружает её и видит ту же настройку. Следовательно, нужны регистрация или фабрика компонента, сериализация authoring полей, валидация значений и UI. Runtime кэш, например pointer на room state, не записывай в JSON: при загрузке он станет недействительным. Сохраняй stable ID комнаты и восстанавливай связь. После изменения формата сцены реши, что делать со старым JSON: default для новых полей, версия схемы и сообщение об ошибке для невозможных комбинаций.

Проверка называется round trip: создай два датчика, задай разные пороги, сохрани, закрой сцену, загрузи и сравни поля. Затем перейди в Play, измени температуру и вернись в Editor; авторское значение порога должно остаться. Если сохраняется и runtime `failed`, объясни, является ли это частью authoring или игрового save. Этот вопрос надо решить до реализации, а не после первого повреждённого файла.

Разбор 3. Сцена и редактор как инструменты обучения

3.1. Что должно быть видно в Inspector

Inspector ценен, когда показывает причину поведения. У ремонтируемой цели нужны severity, required progress, текущий progress, WorkZone, claim owner и статус доступности. У NPC — фракция, роль, выбранная работа, маршрут, state и причина остановки. У двери — питание, команда

открытия, фактический угол створки, collider state и связь комнат. Часть этих полей авторская и редактируется; часть только runtime и должна быть read-only. Если всё доступно для произвольной записи, можно создать комбинацию, невозможную для игровых систем: визуально открытая дверь с закрытым collider и герметичной атмосферной связью.

Не пытайся исправить рассинхронизацию одним универсальным bool `isOpen`. Разные подсистемы могут завершать переход в разные моменты: команда поступила, анимация началась, collider освободил проём, room connection открылась, NavMesh обновился. Для игрока это одна дверь; для разработчика — последовательность согласованных состояний. Хороший debug Inspector показывает эти состояния отдельно и при необходимости кнопку «проверить инварианты».

3.2. Минимальная игровая сцена

Построй корабль из двух комнат, двери, генератора, одной пробоины, инженера, игрока и врага. Сначала без красивых моделей — простые shapes и подписи помогают видеть правила. Сохранение сцены до Play фиксирует авторскую основу. Затем подключай механику по одной: дверь и её питание; пробоина и давление; назначение инженера; маршрут; бой. После каждого шага запиши ожидаемое состояние и проверь его. Если начать с большого уровня, каждый баг будет иметь десятки возможных причин.

Для первой миссии задай цель «восстановить питание и удержать герметичность». Trigger Zone сообщает игроку о задаче; игровые системы определяют её выполнение. Победа не должна зависеть от того, открыт ли Inspector или нарисован ли HUD. После загрузки сохранения миссия восстанавливает стадию по собственным данным или выводит её из фактического состояния мира. Этот маленький срез связывает редактор, ECS и игрока в одну игру.

3.3. Что должен уметь читатель после части II

Самостоятельно создай Entity, добавь два компонента, найди её через `ForEach`, измени состояние в отдельной системе, проверь невозможное значение, покажи его в Inspector, сохрани и загрузи сцену. Затем уничтожь Entity и убедись, что не осталось Bullet body, UI ссылки, claim или async job, обращающегося к прежним данным. Если этот путь работает, можно добавлять сложные системы. Если нет, сначала исправь контракт жизненного цикла: сложный NPC только умножит скрытую ошибку.

9. C++: значения, ссылки, указатели и RAII

9.1. Значение

```
TransformComponent transform;
```

Объект живёт в своей области видимости. Для ECS-компонента это обычно означает хранение внутри `unordered_map<Entity, T>`.

9.2. Ссылка

```
void Update(ECSRegistry& registry, float dt);
```

Ссылка обязательна и не бывает `nullptr`. Она не передаёт владение.

9.3. Указатель

```
TransformComponent* transform = registry.GetComponent<TransformComponent>(entity);
if (!transform)
    return;
```

Указатель выбран, потому что компонент может отсутствовать. Проверка `nullptr` — часть контракта.

9.4. RAII

RAII связывает ресурс со временем жизни объекта. В движке это особенно важно для:

- COM resources (`ID3D11Buffer`, `ID3D11ShaderResourceView`);
- Bullet body/shape/motion state;
- worker threads;
- файлов;
- временных GPU states.

`RigidBodyComponent` пытается реализовать RAII и запрещает копирование. Это правильное направление. Но move-конструктор обязан переносить **каждое** поле владения; пропущенный raw pointer создаёт leak или double-delete.

9.5. Rule of zero / five

Если класс владеет ресурсом, есть два безопасных пути:

1. хранить ресурс в готовом RAII-типе и использовать rule of zero;
2. явно реализовать destructor, move constructor и move assignment, а copy удалить.

Пустой copy constructor хуже удалённого: он компилируется и создаёт объект с неинициализированным состоянием. На срезе такие конструкторы есть у некоторых legacy-классов, включая камеру/D3D/SystemClass.

10. Как устроена ECS

10.1. Entity

```
using Entity = std::uint32_t;
```

Registry выдаёт монотонно растущий ID. Entity не содержит данных; это ключ для component pools.

10.2. Component pool

Для обычного `T`:

```
ComponentManager<T>
  └─ unordered_map<Entity, T>
```

Для `ActiveComponent` используется `unordered_set<Entity>`, потому что tag не хранит полезных полей.

10.3. Type erasure

Registry хранит разные pools в одной таблице:

```
unordered_map<type_index, unique_ptr<IComponentPool>>
```

`IComponentPool::Remove(Entity)` позволяет `DestroyEntity` удалить ID из каждого пула без знания конкретного `T`.

10.4. Добавление компонента

Есть copy-версия для копируемых типов и move-версия для owning-компонентов:

```
registry.AddComponent<TransformComponent>(entity, transform);
registry.AddComponent<RigidBodyComponent>(entity, std::move(body));
```

Для raw-resource компонента второй вариант обязателен.

10.5. Сложность

Операция	Ожидаемая сложность
`Get/Has/Add/Remove` одного компонента	O(1) average
`DestroyEntity`	O(number of component types)
`ForEach<T>`	O(count(T))
`ForEach<T1,T2>`	O(count(T1)) с lookup во втором pool

10.6. Подтверждённые ограничения

1. Нет generation counter. После будущего reuse старый Entity сможет указывать на новый объект.
2. Нет явного alive set; `DestroyEntity` только удаляет компоненты.
3. `GetComponent` и `HasComponent` создают пустой pool при первом чтении отсутствующего типа.
4. `ForEach` идёт напрямую по `unordered_map`; добавление/удаление того же типа из callback может инвалидировать iterator.
5. Нет deferred command buffer для структурных изменений.

Целевая Entity:

```
64 bits = generation | index
```

А структурные изменения должны накапливаться в command buffer и применяться после system phase.

11. Как добавить компонент и систему

11.1. Компонент — только данные

```
struct HeatComponent
{
    float kelvin = 293.15f;
    float heatCapacityJPerK = 1000.0f;
};
```

У полей должны быть единицы, диапазон и значение по умолчанию.

11.2. Система — одно правило изменения

```
class HeatSystem
{
public:
    static void Update(ECSRegistry& registry, float dt)
    {
        registry.ForEach<HeatComponent>(
            [dt](Entity, HeatComponent& heat)
            {
                heat.kelvin = std::max(0.0f, heat.kelvin - 0.2f * dt);
            });
    }
};
```

11.3. До подключения определить

- phase кадра;
- scaled или unscaled time;
- main thread или worker;
- источник истины;
- сериализацию;
- debug representation;
- минимум один детерминированный тест.

Часть III. Математика игры: от чисел до преобразований

Эта часть начинается до формул. В программе число полезно только вместе с вопросом, на который оно отвечает: это положение или скорость, метры или секунды, локальная или мировая координата? Большинство ошибок движения, прицеливания и рендера возникает ещё до вызова сложной функции. Поэтому в каждом разделе сначала строится смысл величин, затем формула, ручной расчёт, программная реализация и проверка в сцене. Читай последовательно. Если не получается самостоятельно предсказать ответ до запуска игры, вернись к предыдущему шагу.

В записи десятичная дробь использует запятую, а координаты векторов разделены точкой с запятой: (0,6; 0; 0,8). В коде C++ дробь записывается точкой: `0.6f`. Если указано `A=(2; 0; 3) м`, все три компонента имеют единицу «метр». Угол для вычислений записываем в радианах, даже если интерфейс показывает градусы. Геометрические иллюстрации в тексте используют оси X вправо, Y вверх, Z вдоль палубы. Реальный знак поворота проверяй по матрице и импорту модели.

12. Числа, единицы и координатные пространства

12.1. Почему `float` не сообщает смысл числа

Допустим, в коде встречается `float speed = 2.0f`. По одной строке нельзя узнать, это 2 метра в секунду, 2 метра за кадр или коэффициент анимации. Если затем написать `position.x += speed`, объект будет проходить 2 условные единицы за каждый кадр и при 120 FPS окажется вдвое быстрее, чем при 60 FPS. Правильное физическое выражение `position += velocity * dt`: скорость имеет размерность м/с, `dt` — с, произведение — м. Проверка размерности похожа на проверку типов компилятором, но выполняется человеком при проектировании игрового правила.

Таблица основных величин для учебной сцены: позиция и длина — м; время — с; скорость — м/с; ускорение — м/с²; сила — Н = кг·м/с²; импульс — Н·с; мощность — Вт = Дж/с; энергия — Дж или Wh после явного пересчёта; давление — Па = Н/м²; поток воды — м³/с. Параметр `repairProgress` может быть безразмерным, если его диапазон 0…1, но `repairRate` тогда имеет единицу 1/с. Если `requiredProgress` измеряется в «единицах работы», явно запиши это; скорость работы имеет единицу «работа/с». Нельзя без пояснения сложить процент здоровья, литры кислорода и расстояние до цели, даже если все они хранятся в `float`.

Ручная проверка: при скорости 3 м/с за 0,2 с ожидаемое смещение 0,6 м. Если код выдаёт 15 м/с после деления скорости на `dt`, он спутал путь с мгновенной скоростью. Для аккумулятора 100 Вт на протяжении 10 с расходует 1000 Дж; это 1000/3600≈0,278 Wh. Ошибка в пересчёте Wh и Дж выглядит в игре как батарея, работающая в 3600 раз дольше или меньше. В математическом журнале рядом с любым числом храни единицу и источник: Inspector, scene JSON, системный расчёт или замер.

12.2. Точка, вектор смещения и направление

Точка A=(2; 0; 3) м обозначает место в мире. Точка B=(8; 0; 11) м — другое место. Разность точек B−A=(6; 0; 8) м является смещением: она отвечает, сколько нужно пройти по осям, чтобы попасть из A в B. Сложение двух точек не имеет такого же ясного геометрического смысла, зато к точке можно добавить смещение. Направление — смещение после деления на длину, у него нет единицы измерения. Поэтому `(6; 0; 8)` и `(0,6; 0; 0,8)` смотрят в одну сторону, но первое хранит расстояние 10 м, а второе — только ориентацию.

В игре эти различия полезно выразить хотя бы именами: `worldPosition`, `toTarget`, `unitDirection`, `velocity`, `desiredDisplacement`. `XMFLOAT3` сам по себе не защитит от передачи локальной координаты кости в функцию, ожидающую мировую позицию. Если проект разрастётся, можно добавить небольшие обёртки типов для пространств координат. До этого договорись, что сигнатура метода и комментарий указывают пространство и единицу. Для `muzzleWorld` это особенно важно: локальный socket оружия нельзя вычитать из мировой позиции игрока, пока socket не прошёл через анимацию, модель и world transform.

12.3. Четыре пространства одного выстрела

Представь точку на дуле оружия. В координатах меша она может быть `(0; 0; 0,6)`. Рука персонажа задаёт ей пространство кости; анимация меняет transform кости; transform персонажа переводит точку в мир. Камера затем переводит мир в view space, а проекция — в clip space для изображения. Пуля же начинает raycast в world space. Если взять clip space координаты потому, что они уже посчитаны для рендера, физический луч пройдёт в бессмысленном направлении. На бумаге подпиши каждую стрелку: `model → bone/skin → world → view → clip → NDC → pixel`. Не все стрелки нужны каждой системе. Physics останавливается на world; UI projection идёт до pixel.

Проверка: поставь персонажа у начала координат и выведи `muzzleWorld` debug точкой. Перенеси персонажа на +10 м по X. Мировая точка должна сместиться примерно на +10 м, локальная точка socket — остаться прежней. Поверни персонажа, не двигая. Расстояние между плечом и дулом должно сохраниться, а направление дула повернуться. Если какое-то свойство нарушено, выясни, в каком пространстве впервые возникло расхождение.

12.4. Относительное и абсолютное время

Игровой кадр сообщает `dt` — длительность предыдущего шага, а не общее время с начала игры. Если cooldown оружия равен 0,2 с, можно уменьшать оставшееся время на `dt` и разрешать выстрел при достижении нуля. Для длительного журнала событий удобнее абсолютная метка simulation time: `nextShotTime = now + cooldown`. Обе модели работают, но не смешивай wall clock и pause-aware simulation time. При паузе симуляции лечение и ремонт должны остановиться, а редакторский UI может продолжать работать.

Упражнение. Два обновления длились 0,016 с и 0,033 с. NPC со скоростью 2 м/с должен запросить суммарно 0,098 м движения вдоль единичного направления. Проверь: 2·(0,016+0,033)=0,098. Если

получилось 0,064 м, значит один из кадров ошибочно заменён фиксированной величиной 0,016. Если получилось 4 м, скорость прибавлена один раз на кадр без времени.

13. Векторы: движение, обзор и геометрия

13.1. Длина из теоремы Пифагора

В двух измерениях прямоугольный треугольник с катетами 6 и 8 имеет гипотенузу $\sqrt{(6^2+8^2)}=10$. В трёх измерениях добавляется третий квадрат: для `(x;y;z)` длина `|v|=√(x²+y²+z²)`. Это формула не «для движка», а способ измерить длину смещения в выбранной евклидовой системе координат. Если NPC движется только по палубе, для оценки горизонтального расстояния исключи Y: `√(dx²+dz²)`. Для проверки дистанции выстрела или объёма взрыва обычно нужна вся трёхмерная длина. Прежде чем удалить Y, спроси, является ли задача плоской.

Часто нужен только ответ «ближе ли цель 10 м?». Вместо `sqrt(dx²+dy²+dz²) < 10` сравни квадраты `dx²+dy²+dz² < 100`. При положительных числах квадратный корень монотонен, поэтому ответ тот же. Для 100 NPC и десятков целей это снижает стоимость отбора. Если нужен сам метраж для HUD или времени прибытия, корень всё равно вычисли. Сравнение квадратов требует одинаковых единиц: левая часть в м², правая тоже.

13.2. Нормализация и проблема нуля

Пусть `d=(6;0;8)` м. Делим на длину 10 м и получаем `n=(0,6;0;0,8)`. Проверка: `0,6²+0,8²=1`. Единичный вектор позволяет независимо задать скорость: для 2 м/с получаем `(1,2;0;1,6)` м/с. Без нормализации `d*2` дал бы длину 20 м/с — далёкая цель внезапно заставила бы NPC бежать быстрее. Нормализуя, ты устраняешь влияние расстояния на длину скорости.

Если A=B, длина равна нулю и деление невозможно. В реальном коде ещё опаснее почти нулевое число: `1e-8` после деления усилит шум округления. Сначала проверь `lengthSquared > epsilon²`, где epsilon выбран в единицах задачи. Для мировых метров epsilon 1 мм означает `epsilon²=1e-6 м²`; для quaternion и unit direction применяются иные критерии. При совпавших точках не нужно придумывать случайное направление: NPC уже прибыл и должен перейти к действию или удержать предыдущий forward. Кодовый шаблон: `if (d2 <= eps*eps) return Arrived; direction = d / sqrt(d2);`.

13.3. Dot product как сумма вкладов осей

Скалярное произведение `dot(a,b)=ax·bx+ay·by+az·bz`. Если `a=(1;0;0)` и `b=(0;0;1)`, произведение равно нулю: направления перпендикулярны. Если `b=(1;0;0)`, результат 1 для единичных векторов: совпадают. Если `b=(-1;0;0)`, результат −1: противоположны. Геометрическая запись `dot(a,b)=|a||b|cosθ` связывает расчёт по компонентам с углом. Нормализуй оба направления, тогда dot находится в пределах −1…1 и сразу равен cosθ. Значение чуть выше 1 из-за float перед `acos` следует ограничить `clamp(dot,-1,1)`.

Рассчитаем обзор врага. Его горизонтальный forward `(0;0;1)`, направление на цель до нормализации `(3;0;4)`. Длина равна 5, unit to target `(0,6;0;0,8)`. Dot равен 0,8; значит угол ≈36,87°. При полном FOV 70° допустимо ±35°, порог `cos35°≈0,819`: цель с dot 0,8 снаружи. При полном FOV 90° допустимо ±45°, порог ≈0,707, цель внутри. Сравнивать нужно `dot >= cos(fullFov/2)`. Если использовать cos полного угла 70°, фактический конус расширится до 140°. Если забыть нормализацию, dot с `(3;0;4)` равен 4 и тест бессмыслен.

FOV отвечает только на вопрос угла. Стена в центре конуса остаётся стеной: после дешёвых distance и FOV нужен raycast к одной или нескольким точкам цели. Для NPC на другом этаже не отбрасывай Y бездумно. Горизонтальный FOV можно считать по XZ, а вертикальный отдельно по углу elevation. При нулевой горизонтальной проекции цель строго над головой: горизонтальное направление неопределено, значит нужен отдельный случай.

13.4. Cross product: сторона, нормаль, winding

Векторное произведение создаёт вектор, перпендикулярный двум исходным. Для `a=(1;0;0)` и `b=(0;0;1)` по стандартной алгебраической формуле `cross(a,b)=(0;-1;0)`. Переставь операнды — знак изменится. Именно поэтому выбор стороны поворота нельзя отлаживать словами «лево» и «право» без test scene: знак зависит от порядка cross, выбранного forward и соглашения о матрицах. Возьми NPC, смотрящего в +Z, и цель в +X; выведи Y-компонент `cross(forward,toTarget)` и проверь, какой знак в твоём коде действительно вращает его к цели.

Для треугольника с вершинами A, B, C нормаль строится из `cross(B−A,C−A)`. Если A=(0;0;0), B=(1;0;0), C=(0;0;1), нормаль `(0;-1;0)`. Переставив B и C, получишь `(0;1;0)`. Это меняет не только освещение: back-face culling может скрыть поверхность. Поэтому при «пропавшем» меше проверяй winding и cull state прежде, чем менять pixel shader. Если треугольник вырожден и точки лежат на одной прямой, длина cross около нуля, нормаль определить нельзя.

13.5. Проекция на направление и нормаль

Для единичного n проекция v на n равна `dot(v,n)·n`. Например, скорость `(3;2;0)` м/с вдоль пола с нормалью `(0;1;0)` содержит вертикальную часть `(0;2;0)` и горизонтальную `(3;0;0)`. Формула удаления нормальной части: `v_tangent=v−dot(v,n)n`. Она полезна при движении вдоль наклонной поверхности, скольжении и построении касательной basis для шейдера. Но не превращай любую скорость в «горизонтальную» каждый кадр: для прыжка вертикальная компонента физически нужна.

При движении к WorkZone проекция помогает понять прогресс: `dot(actualDisplacement,desiredDirection)` положителен, если NPC продвинулся в нужную сторону. Это не то же самое, что длина фактического шага: персонаж может двигаться вдоль стены поперёк цели. Для детектора застревания сравни дистанцию к следующему waypoint через интервал, а также факт реального перемещения Bullet. На границе прибытия введи гистерезис: начать работу при ≤1,5 м, прекратить лишь при >1,8 м. Иначе физические колебания 1,49↔1,51 м будут переключать состояние каждый тик.

13.6. Самостоятельная проверка раздела

Реши до запуска: A=(−2;1;4), B=(1;5;8). Найди B−A, длину и единичное направление. Ответ: `(3;4;4)`, длина `√41≈6,403`, направление `≈(0,4685;0,6247;0,6247)`. Если задача ограничена палубой XZ, горизонтальная длина 5 м, направление `(0,6;0;0,8)`. Второй ответ не противоречит первому: это разные геометрические вопросы.

В движке нарисуй стрелки world forward NPC, to target и normal пола. Рядом покажи значения длины, dot и cross. Поверни NPC на 90°, затем перемести цель этажом выше. По числам объясни каждое изменение до того, как смотришь на анимацию. Если числа верны, а модель смотрит не туда, ищи offset импорта или порядок transform, не переписывай формулу dot.

14. Матрицы и преобразование пространства

14.1. Откуда берётся матрица 4×4

Трёхмерная точка имеет три координаты. Для обычного вращения достаточно матрицы 3×3, но перенос нельзя выразить одним умножением тройки на 3×3: перенос добавляет постоянное число. Однородные координаты дописывают четвёртую компоненту `w`. Точка имеет `(x;y;z;1)`, направление `(x;y;z;0)`. При умножении на матрицу переноса добавка из последней строки действует только на w=1. Поэтому world position переносится, а world direction остаётся направлением. Нормаль также не должна получать перенос.

В DirectXMath используются векторы-строки и композиция слева направо: `p_world = p_local · S · R · T`, если сначала масштабируем вокруг локального начала, затем вращаем и переносим. Это следует из порядка применения: выражение `(p·S)·R` сначала вычисляет результат масштаба. В `pbr.vs` проекта этот контракт виден в последовательных `mul(finalPos,worldMatrix)`, `mul(worldPos,viewMatrix)`, `mul(viewPos,projectionMatrix)`. На CPU `PBRShaderClass.cpp` перед записью в constant buffer вызывает `XMMatrixTranspose` из-за соглашения о размещении HLSL матрицы в памяти. Транспонирование при загрузке не означает, что надо поменять порядок мировых действий.

14.2. Порядок операций считается руками

Локальная точка `(1;0;0;1)` масштабируется по X в 2 раза и переносится на +5 по X. При `p·S·T` получаем сначала `(2;0;0;1)`, затем `(7;0;0;1)`. При `p·T·S` сначала `(6;0;0;1)`, затем `(12;0;0;1)`. Разница 5 м — не ошибка округления, а другая последовательность действий. Если дверь вращается вокруг петли, помести pivot у петли или составь перенос к pivot, вращение и обратный перенос. Вращение вокруг центра створки может выглядеть похоже лишь при малом угле, но на 90° станет очевидной ошибкой.

Родительская и дочерняя матрицы подчиняются тому же правилу. При строковом соглашении локальная точка предмета проходит `p·M_itemLocal·M_characterWorld`; для socket кости между ними

есть локальные transforms и skinning. Проверка: поворот корабля должен поворачивать предмет вместе с ним, сохраняя относительное расстояние. Если ты умножил родителя раньше дочернего, предмет начнёт вращаться вокруг неожидаемой точки. Запиши маленький тест на простом объекте у `(1;0;0)`, не проверяй впервые на сложном скелете.

14.3. View и projection не меняют мир

World matrix отвечает, где объект находится в сцене. View matrix отвечает, как мир выглядит из камеры, и геометрически является обратным transform камеры. Если камера переместилась вправо на 5 м, статичная точка мира в view space сместится влево на 5 м. Projection matrix переводит view координаты в clip coordinates. После деления на w получается NDC, из которого viewport строит пиксель. Смена камеры не должна менять физическое положение Bullet тела. Если игра меняет world Transform в ответ на поворот камеры только ради картинки, физика и AI будут расходиться с рендером.

При перспективе объект на расстоянии 20 м выглядит примерно вдвое меньше того же объекта на 10 м при прочих равных. Это связано с делением x и y на величину, зависящую от глубины. При ортографической проекции размер не меняется с глубиной. Для UI picking эти две модели требуют разных лучей: перспективные лучи расходятся от камеры; ортографические параллельны и отличаются началом. Не подставляй одну формулу origin для обоих случаев без проверки типа камеры.

14.4. Обратная матрица и её границы

Обратная матрица M^{-1} отменяет преобразование: $(p \cdot M) \cdot M^{-1} = p$. Если известна мировая точка попадания и нужно записать её в координатах модели для decal, примени inverse world. Если world scale по оси равен нулю, информация об этой оси уничтожена; обратной матрицы не существует. Поэтому до инверсии проверяй determinant и валидность результата. Матрицу камеры также нужно обратить при построении picking ray: экранная точка проходит обратный путь `clip → view → world`.

При преобразовании луча начальная точка имеет w=1, направление w=0. Если применить перенос к направлению, raycast будет зависеть от того, где расположен объект, даже при неизменной ориентации. После неравномерного масштаба длина направления может измениться, и её следует нормализовать, если API ожидает единичный вектор. Длину луча пересчитай отдельно: нормализация локального направления меняет параметризацию расстояния.

14.5. Почему нормаль требует inverse transpose

Нормаль определяется как вектор, перпендикулярный поверхности. Возьми касательную `t=(1;1;0)` и нормаль `n=(1;−1;0)`: dot равен 0. Масштаб X в 2 раза переводит касательную в `t'=(2;1;0)`. Если к нормали применить тот же масштаб, `n_bad=(2;−1;0)` и dot равен 3: условие перпендикулярности нарушено. Для нормали применяем inverse transpose линейной части

масштаба: `n'=(1/2;-1;0)`, и dot с t' снова 0. Затем нормализуем n'. Это не хитрость ради освещения, а следствие сохранения уравнения плоскости при преобразовании координат.

В коде рендера проверь нормаль вместе с tangent и binormal. Normal map задаёт направление относительно TBN basis; если модель растянута или анимирована, один исправленный normal при неверных tangent всё ещё даёт неправильное освещение. При простом вращении inverse transpose совпадает с вращением, поэтому ошибка долго скрывается. Проверяй на преднамеренно растянутом кубе: грань должна оставаться освещённой по её геометрической ориентации.

14.6. Контрольный опыт для матриц

Поставь вершину в `(1;0;0)`, объект перенеси в `(5;0;2)`, камеру установи в известную точку и временно выведи `model`, `world`, `view`, `clip` для этой вершины. Сначала включи только перенос: world должен быть `(6;0;2)`. Затем включи масштаб X=2: при `S·T` world X станет 7. Затем поверни объект: расстояние вершины от pivot после масштаба должно оставаться 2 м. Если world уже неверен, нет смысла искать ошибку в pixel shader. Если world верен, но clip неверен, переходи к view/projection, aspect ratio и near/far planes.

15. Углы, вращения и интерполяция

15.1. Тригонометрия начинается с окружности

Угол сообщает, насколько повернулось направление. Полный оборот — 360° или 2π радиан; пол-оборота — 180° или π; четверть — 90° или π/2. `sin` и `cos` связывают угол с координатами единичной окружности: при 0° направление вдоль X имеет `(cos0,sin0)=(1,0)`, при 90° — `(0,1)`. Это позволяет построить направление по углу, но только если соглашение об осях и нулевом направлении задано явно. В C++ `std::sin`, `std::cos` и функции DirectXMath работают с радианами. В редакторе человеку удобны градусы; перевод `radians=degrees·π/180` должен происходить ровно один раз на границе UI.

Проверка на конкретном числе: 90°·π/180=π/2≈1,5708 рад. Если передать в `sin` число 90 без перевода, результат не равен 1; функция интерпретирует 90 как радианы. Аналогично ошибка двойного перевода 1,5708 ещё раз умножит на π/180 и даст крошечный поворот. Подписывай поля `yawDegrees` и `yawRadians`, пока данные не прошли границу преобразования. Угол обзора в Inspector может храниться в градусах, а порог FOV вычисляется как `cos(degToRad(fullFov*0.5))`.

15.2. `atan2` и направление поворота

Для направления на плоскости XZ удобно вычислять угол по `atan2(dx,dz)`, если нулевой forward выбран как +Z и знак соответствует твоему Transform. Преимущество `atan2` перед `atan(dx/dz)` — он учитывает квадрант и не делит на dz=0. Но написание `atan2(dz,dx)` меняет нулевую ось; обе формулы могут быть полезны при разных соглашениях. Сначала проверь четыре цели: +Z, +X, −Z,

−X. Ожидаемые углы относительно +Z: 0°, примерно 90°, ±180°, примерно −90° в выбранной системе знаков. Если модель импортирована с локальным forward −Z, добавь один явный model offset, не меняй AI угол в нескольких местах.

Разность углов через границу −π/π нужно завернуть в интервал [−π,π]. От 179° до −179° кратчайший переход составляет +2° или −2° в зависимости от соглашения, а прямое вычитание даёт −358°. Практический алгоритм: `delta = remainder(target-current, 2π)` либо эквивалентное безопасное wrap; затем ограничить шаг `maxTurnRate*dt`. При нулевом направлении на цель угол неопределён — NPC остаётся с прежней ориентацией. Если он непрерывно вращается на месте, смотри не только знак delta, но и состояние прибытия, анимационное root motion и то, кто последним записывает Transform.

15.3. Euler углы и порядок осей

Yaw, pitch и roll — три удобных числа для редактора. Проблема в том, что вращения вокруг разных осей не коммутируют. Поверни книгу сначала вокруг вертикальной оси, затем наклони; повтори в обратном порядке — обложка окажется направлена иначе. Матрицы `Rx·Ry` и `Ry·Rx` обычно различны. Поэтому фраза «поворот (30°,20°,0°)» неполна без порядка применения и соглашения об осях. При некоторых ориентациях две оси Euler оказываются совмещены, и один вид движения теряет независимость. Для камеры с ограниченным pitch и свободным yaw Euler модель часто достаточно удобна; для произвольной ориентации корабля, костей и плавного смешения анимации применяй кватернион.

15.4. Кватернион как запись оси и угла

Единичный кватернион для поворота вокруг нормализованной оси u на угол θ записывается `q=(ux·sin(θ/2);uy·sin(θ/2);uz·sin(θ/2);cos(θ/2))`. Половинный угол появляется из алгебры композиции вращений; не заменяй его на θ в коде. Для 180° вокруг Y получаем `(0;1;0;0)`. Длина правильного q равна 1. При многократных численных операциях она может немного уйти от 1, поэтому периодически нормализуй, если API не делает это гарантированно.

Кватернионы q и −q задают одинаковую ориентацию. Это важно для интерполяции: если `dot(q0,q1)<0`, замени один на отрицательный, чтобы идти по короткой дуге. `slerp` даёт более равномерную угловую скорость; нормализованный lerp дешевле и годится для малых углов, когда визуальное различие несущественно. Кость персонажа требует не только ориентации: её локальная позиция, scale, parent transform и bind pose участвуют в skinning. Кватернион не заменяет всю матрицу кости и не исправит неверный порядок parent-child.

15.5. Интерполяция, которая сохраняет время

Обычный `lerp(a,b,t)=a+(b−a)t` предполагает, что t — доля пути от 0 до 1. Если за каждый кадр брать постоянное t=0,1, при 120 FPS объект делает вдвое больше шагов, чем при 60 FPS, и догоняет цель быстрее. Для сглаживания с постоянной скоростью сближения по реальному

времени используй alpha=1−exp(−λ·dt), затем value=lerp(value,target,alpha). За суммарное время T остаточная ошибка умножится примерно на exp(−λT) независимо от разбиения на кадры.

Например, λ=4 $с^{-1}$ и dt=1/60 с дают alpha≈0,0645; при dt=1/30 с alpha≈0,1248. За 1 с оба случая оставят около 1,8% исходной ошибки. Для физического движения к цели этот сглаживатель не заменяет заданную скорость в м/с: он хорош для камеры, UI и некоторых поворотов. Если персонаж обязан пройти строго 2 м/с, ограничь перемещение величиной speed*dt и не переходи за цель.

15.6. Практика раздела

Задай NPC поворот на 90° к цели, потом измени FPS 30→120. Сравни время поворота при angle += 3°/frame и при angle += 180°/s*dt. Первый вариант закончит за разное реальное время; второй — за 0,5 с в обоих режимах. Затем поставь целевой yaw −179° при текущем +179° и проверь путь. Результат должен быть около 2°, а не почти полный оборот. Для костей проверь q и −q: итоговая поза совпадает, а интерполяция при корректном выборе знака не делает неожиданный длинный поворот.

16. Камера, лучи, столкновения и границы

16.1. Перспектива и превращение пикселя в направление

Пиксель экрана не является координатой мира. Для viewport шириной W и высотой H точка мыши (x,y) превращается в NDC: x_ndc=2x/W−1, y_ndc=1−2y/H. Минус у Y нужен потому, что экранные пиксели обычно растут вниз, а NDC растёт вверх. Для 1920×1080 центр (960,540) даёт (0,0). Точка (1440,270) даёт (0,5;0,5). Это только положение в проецированном квадрате, не готовый вектор Bullet.

Возьми ближнюю и дальнюю точки clip/NDC с одинаковыми x и y, подходящими z для DirectX depth, примени inverse projection и inverse view, затем раздели координаты на w. Получишь nearWorld и farWorld; направление normalize(farWorld−nearWorld). В перспективной камере лучи расходятся от камеры, в ортографической — параллельны и имеют разное начало. После построения сделай debug опыт: центральный луч совпадает с camera forward, лучи по углам идут через соответствующие углы экрана, а изменение FOV меняет угол крайних лучей. Если вертикальная ось отражена, первым делом проверь 1−2y/H.

16.2. Уравнение луча и попадание в плоскость

Луч задаётся началом O, единичным направлением D и неотрицательным расстоянием t: P(t)=O+tD. Для плоскости с точкой P0 и нормалью N условие попадания dot(P(t)−P0,N)=0. Подстановка даёт t=dot(P0−O,N)/dot(D,N). Если знаменатель близок к нулю, луч почти параллелен плоскости: делить нельзя. Если t<0, пересечение позади луча. Если t больше максимальной длины оружия, физически попадания нет. Эта ручная формула полезна для понимания picking и

диагностики, но реальный мир содержит collision shapes, поэтому Bullet raycast выполняет поиск по телам и фильтрам.

Пример: O=(0;2;0), D=(0;−1;0), пол Y=0, нормаль N=(0;1;0), P0=(0;0;0). Числитель −2, знаменатель −1, t=2 м. Если D=(1;0;0), знаменатель 0: луч идёт параллельно полу. Прежде чем передать результат в игровое действие, проверь Entity владельца collider, collision mask, исключение стрелка и тип поверхности. Геометрическое первое касание не обязано быть ремонтируемым объектом.

16.3. Глаз, дуло и центр цели не взаимозаменяемы

Враг может видеть голову игрока поверх ящика, но его ствол остаётся ниже края. Raycast от глаза к голове докажет видимость; raycast от дула к aim point должен запретить выстрел. Если использовать один луч для обоих решений, NPC будет стрелять сквозь собственное укрытие. Аналогично от центра груди до центра груди низкий забор может перекрыть контакт, хотя голова видна. Выбирай несколько точек цели, но не превращай любую видимую точку в идеальное попадание: оружие имеет spread и muzzle clearance.

Для физического снаряда, в отличие от hitscan, цель успевает сместиться. Пусть r=q−p — вектор от стрелка к цели, v — скорость цели, s — скорость снаряда. Равенство расстояний $|r+vt|=st$ после возведения в квадрат даёт $(v\cdot v-s^2)t^2+2(r\cdot v)t+r\cdot r=0$. Нужен минимальный корень t≥0. Если дискриминант отрицателен, перехвата при этих скоростях нет. Для hitscan время полёта по игровой модели равно нулю, и искусственное упреждение будет ошибкой. Сначала выбери модель оружия, потом формулу.

16.4. Ограничивающие объёмы и геометрия поверхности

Sphere быстро проверяет расстояние до центра, AABB — интервалы по мировым осям, OBB учитывает ориентацию, capsule подходит для персонажа с постоянным радиусом и протяжённостью по высоте. Render mesh может иметь тысячи треугольников, но collision shape обычно проще. Поэтому видимая щель не обязана быть проходом для Bullet капсулы, а путь NavMesh должен учитывать фактический радиус агента. Отладка «NPC не проходит» начинается с трёх наложенных картин: render geometry, collision geometry и NavMesh corridor.

Для AABB двух объектов пересечение существует, если интервалы по каждой оси пересекаются. Это дешёвый broadphase filter, но ещё не ответ о контакте сложных shapes. Для ray-AABB по каждой оси получают диапазон t входа и выхода; итоговое пересечение есть, если диапазоны всех осей имеют общую часть и t≥0. Если компонент направления близок к нулю, луч параллелен плоскостям этой оси: проверь, находится ли origin внутри соответствующего интервала. Bullet делает подобные низкоуровневые проверки за тебя, но понимание геометрии помогает диагностировать collision mask, неправильные bounds и неожиданные «невидимые стены».

16.5. Упражнение раздела

Камера смотрит на плоскую стену, а мышь стоит в центре экрана. Нарисуй near и far world точки и физический луч. Затем перенеси камеру вправо на 2 м без поворота: центральное направление должно остаться тем же, origin сдвинется. После поворота камеры вправо направление изменится. Для ортографического режима сравни лучи через центр и левый край: направления совпадают, начала различаются. Проверь три случая выстрела: пустое поле, стена перед целью и стенка прямо перед muzzle. Подпиши, какая проверка разрешает или запрещает действие.

17. Численные методы и математика систем корабля

17.1. Почему формула движения зависит от шага

При постоянном ускорении `a` и начальной скорости `v0` аналитическое положение за время t равно `x=x0+v0t+0,5at²`. Игровая симуляция обычно идёт маленькими шагами: обновляет скорость `v += a·dt`, затем положение `x += v·dt` или использует другой интегратор. Это приближение, потому что скорость в течение шага меняется. При большом dt погрешность растёт, а столкновение можно пропустить. Поэтому physics world часто использует fixed step и несколько substeps при медленном рендере. Не передавай переменный `dt` в каждый слой как будто результат гарантированно одинаков при 30 и 120 FPS.

Простой пример: объект падает с высоты 5 м с нулевой скоростью при g≈9,81 м/с². За 0,1 с аналитическое смещение ≈0,04905 м. Схема `v+=gdt; x+=vdt` даёт ≈0,0981 м за один крупный шаг — вдвое больше. Если разделить 0,1 с на десять шагов по 0,01 с, результат приблизится к аналитическому. Это не значит, что надо вручную двигать Bullet тело по аналитической формуле; пример показывает, почему размер шага имеет значение и почему нельзя судить о правдоподобии только по одному FPS.

17.2. Ограничение шага и интерполяция картинки

Fixed-step loop накапливает реальное время: `accumulator += frameDt`, затем выполняет `while accumulator >= fixedDt` с ограничением числа итераций. После шагов рендер может интерполировать между предыдущим и текущим физическим transform с коэффициентом `accumulator/fixedDt`. Так симуляция остаётся дискретной и предсказуемой, а картинка не дёргается на мониторе с другой частотой. Если кадр завис на секунду, бесконечное догоняние создаст spiral of death; поэтому ограничивают максимальный frameDt или число substeps и фиксируют потерю времени в диагностике. Решение влияет на честность симуляции и должно быть документировано.

17.3. Давление, плотность и затопление

Гидростатическое давление снаружи корпуса в воде растёт с глубиной: `p=p_surface+ρgh`, где ρ — плотность кг/м³, g — м/с², h — м. Проверка размерности: `(кг/м³)(м/с²)(м)=кг/(м·с²)=Па`. При

ρ≈1000 кг/м³ и h=10 м добавка ≈98 100 Па. В коде `PressureSystem` использует атмосферное давление плюс `ρgh`; это модель, и она требует ясного определения, где нулевая глубина и какой знак у вертикальной координаты. Не подставляй 10 в формулу, если координаты сцены измеряются условными единицами без масштаба.

Если входящий поток воды `q_in` и откачка `q_out` измерены в м³/с, изменение объёма за шаг `ΔV=(q_in−q_out)dt`. При `q_in=0,04`, `q_out=0,01` и dt=0,5 с получаем 0,015 м³. Новый объём не может быть меньше нуля или больше физического объёма комнаты; после clamp нужно решить, куда делась лишняя вода и что означает переполнение. Для нескольких комнат проверяй сохранение: внутренний переток уменьшает воду в одной комнате и увеличивает в другой на ту же величину, если потери не предусмотрены. Этот инвариант сильнее визуального эффекта течи.

17.4. Энергия, мощность и приоритеты

Генератор выдаёт мощность в ваттах. Потребители просят мощность; аккумулятор хранит энергию. Если устройство потребляет 200 Вт 5 секунд, оно использовало 1000 Дж. Если аккумулятор хранит 1 Wh, в нём 3600 Дж, значит при идеальной модели хватит на 18 с при 200 Вт. Реальный `ShipPowerGridSystem` распределяет доступную мощность по priority и вычисляет power ratio; устройство с требованием непрерывной полной мощности может считаться powered только при ratio около 1. Это игровое правило, а не арифметическая необходимость. Разрешать ли частичную работу двери при 70% питания, определяет дизайн системы.

Перед распределением запиши инвариант: сумма выделенной мощности не превышает генерацию плюс допустимую отдачу батареи; заряд батареи неотрицателен; при отсутствии генерации и пустой батарее потребитель не получает энергию. Если видишь `hasPower=true` при нулевом supply, проверь старый флаг после загрузки, порядок обновления сетки и значение ratio. Если батарея растёт при выдаче энергии, проверь знак потока. Математика здесь не отделена от архитектуры кадра: даже правильная формула в неправильном порядке систем даёт неверный мир.

17.5. Вероятность, случайность и повторяемость

Разброс оружия и поведение NPC могут использовать случайные числа. Вероятность 20% не гарантирует ровно два срабатывания из десяти, но при большом числе независимых попыток доля стремится к 0,2. Для отладки используй фиксированный seed, чтобы один и тот же бой можно было воспроизвести. Визуальный трассер и физический луч должны получить одно случайно выбранное направление, а не два независимых броска. Иначе игра покажет выстрел в одно место, а урон нанесёт в другом.

При случайном направлении внутри конуса нельзя просто равномерно выбрать yaw и pitch, если требуется равномерная плотность по телесному углу: площадь у полюсов и экватора различается. Для большинства игровых оружий достаточно понятной модели разброса по экранной/угловой плоскости, но её нужно назвать и проверить распределением попаданий. Сохрани 1000 точек на

стене и посмотри, образуют ли они ожидаемое пятно. Настройка «accuracy» без измерения распределения часто скрывает систематический offset muzzle или ошибку единиц градусов.

17.6. Финальный математический эксперимент

На одной тестовой сцене запусти ремонт, бой, затопление и энергосеть при 30, 60 и 120 FPS на одинаковое simulation time. До запуска выпиши прогноз: расстояние NPC, время до готовности ремонта, число выстрелов с заданным cooldown, изменение воды, расход энергии. После прогона сравни цифры и укажи допустимые отклонения для дискретной физики. Если результат отличается кратно FPS, ищи применение величины «за кадр» вместо «за секунду». Если только physics contact отличается на малую величину, сравни substeps, collision geometry и порядок обновления. Если лишь HUD показывает другой ответ, отдели вычисление симуляции от форматирования интерфейса.

Проверка знания: ты должен уметь назвать пространство и единицу каждого аргумента, объяснить причину нормализации, вручную посчитать один dot/FOV, предсказать эффект перестановки матриц, восстановить world ray из пикселя и проверить сохранение массы или энергии. После этого переходи к реализации, а не к запоминанию новых формул списком.

Математическая мастерская. Один игровой эпизод, рассчитанный вручную

Теперь свяжем формулы в один эпизод. Это важно: в настоящей игре NPC не решает отдельно «задачу на вектор», потом отдельно «задачу на мощность». Он живёт в мире, где несколько численных моделей меняют друг друга. Построй такую же сцену и сравни рассчитанные значения с debug overlay. У каждого шага есть входы, вычисление, единица и игровой вывод. Не начинай следующий шаг, пока не можешь повторить предыдущий без подсказки.

M1. Путь инженера и цена прямой линии

Инженер находится в A=(2;0;3) м, рабочая точка пробоины B=(8;0;11) м. Разность `(6;0;8)` и длина 10 м. Он прекращает движение при 1,5 м, поэтому в пустой комнате нужен путь около 8,5 м. При скорости 2 м/с минимальное время 4,25 с. Поставь стену, заставив NavMesh путь пройти через дверь с длиной 15 м. Если рабочий радиус вычитается вдоль последнего сегмента, оценка около `(15−1,5)/2=6,75 с`. Это нижняя оценка: дверь может открываться 0,8 с, повороты и collision contacts добавят задержку. Раздели лог `straightDistance`, `pathLength`, `waitTime`, `actualTravelTime`. Если AI выбирает A вместо более доступной цели только потому, что A ближе по прямой, математика отбора использует неправильную величину для этой задачи.

Проверь координаты после одного шага 0,1 с без препятствий: единичное направление `(0,6;0;0,8)`, желаемая скорость `(1,2;0;1,6)` м/с, смещение `(0,12;0;0,16)` м, новая точка `(2,12;0;3,16)` м. Если фактическая капсула прошла 0,08 м, не подменяй её позицию расчётной:

Bullet мог встретить collider. Выведи contact и сравни фактическую проекцию движения на желаемое направление.

M2. Ремонт: задание, скорость и независимость от FPS

Пробоине нужно 120 единиц работы. У NPC `repairSkill=1,2`, базовая длительность при skill=1 — 12 с. Модель `rate=requiredProgress·skill/repairDuration` даёт `120·1,2/12=12 единиц/с`. На тике 0,1 с прибавка 1,2; на тике 1/60 с — 0,2. За 10 с оба режима должны накопить около 120, если NPC находился в WorkZone всё время. Если при 120 FPS работа завершается вдвое быстрее, где-то добавляется фиксированная порция «за кадр» вместо rate·dt.

Когда WorkZone покинута на 2 с, реши правило: progress сохраняется, медленно уменьшается или сбрасывается. Это дизайн, не математический факт. Но любой вариант должен быть записан и проверен. Если resource тратится на каждую единицу работы, при завершении должно уйти 120 соответствующих долей; если одна заплатка расходуется на весь ремонт, списывай её один раз при начале или завершении с безопасным rollback. Без этого save/load может размножить материалы.

M3. Мощность двери и аккумулятор

Генератор выдаёт 4 МВт, критический потребитель просит 3 МВт, дверь 2 МВт обычным приоритетом. После критического остаётся 1 МВт, дверь получает powerRatio=0,5. Текущий `hasPower` по порогу 0,999 будет false. Если батарея способна выдать дополнительный 1 МВт в течение 5 с, дверь получит полные 2 МВт и откроется. На этот эпизод батарея отдаст 5 МДж, то есть около 1,389 кВт·ч. Проверь, что расчёт в коде использует ту же единицу capacity: названия полей с `Mw` относятся к мощности, не к запасу энергии.

Сценарий: открой дверь при полной батарее, затем повтори при пустой. В первом случае инженер проходит; во втором coordinator должен объяснить недоступность маршрута. Если оба проходят, физический collider или связь двери не слушает энергосистему. Если оба стоят, возможно, дверь открыта в gameplay, но NavMesh или Bullet всё ещё блокирует проём. Так численный расчёт 0,5 ratio приводит к проверке архитектурной связи.

M4. Конус зрения после поворота

Враг сначала смотрит вдоль +Z. Игрок относительно него расположен в направлении `(3;0;4)`, то есть unit direction `(0,6;0;0,8)`. Dot равен 0,8, угол около 36,87°. При полном FOV 70° игрок снаружи на 1,87°; при 90° внутри на 8,13°. Теперь враг повернулся к игроку на 20° в нужную сторону. Угловое расхождение стало около 16,87°, dot≈cos16,87°≈0,957. Он проходит оба FOV. Проверка состоит из двух разных чисел: `forward` после поворота и `toTarget` после нормализации. Если dot не изменился, CharacterFacingSystem/Transform не записал новое направление либо perception читает старый forward.

Потом поставь стену между ними. Dot остаётся 0,957, а LOS становится false. Это наглядно показывает, почему одна формула обзора не может заменить raycast. Если стена прозрачная, реши, блокирует ли она зрение и пулю: visual transparency и collision mask не одно и то же. Явно назови материал или слой, который разрешает зрение, чтобы враг не видел через металл только потому, что он нарисован полупрозрачным shader.

M5. Луч выстрела и ограничения

Враг имеет eye world point `(0;1,7;0)` м и muzzle `(0,3;1,3;0,5)` м. Голова игрока в `(0;1,7;10)` м. Луч зрения от глаза почти горизонтален; луч от дула имеет смещение `(−0,3;0,4;9,5)` и длину около `√(0,09+0,16+90,25)=√90,5≈9,513 м`. Его единичное направление приблизительно `(−0,0315;0,0420;0,9986)`. Если низкая стенка высотой 1,5 м находится в 1 м от дула, этот луч ещё не поднялся выше 1,5 м: при t≈1 м его Y≈1,342. Глаз на высоте 1,7 м видит, но дуло упирается. Это не баг, а геометрический результат двух разных начал.

Покажи оба луча разными цветами и подпиши first hit. Затем перемести NPC назад: линия от дула может пересечь край стенки выше, и выстрел станет допустимым. Так AI может выбрать другую точку выглядывания по геометрии, а не получить специальное исключение «можно стрелять сквозь cover». После попадания проверь, что визуальный tracer совпадает с реальным направлением с учётом spread.

M6. Случайная ошибка прицела как измеряемое распределение

Пусть разброс задан как угловая ошибка с типичным масштабом 2°. На дистанции 10 м поперечное отклонение малых углов примерно `d·tan(2°)≈10·0,0349≈0,349 м`. На 30 м — около 1,047 м. Если игрок жалуется, что оружие «случайно промахивается в упор», выведи фактический угол и расстояние, проверь единицы (2° против 2 рад ≈114,6°) и muzzle offset. Сначала определи, что означает параметр: максимальный радиус конуса, стандартное отклонение или половинный угол квадратного распределения. Эти варианты дают разные частоты попаданий.

Для теста зафиксируй random seed и собери 1000 hits на плоской стене. Посчитай среднее смещение и долю попаданий внутри радиуса 0,35 м на 10 м. Среднее должно быть около центра, если нет систематической ошибки прицела. Равномерное пятно, плотный центр и обрезанный круг соответствуют разным моделям spread; выбери одну сознательно. При изменении расстояния средний угловой разброс остаётся прежним, а линейное пятно растёт.

M7. Корабль в воде и корабль в вакууме

Для подводного режима на глубине 10 м внешнее давление около 199 425 Па при `p0=101325 Па`. Вода течёт через отверстие по отдельной модели, а объём в комнате ограничен её capacity. Для космического режима наружное давление близко к нулю, а внутренний воздух уходит из отсеков; формула `p0+ρgh` здесь неприменима. Не подставляй отрицательную «глубину» или отрицательную плотность, чтобы получить вакуум. Это другая физическая модель с иным владельцем состояния и иной причинной цепью.

Учебный космический пример: две комнаты одинакового объёма при одной температуре содержат 100 и 50 условных единиц газа. Открытая дверь переносит 10 единиц из первой во вторую за тик: 90 и 60, сумма по-прежнему 150. Если вторая комната имеет отверстие наружу и теряет 5 единиц, в комнатах остаётся 145, а счётчик `ventedToSpace=5`; общая масса системы с внешней средой сохранена. Давление каждой комнаты вычисляй из количества газа, объёма и температуры, не обновляй независимо «по ощущению». Так можно проверить симуляцию числами до добавления частиц декомпрессии.

M8. Ошибка вычисления и допустимое отклонение

Числа `float` имеют конечную точность. Если многократно прибавлять маленькое dt, итог может отличаться на несколько последних разрядов от прямого вычисления. Поэтому проверка `position == expected` для физики почти всегда слишком строга. Но «плавающая точка» не оправдывает крупную ошибку. Для пути 10 м допуск 1 мм может быть разумным для чистой векторной формулы, а для Bullet контакта после серии столкновений нужен иной критерий, основанный на радиусе капсулы и игровом эффекте. Заранее запиши tolerance вместе с единицей и объясни, почему он допустим.

Проверяй инварианты, не зависящие от точного последнего бита: единичное направление имеет длину близко к 1; после rigid rotation расстояние между двумя точками сохраняется; распределённая мощность не превышает supply; объём воды находится в [0,capacity]; один эксклюзивный claim имеет не более одного владельца. Если тест не проходит на 20%, не увеличивай epsilon до 20%: найди ошибку в единицах, порядке систем или состоянии.

M9. Самостоятельные задания с ответами

Задание A. NPC стоит в `(-3;0;4)`, цель `(1;0;7)`, скорость 2,5 м/с. Смещение `(4;0;3)`, длина 5 м, единичное направление `(0,8;0;0,6)`, velocity `(2;0;1,5)` м/с. За 0,2 с желаемый шаг `(0,4;0;0,3)` м. Если WorkZone начинается за 1 м до цели, минимальное время по прямой `(5−1)/2,5=1,6 с`.

Задание B. Forward `(1;0;0)`, цель в направлении `(−1;0;1)`. После нормализации toTarget≈`(−0,707;0;0,707)`, dot≈−0,707, угол 135°. При FOV 120° половина 60°, порог 0,5, цель вне обзора. Если ответ «внутри» получился из сравнения −0,707 с 120, смешаны косинус и градусы.

Задание C. Комната получает 0,08 м³/с воды, насос удаляет 0,03 м³/с, dt=0,25 с. Чистый прирост `(0,08−0,03)·0,25=0,0125 м³`. За 100 шагов при неизменных потоках прирост 1,25 м³. Если насос выключен половину шагов, посчитай отдельно первые 50 и вторые 50, затем проверь clamp по объёму комнаты.

Задание D. Аккумулятор содержит 3,6 МДж и питает один прибор мощностью 0,2 МВт без потерь. Время `3,6/0,2=18 с`. Если генератор одновременно даёт 0,1 МВт, батарея покрывает лишь 0,1 МВт и работает 36 с. Если приложение показывает 18 с во втором случае, вероятно, discharge берёт полный demand вместо дефицита.

Задание E. Локальная точка `(1;0;0)` сначала масштабируется по X на 2, потом переносится на +5. Ответ world X=7. Если сначала переносить, потом масштабировать, X=12. Поставь модель в сцене и проверь обе композиции как отдельные конфигурации. Объясни, почему одна из них соответствует выбранному pivot, а другая — другой последовательности действий.

M10. Как понять, что математическая часть усвоена

Возьми незнакомый числовой параметр из Inspector. Без открытия реализации предположи его единицу, диапазон, пространство и владельца. Открой код и проверь догадку. Затем найди место, где параметр соединяется с dt или матрицей, и предскажи один численный результат. Нарисуй debug представление, сравни ручной расчёт и измерение. Если не совпало, напиши три проверяемые гипотезы: неверная единица, неверное пространство, другой порядок обновления. Такой порядок работы и есть практическая математика разработчика движка; формула без проверки ещё не знание.

Часть IV. DirectX 11 и рендер

Рендер нужен игре, чтобы показать состояние мира. Он читает Transform, mesh, материал, свет и камеру, но не должен определять игровые правила. Вершинный шейдер отвечает за положение вершин и передаваемые атрибуты; пиксельный — за цвет видимого фрагмента. Между ними GPU растеризует треугольники и интерполирует атрибуты. Эта часть даёт маршрут одного пикселя и способ отладки, когда он неверен.

Разбор 18. CPU, GPU и жизнь одного draw call

18.1. Что подготавливает CPU

Renderer получает геометрию, выбирает pipeline state, связывает vertex/index buffers, input layout, shaders, constant buffers, textures и samplers, затем отправляет draw call. GPU не знает, что Entity называется «генератор» или что ему нужен ремонт: ему переданы массив вершин, индексов и числовые параметры. Если объект отсутствует на экране, начни с CPU границы: был ли он отобран на рендер, есть ли валидные buffers, какая world matrix отправлена, какая camera active и какие ресурсы привязаны к slots. Попытка лечить отсутствие draw call изменением освещения в pixel shader ничего не даст.

Input layout связывает структуру vertex buffer с семантиками HLSL: position, normal, texcoord, tangent, bone indices и weights. Если layout ожидает float3 normal, а CPU записал другое смещение, vertex shader получит мусор даже при успешной компиляции. Введи контрольный mesh из одного треугольника с известными координатами и цветом. С ним проще отделить ошибку загрузки модели от ошибки pipeline. После него проверь куб, затем skinned NPC. Сложность должна расти по одному измерению за раз.

18.2. Вершинный шейдер и пространства координат

В `src/Render/Shader/Vs/pbr.vs` позиция после возможного skinning проходит три операции: `mul(finalPos, worldMatrix)`, `mul(worldPos, viewMatrix)`, `mul(viewPos, projectionMatrix)`. World position также нужен pixel shader для освещения. `SV_POSITION` — clip position до растеризации; viewport и perspective divide определяют пиксель на экране. Не передавай `SV_POSITION` как будто это world position для расчёта расстояния до источника света. Если освещение «плывёт» при перемещении камеры, первым делом проверь, в каком пространстве находятся light position, surface position и normal.

Для skinned mesh вершинный шейдер умножает позицию на bone transforms с весами. Веса должны быть неотрицательны и суммироваться примерно в 1; если часть костей отсутствует или индекс выходит за массив, вершина улетит в неожидаемую точку. Отдельно проверь normal/tangent: они не должны получать перенос. Bind pose, inverse bind matrices и текущие transforms костей должны использовать совместимое соглашение. Начни с одного жёсткого веса 1,0 для одной кости, затем добавь две кости и смесь, чтобы видно было, на каком этапе появляется ошибка.

18.3. Растеризация, depth и culling

После vertex shader треугольник попадает в rasterizer. Winding определяет лицевую сторону для back-face culling. Depth test решает, виден ли фрагмент за уже нарисованной поверхностью. Поэтому «модель не отображается» может означать невидимость из-за камеры, неправильное winding, clipping по near/far, depth state или прозрачный материал. Проверяй по порядку: wireframe/culling off на тестовой сцене, отладочный одноцветный shader, depth buffer и затем материал. Не отключай depth навсегда только потому, что объект появился: это подтвердило слой проблемы, но не исправило его.

Разбор 19. Пиксельный шейдер и физический смысл материала

19.1. Albedo, normal, metallic и roughness

`pbr.ps` проекта читает albedo, normal, metalness и roughness карты. Albedo задаёт базовое отражение поверхности; normal map меняет локальное направление микрогеометрии для освещения, но не форму collider и силуэт. Metallic определяет, ведёт ли материал себя как металл; roughness задаёт ширину и характер блика. Если normal map хранит значения 0…1, их переводят в диапазон −1…1 и преобразуют из tangent space через TBN basis в world space. Если перепутать цветовое пространство или каналы, металл станет пластиком, а углубления будут выглядеть выпуклыми.

Не все текстуры представляют цвет. Albedo обычно требует учёта sRGB при чтении; roughness, metallic, normal и masks являются линейными данными. Применить к roughness sRGB преобразование означает изменить числовой параметр материала. При импорте GLB проверь не

только путь к текстуре, но и семантику канала. В `pbr.ps` есть отдельная логика чтения roughness и обработка вариантов glTF/Assimp; при подозрительном блеске сначала выведи сырой sample и final roughness, а не подбирай интенсивность света на глаз.

19.2. От Lambert к микрофасеткам

Для идеальной матовой поверхности вклад света зависит от `max(dot(N,L),0)`: поверхность, обращённая к свету, получает больше энергии, а развернутая от света — ноль. Это простая модель диффузного отражения. Блик требует учёта направления к камере V и распределения микроскопических нормалей. PBR шейдер использует функции GGX distribution, geometry term и Fresnel. Их смысл: сколько микрофасеток правильно ориентировано, какая часть не закрыта собственной поверхностью и какая доля энергии отражается на границе материала. Числа нельзя интерпретировать как независимые «эффекты красоты»; вместе они ограничивают характер отражения.

Для проверки отключай слои по очереди. Сначала выведи world normal цветом `(N*0,5+0,5)`, затем NdotL, затем roughness и metallic, потом итоговую diffuse/specular сумму. Если на кубе нормали неправильны уже на первом шаге, сложная BRDF не исправит результат. Если normal верна, а блик вращается вместе с камерой некорректно, сравни пространства N, V и L. Если материал слишком тёмный только при включении normal map, проверь TBN и направление Y канала карты.

19.3. Свет, тени и эффекты повреждения

Directional light задаёт одно направление для всей сцены; point/spot lights зависят от расстояния и геометрии конуса. Shadow map хранит глубину от источника света; сравнение глубины текущего фрагмента с сохранённой решает, закрыт ли он для света. Типичная ошибка тени — acne или отрыв от объекта из-за bias. Подбирай bias в единицах глубины и проверяй на нескольких масштабах, не скрывай проблему огромным смещением, от которого NPC начинает «парить» над собственной тенью.

Трещина, мокрое пятно и подсветка выбора — визуальное следствие gameplay события. В проекте `pbr.ps` содержит crack map и дополнительные параметры повреждения. Источником правды о пробоине остаётся gameplay/physics, а shader получает отображаемые параметры. Если shader рисует пробоину без `HullBreachComponent` или collider, игрок увидит дырку, через которую не течёт вода или воздух. Для новой визуальной механики опиши событие, которое устанавливает параметр материала, и момент, когда он снимается после ремонта.

Разбор 20. Как добавить новый шейдер безопасно

Начни с одного видимого результата и входных данных. Например, аварийная панель должна пульсировать красным при падении давления. Gameplay вычисляет severity в диапазоне 0…1; renderer передаёт её в constant buffer; pixel shader смешивает исходный цвет с аварийным по функции времени. Время для пульса может быть render time, потому что эффект декоративный.

Состояние «давление низкое» должно оставаться в CPU системе: иначе его нельзя сохранить, проверить тестом или использовать NPC без GPU.

Опиши slots до компиляции: какой `register(bN)` занимает новый constant buffer и какие `register(tN)` уже использованы PBR шейдером. Структуры C++ и HLSL должны согласовывать порядок полей и выравнивание. Для constant buffer в D3D11 размер кратен 16 байтам; `float3` рядом с `float` требует внимания к упаковке. Выведи один контролируемый параметр как сплошной цвет, чтобы проверить передачу данных, затем добавляй текстуру и освещение. Сохрани компиляционные ошибки и проверяй shaders в Debug/Release.

Практика. На одном объекте отобрази `severity=0`, `0,5`, `1` тремя кадрами; для каждого заранее предскажи смешанный цвет. Перенеси камеру и объект: цвет от severity не должен меняться, если он не зависит от освещения. Затем отключи текстуру: fallback материал должен остаться видимым. После этого подключи эффект к реальной пробоине через событие gameplay и проверь, что ремонт убирает визуальный след в нужный момент.

18. Ресурсы и состояния DirectX 11

DirectX 11 — stateful API. Результат draw call зависит не только от переданных вершин, но и от всего состояния context: render targets, depth, blend, rasterizer, shaders, buffers, SRV и samplers.

18.1. Базовые ресурсы

Ресурс	Роль
`IDXGISwapChain`	очередь backbuffers и Present
`ID3D11RenderTargetView`	поверхность, куда пишет pixel shader
`ID3D11DepthStencilView`	depth/stencil testing
`ID3D11Buffer`	vertices, indices, constants
`ID3D11ShaderResourceView`	чтение texture/buffer в shader
`ID3D11SamplerState`	фильтрация и addressing
rasterizer state	culling, fill, bias, scissor
depth-stencil state	depth compare/write и stencil
blend state	смешивание результата pixel shader

Текущий backbuffer — `R8G8B8A8_UNORM`, depth — `D24_UNORM_S8_UINT`, MSAA выключен, основной rasterizer использует `CULL_NONE`.

18.2. Constant buffer alignment

HLSL constant buffer упаковывается блоками по 16 байт. CPU-структура должна совпадать по размеру и layout. Ошибка «создали 64 байта, записали 144» не обязана сразу упасть: она повреждает mapped memory и проявляется случайно.

> **ОГРАНИЧЕНИЕ.** В `SkyShaderClass` существует неиспользуемый legacy-метод, который может записать структуру больше созданного matrix buffer. Текущий рабочий path пишет только матрицу, поэтому дефект латентный.

18.3. COM lifetime

COM-объект освобождается `Release`, но ручной код уязвим к раннему `return`. Целевая форма:

```
Microsoft::WRL::ComPtr<ID3D11Buffer> buffer;
```

`ComPtr` автоматически балансирует reference count и делает partial initialization безопаснее.

18.4. Полностью заданное состояние

Каждый render pass должен явно отвечать:

1. какой RTV/DSV связан;
2. какой viewport;
3. depth test и depth write;
4. blend;
5. culling/fill;
6. какие SRV и samplers заняты;
7. какие slots очищаются после pass.

Иначе следующий pass наследует невидимый остаток предыдущего.

19. Фактический render pipeline

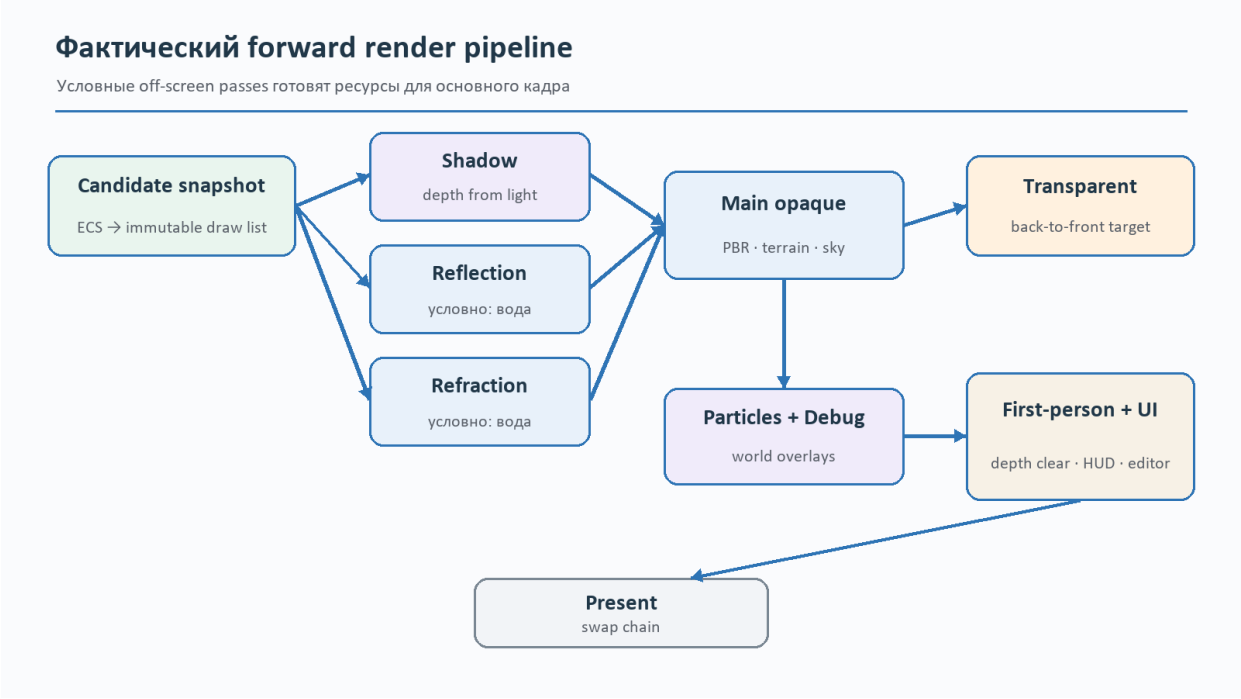


Рисунок 4. Основные проходы forward-рендера.

`Renderer::RenderFrame` реализует forward renderer. G-buffer/deferred lighting в проекте нет.

```
RenderCandidate snapshot/request
  ↓
Sun + environment update
  ↓
Directional shadow map
  ↓
Reflection texture (если нужна)
  ↓
Refraction texture (если нужна)
  ↓
Editor preview (если нужен)
  ↓
Backbuffer clear + equirectangular sky
  ↓
Opaque scene + grass
  ↓
World sprites
  ↓
Transparent scene
  ↓
Particles
  ↓
Debug geometry
  ↓
Depth clear + first-person geometry
  ↓
UI/HUD + Present
```

19.1. Почему first-person рисуется отдельно

Оружие и руки не должны проваливаться в стены мира. Отдельный pass после очистки depth и с собственной projection делает view model визуально стабильной. Цена — нужны отдельные

правила shadows, FOV и clipping; объект first-person нельзя случайно отрисовать ещё раз в world pass.

19.2. Candidate culling

Main thread копирует минимальные render-данные из ECS, worker проверяет frustum, main thread использует только подходящий по camera matrices результат. Это хороший пример безопасной параллельности: worker не трогает D3D immediate context и не обходит живой ECS.

19.3. Room culling

Кроме frustum, `RoomCullingSystem` определяет видимые помещения через связи комнат/дверей. Это portal-like gameplay culling. Оно не должно влиять на саму симуляцию: закрытая за дверью комната может не рисоваться, но oxygen/power/NPC продолжают обновляться.

19.4. Draw context как целевой интерфейс

Сейчас функции получают длинные наборы matrices, arrays и pointers. Безопаснее один объект:

```
struct RenderViewContext
{
    XMMATRIX view;
    XMMATRIX projection;
    XMFLOAT3 cameraPosition;
    LightPacket lights;
    FogPacket fog;
    RenderPassKind pass;
};
```

Так компилятор и validation layer не позволят refraction забыть camera position или light packet.

20. PBR: математика Cook–Torrance

Основной `pbr.ps` реализует metallic-roughness Cook–Torrance с GGX, Fresnel-Schlick и Smith geometry.

20.1. Векторы

```
N = surface normal
V = direction surface → camera
L = direction surface → light
H = normalize(V + L)
```

Все dot products ограничиваются снизу нулём:

```
NdotL = max(dot(N,L), 0)
NdotV = max(dot(N,V), epsilon)
```

20.2. Roughness и GGX distribution

Код использует:

```
a = roughness²
D_GGX = a² / (π · ((N·H)²(a² - 1) + 1)²)
```

`D` описывает долю микрограней, ориентированных как `H`. При малой roughness пик узкий; при большой — широкий.

20.3. Fresnel-Schlick

```
F = F0 + (1 - F0)(1 - cosTheta)⁵
F0 = lerp(0.04, albedo, metalness)
```

Диэлектрик отражает около 4% при прямом взгляде; металл берёт цвет отражения из albedo.

20.4. Smith/Schlick-GGX geometry

```
k = (roughness + 1)² / 8
G1(x) = x / (x(1-k) + k)
G = G1(N·V) · G1(N·L)
```

`G` учитывает masking/shadowing микрограней.

20.5. Specular BRDF

```
specular = D · G · F / max(4(N·V)(N·L), epsilon)
```

Diffuse сохраняет энергию:

```
kS = F
kD = (1 - kS)(1 - metalness)
diffuse = kD · albedo / π
```

Итог одного света:

```
Lo += (diffuse + specular) · radiance · (N·L)
```

20.6. Точечное и spot-затухание

Точечный свет использует художественную модель конечного радиуса:

```
attenuation(d) = (1 - d/R)², если d < R; иначе 0
```

Это не физический `1/d²`, зато свет строго заканчивается на `R` и удобен редактору.

Spot factor:

```
smoothstep(cosOuter, cosInner, coneDirection · L)
```

20.7. Ограничение количества lights

Shader поддерживает одно солнце и четыре point/spot lights.
`SceneManager::GatherLightsForShader` берёт первые четыре активных ECS-источника, а не ближайшие/важнейшие. При пятом источнике результат зависит от порядка `unordered_map` и может «прыгать». Целевая политика: score по влиянию на объект/камеру и стабильная сортировка.

21. Цвет, HDR и IBL

21.1. Linear и sRGB

Физика света выполняется в linear space. Обычная цветовая текстура хранится в sRGB и декодируется при sample; normal/metal/roughness — linear data.

Правильный pipeline:

```
sRGB textures → decode to linear
lighting/blending in linear HDR
exposure + tone mapping
linear → sRGB output
```

21.2. Текущий pipeline

PBR shader выполняет ACES-like tone mapping и gamma внутри каждого draw, а пишет сразу в UNORM backbuffer. Следствия:

- прозрачность смешивается после gamma;
- невозможно корректно применить общий exposure;
- bloom/postprocess не видит исходную HDR-энергию;
- разные shader families могут иметь разные color transforms.

Целевая модель: общий HDR scene target и ровно один финальный postprocess.

21.3. Environment map

Текущий sky — один equirectangular HDR. PBR читает его как environment reflection, но не использует:

- diffuse irradiance convolution;
- prefiltered specular mips;
- BRDF integration LUT.

Roughness поэтому в основном ослабляет отражение, а не физически размывает его.

Split-sum IBL:

```
ambientDiffuse  = irradiance(N) · albedo
ambientSpecular = prefiltered(R, roughness) · (F · brdf.x + brdf.y)
```

21.4. Несовпадение ориентации HDR

Sky shader добавляет к U примерно `+0.25`, а PBR direction→equirect UV — нет. Видимая панорама и отражение повернуты относительно друг друга примерно на 90°. Должна существовать одна общая функция mapping и один rotation parameter.

22. Тени

22.1. Текущая модель

- один directional light;
- orthographic shadow map 1024×1024;
- 2×2 PCF;
- receiver slope bias.

Light matrices:

```
V_light = LookAtLH(center - direction·distance, center, up)
P_light = OrthoLH(size, size, near, far)
```

Receiver bias:

```
bias = max(0.00015, 0.0008 · (1 - N·L))
```

22.2. Подтверждённые ограничения

1. Shadow volume центрирован возле world origin, а не camera frustum.
2. Casters предварительно отсекаются main-camera frustum; объект за камерой может давать видимую тень, но быть удалён.
3. Нет texel snapping — возможен shimmering.
4. Shadow pass не skinning-aware — animated mesh даёт bind-pose shadow.
5. Нет alpha cutout.
6. PCF предполагает фиксированное разрешение.
7. Grass не включён полностью в shadow model.

22.3. Путь развития

Сначала camera-fitted stabilized single cascade. Только после стабильности — CSM:

```
near cascade: высокая детализация
middle: средняя
far: большая площадь
```

23. Прозрачность, reflection и refraction

23.1. Правильная alpha transparency

Для обычного `srcAlpha / (1-srcAlpha)`:

```
opaque:       depth test ON, depth write ON
transparent:  depth test ON, depth write OFF, sort back-to-front
```

На срезе transparent objects не сортируются и продолжают использовать depth-write state. Если модель имеет хотя бы одну прозрачную embedded part, весь объект может уйти в transparent pass, включая непрозрачные части.

23.2. Refraction defect

В refraction path локальные arrays lights объявлены, но не заполняются до передачи в render. Camera position также остаётся default `(0,0,0)`. PBR specular/Fresnel получают случайный или неверный input. Это P0-дефект, потому что Debug/Release могут показывать разные картинки.

23.3. Reflection camera

Для горизонтальной плоскости y=h:

```
C'_y = 2h - C_y
```

View matrix отражается, но PBR должен получить и отражённую camera position. Текущий исключающий scene path теряет входную позицию и передаёт ноль.

23.4. Вода

Текущий water shader смешивает reflection/refraction почти постоянным коэффициентом. Более полная модель использует Fresnel:

```
F = F0 + (1-F0)(1-N·V)⁵
color = lerp(refraction · absorption, reflection, F)
```

Дальше добавляются depth/thickness absorption, normal waves и foam. Но сначала нужно исправить cameras, clipping и render states.

24. Частицы и взрывы

24.1. CPU simulation

Максимум около 4096 частиц. Интегрирование:

```
v_next = (v + a·dt) · exp(-drag·dt)
x_next = x + v_next·dt
```

Emission accumulator:

```
A += rate·dt
spawn = floor(A)
A -= spawn
```

Это сохраняет дробную часть и делает среднюю скорость emission устойчивой к FPS.

24.2. Billboard

Right/up извлекаются из inverse view. На каждую частицу строится quad из шести CPU-вершин. Это просто и понятно, но масштабируется хуже instancing.

24.3. Случайное распределение

Нормализованная точка равномерного куба **не** равномерна на сфере. Корректный способ:

```
z = 1 - 2u
phi = 2πv
dir = (sqrt(1-z²)cos(phi), z, sqrt(1-z²)sin(phi))
```

Для равномерного объёма шара радиус:

```
r = R · cbrt(u)
```

а не $R \cdot u$.

24.4. Сортировка

Renderer сначала сортирует alpha particles far-to-near, но затем группирует их по texture через map. Между разными текстурами глобальный порядок теряется. Нужен либо stable batching с ограничениями, либо OIT.

24.5. Explosion impulse

```
J = J0 · (1 - d/R)²
```

Текущая модель не проверяет line of sight, поэтому импульс проходит через стену. Для engine-level explosion query нужны physics overlap + ray/occlusion и точка приложения силы.

25. Скелетная анимация

25.1. Local и global bone transforms

Position/scale интерполируются линейно, rotation — SLERP. Local matrix:

```
M_local = S · R · T
M_global = M_local · M_parent
```

Skin matrix в row-vector convention:

```
M_skin = M_offset · M_animatedGlobal
```

25.2. Vertex blending

До четырёх bone influences:

```
p_skinned = Σ weight_i · (p · M_skin_i)
```

Weights нормализуются importer-ом.

25.3. Ограничения

- HLSL buffer рассчитан на 128 bones;
- importer не всегда выдаёт явную ошибку при превышении;
- bone ID ≥128 пропускается;
- shadow/refraction/transparent passes не везде получают current bone buffer;
- в `assimpmodelclass.cpp` остаётся legacy hierarchy path с другим порядком matrices.

Проверка должна одновременно показывать персонажа напрямую, в reflection/refraction и shadow.

Часть V. Bullet и симуляция мира

Bullet отвечает за столкновения и движение физических тел, но не знает игровых понятий «враг», «герметичная дверь» и «пробоина». Движок соединяет Bullet object с Entity и решает, какая система владеет позицией. Физика не должна заменять AI, а AI не должен телепортировать тело сквозь collision shape ради достижения waypoint.

Разбор 21. Тело, форма и мир физики

21.1. Static, dynamic и kinematic

Static body имеет нулевую массу и служит неподвижной геометрией. Dynamic body имеет положительную массу, скорость и откликается на силы и столкновения. Kinematic body перемещается по заданной траектории и влияет на другие тела, но не ведёт себя как обычное динамическое тело. Дверь на анимации может потребовать kinematic подхода; ящик, который отбрасывает взрыв, — dynamic; палуба — static. Выбирай тип по тому, кто является владельцем transform. Если вручную каждый кадр ставить dynamic body в Transform, а Bullet в том же кадре решает столкновения, два владельца будут бороться за позицию.

Collision shape не обязан совпадать с render mesh. Box, sphere и capsule дешевле и устойчивее для большинства предметов. Для сложной статичной палубы возможен треугольный mesh collider, но движущийся сложный mesh требует другого подхода. При создании dynamic body вычисляют локальную инерцию по форме и массе; одна лишь масса без корректной shape/inertia меняет вращение. В debug режиме одновременно показывай render bounds, collision shape и origin тела. Если NPC застрял у визуально широкого проёма, возможно, фактическая капсула шире доступного прохода.

21.2. Шаг и синхронизация

`btDiscreteDynamicsWorld::stepSimulation` продвигает физическое состояние. Для плавной и устойчивой игры важно согласовать его с `dt` и внутренним fixed step. Обновление, которое каждый

кадр вызывает physics с `dt=1/60`, даже когда реально прошло 1/30 с, замедлит мир вдвое. Напротив, один огромный шаг после зависания увеличивает риск пропущенного столкновения и неустойчивости constraint. Выбери лимит substeps и показывай диагностическую метрику «сколько шагов выполнено, сколько времени потеряно».

После физического шага прочитай world transform тела для рендера и gameplay, если body является владельцем движения. Для kinematic персонажа порядок может быть обратным: controller задаёт желаемое перемещение, Bullet разрешает его с учётом контактов, затем фактическая позиция записывается в ECS. Факт «StateSystem хочет идти» не гарантирует «NPC сдвинулся». Логируй желаемую скорость и реальное смещение раздельно. Если они расходятся, переходи к коллизии и NavMesh, а не только к AI.

21.3. Контакты, raycast и фильтры

Broadphase быстро отбрасывает пары, чьи bounds не пересекаются; narrowphase вычисляет конкретный контакт shapes. Collision group/mask решают, какие пары и лучи вообще проверяются. Если луч оружия проходит сквозь дверь, выясни, отсутствует ли collider, неверна ли mask, исключён ли объект фильтром или ray начинается за дверью. Не предполагай сразу ошибку Bullet. Для выстрела физический hit возвращает тело/форму и точку; игровая система преобразует это в Entity, затем определяет материал, урон и событие.

Собственное тело стрелка обычно исключают из raycast. Но исключение не должно случайно удалить из проверки союзника или дверь. Раздели луч зрения, muzzle clearance и игровой damage ray: у них могут быть разные начала и допустимые collision filters. Для взрыва используй overlap query в радиусе, затем рассчитай направление и импульс для каждого подходящего тела. В центре взрыва длина вектора к телу может быть нулевой; выбери безопасное fallback направление или не применяй импульс, а не дели на ноль.

Разбор 22. Персонаж, дверь и разрушение

22.1. Контроллер персонажа и NavMesh

NavMesh отвечает, где можно пройти, а character controller — прошёл ли персонаж фактически. Параметры Recast agent radius/height должны соответствовать Bullet капсуле, иначе путь проходит слишком близко к стене. Если дверь ещё закрывает проём collider, даже корректный путь через открытую графовую связь не позволит пройти. Три диагностических изображения — NavMesh corridor, желаемое направление контроллера и реальная capsule position — быстро указывают, на каком слое ошибка.

При подходе к ступени controller должен согласовать step height и физическую геометрию. Слишком большой step height позволяет подниматься на непредусмотренные препятствия; слишком малый делает даже маленький порог двери непроходимым. Проверяй одну ступень известной высоты и один проход ширины, равной диаметру капсулы плюс запас. Если NPC

пробегает сквозь визуальную стену, проверь collision registration и owner transform. Если стена есть в Bullet, но отсутствует в NavMesh, путь строится через неё и NPC будет застревать.

22.2. Разрушение и обновление нескольких миров

Разрушение корпуса или двери меняет render mesh, collision geometry, NavMesh, room/pressure connections и игровой список задач. Эти изменения нельзя считать одной операцией «удалить треугольники». Пока новый mesh строится асинхронно, должен существовать безопасный промежуточный статус: текущая collision shape ещё действует или уже заменена; NPC получает определённую версию проходимости; атмосфера знает, открыта ли связь. При завершении worker job изменения Bullet и ECS применяются на согласованной границе main thread. Старые shape и body удаляются после того, как мир перестал их использовать.

Построй конечный автомат `Intact → Damaged → GeometryPending → Breached → RepairPending → Repaired`. Для каждого состояния запиши collision, визуальное представление, airflow и доступность ремонта. Так можно обнаружить неочевидную ошибку: трещина нарисована, но воздух ещё не течёт; или пробоина уже закрыта gameplay, но collider ещё открыт. Не требуй идеально атомарного обновления GPU и Bullet в одном такте, но обязательно определи, что видит игрок в промежуточный момент.

Разбор 23. Физические проверки, которые нужны перед игрой

Создай тестовую комнату с палубой, ящиком, дверью и NPC. Проверь: ящик падает на палубу, не проваливается и не «взрывается» от ошибочной инерции; NPC останавливается перед закрытой дверью; после открытия проём физически свободен; raycast от глаза и от дула возвращают разные результаты при низком укрытии; удаление ящика убирает Bullet body; reload сцены не оставляет невидимую копию collider. Каждому случаю соответствует одна ожидаемая геометрия и один владелец transform.

После функциональной проверки меняй частоту рендера и длительность кадров. Измеряй число physics substeps и фактическое положение тела через одинаковое simulation time. Малые численные различия допустимы, но зависание ящика в воздухе при другом FPS говорит о неверном времени или синхронизации. Не расширяй сцену до десятков динамических обломков, пока базовый цикл создания, разрушения и повторной загрузки не стабилен.

Разбор 24. Корабельные сети как связанные правила

Космический симулятор интересен, когда авария меняет не только число HP. Повреждение корпуса влияет на атмосферу, потеря питания закрывает путь и выключает насос или вентилятор, экипаж меняет приоритет, а игрок вынужден принять решение. Чтобы такая цепочка работала, каждому звену нужен владелец данных, единица измерения, порядок обновления и видимый для игрока результат. Отдельная эффектная анимация без этих связей не создаёт симуляцию.

24.1. Энергосеть на конкретном примере

`ShipPowerGridSystem` собирает сети по root Entity. Генераторы добавляют `currentOutputMw`, потребители требуют `requiredMw`, затем сеть распределяет доступное по приоритетам от 0 до 3. Внутри одного уровня приоритета `SupplyEvenly` делит мощность между ещё неудовлетворёнными потребителями и повторяет деление после насыщения тех, кому нужно меньше. Это важно: «сначала по приоритету» не означает «первому Entity всё, остальным ничего». После расчёта у каждого потребителя есть `suppliedMw`, `powerRatio` и `hasPower`; текущий код считает `hasPower=true` при ratio≥0,999.

Пример. Генератор выдаёт 8 МВт. Критические устройства просят 3 и 2 МВт; обычные — 4 и 1 МВт. Первые получают полные 5, остаётся 3. Обычные сначала получают по 1,5, но второму нужен только 1, поэтому остаётся 0,5, которое достаётся первому: он получает 2 МВт из 4. Итого 3+2+2+1=8. Для первого обычного `powerRatio=0,5`, значит `hasPower=false` по текущему контракту, хотя часть мощности выделена. Если нужна частичная работа, например тусклый свет при 50%, потребитель должен явно использовать ratio, а не bool.

Батарея покрывает нехватку через `EmergencyBatterySystem::Discharge`, затем избыток генерации может её заряжать. Не смешивай мощность и энергию: мощность измеряется в МВт, количество энергии — в МВт·с, МДж или MWh после пересчёта. При 2 МВт на протяжении 3 с потрачено 6 МДж. Для каждой сети проверь инвариант: выделенная мощность ≤ доступной, запас батареи не выходит за [0,capacity], устройства разных root не питают друг друга. Отладочный экран должен показать генерацию, спрос, выдачу батареи, brownout и список потребителей по приоритету.

24.2. Дверь как потребитель и проход

Если дверь требует мощность, её логика не должна смотреть только на наличие GeneratorComponent где-то в сцене. Она читает именно свой `PowerConsumerComponent` после обновления сети. Важно выбрать семантику: мощность нужна только для запуска привода или для удержания открытого состояния? Что происходит при отключении во время движения — дверь замирает, закрывается пружиной или остаётся открытой? Ответ влияет на физический collider, NavMesh, атмосферу и бой. Запиши конечный автомат и переходы при потере питания. Без этого разные системы начнут независимо придумывать поведение.

Проверка: с двумя кораблями и разными root отключи генератор первого. Потребители второго должны остаться запитанными. Затем на первом восстанови питание и открой дверь. NPC должен пройти только после фактического освобождения проёма, а не в момент изменения команды `Open`. Если атмосфера начинает смешиваться в момент начала анимации, это тоже выбранное правило, которое надо показать игроку и проверить на сохранении.

Разбор 25. Давление, затопление и пространство

25.1. Текущая модель давления — водная

В исходном `PressureSystem.cpp` внешнее давление вычисляется как `101325 + ρgh`, где `depth=max(0,waterLevel−bodyY)`, ρ=1000 кг/м³ и g=9,81 м/с². Это модель погружения в воду. Она не описывает вакуум вокруг космического корабля. В `FloodingSystem` также есть water volume, pump rate и buoyancy. Если цель игры — именно космос, эти системы можно использовать как отдельный режим планеты/подводного участка, но нельзя выдавать их за готовую симуляцию разгерметизации в вакууме. Для космоса нужна отдельная модель внутренних газов, наружного давления, связи комнат и потоков через отверстия.

Математика водной модели: на глубине 10 м добавка `1000·9,81·10=98 100 Па`; полное давление около 199 425 Па. Код сравнивает его с `maxPressure` корпуса и наносит урон за секунду при превышении. Проверь границы: maxPressure должен быть положительным до деления, здоровье ограничено [0,100] или другим документированным диапазоном, `dt` не отрицателен. При разрыве `FloodingSystem` переводит локальную позицию пробоины в world, определяет глубину и поток. После притока и откачки обновляет `waterVolume += (inflow−outflow)dt` и ограничивает объём комнаты.

Заметь, что в разных местах текущего кода давление и поток могут использовать разные игровые коэффициенты. Не предполагай, что все они уже образуют физически точную модель. Перед балансом выпиши единицы каждого поля и прогони численный опыт: одна пробоина, известная глубина, нулевая откачка, объём комнаты 10 м³, шаг 1 с. Проверь, соответствует ли прирост воды заявленному flowRate. Если нет, исправь соглашение о единицах или назови коэффициент игровым, не скрывая его под физическим именем.

25.2. Проектируем разгерметизацию космического отсека

Для нового космического режима начни с простого состояния комнаты: объём V, количество газа n или масса m, температура T, давление p. Идеальный газ даёт `pV=nRT`, но только если единицы согласованы и температура в кельвинах. Для учебного шага можно считать T постоянной; тогда при уменьшении n давление пропорционально количеству газа. Связь двух комнат или отверстие наружу переносит газ от высокого давления к низкому. Точную газодинамику с критическим течением добавляй только если она нужна геймплею: для первой версии ограниченная функция потока с сохранением массы и ясными коэффициентами лучше сложной формулы без проверки.

Предположим две равные комнаты по 100 м³ при одинаковой температуре: A имеет условно 100 единиц газа, B — 50. После открытия двери и обмена 10 единицами A станет 90, B — 60; сумма 150 сохраняется. Если B соединена с вакуумом, потерянная масса должна появиться в отдельном счётчике `ventedToSpace`, иначе инвариант не проверить. Давление после шага выводи из нового количества газа, не меняй массу и давление независимо. Для NPC атмосферная опасность должна

быть игровым входом: доступен ли скафандр, сколько времени можно работать, какую дверь следует закрыть прежде ремонта. Так математика становится причиной решений.

25.3. Порядок обновления среды

Выбери единый порядок: команды двери и её питание → фактическое состояние проходов → обмен воздухом/водой → урон и состояние оборудования → уведомление экипажа → выбор работы. Если NPC принимает решение до обновления среды, допустима задержка на один кадр, но UI и debug должны позволить её объяснить. При разных частотах обновления, например среда 10 Гц и AI 5 Гц, сохраняй метки времени и не применяй один и тот же поток дважды. При загрузке сцены восстанови связи комнат до первого тика обмена, иначе один кадр может мгновенно переместить газ через неинициализированную дверь.

Разбор 26. Авария, ремонт и наблюдаемый результат

Пробоина начинается как событие повреждения с мировыми координатами и связью с комнатой или корпусным сегментом. Создай `RepairableComponent`, подходящий WorkZone и визуальный след; затем сообщи координатору экипажа. Ремонтник выбирается по severity, расстоянию, роли, skill и доступности. Работа наращивает progress только в пределах рабочей зоны, при достаточном ресурсе и допустимой среде. Завершение закрывает leak, удаляет или переводит цель в Repaired, обновляет геометрию и collider, затем публикует событие для HUD/звука. Если одна из этих операций асинхронная, введи состояние `RepairPending`, чтобы не показать герметичность раньше физического завершения.

На одном тесте измерь цепочку: пробоина появляется на отметке t=0; давление комнаты начинает падать; инженер получает claim и route; дверь открывается; инженер входит в WorkZone; progress растёт с ожидаемой скоростью; по окончании утечка прекращается и давление стабилизируется. Каждое звено должно иметь время, Entity и причину перехода. Если пробоина исчезла визуально, но давление продолжает падать, ищи не «баг NPC», а несогласованный статус утечки. Если pressure исправилось, но Collider остался отверстием, ищи этап rebuild геометрии. Этот метод проверки используется дальше для любой новой механики корабля.

26. Bullet Physics и синхронизация

26.1. World settings

```
gravity = (0, -9.81, 0) m/s²
fixed step = 1/60 s
solver iterations = 50
split impulse = enabled
ERP ≈ 0.2
```

26.2. Типы тел

Тип	Кто задаёт transform
static	создаётся один раз; мир не двигает
kinematic	ECS/editor задаёт каждый кадр, Bullet только сталкивает
dynamic	Bullet — источник истины; ECS получает результат

Нельзя одновременно каждый кадр телепортировать dynamic Transform из ECS и ожидать честную физику. В проекте есть детектор внешнего изменения: editor teleport обновляет Bullet и сбрасывает velocities.

26.3. Collision shapes

Поддерживаются box, sphere, cylinder, capsule, plane, convex hull, triangle/GImpact mesh, compound, StairRamp и объявленный VHACD.

Правила:

- static concave: triangle mesh допустим;
- dynamic concave: GImpact дорог, convex decomposition предпочтительнее;
- capsule: cylinder height не может быть отрицательной;
- sphere при non-uniform scale либо использует max scale, либо перестаёт быть sphere;
- compound child должен учитывать local center/rotation.

> **ОГРАНИЧЕНИЕ.** VHACD присутствует в enum/editor, но основной path не создаёт полноценную shape во всех ветках. Функция выглядит доступной, хотя физически может отсутствовать.

26.4. Настройки компонента

`RigidBodyComponent` хранит friction, damping, CCD и angular factor, однако create path на срезе часть значений задаёт константами. UI-поле, которое не влияет на runtime, опаснее отсутствующего поля: разработчик верит ложной настройке.

26.5. Interior collider

Код создаёт `interiorBody`, но аудит не нашёл регистрацию `world->addRigidBody(interiorBody, ...)`. Выделенный объект без регистрации не участвует в collision detection.

26.6. Ownership

Безопасный teardown:

```
world.removeConstraint/removeAction/removeRigidBody
→ delete controller/constraint/body motion state
```

```
→ delete body
→ delete shapes/triangle meshes
```

Прямой reset Registry опасен для character ghost/action и constraints, потому что их destructors не все снимают registration.

27. Character controller

27.1. Геометрия

Персонаж — вертикальная capsule. Полная высота:

```
H_total = H_cylinder + 2R
```

Центр capsule смещён относительно feet Transform. Поэтому попытка «починить провал» случайным подъёмом всей capsule ломает ступеньки, camera anchor и reach.

27.2. Ground test

Поверхность walkable, если:

```
dot(normal, up) >= cos(maxSlope)
```

27.3. Auto-step

Последовательность sweep:

```
up → forward → down → validate slope
```

Это хорошая engine-функция: один controller используется игроком и NPC, поэтому они сталкиваются с миром одинаково.

27.4. Stabilization

Код содержит post-step ground correction и dead zone, появившиеся из реального бага дрожания. Важный урок: прежде чем добавлять smoothing, определить источник:

- conflict ECS/Bullet ownership;
- contact normal noise;
- capsule center offset;
- camera reading pre-physics pose;
- animation root motion.

27.5. Прыжок

Сейчас `wantJump` формируется из `GameInputFrame::Down(GameAction::Jump)`. После приземления удержанная кнопка может запустить новый прыжок. Если нужен прыжок только по новому нажатию, замени это место на `Pressed` и проверь поведение при паузе и F2.

28. Давление, затопление и плавучесть

28.1. Гидростатическое давление

```
P = P0 + ρgh
```

Где:

- `ρ ≈ 1000 kg/m³` для воды;
- `g = 9.81 m/s²`;
- `h` — глубина в метрах;
- результат — Pa.

`PressureSystem` следует этой модели.

28.2. Скорость через пробоину

Из Bernoulli/Torricelli:

```
ΔP = ρgh
v = sqrt(2ΔP/ρ) = sqrt(2gh)
Q = Cd · A · v
```

Если `size` — радиус:

```
A = πr²
```

28.3. Подтверждённая ошибка единиц

`FloodingSystem` вычисляет:

```
pressure = depth · 9.81
velocity = sqrt(2 · pressure / 1000)
```

В первой строке отсутствует `ρ`, а во второй оно делится ещё раз. Полученная скорость примерно в `sqrt(1000) ≈ 31.6` раза меньше Torricelli.

Пример при `h=10m`:

```
правильно: sqrt(2·9.81·10) ≈ 14.0 m/s
текущий код: sqrt(2·9.81·10/1000) ≈ 0.443 m/s
```

28.4. Water volume

```
V_next = clamp(V + (Q_in - Q_pump)dt, 0, V_room)
waterLevel = (V / V_room) · roomHeight
```

Нужен guard `V_room > epsilon`, иначе division by zero.

28.5. Архимедова сила

```
F_b = ρwater · g · V_displaced
```

Текущая модель:

```
submersion = clamp((waterLevel - bottom)/height, 0, 1)
```

```
F_b = ρgV · submersion · multiplier
F_net_y = F_b - ρwater·V_inside·g
```

28.6. Двойной учёт воды

`FloodingSystem` уменьшает `buoy.volume` на внутренний объём воды. `PhysicsSystem` одновременно оставляет отдельный вес внутренней воды. Физически затопление должно либо:

- уменьшать эффективную герметичную displacement volume;
- либо добавлять массу воды;
- либо использовать подробно моделируемый flooded volume,

но не применять упрощённые эффекты дважды без вывода. На срезе потеря плавучести и дополнительный вес суммируются.

28.7. Центр плавучести

Сила сейчас прикладывается в центре массы, поэтому не создаёт восстанавливающий момент. Более выразительная модель использует несколько sample points или вычисленный center of buoyancy:

```
τ = (centerBuoyancy - centerMass) × F_b
```

Асимметричное затопление тогда создаёт крен.

29. Комнаты и атмосфера

29.1. Текущая игровая модель

`RoomComponent` хранит normalized oxygen/pressure, temperature, bounds и связи, но не массу газа и физический volume. Поэтому это **игровая диффузионная модель**, не CFD и не thermodynamics solver.

29.2. Парный перенос

Для двух комнат:

```
transfer = (A - B) · rate · doorFlow · dt
```

Transfer ограничивается половиной разности, а изменения сначала собираются в список и применяются затем. Это даёт два плюса:

- A→B и B→A не зависят от порядка обхода;
- закрытая пара сохраняет сумму величины до clamp/источников.

29.3. Leak

Пробоина использует fraction loss:

```
lost = clamp(leak · dt, 0, 0.95)
pressure *= (1 - lost)
oxygen   *= (1 - lost)
```

Это понятно для gameplay, но не учитывает physical volume и нелинейный массовый расход.

29.4. Breathing и generators

Breathing вычитает normalized oxygen на entity, а generator равномерно раздаёт output активным комнатам. Комната в 10 м³ и 1000 м³ реагирует одинаково. Если понадобится физичность, сначала добавить `volumeM3` и хранить moles/mass, затем вычислять fractions.

29.5. Conservation test

Для двух закрытых комнат без source/sink:

```
sumBefore = amountA + amountB
simulate 10 minutes
abs(sumAfter - sumBefore) / sumBefore <= 0.001
```

Если используются normalized fractions, test должен весить комнаты по volume.

30. Энергетика

30.1. Power grid

Каждый ship root имеет свою сеть. Генераторы дают MW, consumers требуют MW, priority определяет порядок. Внутри одного priority используется water-filling: available power делится поровну, полностью насыщенные consumers удаляются, остаток перераспределяется.

30.2. Battery units

Power — MW, energy — MWh:

```
ΔE_MWh = P_MW · dt_seconds / 3600
```

Код `EmergencyBatterySystem` использует именно это преобразование для charge/discharge. Это пример хорошего явного unit contract.

30.3. Brownout

```
powerRatio = suppliedMW / requiredMW
hasPower = powerRatio >= 0.999
brownout = totalSupplied < totalDemand
```

Gameplay subsystem не должен самостоятельно забирать мощность «мимо» grid; он читает `powerRatio`.

30.4. Engine-level развитие

- stable consumer IDs;
- event на переход powered↔brownout, а не каждый кадр;
- граф dependency/isolated grids;
- telemetry history;
- deterministic allocation независимо от unordered order.

31. Damage, cutting и repair

31.1. Damage flow

```
Weapon/raycast
→ DamageEvent
→ DamageSystem
→ health / hull panel / decals / particles / world event
→ breach or destroyed state
```

31.2. Runtime cutting

F6 строит picking ray и вызывает `MeshCuttingTest::CutByRayAsync`. Старое руководство, где указан синхронный `CutByRay`, устарело.

Концептуальные этапы cutting:

1. ray/triangle intersection;
2. выбор области/плоскости разреза;
3. clipping polygons;
4. interpolation новых attributes;
5. triangulation;
6. rebuild CPU/GPU mesh;
7. rebuild physics shape;
8. запись operation ID/history;
9. создание breach/damage representation.

31.3. Интерполяция атрибутов

Для точки на ребре:

```
p(t) = a + t(b-a)
uv(t) = uvA + t(uvB-uvA)
n(t) = normalize(nA + t(nB-nA))
```

Если используется barycentric interpolation внутри triangle:

```
p = uA + vB + wC,  u+v+w=1
```

31.4. Асинхронность

В текущем коде async cutting уже разделён на две фазы. Main thread выбирает цель и снимает CPU-snapshot вершин и индексов. Worker выполняет clipping и triangulation только на этих копиях. Completion возвращается на main thread, проверяет `ActiveComponent` и то, что entity всё ещё ссылается на ту же model, и только после этого меняет D3D buffers, history и Bullet collider.

Async repair добавляет более сильную optimistic validation: worker запоминает `expectedRevision`, а main-thread completion до первой мутации проверяет revision и наличие repair operation у всех целей. Это защищает от применения устаревшей геометрии.

Scene/entity generation token пока не стал общим engine-контрактом. Локальные проверки model pointer и revision закрывают текущие cutting/repair paths, но новая async-система должна либо получить generation token, либо доказать эквивалентную проверку lifetime. Общая модель worker/completion разобрана в главе 35.

31.5. Repair invariants

- одна operation repair не применяется дважды;
- удалённая entity не получает completion;
- GPU buffer меняется только main thread;
- collider перестраивается после mesh;
- save во время repair имеет определённое состояние;
- visual crack, breach simulation и geometry history согласованы.

Часть VI. NPC, данные и инструменты

NPC кажется одним персонажем, но в коде его поведение получается из последовательности систем. Чтобы он чинил, атаковал и помогал игроку предсказуемо, раздели четыре вопроса. Что NPC знает о мире? Какую цель он выбирает? Как он превращает решение в движение или выстрел? Как игра показывает результат? Ошибка любого этапа выглядит одинаково — персонаж «тупит», — поэтому обучение строится вокруг явных контрактов и диагностики.

Разбор 27. Собираем NPC как сущность

27.1. Минимальный состав зависит от роли

Видимый humanoid обычно имеет Transform, модель, анимацию и состояние активности. Движущийся NPC дополнительно нуждается в данных движения, controller/capsule и навигационном доступе. Дружественный член экипажа имеет роль, skill и право получать задания. Боевой NPC требует фракцию, combat data, здоровье и оружие либо melee правило. Один

NPCComponent не создаёт автоматически все остальные связи. Поэтому для каждой фабрики задай профиль: CrewEngineer, EnemyRifleman, NeutralCivilian. Профиль указывает обязательные компоненты, допустимые значения и fallbacks, а загрузка сцены сообщает о неполном объекте.

Проверь одного NPC без визуальной модели: если state и movement работают, ошибка изображения отделена от AI. Затем добавь модель без анимации, потом clips, потом оружейный socket. Такой порядок предотвращает распространённую ловушку: красивый персонаж, у которого отсутствует collider или Faction. В Editor сделай read-only diagnostic panel с паспортом NPC. Красная строка «нет CharacterController» полезнее, чем десяток параметров, которые пользователь может менять, но которые никто не читает.

27.2. Фракция как игровое отношение

Разные сущности могут быть близко, но это не делает их врагами. FactionComponent и правило AreEnemies задают право выбирать цель; здоровье и active state затем отсекают недопустимые цели. В текущем коде neutral не должен автоматически превращаться в противника только потому, что его ID отличается. Старое поле isHostile встречается для совместимости и разграничения utility поведения, но новую боевую логику лучше строить через Faction. При создании врага проверь оба слоя: не оказался ли он также членом crew utility и не получает ли ремонтную работу во время боя.

Отношения фракций могут позже стать сложнее простого «разные ненейтральные — враги»: репутация, временный союз, квестовая нейтральность. В таком случае не рассыпай сравнение enum по десяткам систем. Сохрани одну функцию разрешения отношений и тестовую матрицу пар. UI должен показывать фактический результат отношения, а не только номер фракции, иначе игрок и разработчик не поймут, почему охранник стреляет в союзника.

Разбор 28. Решение NPC — последовательность, а не один enum

28.1. Наблюдение и память

Вражеский NPC может лично видеть цель, услышать шум, получить сообщение союзника или помнить последнее место контакта. Это разные свидетельства. Личное зрение после FOV и LOS даёт точную текущую точку с короткой давностью. Слух даёт область и меньшую точность. Сообщение от товарища содержит его источник и время; оно не должно превращаться в личную видимость. Последнее известное положение перестаёт обновляться, как только цель скрылась. Если NPC продолжает поворачивать голову к невидимому игроку по его текущему Transform, у него фактически есть всеведение, независимо от красивой анимации поиска.

Для каждого свидетельства храни источник, время, позицию/область и срок жизни. Решение может оценивать доверие к сведениям. Например, увиденное 0,2 с назад важнее звука 4 с назад, но взрыв прямо за дверью может изменить маршрут. Не обязательно вводить сложную вероятностную модель на первом этапе; явные правила переходов Seen → Lost → Search → GiveUp

уже дают честное поведение. На debug карте рисуй разные цвета для личного луча, lastKnown point и слуховой области.

28.2. Цель, действие и исполнение

`NpcCombatSystem` выбирает допустимого врага и боевое состояние; `NpcCoverSystem` может выбрать укрытие; `StateSystem` исполняет итоговое состояние через NavMesh и movement; `NpcRangedCombatSystem` позже разрешает фактический выстрел. Для экипажа `CrewTaskCoordinator` выдаёт работу, `NpcUtilitySystem` выбирает её как действие, а `StateSystem` доводит NPC до WorkZone и наращивает ремонт. Эти системы не должны по очереди бесконтрольно перезаписывать один `brain.state`. Задай приоритеты: смерть/disabled, ручной приказ, немедленная опасность, бой, критическая авария, обычная работа, idle. У каждого перехода есть причина и правило отмены старого назначения.

Проведи таблицу переходов для одного NPC: `Idle → MoveToRepair` при принятой задаче, `MoveToRepair → WaitingForDoor` при закрытом проходе, `WaitingForDoor → MoveToRepair` после открытия, `MoveToRepair → Repairing` в WorkZone, `Repairing → Idle` после завершения. Для каждого перехода подпиши владельца записи и действие очистки: сброс пути, освобождение claim, остановка анимации, отмена таймера. Если у состояния нет выхода при ошибке пути, NPC рано или поздно зависнет навсегда.

28.3. Команда игрока и автоматическое поведение

Ручная команда не должна быть ещё одной слабой «целью» в utility score, если интерфейс обещает приказ. При принятии команды запиши источник Manual, цель, момент и условия отмены. Coordinator освобождает старый automatic claim; маршрут к прежней работе очищается. Если приказ невыполним — цель уничтожена, нет ресурса, проход закрыт навсегда — UI возвращает причину. После `ResumeAutomatic` NPC не должен продолжать старое manual состояние из-за сохранённого target или waypoint. Проверяй эту цепочку на одном инженере и двух пробоинах до появления сложного командного меню.

Разбор 29. Маршрут, дверь и фактическое движение

Recast строит проходимую поверхность для агента заданного радиуса и высоты; Detour ищет путь. `NpcNavMeshSystem` выдаёт маршрут, `StateSystem` выбирает следующий waypoint, `CharacterControllerSystem` пытается переместить Bullet капсулу. Слова «путь есть» и «персонаж пришёл» описывают разные уровни. Динамический ящик может появиться после построения NavMesh, створка может ещё двигаться, capsule может быть шире агента. Поэтому застревание измеряется фактической позицией за интервал, а не числом waypoints.

У двери есть минимум три состояния, значимые для AI: связь есть и проход свободен; связь есть, но нужно дождаться открытия; связь недоступна из-за питания или блокировки. При последнем случае нельзя бесконечно вызывать `TryOpen` каждый кадр. Нужны backoff, альтернативный

маршрут или отказ от задачи. Если дверь открылась, но физический collider не освободил проход, NPC правильно продолжает ждать по физическому факту, даже если UI уже показывает `Open`. Измеряй время ожидания, попытки и последнюю причину отказа.

Практика. Создай коридор с одной дверью, второй обходной дорогой и физическим ящиком. Проверь четыре сценария: дверь открыта, закрыта с питанием, закрыта без питания, открыта, но путь перекрыт ящиком. Для каждого заранее выпиши ожидаемые значения `hasPath`, `doorStatus`, `desiredVelocity`, `actualDisplacement`. Только после такой таблицы меняй алгоритм перепланирования. Иначе можно случайно «исправить» AI, разрешив капсуле проходить сквозь дверь.

32. NPC AI pipeline

Фактический порядок:

```
NpcUtilitySystem
→ CrewTaskCoordinator
→ StateSystem
→ CharacterFacingSystem
→ CharacterControllerSystem
→ AnimationController
```

32.1. Utility AI

Idle score примерно `0.1`. Repair breach score сочетает severity и distance; equipment repair — damage, importance и distance. Utility выбирает не только пробоины, как говорит старый NPC guide, но и повреждённое оборудование.

Общая нормализация:

```
score = Σ wi · normalizedCriterion_i
Σ wi = 1
```

Distance utility, например:

```
distanceScore = 1 - clamp(distance / detectionRange, 0, 1)
```

32.2. Hysteresis

Без hysteresis две близкие задачи меняются местами каждый кадр:

```
switch only if newScore > currentScore + threshold
```

Также полезны minimum commitment time и cooldown.

32.3. Manual command

Debug/manual command имеет приоритет. Utility продолжает считать scores для UI, но не перехватывает управление. Это хорошее разделение decision telemetry и authority.

32.4. Чего пока нет как законченной системы

Старая документация описывает fatigue, fear, перенос инструментов и общий World Task Board так, будто они реализованы. На текущем срезе это всё ещё целевая модель, но reservation ремонта уже появился.

`CrewTaskCoordinator` собирает активные `RepairableComponent`, сортирует NPC по `Entity`, исключает hostile/off-duty/manual-command персонажей и закрепляет цель через `CrewTaskClaimComponent`. `CrewMemberComponent` хранит роль, навык, текущую работу и счётчик завершённых работ. Это уже предотвращает ситуацию, когда два дружественных NPC одновременно выбирают одну пробоину или одно оборудование.

Важно точно различать два уровня:

- `NpcUtilitySystem` оценивает индивидуальную полезность ремонта и оставляет диагностический score;
- `CrewTaskCoordinator` выполняет межагентный арбитраж и назначает уникальные цели;
- общего task board для пожаров, лечения, переноски ресурсов и охраны пока нет;
- claim существует только для ремонтируемой цели и пересоздаётся из состояния мира, а не является полноценной долговременной задачей.

Поэтому новый тип коллективной работы следует добавлять либо расширением координатора, либо будущим единым task board. Нельзя создавать второй скрытый `unordered_map<Entity, Entity>` рядом: тогда UI, save/load и другие AI-системы перестанут видеть единственный источник истины.

33. StateSystem и исполнение

Utility отвечает «что делать», StateSystem — «как выполнить».

Типичный repair state:

```
Acquire target
→ build room route
→ build/follow NavMesh path
→ request/open door
→ approach work zone
→ execute timed repair
→ validate target
→ success/fail/cancel
```

State должен иметь:

- enter/exit;
- timeout;
- reason code;
- target validation;
- cancellation cleanup;
- debug-readable progress.

Файл `StateSystem.h` вырос примерно до 1845 строк. Разделение по state handlers уменьшит coupling и compile time.

34. Recast/Detour NavMesh

34.1. Параметры build

```
cellSize = 0.20 m
cellHeight = 0.10 m
agentHeight = 1.8 m
agentRadius = 0.42 m
agentClimb = 0.30 m
maxSlope = 45°
```

Они должны соответствовать capsule controller. Если radius NavMesh меньше физической capsule, найденный путь может не пройти.

34.2. Pipeline

```
input triangles
→ rasterized heightfield
→ compact heightfield
→ erode by agent radius
→ regions
→ contours
→ polygon mesh
→ detail mesh
→ Detour nav data
```

34.3. Query

```
findNearestPoly(start/target)
→ findPath (A*)
→ findStraightPath (funnel-like corners)
```

Лимиты на срезе: около 256 polygon refs и 64 straight points. Длинный путь может быть обрезан.

34.4. Ограничения

- obstacle footprint строится из world AABB, а не oriented geometry;
- doors исключаются из baked geometry;
- нет tile cache/dynamic carving;
- нет явных off-mesh door links;
- fallback может построить только пол без препятствий;
- локальное avoidance выбирает ближайший dynamic AABB, но это не RVO crowd.

34.5. Debug colors

NavMesh/path/obstacles рисуются one-frame debug lines. Цвет fallback-сетки должен отличаться от полноценной, чтобы «путь сквозь мебель» не выглядел загадочным AI-багом.

35. JobSystem и асинхронные конвейеры

> ФАКТ. Эта глава актуализирована после engine-maturity коммита 9913264. Асинхронность в этом движке — не «запустить всё в другом потоке», а явно разделить CPU-подготовку и применение результата к живому engine state.

35.1. Thread affinity: почему нужны две очереди

Worker безопасен только тогда, когда работает с независимыми данными. Сам факт переноса функции в другой поток не делает ECS, Direct3D или Bullet thread-safe.

Работа	Поток	Почему
чтение/parse файла, clipping, frustum test, CPU mesh processing	worker	работа идёт с immutable snapshot или локальным result
публикация CPU-result в completion queue	worker	передаёт владение результатом, но не меняет engine state
ECS structural change	main	component pools и entity lifetime принадлежат main thread
D3D immediate context/GPU buffer creation	main	immediate context и render state имеют одного владельца
Bullet world mutation	physics/main owner thread	world хранит внутренние ссылки на bodies, actions и constraints
ImGui	main	UI живёт в main-frame lifecycle

Отсюда следует главный контракт:

```
main snapshot → worker CPU result → main-thread validation → ECS/D3D/Bullet apply
```

35.2. Владение и lifecycle JobSystem

`ApplicationClass` хранит `JobSystem` как value member и запускает его с четырьмя workers. Число «4» — это конфигурация приложения, а не ограничение класса: `Initialize(workerCount)` принимает любое положительное число.

Важные гарантии lifecycle:

1. Новый или уже остановленный `JobSystem` имеет `stop=true`, поэтому отвергает job, а не кладёт его в очередь без workers.
2. `Initialize(0)` и отрицательное число workers отвергаются.
3. Повторный валидный `Initialize` сначала выполняет `Shutdown`, затем сбрасывает stop/active count и создаёт новые потоки.
4. Если создание одного из потоков бросит исключение, уже созданные workers останавливаются и join-ятся, а `Initialize` возвращает `false`.
5. Деструктор вызывает idempotent `Shutdown`; копирование и перемещение запрещены.

Концептуальная машина состояний:

```
Stopped/rejecting
      | Initialize(N > 0)
      ▼
Running/accepting — Shutdown ──► Draining queued jobs — join ──► Stopped/rejecting
```

35.3. Как worker извлекает job

Worker ждёт condition variable по предикату `stop || !jobs.empty()`. Очередь `jobs` и флаг `stop` защищены одним `queueMutex`. Job извлекается по FIFO, счётчик `activeJobs` увеличивается, после чего мьютекс отпускается. Тяжёлая функция выполняется вне lock, иначе все workers фактически стали бы одним потоком.

```
lock → wait → pop → activeJobs++ → unlock
                        │
                        └→ execute job
                                │
lock ← notify Wait if idle ← activeJobs-- ↵
```

`Shutdown` ставит `stop=true` и будит workers. Worker завершается только при `stop && jobs.empty()`: уже принятые CPU-jobs дренируются, но новые `SubmitJob` отвергает.

35.4. Wait не равен completion fence

В движке две независимые очереди:

- `SubmitJob` кладёт CPU-работу в worker queue;
- `SubmitMainThread` кладёт completion в main-thread queue.

`Wait()` ждёт только `jobs.empty() && activeJobs == 0`. Он не запускает main-thread callbacks. Поэтому «`Wait` вернулся» означает «CPU-фаза закончена», но не «результат уже применён к

сцене». Вызывать `Wait()` из worker job нельзя: текущий job сам учтён в `activeJobs`, поэтому будет ждать собственного завершения.

`ExecuteMainThreadJobs()` под lock меняет shared queue на локальную, затем выполняет callbacks без удержания mutex. Callback, добавленный во время этого прохода, останется до следующего вызова. Это убирает re-entrancy и долгое удержание `mainThreadMutex`.

35.5. Фактические async-конвейеры

Потребитель	Worker-фаза	Main-thread применение	Защита от устаревшего результата
`AssetLoadSystem`	Assimp/file parse и CPU mesh	completion ставит `CpuLoaded`, `Update()` создаёт GPU buffers	tracker и state; completion захватывает raw `this`, поэтому важен shutdown-порядок
async ship import	Assimp читает GLB, готовит nodes, collision proxies и panel batches	ECS/Bullet/GPU создаются фазами с бюджетом 4 ms на кадр	state живёт в `shared_ptr`; `Clear()` отцепляет старый state от новой сцены
`RenderCandidateSystem`	frustum test по snapshot transform/bounds	completion публикует cache visible entities	cache используется только при близких view/projection; иначе fail-open считает entity кандидатом
async cutting	clipping копии CPU mesh	проверка entity/model, GPU apply, history и collider rebuild	active entity + тот же model pointer
async repair	копирование baseline и replay оставшихся cuts	валидация всех целей, GPU apply и collider rebuild	`expectedRevision`, operation ID/state и per-model pending lock

`AssetLoadSystem::RequestMeshLoad` реализован, но в текущем editor spawn path флаг `useAsyncMesh` выключен. Поэтому generic mesh loader — готовый, но не основной runtime path. Активные async-пути сегодня — ship import, render candidates, cutting и repair.

35.6. Порядок внутри кадра и shutdown

В начале `ApplicationClass::Frame` порядок такой:

1. `ExecuteMainThreadJobs()` применяет готовые completions.
2. `AssetLoadSystem::Update(device)` создаёт GPU buffers для состояния `CpuLoaded`.
3. `UpdateAsyncShipImport(..., 4.0f)` тратит ограниченный main-thread budget на текущую фазу импорта.

Порядок shutdown ещё важнее:

```
JobSystem.Shutdown()          // reject new jobs, drain accepted jobs, join workers
ExecuteMainThreadJobs()       // apply every completion produced before join
SceneManager.Clear()          // release ECS/Bullet runtime owners
PhysicsWorld.Shutdown()
AssetLoadSystem.Shutdown()
```

Так raw `this` в asset completion не становится dangling в штатном `ApplicationClass` lifecycle: workers уже join-нуты, completions выполнены, и только затем уничтожаются scene/assets. Вынос `AssetLoadSystem` в другой lifetime потребует новой гарантии.

35.7. Проверяемые контракты

`tests/CoreContractsTests.cpp` теперь доказывает базовый lifecycle `JobSystem`:

- job отвергается до `Initialize`;
- неположительное число workers отвергается;
- запущенные workers выполняют все принятые jobs;
- после `Shutdown` новая работа отвергается;
- повторный `Initialize` снова принимает и выполняет job.

Контракты запускаются для Debug и Release x64 через:

```
.\scripts\verify_engine.ps1 -Configuration All
```

Это уже автоматический test target, но не полное async test suite и не внешний CI.

35.8. Текущие ограничения

1. Исключение из worker job не перехватывается `JobSystem`: обычно это завершит process, а `activeJobs--` не будет выполнен.
2. Исключение из main-thread completion также не изолировано и выйдет в `ApplicationClass::Frame` или `Shutdown`.
3. Нет cancellation token: принятый job при shutdown дренируется, а не отменяется.
4. Нет priority queue, dependency graph, task handle/future и общего канала ошибок.
5. Приложение всегда запускает четыре workers и не учитывает `hardware_concurrency` и нагрузку на render thread.
6. `SubmitMainThread` не проверяет stop-state и не возвращает status; вне штатного application lifecycle completion может остаться без consumer.
7. Нет универсального scene/entity generation token. Cutting и repair имеют локальную валидацию, ship import изолирует state через `shared_ptr`, но новый async-consumer не получает эту защиту автоматически.
8. Static cache `RenderCandidateSystem` не имеет scene generation и не сбрасывается в `SceneManager::Clear`; матричный fail-open снижает риск невидимого объекта, но не заменяет lifecycle token.

Целевая форма completion для новых систем:

```
RequestId + sceneGeneration + entityGeneration/revision + immutable CPU result + error
```

36. Scene save, runtime save и Entity remap

36.1. Текущее scene save

Schema `version=1`, объекты собираются вручную из нескольких pools. `SerializeObject` покрывает много компонентов, затем `LoadScene` создаёт новые Entity и строит map:

```
oldEntityId → newEntityId
```

После создания выполняется `RemapEntityReferences`.

36.2. Подтверждённые пробелы

- loader читает `world`, saver его не пишет;
- `WorldAreaComponent` не сохраняется;
- `SerializeMaterial` — пустой stub;
- sprites не входят в полноценный round trip;
- одна entity может попасть в objects через несколько component pools;
- JSON exception не перехватывается;
- текущая сцена сбрасывается до полной проверки новой;
- прямой Registry reset обходит специальный Bullet teardown;
- version не мигрируется.

36.3. Транзакционная загрузка

```
read bytes
→ parse JSON
→ validate schema/version
→ build temporary scene/registry
→ validate references/resources
→ atomically swap current scene
→ destroy old scene safely
```

36.4. Атомарная запись

```
write save.tmp
→ flush + close
→ rename/replace save.json atomically
```

Повреждение `save.tmp` не уничтожает предыдущий рабочий save.

36.5. Scene и runtime save

Scene asset хранит authoring data. Runtime save дополнительно хранит:

- world tick/time;
- velocities;
- health/damage;
- atmosphere/water/power;
- AI tasks/memory/reservations;
- in-progress operations;
- random seeds.

Смешивать два формата без явной границы опасно.

37. Editor, picking и диагностика

37.1. Редактор

`EditorUI.cpp` — крупнейший hotspot. Он показывает components, создаёт requests, управляет inspectors и debug panels. Хороший принцип уже используется: тяжёлая операция из UI оформляется request и исполняется application/scene layer, а не внутри ImGui draw.

37.2. Gizmo и physics

Editor меняет Transform. Если entity dynamic, physics layer должен распознать teleport, обновить body/AABB, сбросить или сохранить velocities по явной политике. Изменение scale требует rebuild collision shape.

37.3. Диагностический минимум объекта

```
Entity + generation/alive
Name + Active
Transform + units
world bounds
material/shader/textures
Bullet body/shape/type/mass/velocity
room/ship membership
AI state/target/path/reason
save support yes/no
last system that modified critical value
```

37.4. Debug UI не должен быть логикой

UI может вызвать command, но правило симуляции должно жить в system. Иначе headless test и NPC не смогут использовать ту же механику.

Часть VII. Срез качества и диагностика

Почти готовая игра должна объяснять свои сбои. Когда NPC стоит, дверь не открывается или пробоина не закрывается, разработчик должен увидеть место разрыва цепочки данных за

минуты, а не менять параметры наугад. Диагностика является частью архитектуры: после релиза на машине игрока Inspector не будет, поэтому основные причины отказа надо уметь записать в лог или показать ограниченным debug overlay.

Разбор 35. От симптома к нарушенному контракту

35.1. Найди первую неверную границу

Представь, что инженер не ремонтирует пробоину. Возможные границы: объект не загружен в Registry; нет `RepairableComponent`; coordinator не видит исполнителя; claim занят; utility AI выбрал другое действие; NavMesh не дал путь; дверь не открылась; controller не сдвинул капсулу; WorkZone недоступна; progress не растёт; `RepairSystem` не завершает работу; geometry job не применил результат. Проверяй в этом порядке и записывай первое место, где ожидание расходится с фактом. Если progress уже достиг порога, дальнейшая настройка AI score бесполезна. Если target никогда не назначен, анимация ремонта ни при чём.

Сделай диагностический снимок Entity: живость, компоненты, role, active, faction, state, command source, target, claim, route status, desired velocity, actual displacement, current animation и last error. Снимок одной строки полезен при репродукции; поток лога каждого кадра быстро станет нечитаемым. Пиши сообщение при переходе состояния или изменении причины, при необходимости добавляй ограниченное семплирование раз в секунду.

35.2. Различай ошибку данных, порядка и модели

Ошибка данных: у NPC нет оружия или у двери нет collider. Ошибка порядка: питание посчитано после решения двери, а NPC ждёт старого состояния; анимация обновлена после muzzle socket. Ошибка модели: формула ремонта даёт слишком быстрый прогресс или enemy aim использует невидимую текущую позицию игрока. Эти категории требуют разных исправлений. Добавление недостающего компонента не решит неправильную формулу; перестановка систем не поможет, если объект изначально не был создан. В bug report укажи первую неверную границу, owner данных и ожидаемый момент обновления.

35.3. Debug рисунок имеет единицы и масштаб

Линии NavMesh, raycast зрения, muzzle ray, capsule, WorkZone и cover points должны иметь легенду и ограничение времени жизни. Цвет не должен быть единственным носителем смысла: подписи `LOS`, `Fire`, `Path`, `Blocked` помогают при разных фонах. Если рисуешь длину 10, подпиши «10 м». Debug line из локального socket в мировую цель выдаёт ложный след; преобразуй оба конца в world. Определи, какие overlay доступны только в Editor, какие в Development build и какие безопасно оставить в Release через выключенный по умолчанию режим диагностики.

Разбор 36. Тестирование без копирования реализации

36.1. Проверяй поведение и инварианты

Тест, который буквально повторяет ветвление исходной функции, обнаружит мало ошибок: он согласится с той же ошибочной идеей. Лучше задать независимые инварианты. После завершения ремонта leak прекращается и claim освобождён. Сумма выделенной мощности не превышает доступной. У мёртвого NPC нет активного выстрела. NPC без личной видимости не получает точные координаты скрытого игрока. После scene reload число Bullet тел соответствует живым объектам. Эти утверждения относятся к игровому правилу, поэтому переживают рефакторинг.

Для математики применяй небольшие известные случаи: нормализованный вектор имеет длину 1; поворот без масштаба сохраняет длину; inverse transform возвращает исходную точку; внутренний обмен газом сохраняет общую массу; постоянная скорость за одинаковое время даёт близкий путь при разных FPS. Дробные вычисления сравнивай с допуском в нужной единице, а не с абсолютным `==`. Угол 0,001 рад может быть большим для прицела, но ничтожным для декоративной камеры; epsilon выбирается по смыслу.

36.2. Один тестовый уровень лучше сотни случайных состояний

Сделай отдельную диагностическую сцену из двух комнат. Она содержит дверь, генератор, батарею, пробоину, WorkZone, двух инженеров, врага, союзника и укрытие. Каждый эксперимент начинается с известного сохранённого состояния. Варианты создаются параметрами или отдельными минимальными сценами: `door_unpowered`, `cover_low`, `ally_in_fire_lane`, `repair_target_removed`. После каждого изменения запускай связанные сценарии. Это сокращает время поиска причины, потому что у всех участников известные координаты и компоненты.

36.3. Производительность измеряется по слоям

Частота кадров сообщает только итог. Для оптимизации нужны CPU времена input/editor, AI, NavMesh, physics, scene systems, render preparation и GPU время основных passes. Если сцена медленна, сначала определи слой и размер нагрузки: число активных NPC, целей, path queries, draw calls, triangles, lights, shadow passes, Bullet objects. Затем меняй один алгоритм. Например, кэш маршрутов с revision двери может снизить дорогие path queries; ранняя фильтрация целей по distance² уменьшит LOS raycasts; batching может снизить draw calls. Но отключение симуляции за пределами камеры может сломать пожар и давление — состояние мира обязано продолжать жить, даже когда его не видно.

Разбор 37. Критерий почти готовой игры

Выбери один эпизод на 15–30 минут. Игрок получает ясную цель, обучается управлению, сталкивается с аварией и врагом, отдаёт приказ экипажу, видит последствия питания и

атмосферы, сохраняет прогресс и может выиграть или проиграть. Результат должен запускаться из отдельного каталога без исходников и редактора. Не называй готовностью одно успешное `Build succeeded`: внешний игрок должен пройти путь без объяснений автора. Если он не понимает, почему дверь закрыта или почему NPC стоит, это дефект обратной связи независимо от корректности внутреннего состояния.

Проведи три цикла наблюдения: сам разработчик с debug overlay, другой человек без debug и чистая машина с Release. После первого исправляй нарушение инвариантов; после второго — цели и подсказки; после третьего — паковку, зависимости, пути к ресурсам и сохранения. Для каждого дефекта запиши шаги воспроизведения, ожидаемое и фактическое состояние, а также приоритет. Только после исправления блокирующих ошибок полируй дополнительные эффекты.

38. Сильные стороны движка

1. Порядок кадра виден в одном координаторе.
2. ECS отделяет данные от большинства систем.
3. Bullet использует fixed step и accumulator clamp.
4. Character controller общий для player/NPC.
5. Async render culling работает по snapshot, не по живому ECS.
6. CPU asset loading отделено от GPU upload.
7. Editor/Play — явные режимы.
8. PBR shader содержит реальную Cook–Torrance математику.
9. Render pipeline уже многопроходный.
10. Entity references remap-ятся после load.
11. Корабельные systems образуют причинные цепочки.
12. Debug overlays существуют для physics/NavMesh/objects.

39. Реестр слабых мест

Приоритет	Слабое место	Симптом	Главная проверка
P0	Release x64 не настроен	C1083/link/subsystem	clean Debug+Release matrix
P0	partial init raw pointers	случайный crash в Shutdown	fail Initialize после каждого шага
P0	scene reset обходит Bullet teardown	crash/dangling после load	100 load/unload + counters
P0	FileSystem path traversal	delete вне assets	`..`/absolute/symlink tests
P0	refraction uninitialized lights/camera	случайный PBR	deterministic render capture
P0	Euler order render/Bullet	collider не совпадает	random compound rotation test

Приоритет	Слабое место	Симптом	Главная проверка
P0	flooding units/double count	слишком медленный/тяжёлый flood	analytical Torricelli/buoyancy tests
P1	transparent depth/order	исчезающее стекло	creation-order invariant image
P1	forces outside fixed tick	FPS-dependent physics	30/60/144 comparison
P1	incomplete serialization	потеря world/material/area	round-trip + corruption test
P1	Entity without generation	stale ID becomes valid	reuse test
P1	normal matrix	неверный свет при scale	inverse-transpose reference
P1	shadow at origin/no skinning	тени исчезают/старая поза	long-distance animated test
P1	current-directory assets	EXE не запускается отдельно	launch from output/temp
P1	jobs without exception/cancel	terminate/hanging Wait	throwing/cancel stress test
P2	monolithic coordinators	дорогие правки/merge conflicts	module dependency budget
P2	duplicate vendored ImGui	drift/size	single dependency source
P2	mixed encoding	C4566/mojibake	UTF-8 compile policy
P2	узкое test coverage / no CI	сложные регрессии всё ещё узнаются вручную	расширять contracts/integration tests + CI gate

39.1. Самые важные пять

Если времени только на пять исправлений:

1. воспроизводимый Release/runtime package;
2. безопасный scene/Bullet teardown;
3. тестовый target;
4. единые rotation/time/fixed-step contracts;
5. refraction/transparency/render-state correctness.

40. Производительность без угадывания

40.1. CPU budgets

Измерять отдельно:

- input/editor;
- AI/NavMesh;
- fixed physics;
- scene systems;
- render preparation/culling;
- draw submission;
- UI;
- main-thread asset upload.

40.2. GPU budgets

- shadow;
- reflection/refraction;
- opaque;
- transparent;
- particles;
- first person;
- postprocess.

40.3. Counters

```
entities/components
Bullet bodies/actions/constraints
render candidates/culled/draw calls/triangles
jobs queued/active/failed
assets queued/ready/failed
GPU buffers/textures/SRV
```

40.4. Hotspot strategy

Сначала profiler, затем изменение. Например, `CULL_NONE` удваивает часть raster workload, но его нельзя просто включить глобально: импортированные winding/inside surfaces должны иметь явный material culling mode.

41. Как диагностировать симптом

Объект не виден

1. Active?
2. Transform finite и scale > 0?
3. model/buffers ready?
4. material/shader/SRV?
5. main/room/frustum culling?
6. transparent queue/depth?
7. camera near/far?

Объект дрожит

1. два владельца Transform?
2. force зависит от render FPS?
3. fixed pose не интерполируется?
4. camera читает pre-physics?
5. animation root motion спорит с controller?
6. collider/pivot offset?

NPC не идёт

1. Utility выбрала target?
2. manual command блокирует?
3. State reason?
4. start/target project on NavMesh?
5. path не обрезан?
6. door links/room route?
7. controller capsule проходит физически?
8. fallback mesh не игнорирует obstacles?

После load crash

1. JSON parse/version;
2. old scene destroyed слишком рано;
3. Bullet action/constraint остался;
4. async completion пришёл из старого generation;
5. Entity reference не remap;
6. material/model pointer уничтожен;
7. static render/nav cache не invalidated.

Значение зависит от FPS

Искать:

- константу «на кадр» без `dt`;
- `lerp(..., 0.3)`;
- силы вне fixed tick;
- random calls разное число раз на frame;
- clamp после интегрирования;
- unordered iteration, влияющий на решение.

42. Практикум: безопасное добавление engine-функции

42.1. Шаблон design note

```
Проблема:
API/данные:
Источник истины:
Владелец:
Поток:
Фаза кадра:
Единицы:
Инварианты:
Сериализация:
Диагностика:
Автотест:
Manual test:
Rollback:
```

42.2. Пример: Heat как engine subsystem

Не начинать с fire particle. Сначала:

1. `HeatComponent` с Kelvin/J/K;
2. `HeatTransferSystem` с pairwise order-independent transfers;
3. fixed/scaled time contract;
4. conservation test;
5. debug heat overlay;
6. serialization;
7. только затем fire gameplay и visual particles.

42.3. Definition of Done системы

- API минимален;
- ownership ясен;
- units названы;
- phase зарегистрирована;
- test падает при намеренной ошибке;
- debug UI объясняет состояние;
- save/load round trip;
- нет D3D/Bullet warnings;
- performance budget известен;
- known limitation записано.

43. Карта чтения кода

Маршрут «запуск и кадр»

```
src/Core/main.cpp
→ src/Core/SystemClass.cpp
→ src/Core/ApplicationClass.cpp::Initialize/Frame/Shutdown
→ src/Scene/SceneManager.cpp::UpdateAll
→ src/Render/Renderer.cpp::RenderFrame
```

Маршрут «сущность»

```
src/ECS/Components/ECSTypes.h
→ ComponentManager.h
→ ECSRegistry.h
→ Components.h
→ SceneManager::SpawnObject/RemoveObject/SerializeObject
```

Маршрут «физика персонажа»

```
CharacterMovementSystem
→ CharacterFacingSystem
→ CharacterControllerSystem
→ PhysicsWorld
→ PhysicsSystem post-step sync
```

Маршрут «NPC»

```
NPCComponents.h
→ NpcUtilitySystem.h
→ NpcCommandSystem.h
→ StateSystem.h
→ NpcNavMeshSystem.cpp
→ CharacterControllerSystem
```

Маршрут «PBR frame»

```
Renderer::RenderFrame
→ SceneManager::RenderAllECS
→ RenderMesh.h
→ MaterialClass
→ PBRShaderClass
→ pbr.vs / pbr.ps
```

Маршрут «корабельная авария»

```
WeaponSystem/DamageSystem
→ HullPanel/Breach
→ FloodingSystem/RoomEnvironmentSystem
→ PhysicsSystem/SubmarineSystem
→ ShipPowerGrid/OxygenGenerator
→ ShipAlert/NpcUtility/RepairSystem
```

Часть VIII. Работа с кодом и практикум по NPC

Цель этой части — проследить одного инженера от первой пробоины до восстановленного корпуса, затем повторить сценарий с двумя работниками, дверью, нехваткой ресурса и ручной командой. Важна не одна функция `Update`, а договор между обнаружением, назначением, движением, работой и завершением. Если любое звено скрыто, следующий сбой снова будет казаться «AI не работает».

Разбор 30. Что считается ремонтной задачей

30.1. Цель и исполнитель имеют разные данные

Ремонтируемый объект хранит, что с ним случилось и сколько работы осталось: `RepairableComponent`, данные пробоины или оборудования, WorkZone и принадлежность комнате. Исполнитель хранит способность действовать: роль, `repairSkill`, режим службы, состояние мозга, ресурс и движение. Coordinator связывает их временным claim. Нельзя хранить всю логику только в NPC: если цель исчезнет, другие системы должны узнать об этом без поиска всех персонажей. Нельзя хранить всё только на цели: состояние маршрута и анимации относится к исполнителю.

Сначала создай одну пробоину и одного инженера. В debug выведи target Entity, `requiredProgress`, текущий progress, WorkZone bounds, выбранную рабочую точку, claim owner и state инженера. Прежде чем запускать Play, объясни ожидаемые переходы: Idle → назначение → путь → вход в рабочую зону → накопление progress → закрытие пробоины → освобождение claim. Если пробоина на стене, её геометрический центр может быть внутри collider. WorkZone должна давать

доступную позицию перед стеной; `StateSystem` умеет искать достижимую точку зоны, а не обязан заставлять капсулу идти точно в центр повреждения.

30.2. Назначение с численным расчётом

В `CrewTaskCoordinator` базовая оценка ремонтной цели строится из severity и близости: `(severity·0,65 + distanceScore·0,35)·roleAffinity·skill` с ограничением результата. Engineer имеет role affinity 1,25; Operations 1,05; Security 0,72; Medic 0,62. Skill ограничивается диапазоном 0,25…2,0. Severity для пробоины выводится из leak rate с clamp, а для повреждённого оборудования может выводиться из доли потерянного здоровья. У разных типов задач есть дополнительные поправки; при расчёте в конкретной сцене смотри ветку типа, а не только общую формулу.

Пример без дополнительных поправок. Пробоина имеет severity 0,8, близость 0,6. База `0,8·0,65+0,6·0,35=0,73`. Инженер skill 1 получит до clamp `0,9125`; медик skill 1 — `0,4526`. Это объясняет выбор инженера при одинаковом пути. Но если инженер находится за недоступной дверью, высокий score не должен удерживать бесполезный claim. Оценка должна сопровождаться проверкой доступности или явным быстрым отказом. Прямая дистанция через стену не равно цене маршрута. Для второй версии можно использовать кэш path length и учитывать время ожидания двери.

Claim ставится на цель, чтобы два исполнителя не получили её одновременно. Если цель должна поддерживать совместную работу, это отдельное игровое правило с capacity и несколькими WorkZone, а не случайное отсутствие проверки. Координатор чистит неверные claims, но все переходы жизненного цикла всё равно нужно испытать: смерть исполнителя, ручной приказ, цель уничтожена, работа завершена, scene reload и потеря пути. Debug строка «занято другим» должна показывать конкретный assignee и время назначения.

Разбор 31. Исполнение работы в `StateSystem`

31.1. От назначения к физическому движению

`NpcUtilitySystem` может установить `brain.repairTarget` и состояние, но он физически ничего не чинит. `StateSystem::UpdateRepairTask` проверяет цель, WorkZone, дистанцию и маршрут. `CharacterControllerSystem` преобразует желание движения в фактическое положение капсулы. Если инженер стоит, изучай в этом порядке: активен ли он; на службе ли; есть ли target; не забрал ли claim другой; существует ли WorkZone; получен ли путь; не ждёт ли он дверь; двигается ли капсула. Проверять animation clip первым бессмысленно, если ни одна цель не была назначена.

Горизонтальная дистанция до рабочей точки и трёхмерная дистанция до пробоины отвечают на разные вопросы. Для движения по палубе сравнивай XZ, но для безопасности работы дополнительно проверь комнату и физическую достижимость: объект этажом выше может иметь ту же XZ позицию. Порог входа в Repairing должен учитывать радиус капсулы и форму WorkZone.

Для выхода применяй гистерезис, например войти при ≤1,5 м, выйти только при >1,8 м, чтобы физические колебания не обнуляли progress.

31.2. Скорость ремонта и единицы

В текущем NPC пути прирост work progress зависит от `requiredProgress`, `repairSkill`, `dt` и `repairDuration`. Если `requiredProgress=100`, skill=1, duration=10 с, ожидаемая скорость около 10 единиц работы/с; за dt=0,1 с прирост около 1. Skill=1,5 сокращает время до примерно 6,67 с при той же формуле, если нет дополнительных ограничений. Не путай это с ручным путём ремонта игрока, где может применяться отдельный `repairRate·dt`. Если балансируешь длительность, выпиши обе ветки и объясни, должны ли игрок и NPC работать одинаково.

Перед добавлением прогресса проверь, не отменена ли задача и не потерян ли ресурс. После `RepairSystem::ApplyRepairProgress` не доверяй старой ссылке на компонент: завершение может удалить Entity пробоины либо изменить геометрию асинхронно. В event log запиши «работа завершена», а HUD и звук получат событие. Не определяй завершение только по отсутствию mesh: mesh может обновиться с задержкой.

31.3. Команда игрока и отказ

Игрок выбирает инженера и указывает повреждённый объект. Если объект не ремонтируем, нет пути, нет ресурса или опасная среда запрещает работу, интерфейс возвращает конкретную причину. Если команда принята, прежний automatic claim снимается, старый route очищается, новый target фиксируется как manual. При `ResumeAutomatic` ручной target и временное состояние удаляются, после чего coordinator может назначить новую задачу. Тестируй две пробоины: автоматический ремонт A, ручной переход на B в середине progress, возврат в automatic. На каждом шаге проверяй claim A и B, а не только анимацию инженера.

31.4. Полная лабораторная работа

Поставь две пробоины и двух инженеров. A тяжелее, но ближе к инженеру 1; B легче и ближе к инженеру 2. Заранее рассчитай приближённый score каждого сочетания. Запусти игру и сравни назначения. Убей первого инженера: его claim освобождается, второй выбирает A или сохраняет B по документированному правилу. Затем восстанови сцену, закрой дверь без питания, заставь одну пробоину стать недостижимой и проверь, что свободный инженер не повторяет одну и ту же неудачу каждый кадр. Наконец, выдай ручной приказ, завершай работу и перезагрузи сцену. Результат считается верным, если каждый переход объясняется данными и в мире не осталось скрытого claim.

44. Как изучать собственный движок, а не просто читать файлы

Эта часть превращает справочник в рабочий учебник. Её задача — научить не запоминать тысячи строк, а восстанавливать путь данных и находить правильное место изменения.

44.1. Пять вопросов перед чтением любой системы

Перед тем как разбирать метод, ответь на пять вопросов:

1. **Какие данные он читает?** Например, `NpcUtilitySystem` читает позиции, повреждения, пробоины и настройки мозга.
2. **Какие данные он меняет?** Например, он записывает `repairScore`, `repairTarget`, `state` и `status`.
3. **Кто вызывает метод?** Система сама по себе не работает; важна её позиция в `ApplicationClass::Frame` или `SceneManager::UpdateAll`.
4. **Кто использует результат?** `StateSystem` читает решение utility, а `CharacterControllerSystem` читает уже готовое движение.
5. **Что является источником истины?** Для движения это `CharacterMovement` до физического шага и Bullet-transform после него; для враждебности — `NPCComponent::isHostile`; для назначения ремонта — claim на цели.

Если на эти вопросы нет ответа, изменение пока рано писать. Большинство трудноуловимых багов движка возникает не внутри формулы, а между двумя владельцами одного состояния.

44.2. Чтение по вертикальному пути

Не нужно читать каталог подряд. Выбирай одно наблюдаемое поведение и иди от входа до результата.

Пример для движения NPC:

```
NPCComponent / NpcBrainComponent
→ NpcUtilitySystem или NpcCommandSystem
→ StateSystem
→ CharacterMovement
→ CharacterFacingSystem
→ CharacterControllerSystem
→ Bullet fixed step
→ PhysicsSystem sync/interpolation
→ TransformComponent
→ AnimationController
→ Renderer
```

Пример для ремонта:

```
DamageSystem создаёт повреждение
→ RepairableComponent описывает работу
→ CrewTaskCoordinator закрепляет цель
→ NpcUtilitySystem считает полезность
→ StateSystem ведёт NPC к WorkZone
→ RepairSystem применяет прогресс
→ WorldEventLog записывает результат
→ HUD/debug объясняет состояние
```

Такой маршрут даёт контекст. Отдельный метод без вызывающей и принимающей стороны почти всегда вводит в заблуждение.

44.3. Метод изучения за 30 минут

Для каждой новой подсистемы используй один и тот же цикл:

1. Найди типы данных через `rg -n "ComponentName|SystemName" .`.
2. Найди место вызова `Update`, `Spawn`, `Destroy` или `Render`.
3. Выпиши producer и consumer каждого изменяемого поля.
4. Включи debug UI и сопоставь поля с наблюдаемым поведением.
5. Измени один параметр через инспектор, не меняя код.
6. Предскажи результат до запуска.
7. Запусти и сравни предсказание с фактом.
8. Верни параметр и запиши вывод в `DEV_NOTES_OFFLINE.md`.

Этот способ полезнее пассивного чтения: ты строишь причинную модель движка и сразу проверяешь её.

44.4. Как отличать реализованное от проектируемого

В документации встречаются три типа утверждений:

- **факт текущего кода** — можно найти тип и место вызова;
- **ограничение** — код работает, но имеет известную границу;
- **целевая архитектура** — идея для следующего этапа, которой ещё нет в runtime.

Пример: `CrewTaskClaimComponent` — реализованный reservation ремонта. Общий task board пожаров, лечения и снабжения — целевая архитектура. Нельзя писать новую систему, исходя из того, что целевая часть уже существует.

44.5. Минимальная design note перед новой механикой

```
Наблюдаемое поведение:
Компоненты данных:
Система, принимающая решение:
Система, исполняющая решение:
Источник истины:
Место в кадре:
Время: real / simulation / fixed:
Что происходит при pause:
Как обрабатывается удаление target:
Что сохраняется в JSON:
Что видно в debug UI:
Минимальная тестовая сцена:
Критерий готовности:
Что намеренно не входит в задачу:
```

Если этот список заполнен, код обычно получается меньше и понятнее.

45. Анатомия персонажа в DX Framework

Игрок, дружественный NPC и враг используют общую физическую основу. Различие задаётся набором компонентов и тем, какая система пишет намерение движения.

45.1. Слои сущности персонажа

Слой	Компоненты	Ответственность
Идентичность	`NameComponent`, `NPCComponent`, `PlayerGameTag`	кто это и какую роль играет
Представление	`MeshRendererComponent`, `AnimationComponent`	модель, материал, скелет и клипы
Положение	`TransformComponent`	отображаемая позиция, поворот и масштаб
Намерение	`CharacterMovement`	куда и с какой скоростью персонаж хочет двигаться
Физика	`CharacterController`, `CharacterPhysicsTuningComponent`	Bullet capsule, земля, ступени, прыжок
NPC-настройки	`NPCMovementComponent`	скорость, бег, дистанции прибытия и дверей
AI runtime	`NpcBrainComponent`	состояние, команда, цели, маршрут и debug
Экипаж	`CrewMemberComponent`	роль, смена, ремонтный навык и assignment

Mesh не определяет игровую принадлежность. Одна и та же модель может быть дружественной или враждебной. Источником истины служат gameplay-компоненты.

45.2. `NPCComponent`

`NPCComponent` уже содержит:

- `name` и `description` для инспектора и взаимодействия;
- `health` и `maxHealth` как базовое здоровье NPC;
- `isHostile` как текущий признак враждебности.

Флаг `isHostile` уже:

- редактируется чекбоксом `Hostile` в `EditorUI`;
- сохраняется как `npc_hostile`;
- восстанавливается при загрузке сцены;
- передаётся в `InteractionTargetInfo::isHostileNpc`;
- меняет цвет панели NPC в `GameHud`;
- исключает NPC из автоматического распределения ремонтных задач.

Поэтому отдельный `EnemyComponent` для первого врага не нужен. Он создал бы два источника истины: `isHostile == true`, но `EnemyComponent` отсутствует, или наоборот. Отдельный компонент

станет оправдан только для самостоятельного набора боевых данных, например `HostileAiComponent`, а не как второй флаг принадлежности.

45.3. `NpcBrainComponent`

Brain хранит изменяемое состояние, но не обязан содержать всю будущую логику. Его важные группы полей:

- текущие `NpcState` и `NpcCommand`;
- цель ремонта и комнатный маршрут;
- активная дверь и состояние прохождения портала;
- utility-настройки и диагностические score;
- таймеры перестроения пути и detector застревания;
- состояние wander;
- debug-команды и объясняющие строки.

Поле `status` предназначено для ответа «что сейчас происходит», а `blockedReason` — «почему система не может продолжить». Они не должны быть условием gameplay-логики.

45.4. `CharacterMovement` как граница между AI и физикой

AI не должен напрямую двигать `TransformComponent` или вызывать Bullet `warp` для обычной ходьбы. Он записывает намерение:

```
walkDirection
currentSpeed
isMoving
wantsRun / isRunning
wantJump
locomotionState
```

После этого общие системы выполняют намерение. Благодаря такой границе:

- игрок и NPC используют одну collision-модель;
- анимация получает одинаковые состояния locomotion;
- враг не проходит сквозь препятствия особым способом;
- AI можно тестировать отдельно от Bullet.

45.5. Создание runtime-контроллера

Добавление `CharacterController` в ECS ещё не создаёт Bullet-объекты. `CharacterSpawnSystem::Spawn`:

1. проверяет `CharacterController` и `TransformComponent`;
2. уничтожает старый runtime controller при повторном spawn;
3. создаёт capsule shape;
4. создаёт ghost object и связывает его с Entity через `userIndex`;
5. создаёт `btKinematicCharacterController`;
6. применяет gravity, slope, jump и penetration settings;
7. регистрирует collision object и action в Bullet world.

Парный `Destroy` обязан удалить action и collision object из мира до освобождения памяти. Простое удаление ECS-компонента без teardown опасно.

45.6. Что делает `CharacterControllerSystem::Update`

Метод обслуживает все сущности с `CharacterController`:

- останавливает и временно снимает Bullet-объекты неактивной сущности;
- лениво добавляет безопасный `CharacterPhysicsTuningComponent` старым сценам;
- вычисляет опору геометрическим sweep, а не доверяет только `onGround()`;
- применяет auto-step;
- объединяет locomotion и внешнюю скорость декомпрессии;
- запускает прыжок;
- переводит неподвижного стоящего персонажа в стабильный physics idle;
- очищает runtime-map для удалённых сущностей.

Публичные диагностические методы `IsAutomaticStepActive`, `HasGroundSupport`, `IsIdlePhysicsLocked` позволяют UI показать внутреннее состояние без дублирования расчётов. `ForceReconfigure` сбрасывает runtime cache после изменения настроек в инспекторе.

45.7. Поворот и анимация

`CharacterFacingSystem` читает горизонтальное направление движения и плавно меняет visual rotation. Он не принимает решение, куда идти.

`AnimationController` читает locomotion-результат и выбирает Idle, Walk, Run или другой доступный клип. AI не должен напрямую запускать Walk каждый кадр: это связывает решение с конкретным ассетом и ломает fallback для моделей с другим набором клипов.

46. Дружественный NPC в текущей архитектуре

46.1. Состав дружественного NPC

Минимальная дружественная сущность содержит:

```
ActiveComponent
TransformComponent
MeshRendererComponent
AnimationComponent
NPCComponent { isHostile = false }
NpcBrainComponent
NPCMovementComponent
CharacterMovement
CharacterController
CharacterPhysicsTuningComponent
```

`CrewMemberComponent` может быть добавлен вручную или будет создан координатором с defaults. Для предсказуемой сцены лучше авторить его явно и сохранять роль, `onDuty` и `repairSkill`.

46.2. `NpcCommandSystem`

Это единая точка выдачи ручного приказа. Команды не исполняют движение сами: они валидируют/записывают `NpcCommand`, увеличивают revision и сбрасывают необходимые runtime-поля.

Назначение методов:

Метод	Смысл	Время жизни команды
`Stop`	запретить автоматике перехватить NPC	пока не будет Resume/другая команда
`Wander`	постоянно выбирать точки блуждания	постоянная
`MoveToPosition`	прийти в мировую точку	снимается по прибытии
`MoveToRoom`	прийти внутрь комнаты	снимается после безопасного прохода портала
`FollowEntity`	следовать за движущейся Entity	постоянная
`RepairEntity`	прийти и починить конкретную цель	снимается после завершения/невалидности
`ResumeAutomatic`	очистить ручной приоритет	немедленно возвращает authority автоматике
`ForceRepath`	инвалидировать маршрут	не меняет смысл текущего приказа

Главный инвариант: `command.type != None` означает, что automatic utility не имеет права менять задачу NPC. Debug UI не должен обходить этот контракт прямой записью `brain.state`.

46.3. `NpcUtilitySystem`

Utility проходит по NPC с brain, movement и transform, оценивает ремонтируемые пробоины и оборудование. При оценке учитываются валидность, расстояние, тяжесть повреждения и чужой claim.

Система всегда может вычислять score для диагностики, но при активной ручной команде не меняет решение. Это позволяет видеть, что автоматика выбрала бы, не отбирая управление у разработчика.

46.4. `CrewTaskCoordinator`

Координатор решает межагентную проблему: несколько NPC не должны чинить одну цель.

Последовательность `Update`:

1. очистить невалидные claims;
2. собрать и отсортировать ремонтируемые цели;
3. собрать активных NPC и отсортировать по Entity для воспроизводимости;
4. исключить hostile, off-duty и NPC с ручной командой;
5. сохранить действующее валидное назначение;
6. оценить свободные цели;
7. добавить `CrewTaskClaimComponent` цели;
8. записать assignment в `CrewMemberComponent` и brain;
9. обновить debug snapshot.

`IsRepairTargetValid` — общий контракт валидности цели. `IsClaimedByOther` используется utility, чтобы не выбирать чужую работу. `GetSnapshot` отдаёт UI количество доступного экипажа, занятых работников и открытых задач.

Роль влияет на score: инженер получает преимущество в ремонте, Operations — дополнительную близость к генераторам и кислородному оборудованию, Security/Medic имеют меньшую склонность к обычному ремонту. `repairSkill` влияет не только на выбор, но и на скорость исполнения ремонта.

46.5. `StateSystem`

`StateSystem` — исполнитель. Он не должен решать стратегическую полезность задачи.

Верхний `Update` проверяет обязательные компоненты и направляет NPC либо в manual-command handler, либо в automatic state. Для movement-задач используется общий `NavigateTo`:

```
определить текущую и целевую комнаты
→ построить маршрут по дверям
→ выбрать approach/exit точки портала
→ запросить открытие двери
→ пройти портал полностью
→ запросить локальную NavMesh waypoint
→ записать CharacterMovement
→ следить за фактическим прогрессом
```

`FollowEntity` уже является полезным прототипом преследования: target transform читается каждый кадр, target room обновляется, а NPC держит дистанцию 1.5 м. Но это ручная бесконечная команда, а не hostile perception/combat AI.

`UpdateRepairTask` унифицирует пробоину и оборудование через `RepairableComponent`. Для оборудования он ищет `WorkZoneComponent`, выбирает достижимую рабочую точку, подходит, учитывает `repairSkill` и вызывает общий ремонтный путь.

46.6. Двери и NavMesh

Комнатный маршрут и локальный NavMesh решают разные задачи:

- граф комнат говорит, через какие двери идти;
- NavMesh говорит, как пройти внутри комнаты вокруг статических препятствий;
- local dynamic avoidance даёт короткий объезд динамического AABB;
- Bullet controller окончательно решает физическую проходимость.

Успешный путь NavMesh не гарантирует, что capsule пройдёт: размеры агента и физического контроллера должны совпадать. При любом застревании сначала смотри `status`, `blockedReason`, door snapshot, current room, waypoint и ground support.

46.7. Что значит «почти идеальный дружественный NPC»

Для текущего масштаба проекта это не NPC с сотней реплик. Это персонаж, который:

- стабильно стоит, ходит, бегает и проходит допустимые пороги;
- не дрожит и не проваливается;
- проходит двери и восстанавливается после временной блокировки;
- выполняет manual-команды;
- самостоятельно получает уникальную ремонтную работу;
- корректно освобождает claim при отмене, off-duty, уничтожении цели или смене команды;
- имеет понятную роль и параметры в инспекторе;
- сохраняет authoring-данные и безопасно пересоздаёт runtime-данные;
- в каждый момент объясняет состояние через UI;
- не зависит от присутствия игрока или камеры.

47. Проектирование врага на существующем NPC pipeline

47.1. Что уже готово

Для врага уже можно переиспользовать:

- `NPCComponent::isHostile` как принадлежность;
- `health/maxHealth` как базовое здоровье;
- `NpcBrainComponent` как общий navigation/debug runtime;
- `NPCMovementComponent` как параметры движения;
- `CharacterMovement`, controller, facing и animation;
- `FollowEntity`/`NavigateTo` как доказанный путь преследования;
- `PlayerGameTag` как способ найти игрока;
- `DamageSystem` как общий путь применения урона;
- scene save для базовых NPC-полей.

Врагу не нужна отдельная физика или особая телепортация. Новая часть — принятие боевого решения и безопасная атака.

47.2. Почему одного `isHostile` недостаточно

`isHostile` отвечает только «враг ли это». Боевые runtime-данные имеют другой смысл и другое время жизни. Их лучше хранить в отдельном компоненте, например концептуальном `HostileAiComponent`:

```
// Учебный эскиз контракта, не готовая вставка.
struct HostileAiComponent
{
    Entity target = INVALID_ENTITY;
    float detectionRange = 12.0f;
    float loseTargetRange = 18.0f;
    float attackRange = 1.6f;
    float attackCooldownSeconds = 1.0f;
    float cooldownRemaining = 0.0f;
    // Состояние perception/combat добавь осознанно.
};
```

Этот компонент не дублирует `NPCComponent::isHostile`. Маркер отвечает за отношение, hostile AI — за настройки и runtime боя.

47.3. Минимальный автомат состояний

Первый враг должен иметь только наблюдаемые состояния:

```
Idle
→ обнаружил игрока
Chase
→ вошёл в attack range
Attack
→ игрок отошёл
Chase
→ игрок потерян или невалиден
Idle
```

Дополнительное `Blocked` можно переиспользовать из общего navigation runtime либо показать как причину, не превращая первую версию в огромный state machine.

47.4. Perception первой версии

Первый шаг — дистанция и валидность Entity. Затем добавляется line of sight через raycast.

Порядок проверки:

1. существует ли игрок;
2. активен ли он и имеет ли Transform/health;
3. находится ли он в detection/lose range;
4. не закрывает ли raycast стену или дверь;
5. нужно ли удержать уже найденную цель по hysteresis.

Два радиуса обязательны:

```
detectionRange < loseTargetRange
```

Иначе враг будет переключать Chase/Idle каждый кадр на границе одного радиуса.

47.5. Преследование

Не копируй навигацию из `StateSystem`. Вынеси или переиспользуй существующий путь так, чтобы hostile decision только сообщал:

```
target entity
desired stop distance
desired movement state
```

Для первого прототипа допустимо доказать маршрут существующей ручной командой `FollowEntity`. Но автоматическая hostile-система не должна каждый кадр заново выдавать command и увеличивать revision: это постоянно инвалидирует маршрут. Ей нужен стабильный combat state или отдельный вызов общего navigation helper.

47.6. Атака

Первая атака должна быть простой melee-атакой с cooldown. Условия применения урона проверяются в момент удара, а не только при входе в Attack:

- target всё ещё валиден;
- target всё ещё в attack range;
- при необходимости line of sight не потерян;
- cooldown завершён;
- урон проходит через общий `DamageSystem`/damage event, а не прямое изменение случайного поля HP.

Cooldown использует `simulationDeltaTime`. На паузе он не уменьшается. Один удар не должен зависеть от FPS и не должен наноситься каждый кадр анимации.

47.7. Порядок hostile-системы в кадре

Боевой decision producer должен работать до facing/controller/animation. Возможный контракт:

```
NpcUtilitySystem (friendly repair score)
→ CrewTaskCoordinator (friendly assignment)
→ Hostile decision/perception
→ StateSystem или общий navigation executor
→ CharacterFacingSystem
→ CharacterControllerSystem
→ AnimationController
→ Damage resolution / particles
```

Главное правило: в одном кадре только один слой имеет authority над `CharacterMovement` конкретного NPC. Для hostile NPC дружественный repair pipeline должен быть выключен — это уже частично обеспечивает `CrewTaskCoordinator`, но `NpcUtilitySystem` тоже нужно проверить на ранний выход для `isHostile`.

47.8. Смерть и потеря цели

Вражеский AI обязан безопасно переживать:

- удаление игрока;
- выход из Play Mode;
- загрузку новой сцены;
- деактивацию врага;
- смерть врага;
- смерть target;
- уничтожение двери/изменение маршрута.

Нельзя хранить raw pointer на компонент цели между кадрами. Храни `Entity`, а компонент получай заново после проверки `IsAlive` и наличия нужного типа.

47.9. Debug-контракт врага

Для выбранного врага показывай:

```
Hostile: yes/no
Combat state
Target Entity
Distance to target
Detection / lose / attack ranges
Line of sight
Cooldown remaining
Current room / target room
Nav status / waypoint
Movement intent
Blocked reason
Last damage result
```

Если враг «не работает», эта панель должна за минуту ответить: он не видит, не решил, не построил путь, физически застрял или не прошёл условие удара.

48. Практическое задание Первый полноценный дружественный NPC

Цель задания — довести одного члена экипажа до состояния, при котором его можно оставить в сцене без постоянного ручного контроля. Код выполняет разработчик; учебник задаёт архитектурные границы и проверяемый результат.

48.1. Сценарий

Создай NPC `Crew_Engineer_01`. Он стоит в жилой комнате, получает роль Engineer, самостоятельно выбирает одну из двух ремонтных целей, проходит в техническую комнату через дверь, чинит цель, освобождает assignment и возвращается в понятное Idle-состояние. Ручная команда временно перехватывает его и после Resume возвращает автоматике.

48.2. Этап 1. Authoring и сохранение

Настрой сущность через редактор:

- читаемые name/description;
- `isHostile = false`;
- корректный набор character/NPC-компонентов;
- роль Engineer, `onDuty = true`, осмысленный `repairSkill`;
- скорости walk/run и arrival distances;
- capsule и physics tuning под размеры модели;
- AnimationComponent с Idle/Walk/Run;
- ActiveComponent.

Сохрани сцену, перезапусти и проверь, что authoring-поля восстановились. Runtime pointer Bullet, текущий путь и временный claim не должны сериализоваться как адреса.

48.3. Этап 2. Физическая стабильность

Проверки:

1. 60 секунд Idle на ровном полу.
2. Проход порогов 0.05, 0.10, 0.20, 0.30 и 0.36 м.
3. Движение по склону до max slope и отказ на слишком крутом.
4. Остановка у цели без колебания вперёд-назад.
5. Исчезновение пола корректно снимает grounded/idle lock.
6. 30, 60 и 144 FPS не меняют заметно скорость и результат.

Запиши capsule bottom, ground Y, grounded, idle lock, auto-step и горизонтальную скорость.

48.4. Этап 3. Команды

Проверь все команды инспектора:

- Stop удерживает NPC;
- Wander выбирает достижимые точки;
- MoveToPosition завершается;
- MoveToRoom проходит портал до безопасной точки;
- Follow player держит дистанцию и не толкает capsule;
- Repair entity работает для пробоины и оборудования;
- Resume auto действительно снимает manual authority;
- Force repath не стирает смысл команды.

Для каждой ошибки сначала исправляй объясняющее состояние, затем поведение. Не оставляй silent failure.

48.5. Этап 4. Автономный ремонт и reservation

Размести две цели: пробоину и повреждённый генератор. Добавь второго простого crew NPC только для проверки arbitration.

Критерии:

- каждый NPC получает максимум одну работу;
- одна цель принадлежит максимум одному NPC;
- Engineer имеет предсказуемое преимущество при близких условиях;
- ручная команда освобождает прежний claim;
- off-duty освобождает claim;
- завершение ремонта увеличивает completed count один раз;
- исчезновение цели не оставляет вечный claim;
- hostile NPC не получает crew assignment.

48.6. Этап 5. Двери, препятствия и восстановление

Проверки:

- закрытая рабочая дверь открывается;
- NPC ждёт створки, а не толкает её;
- после прохода collider двери возвращается;
- временное препятствие вызывает repath/avoidance;
- обесточенная или непроходимая дверь даёт понятный `Blocked`;
- после исправления причины Force repath восстанавливает задачу.

48.7. Definition of Done дружественного NPC

- Debug и Release собираются.
- Нет дрожания в Idle и постоянного толкания двери.
- Все ручные команды имеют предсказуемое завершение.
- Автоматический ремонт работает без камеры и игрока рядом.
- Два NPC не чинят одну цель.
- Save/load сохраняет authoring и не сохраняет опасные runtime pointers.
- В debug UI видно состояние, цель, assignment, путь и причину блокировки.
- Есть отдельная regression-сцена и записанные шаги воспроизведения.
- Известные ограничения перечислены, а не спрятаны.

49. Практическое задание Первый преследующий и атакующий враг

Цель — создать `Enemy_Hunter_01`, который автоматически обнаруживает игрока, преследует его через комнаты и двери, останавливается на дистанции атаки, наносит дискретный урон и безопасно теряет цель.

49.1. Ограничение первой версии

В задачу входят только:

- один игрок;
- один melee-враг;
- Idle, Chase и Attack;
- distance perception и затем line of sight;
- существующая navigation/physics/animation pipeline;
- cooldown и урон;
- debug UI и сохранение настроек.

Не входят patrol graph, ranged weapon, cover, squad tactics, loot, ragdoll, hearing, stealth meter и сложная анимационная синхронизация. Они добавляются после стабильного vertical slice.

49.2. Этап 1. Сущность и данные

Создай NPC через существующий `NPC AI Components`, поставь `Hostile`, настрой mesh, animation, controller и movement. Проверь, что после save/load `npc_hostile` остаётся true и HUD показывает hostile-оформление.

Спроектируй отдельные combat-поля. Перед кодом письменно реши:

- где хранится combat state;
- кто владеет target Entity;
- чем отличаются detection и lose range;
- где хранится attack range, damage и cooldown;
- какие поля authoring, а какие runtime;
- какие из них сохраняются;
- кто имеет authority над `CharacterMovement` hostile NPC.

49.3. Этап 2. Доказать преследование без нового AI

Через debug нажми `Follow player`. Проверь путь на ровном полу, через дверь и между комнатами. Это отделяет проблемы существующей навигации от ошибок будущего combat decision.

Если ручное Follow уже не работает, hostile-код пока не писать. Сначала исправляется общий pipeline.

49.4. Этап 3. Автомат Idle и Chase

Добавь систему принятия hostile-решения. Она обрабатывает только активных NPC с `isHostile == true` и нужными combat-данными.

Требования:

- игрок ищется через `PlayerGameTag`;
- target хранится как `Entity`, не pointer;
- detection использует горизонтальную или трёхмерную дистанцию по явно выбранному правилу;
- захваченная цель теряется только за `loseTargetRange`;
- состояние не переустанавливается каждый кадр без причины;
- изменение target/состояния инвалидирует путь один раз;
- при отсутствии игрока движение полностью останавливается;
- friendly utility не перезаписывает hostile movement.

49.5. Этап 4. Line of sight

Добавь raycast от осмысленной точки врага к осмысленной точке игрока. Ray не должен начинаться в полу или центре capsule, если там мешает собственный collider. Игнорируй entity врага и корректно интерпретируй первое попадание.

Проверки:

- видит игрока в пустой комнате;
- не видит через стену;
- закрытая дверь перекрывает видимость;
- открытая дверь разрешает видимость;
- краткая потеря LOS не вызывает дрожание состояния — при необходимости добавь grace timer;
- debug line показывает реальный луч.

49.6. Этап 5. Attack

Когда дистанция меньше attack range, враг останавливает locomotion и входит в Attack. Удар происходит только по cooldown и повторно проверяет target/range/LOS.

Критерии:

- один удар уменьшает здоровье ровно один раз;
- частота удара одинакова при 30/60/144 FPS;
- pause замораживает cooldown и атаку;
- враг не наносит урон через стену;
- игрок, вышедший из attack range, переводит врага обратно в Chase;
- удалённый/мёртвый игрок очищает target;
- damage идёт через общий механизм, а не обходной прямой доступ к UI-полю;
- friendly NPC не становится жертвой, пока это явно не предусмотрено.

49.7. Этап 6. Анимация и обратная связь

Idle/Walk/Run остаются под управлением общего animation controller. Для первой атаки допустим отдельный trigger/state, но урон нельзя случайно привязать к render frame.

Минимальная обратная связь:

- имя и красный hostile accent;
- combat state в debug;
- attack cooldown;
- короткое событие/лог успешного удара;
- видимое изменение HP игрока;
- понятная причина потери цели.

49.8. Матрица тестов врага

Ситуация	Ожидаемый результат

Ситуация	Ожидаемый результат
Игрок вне detection	Idle, target invalid
Игрок входит в detection	target захвачен, Chase
Игрок между detection и lose после захвата	Chase продолжается
Игрок за стеной	нет захвата или срабатывает grace по выбранному правилу
Игрок в другой связанной комнате	путь идёт через дверь
Игрок ближе attack range	движение остановлено, Attack
Игрок отступил	возврат в Chase
Игрок удалён	target очищен, Idle
Враг деактивирован	Bullet/action и AI не продолжают работу
Игра на pause	движение, cooldown и урон заморожены
Save/load	authoring сохранён, runtime target безопасно сброшен/проверен

49.9. Definition of Done врага

- Враг определяется существующим `isHostile`.
- Combat-данные не дублируют brain/movement без причины.
- Преследование переиспользует общий navigation pipeline.
- Есть hysteresis обнаружения и cooldown атаки.
- Урон применяется дискретно и FPS-независимо.
- Friendly crew pipeline не управляет врагом.
- Невалидная target Entity никогда не разыменовывается.
- Debug UI объясняет perception, decision, navigation и attack.
- Есть тестовая сцена с комнатой, стеной и дверью.
- Все сценарии матрицы проверены и записаны.

50. План одного рабочего дня и порядок сдачи

Если на практику есть один длинный вечер, не пытайся закончить обоих персонажей сразу.

Блок 1. 30 минут — baseline

- собрать Debug;
- сохранить копию тестовой сцены;
- проверить текущего NPC через Follow player и Repair entity;
- записать исходные баги отдельно от новых.

Блок 2. 90 минут — дружественный NPC

- полный authoring;
- роль Engineer;
- команды;
- одна автономная ремонтная задача;
- debug snapshot;
- save/load.

Если на этом этапе есть баг движения или двери, остановись и локализуй его. Враг использует тот же фундамент.

Блок 3. 60 минут — данные врага и Idle/Chase

- `isHostile`;
- отдельный combat runtime;
- target игрока;
- detection/lose hysteresis;
- автоматическое преследование;
- отсутствие конфликтов с crew utility.

Блок 4. 60 минут — Attack

- attack range;
- остановка;
- cooldown;
- общий damage path;
- pause/FPS проверки;
- debug-поля.

Блок 5. 30 минут — фиксация результата

Запиши:

```
Дата:
Сцена:
Изменённые файлы:
Что умеет Crew_Engineer_01:
Что умеет Enemy_Hunter_01:
Какие состояния проверены:
Какие значения ranges/cooldown использованы:
Что происходит при pause:
Что происходит при потере/удалении target:
Известные проблемы:
Следующий маленький шаг:
```

Для code review подготовь короткое видео или последовательность скриншотов и список шагов воспроизведения. Хороший результат — не «вроде бегает», а повторяемый сценарий, в котором каждое состояние можно объяснить данными.

Часть IX. Боевой NPC как учебный проект

Враг должен быть опасен по понятным причинам. Игроку должно помогать укрытие; звук должен создавать догадку, а не телепатию; оружие должно стрелять из реального дула; дружественный персонаж на линии огня должен влиять на решение. Для разработки это означает последовательный автомат: допустимая цель → наблюдение → память → позиция → разрешение выстрела → урон → реакция. Каждый шаг можно испытать отдельно.

Разбор 32. Допустимая цель и честное наблюдение

32.1. До геометрии проверь правила мира

Враг не должен выбирать ближайший Entity без проверки фракции, active state и здоровья. `FactionComponent` определяет отношения; `AreEnemies` решает, допустима ли атака. Старое `isHostile` не является достаточной заменой. После выбора допустимых кандидатов сравни расстояние, угрозу, доступность и устойчивость текущей цели. Если оценки равны, используй стабильное правило разрешения ничьей, чтобы порядок обхода контейнера не менял поведение между запусками.

Сначала проверь квадрат дистанции, затем FOV через dot единичных направлений, затем LOS raycast. Порог для полного 90° обзора равен `cos45°≈0,707`. Отдельные лучи к голове и корпусу снижают ошибки низкого укрытия, но должны исходить из корректной точки глаза. Если pose обновляется позднее, явно выбери fallback или переставь получение eye world point. Выводи причины отказа по каждому кандидату: `TooFar`, `OutsideFov`, `Occluded`, `Inactive`, `Friendly`, `Dead`.

32.2. Потеря контакта и поиск

Когда LOS исчезает, `targetVisible` становится false. `lastKnownPosition` сохраняет место последнего личного наблюдения, а не копирует Transform скрытой цели каждый кадр. Враг идёт туда и ищет ограниченное время. Если он получил сообщение от товарища, это новый факт с чужим источником и давностью, но не личное видение. Слух в будущей версии может сообщать комнату или область; точность звука ниже точности зрения. Тест: игрок пересекает открытое поле, исчезает за дверью и скрытно обходит NPC. Враг ищет у двери; если сразу разворачивается к новой скрытой позиции, установи watchpoint на запись памяти.

Разбор 33. Бой, укрытие и решение о выстреле

33.1. Кто пишет состояние

`NpcCombatSystem` обновляет боевое восприятие и состояние; `NpcCoverSystem` может выбрать укрытие; `StateSystem` исполняет движение. Для каждого brain state запиши владельца записи.

Приоритет укрытия должен иметь короткое время удержания или гистерезис, иначе NPC может каждую секунду менять цель между «атаковать» и «спрятаться». Резерв cover point освобождается при смерти, отмене боя, недостижимости и изменении сцены. `NpcSquadCombatSystem::Update` в текущем `ApplicationClass.cpp` закомментирован: не считай групповое подавление и флангование готовой особенностью. Включай его только после индивидуальных тестов, передавая последнюю известную позицию, а не личный флаг видимости.

33.2. `Attacking` не равно «оружие выстрелило»

`NpcRangedCombatSystem` проверяет фракцию, активность, здоровье, состояние мозга, ручную команду, экипированное оружие, боезапас и reload, личную видимость, время прицеливания, muzzle position, безопасную линию и союзника на ней. Только после этого вызывается `WeaponSystem::TryFire`. Последний управляет cooldown, spread, Bullet ray и damage event. Если NPC стреляет слишком часто, проверь оба таймера — AI burst и weapon cooldown. Если вообще не стреляет, выведи первый отказ. Настройка «больше агрессии» не исправит отсутствующее оружие или неверный socket.

Поставь низкую стенку между стволом и игроком так, чтобы голова врага видела цель. Ожидаемо `targetVisible=true`, но `muzzleBlocked=true`; выстрела нет. Затем поставь союзника между врагом и игроком. Центральная линия удерживает огонь. Однако spread может отправить реальный луч в союзника даже при чистой центральной линии; если дизайн требует почти исключить friendly fire, проверяй расширенный безопасный конус или траектории нескольких выборок. Это отдельное правило, а не свойство одного raycast.

33.3. Прицел, поза и сокет оружия

Сокет дула проходит несколько transform: модель оружия, хват, кость руки, world transform NPC. Вычисляй его после обновления анимационной позы. Если ствол визуально направлен на игрока, но raycast начинается у ног, сравни локальный socket, bone transform, character transform и итоговую world point. Не исправляй это сдвигом цели AI: ошибка пространства координат проявится снова после другого клипа. Добавь debug сферу на muzzle и линию из неё по фактическому fire direction. При повороте NPC сфера должна двигаться вместе с оружием без скачка.

33.4. Урон и обратная связь

Физическое попадание переводится в игровой damage event. `DamageSystem` применяет здоровье и реакцию; NPC может получить память о входящем огне и выбрать укрытие. После смерти необходимо остановить движение и огонь, освободить cover reservation и crew claim, убрать NPC из допустимых целей других бойцов. Визуальные эффекты попадания подписываются на событие; они не должны быть источником урона. Pixel shader может вообще не исполниться для скрытого пикселя, но игровое попадание должно остаться тем же.

Разбор 34. Пять боевых сцен для проверки

Сцена 1 — открытое поле: враг видит игрока, поворачивается, ждёт aim delay и стреляет с правильным cooldown. Сцена 2 — полная стена: FOV проходит, LOS нет, NPC идёт к последнему контакту. Сцена 3 — низкое укрытие: глаз видит, дуло блокировано. Сцена 4 — союзник в огневом коридоре: оружие удерживается и объясняет причину. Сцена 5 — дверь без collider: если враг видит сквозь неё, ошибка не в AI, а в collision setup. Для каждой сцены заранее запиши ожидаемые `visible`, `brain.state`, `rangedStatus`, hit Entity и damage. Только потом запускай Play.

Повтори сцены после scene reload и при другом FPS. Если поведение отличается, сравни время симуляции, update order, физические shape и seed разброса. Один успешный выстрел недостаточен: почти готовая игра требует повторяемого результата и объяснимого отказа. После этого можно добавлять слух, отряд и новые тактики, сохраняя отдельные тестовые сцены для каждого нового правила.

Эта часть продолжает практикум ровно с текущего состояния проекта. Враг уже обнаруживает игрока по дистанции, переходит между `Chasing` и `Attacking`, поворачивает корпус к цели и запрашивает одноразовый клип `Punch`. Наблюдаемая проблема: ноги и корпус смотрят на игрока, но траектория удара уходит вбок. Цель следующих глав - не получить готового врага из одного большого патча, а последовательно построить и доказать собственную боевую архитектуру.

Главный принцип обучения: каждый этап имеет гипотезу, минимальную тестовую сцену, наблюдаемые данные и критерий сдачи. Если критерий не выполнен, следующий слой не добавляется. Так ошибка в анимации не маскируется NavMesh, а ошибка видимости не маскируется сложным utility score.

Статус реализации на 28 октября 2026 года. В этой части групповые роли и координация описаны как учебный проект и возможное развитие. В текущем игровом кадре NpcSquadCombatSystem::Update отключён в ApplicationClass.cpp. Проверяй работающий индивидуальный цикл через NpcCombatSystem, NpcCoverSystem, StateSystem и NpcRangedCombatSystem.

51. Текущее состояние врага и границы следующей версии

51.1. Фактический путь одного кадра

В текущем `ApplicationClass` боевой путь проходит в следующем порядке:

```
NpcUtilitySystem
-> CrewTaskCoordinator
-> NpcCombatSystem::Update
-> StateSystem::Update
-> CharacterFacingSystem::Update
-> выравнивание игрока по камере
-> CharacterControllerSystem::Update
-> PlayerPunchSystem::Update только для игрока
-> AnimationController::Update
-> DamageSystem::Update
```

Для понимания текущего бага особенно важны три факта. `StateSystem` меняет yaw NPC и отправляет `oneShotRequested = true`. `AnimationController` накладывает клип от `mixamorig:Spine`. А

привязка кулака к лучу камеры реализована внутри `PlayerPunchSystem` и требует `PlayerComponent`; NPC через этот код не проходит. Поэтому фраза «удар привязан к прицелу» сейчас верна для игрока, но не для врага.

51.2. Что уже существует

Слой	Реализовано сейчас	Следующее доказательство
Данные	`NPCCombatComponent`: target, attackRange, seeRange, cooldown и таймеры	Отделить authoring от runtime и сохранить настройки
Решение	`NpcCombatSystem` выбирает `Chasing` или `Attacking` по дистанции	Добавить LOS, память и устойчивые переходы
Исполнение	`StateSystem` строит путь, останавливает NPC и поворачивает его	Разделить facing корпуса и aim атаки
Анимация	`oneShotState = Punch` воспроизводится поверх locomotion	Доказать правильный forward клипа и контактную фазу
Навигация	Detour waypoints, двери и dynamic avoidance уже используются	Переиспользовать их для cover и flank destinations
Диагностика	Inspector показывает ranges, target и attack timer	Добавить LOS, yaw error, aim error, cover score и reason

51.3. Подтверждённые ограничения текущего кода

- `NpcCombatSystem` пока воспринимает игрока только по расстоянию; стены и направление взгляда не учитываются.
- В ветке, где цели ещё нет и игрок вне `seeRange`, состояние устанавливается в `Chasing`; это нужно проверить как логическую ошибку до усложнения автомата.
- `NPCCombatComponent` редактируется в инспекторе, но в текущем `SceneManager.cpp` не найдено сохранение его полей. После загрузки значения могут вернуться к defaults.
- В `StateSystem` удар пока описан как запрос анимации. Общая точка контактного урона NPC ещё должна быть спроектирована.
- Дистанция считается в 3D, а поворот - в плоскости XZ. Нужно явно решить, влияет ли разница высоты на detection и melee range.
- Константа разворота модели на PI повторяется в нескольких местах. Пока соглашение не централизовано, легко получить дополнительный поворот на 90 или 180 градусов.

51.4. Учебный контракт

Для каждого этапа создавай короткую design note:

```
Наблюдаемое поведение:
Гипотеза:
Producer данных:
Consumer данных:
Источник истины:
Минимальная сцена:
Что рисуется в debug:
Тест до изменения:
Тест после изменения:
Definition of Done:
Что намеренно не входит в этап:
```

При обращении за помощью присылай эту заметку, небольшой фрагмент кода и один конкретный наблюдаемый симптом. Тогда помощь останется объясняющей, а решение - твоим.

52. Практикум Удар направлен вбок

52.1. Теория трёх направлений

В одном кадре существуют минимум три независимых направления. `bodyForward` задаётся world rotation сущности. `clipForward` зашит в локальной системе скелета и в ключах таза, позвоночника и плеч. `attackDirection` идёт от точки атаки к выбранной цели. Исправление считается верным только тогда, когда эти направления согласованы осознанно.

```
toTarget = targetPosition - attackerPosition
horizontal = normalize(float3(toTarget.x, 0, toTarget.z))
desiredYaw = atan2(horizontal.x, horizontal.z) + modelForwardOffset
yawError = wrapPi(desiredYaw - currentYaw)
canStartAttack = abs(yawError) <= facingTolerance
```

Эта формула поворачивает корпус, но не гарантирует направление кисти. Верхний клип может вращать `Spine`, анимация может быть экспортирована с другой осью forward, а runtime IK может вообще не запускаться для NPC.

52.2. Четыре вероятные причины

7. Клип `Kevin_Punch.1` сам направлен боком относительно локального forward модели.
8. Клип корректен, но при смешивании от `mixamorig:Spine` его локальный поворот применяется поверх уже развёрнутого корпуса не в ожидаемом пространстве.
9. Корпус NPC смотрит правильно, но система aim и IK существует только для игрока, поэтому кисть NPC не получает цель.
10. В разных местах используется разный model forward offset: один слой добавляет PI, а другой считает, что модель смотрит по +Z без поправки.

Не выбирай причину по ощущению. Следующие проверки должны исключить три неверных варианта.

52.3. Минимальная диагностическая сцена

- Ровный пол без стен, один игрок, один `Enemy_Hunter_01`, одна камера сверху или сбоку.
- Отключи перемещение игрока и автоматическое изменение yaw камеры во время серии.
- Поставь врага в начало координат с yaw = 0. Игрока последовательно ставь в +Z, +X, -Z и -X.

- Используй один и тот же вариант удара и один и тот же кадр контакта.
- Сохраняй для каждого направления: target yaw, current yaw, yaw error, world position плеча, кисти и цели.

Позиция цели	Ожидаемое	Если кулак всё равно уходит вправо
+Z	корпус и кулак по +Z	постоянная локальная ошибка клипа или IK отсутствует
+X	корпус и кулак по +X	если ошибка повернулась вместе с NPC, проблема в model space
-Z	корпус и кулак по -Z	если ошибка осталась в world space, ищи неверную world transform
-X	корпус и кулак по -X	сравни знак atan2 и повторное добавление offset

52.4. Сделай ошибку видимой

Добавь временную debug-визуализацию, а не новый gameplay-код:

- зелёная линия - `bodyForward` от позиции NPC;
- жёлтая линия - горизонтальный `toTarget`;
- синяя линия - направление от плеча к текущей кисти;
- красная линия - от кисти к желаемой contact point;
- текст: `currentYaw`, `desiredYaw`, `yawErrorDeg`, `oneShotTimeRemaining`, активный clip index.

Если зелёная и жёлтая линии совпадают, поворот ног уже исправен и трогать `TransformComponent` нельзя. Если синяя линия стабильно отклонена на одинаковый угол, проблема находится после facing: в ассете, смешивании или IK.

52.5. Развилка ассет или runtime

Сначала открой клип в Animation Studio без AI и проверь его на yaw 0. Затем поверни всю сущность на 90 градусов и повтори.

Результат проверки	Диагноз	Учебное решение
Клип уже в Studio бьёт боком	ошибка ориентации Action или ключей верхнего тела	исправить Action в Blender или локальную pose correction; не компенсировать AI углом
Studio верна, NPC неверен	ошибка runtime binding, layer root или отсутствующий aim	проверить binding `Punch`, `upperBodyRoot`, clip index и generic aim path

Результат проверки	Диагноз	Учебное решение
Только игрок верен	player-only camera aim и IK	вынести общую часть melee aim из `PlayerPunchSystem`
Ошибка меняется при повороте мира	смешаны local/model/world spaces	подписать пространство каждого вектора и добавить явные преобразования

Минимальный ассетный фикс допустим, если Action объективно экспортирован боком. Архитектурный фикс нужен, если враг должен направлять один и тот же удар к разным точкам, высотам и движущимся целям.

52.6. Проектирование общего melee aim

Не копируй весь `PlayerPunchSystem` внутрь NPC. Выдели нейтральный контракт, который не знает о клавиатуре, камере или `PlayerComponent`. Один из вариантов данных для обсуждения:

```
struct MeleeAimRuntime
{
    Entity target = INVALID_ENTITY;
    XMFLOAT3 aimOrigin{};
    XMFLOAT3 aimDirection{0, 0, 1};
    XMFLOAT3 contactPoint{};
    float contactPhase = 0.5f;
    float reach = 1.5f;
    bool active = false;
};
```

Это не готовое требование к имени компонента. Твоё задание - письменно ответить:

11. Кто создаёт aim для игрока и кто создаёт aim для NPC?
12. Кто переводит world contact point в model space перед IK?
13. Кто знает, какой рукой выполняется текущий вариант удара?
14. Кто сбрасывает runtime после окончания one-shot?
15. Какие поля сохраняются в сцену, а какие никогда не сериализуются?

Хорошая граница: player input и NPC decision формируют одинаковое намерение; общая melee-система вычисляет контакт, применяет IK и в contact window создаёт `DamageEvent`. Тогда прицел игрока и target врага различаются только producer-ом.

52.7. Контактная фаза и урон

Не связывай урон с моментом запуска клипа или с конкретным render frame. В текущем игроке уже есть идея `contactPhase`. Для NPC используй нормализованную фазу one-shot и флаг «урон уже применён в этом ударе».

```
phase = elapsed / duration
enteredContactWindow = previousPhase < contactPhase && phase >= contactPhase
if enteredContactWindow and not hitApplied:
    revalidate target, range, line of sight and fist contact
    emit DamageEvent once
    hitApplied = true
```

При смене FPS условие пересечения фазы остаётся устойчивым, даже если точный момент между кадрами был пропущен.

52.8. Definition of Done исправления удара

- Враг поворачивается к игроку с четырёх сторон и не начинает клип раньше допуска по yaw.
- На contact phase кулак находится в допустимом радиусе от целевой точки, а debug показывает ошибку в метрах.
- Один one-shot создаёт максимум один `DamageEvent`.
- При выходе игрока из reach во время замаха урон не проходит.
- Стена между плечом и contact point блокирует урон.
- Результат одинаков по смыслу при 30, 60 и 144 FPS.
- Исправление не ломает удар игрока и first-person view.
- В коде нет случайного `+90°` или `-90°` без имени и объяснения соглашения forward.

53. Восприятие и память цели

53.1. От расстояния к восприятию

Сейчас `seeRange` означает сферу знания: внутри неё враг знает игрока даже через стену. Настоящее восприятие лучше разделить на широкую фазу кандидатов и точную проверку. Сначала distance/FOV быстро отбрасывают лишних, затем raycast доказывает line of sight.

```
candidate -> alive and active
-> within detection range
-> inside horizontal field of view
-> eye-to-chest ray is not blocked
-> visible target observation
```

Для первого варианта достаточно одного игрока, но данные проектируй так, чтобы система могла сравнить несколько целей позднее.

53.2. Память без телепатии

При потере LOS сохраняй наблюдение, а не live transform невидимой цели:

```
lastSeenPosition
lastSeenVelocity optional
lastSeenTime
hasLineOfSight
visibilityGraceRemaining
memoryRemaining
```

Пока цель видима, эти поля обновляются. После потери LOS враг идёт к `lastSeenPosition`, но не читает текущую позицию игрока. Когда memory истекает, target очищается или начинается ограниченный поиск.

53.3. Самостоятельное задание

16. Добавь eye point врага и aim point игрока как вычисляемые world positions.
17. Сделай raycast с игнорированием собственного capsule врага.
18. Раздели `hasLineOfSight` и `targetEntity`: цель может быть известна, но временно невидима.
19. Добавь grace timer 0.15-0.35 секунды, чтобы край двери не дёргал состояние каждый кадр.
20. Покажи луч, первое попадание, last seen marker и возраст наблюдения в debug UI.
21. Исправь ветку без цели: вне detection враг должен оставаться в Idle/Patrol, а не самопроизвольно переходить в `Chasing`.

53.4. Проверки

- Пустая комната: цель видна.
- Стена: цель не видна, но последнее наблюдение ещё помнится.
- Закрытая дверь блокирует LOS; открытая разрешает.
- Краткое пересечение дверной рамой не сбрасывает target мгновенно.
- После истечения memory враг перестаёт знать текущую позицию игрока.
- Save/load не восстанавливает старую runtime-цель как валидный указатель или случайную Entity.

54. Преследование последней известной позиции

54.1. Состояния поиска

После стабильного LOS раздели `Chasing` на смысловые фазы. Имена могут отличаться, но данные должны объяснять поведение.

```
Idle or Patrol
-> Alerted
-> ChaseVisibleTarget
-> MoveToLastSeen
-> SearchArea
-> Return or Idle
```

`StateSystem` по-прежнему должен исполнять цель движения через существующий `NavigateTo`; perception не пишет `CharacterMovement` напрямую.

54.2. Поиск без сложной системы

Первая версия SearchArea может состоять из 3-5 достижимых точек вокруг last seen position. Кандидаты проецируются на NavMesh, отбрасываются при слишком коротком пути или отсутствии связности и посещаются по очереди. Это уже создаёт понятное поведение без behavior tree.

22. Прийти к last seen position.

23. Повернуться в последнее известное направление движения игрока.
24. Выбрать несколько NavMesh-кандидатов в радиусе поиска.
25. Остановиться на короткую scan pause в каждой точке.
26. Немедленно вернуться в ChaseVisibleTarget при восстановлении LOS.
27. После таймаута очистить цель и вернуться к исходному поведению.

54.3. Навигационные инварианты

- Новая perception-цель инвалидирует путь один раз, а не каждый кадр.
- Асинхронный результат проверяет revision/epoch перед применением.
- Если last seen point не проецируется на NavMesh, debug хранит конкретную причину.
- Дверной маршрут и local Detour waypoint остаются общей инфраструктурой.
- Ctuck recovery не меняет стратегическую цель: он только перестраивает способ достижения.

55. Укрытия как данные и запрос

55.1. Что такое укрытие для AI

Укрытие - не просто объект с тегом. Это позиция, из которой геометрия закрывает атакующего от угрозы, но рядом существует достижимая позиция для выхода и атаки. Поэтому система должна оценивать укрытие относительно конкретной угрозы.

Для первого варианта выбери один из двух путей:

- Авторские `CoverPointComponent` и `CoverSlot` на стенах и ящиках. Это проще отлаживать и хорошо подходит текущему редактору.
- Автоматические точки по границам NavMesh. Это масштабируется лучше, но требует устойчивого анализа полигонов и нормалей.

Для учебного проекта начни с авторских точек, а генерацию добавь после работающего поведения.

55.2. Минимальные данные cover slot

```
worldPosition
coverNormal
standOrCrouch
leftPeekPoint and rightPeekPoint
reservedBy Entity
reservationRevision or expiry
enabled
```

`coverNormal` должен смотреть от поверхности в свободное пространство. Для угрозы подходящее укрытие обычно находится с противоположной стороны нормали.

55.3. Оценка кандидатов

```
score =
```

```
    protectionWeight * protection
  - pathWeight * normalizedPathCost
  - distanceWeight * distanceToSlot
  + attackWeight * peekCanSeeThreat
  + spacingWeight * distanceFromAllies
  - occupancyPenalty
  - recentFailurePenalty
```

Сначала фильтруй невозможные точки: нет NavMesh path, slot занят другим NPC, точка находится слишком близко к угрозе, между угрозой и укрытием нет блокирующей геометрии. Только затем вычисляй score.

55.4. Практическое задание

28. Создай в редакторе 6-10 cover slots вокруг двух ящиков и одной стены.
29. Покажи normal, peek points и состояние reservation разными цветами.
30. Напиши read-only запрос `FindBestCover`, который возвращает кандидат и подробный score breakdown, но ещё не двигает NPC.
31. Сравни прямую дистанцию и реальную длину NavMesh path; в score используй выбранное правило осознанно.
32. Добавь reservation по образцу `CrewTaskClaimComponent`, чтобы два врага не выбирали одну точку.
33. Только после устойчивого выбора добавь состояния `MoveToCover` и `InCover`.

55.5. Definition of Done укрытия

- Враг выбирает точку, действительно закрытую от текущей угрозы.
- Путь к cover slot существует и завершается с предсказуемой дистанцией.
- Два врага не занимают один slot.
- Исчезновение или отключение укрытия освобождает reservation.
- В debug видны все кандидаты, причина фильтрации и итоговый score.

56. Атака из укрытия и смена позиции

56.1. Раздели melee и ranged ветки

Текущий `Enemy_Hunter_01` является melee-врагом. Укрытие с выглядыванием имеет полный смысл для ranged-врага, поэтому не смешивай обе модели в одном условии. Общая часть - perception, memory, navigation и reservation. Attack executor различается.

Свойство	Melee	Ranged
Желаемая дистанция	в пределах reach	preferred range с min и max
Укрытие	путь сближения и разрыв LOS	позиция защиты и peek

Свойство	Melee	Ranged
Контакт	fist/contact window	muzzle ray или projectile
Позиционирование	обход препятствия и attack slot	cover, strafe, reposition
Анимация	one-shot punch плюс aim IK	aim pose, recoil, reload

56.2. Peek как временное состояние

```
InCover
-> DecidePeekSide
-> MoveOrLeanToPeekPoint
-> ConfirmLineOfFire
-> AttackBurst
-> ReturnToCover
-> cooldown or reposition
```

Сначала перемещай весь capsule к отдельной peek point. Настоящий procedural lean добавляй позже: он требует согласования collider, камеры, оружия и анимации. Простая позиция легче проверяется и не создаёт визуальный обман, когда модель вышла, а collider остался за стеной.

56.3. Условия смены укрытия

- Текущий slot больше не защищает от новой позиции игрока.
- Peek не даёт line of fire несколько попыток подряд.
- Игрок подошёл ближе минимальной дистанции.
- Укрытие стало занято, разрушено или выключено.
- NPC получил подавление или роль flanker.
- Прошёл минимальный dwell time, чтобы AI не менял решение каждый кадр.

57. Обход и фланг

57.1. Фланг как задача выбора позиции

Фланг - не специальная анимация и не поворот вокруг игрока по окружности. Это выбор достижимой позиции, которая отличается углом атаки от текущей позиции союзника или последней линии огня.

```
candidateAngle = threatYaw + side * flankAngle
candidate = threatPosition + direction(candidateAngle) * flankRadius
project candidate to NavMesh
validate path, exposure and line of fire
score candidate
```

Начни с углов 45-90 градусов и двух сторон. Не генерируй десятки точек до появления debug-визуализации.

57.2. Проверки кандидата

34. Точка проецируется на тот же связный NavMesh-остров.

35. Путь не проходит через закрытую навсегда дверь или запрещённую область.
36. Кандидат не находится внутри минимального радиуса игрока.
37. Угол атаки действительно отличается от текущего, например минимум на 35 градусов.
38. Точка не зарезервирована другим flanker.
39. Ожидаемая линия огня не проходит через союзника.
40. Score содержит штраф за слишком длинный или открытый путь.

57.3. Самостоятельное задание

- Сначала вычисляй flank candidates read-only и рисуй их без движения.
- Покажи зелёным валидные, красным отклонённые и подпиши главную причину отказа.
- Добавь один state `MoveToFlank`, который использует обычный `NavigateTo`.
- При восстановлении хорошей позиции атаки разрешай прервать длинный фланг.
- Добавь cooldown на смену стороны и hysteresis score, иначе NPC будет метаться слева направо.

58. Несколько врагов и распределение ролей

58.1. Почему два хороших одиночных AI могут вести себя плохо

Без координации оба врага выберут лучший cover, один проход или одинаковый фланг. В движке уже есть полезный образец: `CrewTaskCoordinator` сортирует NPC и цели, создаёт claim и сохраняет воспроизводимость. Боевой координатор может использовать тот же принцип.

58.2. Минимальные роли

Роль	Обязанность	Ограничение первой версии
Pressure	держит видимость и атакует из безопасной позиции	не меняет cover без причины
Flanker	ищет другой угол и резервирует flank destination	только один flanker на маленькую группу
Chaser	сокращает дистанцию при потере ranged pressure	не обгоняет всех через один дверной проём
Searcher	проверяет last seen и search points	не получает live position невидимой цели

Не делай role отдельной жёсткой сущностью на всё время жизни. Это runtime assignment с revision и минимальным временем удержания. Роль меняется при смерти агента, исчезновении цели или существенном изменении ситуации, но не из-за малой разницы score.

58.3. Задание на координацию

41. Запусти двух врагов без coordinator и запиши конфликт в видео или лог.
42. Создай общий snapshot цели: visible agents, occupied cover slots и reserved flank sides.
43. Сортируй участников по Entity для воспроизводимости.
44. Сохраняй действующее валидное назначение до перераспределения.
45. Назначай максимум одну роль и одну destination на NPC.
46. Освобождай reservation при смерти, deactivation, scene reload и потере задачи.
47. Покажи role, assignment age и причину смены в debug UI.

Часть X. Атлас классов и маршруты данных

Атлас — не список названий классов. Он помогает ответить: где создаётся значение, кто его меняет, кто только читает, когда оно сохраняется и что нужно проверить при отказе. Ниже маршруты показывают типичное прохождение данных. Перед изменением кода открой соответствующий путь и сравни его с текущей ревизией проекта: названия методов и порядок вызовов могут меняться.

Разбор 38. Маршрут от файла сцены до Entity

`SceneManager` загружает JSON сцены, создаёт Entity в `ECSRegistry` и добавляет компоненты. Assets layer загружает модель и материалы; physics создаёт collider/body; renderer получает визуальные ресурсы. Компоненты authoring должны пережить round trip через `SaveScene`/`LoadScene`; runtime указатели на Bullet и GPU ресурсы должны пересоздаваться. Если после загрузки объект видим, но не взаимодействует, сравни авторские и runtime компоненты до сохранения и после. Если Entity ID меняются, все ссылки на другую сущность требуют remap. Для новой системы заранее укажи, какие поля являются authoring, какие runtime, какие сохраняются как игровое состояние.

Разбор 39. Маршрут от ввода до движения персонажа

Input layer фиксирует команду, player controller превращает её в намерение, character movement/controller взаимодействует с Bullet, фактический Transform становится доступен камере, анимации и рендеру. Не записывай желаемое положение прямо в Transform после physics, если body является источником правды. При «игрок нажал W, но не двигается» последовательно проверь input event, режим Editor/Play, разрешение контроллера, desired velocity, collision contact и actual displacement. При «скользит без анимации» движение работает, а состояние animation system не получило скорость или использует старый кадр.

Разбор 40. Маршрут от аварии до действия экипажа

Damage/pressure/flooding создают повреждение и `RepairableComponent`. `CrewTaskCoordinator` оценивает цели и ставит claim, `NpcUtilitySystem` выбирает ремонт, `StateSystem` исполняет путь и WorkZone, `CharacterControllerSystem` двигает capsule, `RepairSystem` применяет progress и завершение. Scene/mesh job может перестроить геометрию; HUD и effects показывают результат. Если пропустить один из слоёв, авария может существовать лишь визуально, а NPC её не увидит. Debug трасса должна хранить для каждой стадии один timestamp и Entity, чтобы переходы можно было сопоставить.

Разбор 41. Маршрут от вражеского контакта до урона

`FactionComponent` разрешает отношения; `NpcCombatSystem` вычисляет допустимость, FOV, LOS и память; `NpcCoverSystem` выбирает индивидуальное укрытие; `StateSystem` отвечает за движение; `NpcRangedCombatSystem` после позы проверяет оружие, дуло и союзников; `WeaponSystem` делает реальный выстрел; `DamageSystem` меняет HP и реакцию. Renderer рисует muzzle flash, tracers и damage overlays по событиям. Если враг целится, но не наносит урон, начни с ranged status и hit result, не с PBR материала. Если урон есть без видимого выстрела, проверь связь боевого события и визуального эффекта.

Разбор 42. Маршрут от команды до картинки на экране

Для PBR объекта Renderer выбирает mesh и материал, `PBRShaderClass` загружает matrices и параметры в GPU buffers; vertex shader выполняет skinning и model→world→view→clip; rasterizer создаёт фрагменты; pixel shader использует текстуры и свет. Если объект расположен неверно, остановись на world transform; если форма верна, а пиксели не появляются, проверь culling/depth; если виден, но освещён неверно, проверь normals, tangent basis, texture channels и light space. Сохраняй контрольный одноцветный shader: он позволяет отделить геометрию от материала за один запуск.

Разбор 43. Как использовать атлас при новой системе

Для новой пожарной механики нарисуй свой маршрут до написания C++: damage event → FireComponent → FireSystem tick → room temperature/oxygen → coordinator task → NPC path/WorkZone → suppression progress → extinguisher resource → HUD/audio → save/load. Напротив каждой стрелки запиши владельца, единицу и порядок кадра. Потом пройди маршрут руками на одном Entity и лишь затем добавляй Editor UI и массовые сцены. Этот приём масштабируется: чем больше систем связано, тем важнее явный путь данных.

Этот атлас описывает проектные классы, компоненты и сообщения по их фактической роли в текущем коде. Он не перечисляет внутренние типы ImGui, Assimp, DirectX, Bullet, Recast/Detour и tinygltf, а также

однотипные constant-buffer структуры: это чужие или локальные детали реализации. Семейства однотипных шейдеров и простых ECS-компонентов объединены, но все главные точки управления, данные gameplay и системы исполнения разобраны отдельно.

Удобная модель чтения: компонент хранит факт, система читает факты и создаёт изменение, координатор задаёт порядок систем, а renderer только превращает подготовленное состояние в изображение. Если класс одновременно выбирает цель, двигает Entity, включает анимацию и наносит урон, границы уже размыты и тестировать его будет трудно.

60. Как читать атлас и где проходят границы

60.1. Пять ролей типа

Роль	Главный вопрос	Примеры
Данные	Какой факт должен пережить кадр?	`TransformComponent`, `NPCCombatComponent`, `RoomComponent`
Решение	Что нужно сделать и почему?	`NpcUtilitySystem`, `NpcCombatSystem`, `CrewTaskCoordinator`
Исполнение	Как изменить мир безопасно?	`StateSystem`, `CharacterControllerSystem`, `DamageSystem`
Оркестрация	В каком порядке идут подсистемы?	`ApplicationClass`, `SceneManager`, `Renderer`
Представление	Как показать состояние человеку?	`GameHud`, `EditorUI`, debug renderers

60.2. Карточка класса

Для каждого типа фиксируй четыре вещи: владелец времени жизни; входные данные; выход или изменяемые компоненты; следующий потребитель результата. Эта карточка лучше списка методов: она показывает контракт между классами и место ошибки в цепочке.

```
Тип: CharacterFacingSystem
Вход: CharacterMovement.desiredFacingDirection
Выход: Transform.rotation.y
Следующий потребитель: CharacterController / Animation / Renderer
Инвариант: нулевое направление не создаёт NaN; yaw-конвенция едина
```

61. Ядро, цикл приложения и инфраструктура

Класс или тип	За что отвечает	Связи и границы

Класс или тип	За что отвечает	Связи и границы
`ApplicationClass`	Создаёт сервисы, переключает Editor/Game, задаёт порядок update и render.	Владеет большинством систем; не должен становиться местом gameplay-правил.
`SystemClass`	Поднимает Win32-окно и главный цикл приложения.	Передаёт ввод и кадры в `ApplicationClass`; платформенная оболочка.
`TimeManager`	Предоставляет кадровое время и общие timing-данные.	Питает цикл; gameplay должен получать уже выбранный simulation delta.
`GameTimer`	Управляет simulation time, pause и скоростью игрового времени.	Используется перед gameplay-системами; cooldown NPC должен жить в этом времени.
`JobSystem`	Выполняет фоновые задачи и очередь возврата на main thread.	Используется загрузкой ресурсов, ремонтом и NPC; ECS/Direct3D нельзя бездумно менять из worker.
`FileSystem`	Сканирует assets и предоставляет проектный каталог файлов.	Связан с редактором, сценой и загрузчиками; хранит пути, а не GPU-объекты.
`GameSettings`	Собирает настройки окна, fog, physics debug и визуальных режимов.	Читается приложением и renderer; это authoring/config, не runtime state.
`PhysicsWorld`	Инкапсулирует создание и владение миром Bullet.	Базовый сервис для `PhysicsSystem`, raycast и controller; удаление тел должно быть согласовано с ECS.
`InputClass`	Собирает низкоуровневое состояние клавиш, мыши и окна.	Источник для `InputManager`; не должен знать игровые действия.
`InputManager`	Нормализует переходы кнопок и состояние устройств на кадр.	Подаёт данные в `GameplayInputSystem`, editor camera и drag/drop.
`CameraController`	Управляет свободной камерой редактора и её перемещением.	Читает input, меняет `CameraClass`; не равен first-person camera персонажа.
`WorldEventLog`	Хранит человекочитаемые события мира и диагностику.	Получает записи от gameplay; полезен для причин решения NPC и тестов.

61.1. Главная опасность ApplicationClass

Сейчас `ApplicationClass` является composition root и одновременно знает детали оружия, NPC, HUD, загрузки, редактора и подлодки. Это допустимо для прототипа, но при росте лучше выделить

SimulationPipeline, EditorPipeline и RenderPipeline. Тогда порядок систем останется видимым, а главный класс перестанет быть единственным местом, где можно добавить механику.

62. ECS, сцена, редактор и сериализация

Класс или тип	За что отвечает	Связи и границы
Entity / ECSTypes	Идентификатор объекта и базовые ECS-типы.	Идентификатор не гарантирует существование; перед доступом нужна проверка registry.
IComponentPool	Нетипизированный интерфейс пула компонентов.	Позволяет registry хранить разные ComponentManager<T> единообразно.
ComponentManager<T>	Плотно хранит компоненты одного типа и сопоставление Entity-index.	Используется ECSRegistry; специализация ActiveComponent задаёт быстрый фильтр активности.
ECSRegistry	Создаёт и уничтожает Entity, добавляет, ищет и перебирает компоненты.	Общий источник состояния для почти всех систем; не содержит gameplay-решений.
TransformComponent	Позиция, вращение, масштаб и пространственная иерархия.	Пишут movement/physics/editor; читают render, AI, interaction и picking.
NameComponent / ActiveComponent	Имя для человека и признак участия Entity в системах.	Читаются редактором и фильтрами; не заменяют устойчивый save-id.
MeshRendererComponent	Ссылка на mesh/material и параметры отображения.	Готовится сценой/assets, читается render systems и culling.
RigidBodyComponent / StaticColliderComponent	Связь Entity с Bullet-телом или статическим collider.	Создаются physics layer; transform sync должен иметь одного владельца направления.
SceneManager	Владеет registry, сценовыми ресурсами, import/save/load и world settings.	Связывает editor, assets и render; runtime-only ссылки нельзя сериализовать как обычные числа.
GameObject	Старый объектный путь рендера и служебных light-сфер.	Сосуществует с ECS; новые gameplay-объекты лучше не возвращать в двойное владение.
TransformSystem	Обновляет иерархические/world transforms.	Должен выполняться после локальных изменений и до culling/render/picking.
EditorUI	Рисует editor panels и формирует запросы spawn/import/file actions.	Командует SceneManager; UI не должен напрямую владеть runtime-ресурсами.

Класс или тип	За что отвечает	Связи и границы
`ObjectDebugWindow`	Показывает диагностические данные выбранной Entity.	Читает scene/registry; подходящее место для NPC reason, LOS и yaw error.
`PickingSystem`	Строит луч экрана и выбирает объект, свет или sprite.	Использует camera и scene data; результат потребляет editor selection.
`DragDropSystem`	Перетаскивает выбранный объект в мире редактора.	Связывает input, picking, camera и scene transform.

62.1. Семейства компонентов сцены

`InteractionComponent`, `EditorSelectionComponent`, `ShipRootComponent`, `ShipPartComponent`, `BoneAttachmentComponent`, `PlayCameraComponent`, `FirstPersonViewComponent`, `WorldAreaComponent` и `SpotLightComponent` — декларативные метки или настройки. Они не выполняют работу сами: их смысл возникает только в системе-потребителе. При добавлении поля всегда записывай, какая система его пишет, какая читает и сохраняется ли оно в сцену.

62.2. Save/load контракт

- Authoring: диапазоны, скорости, ссылки через устойчивый id, флаги и конфигурация должны сохраняться.
- Runtime: текущая цель, временный путь, cooldown, reservation и указатели Bullet/GPU после загрузки пересоздаются.
- Каждый новый компонент получает schema version, default для старой сцены и round-trip тест save -> load -> save.
- Для `NPCCombatComponent` отдельно решить судьбу `attackRange`, `seeRange`, cooldown и lose-target delay; runtime target не сохранять.

63. Ресурсы и рендеринг

Класс или семейство	За что отвечает	Связи и границы
`AssetLoadSystem`	Асинхронно готовит CPU mesh и создаёт GPU-ресурсы на главном потоке.	Работает через `JobSystem`; состояние запроса отражают tracker/snapshot типы.
`MeshAsset` / `MeshCpuData` / `MeshGpuData`	Разделяют декодированные вершины и готовые D3D buffers.	CPU-часть допустима в worker; GPU-часть создаётся с device на main thread.
`D3DClass`	Владеет Direct3D device, context, swap chain, depth и базовыми render states.	Нижний слой для всех render-классов; gameplay не должен обращаться к нему.

Класс или семейство	За что отвечает	Связи и границы
`Renderer`	Оркестрирует проходы кадра, тени, материалы, debug и итоговую статистику.	Читает scene/camera/settings; не принимает gameplay-решений.
`RenderCandidateSystem`	Готовит и фильтрует список кандидатов на рендер.	Получает ECS snapshot, выдаёт result renderer; удобная граница для многопоточности.
`CameraClass`	Хранит view-состояние и строит лучи из экрана.	Используется renderer, picking, оружием и камерными системами.
`ViewPointClass`	Представляет дополнительную точку зрения/матрицы.	Нужен для специальных проходов; не заменяет gameplay camera ownership.
`FrustumCulling`	Проверяет видимость bounds относительно камеры.	Потребляет world bounds, помогает `RenderCandidateSystem` и renderer.
`ModelClass`	Базовое статическое mesh-представление и buffers.	Низкоуровневый ресурс, используемый render path.
`AssimpModelClass`	Загружает сложные модели, skeleton, clips и bone data.	Источник данных для `AnimationComponent` и character rendering.
`TinyGltfModelClass`	Загружает glTF-модели через tinygltf.	Альтернативный import path; форматные детали не должны течь в gameplay.
`ModelManager`	Кэширует и выдаёт модели по ключу.	Снижает повторные загрузки; политика владения должна быть едина с assets.
`MaterialClass`	Хранит параметры материала и ссылки на текстуры.	Читается shader/renderer; damage visual может передаваться отдельным параметром.
`TextureClass` / `RenderTextureClass`	Загружает обычную texture и предоставляет render target texture.	Используются материалами и off-screen проходами отражения/постобработки.
`LightClass` / `SpotLightComponent`	Представляют старый light-объект и ECS-настройки spot light.	Renderer должен получать единый подготовленный light list.
`ShaderManager`	Создаёт и выдаёт набор shader-программ.	Централизует lifetime; renderer выбирает подходящий shader.
`ShaderBaseClass`	Общий код компиляции, buffers и базовых параметров.	Родитель специализированных shader classes; не знает сцену целиком.
PBR/light/normal/fog shaders	Освещение, материалы, normal maps, fog и damage-параметры.	Получают подготовленные constants от renderer/material.

Класс или семейство	За что отвечает	Связи и границы
Water/glass/reflection/clip shaders	Прозрачность, отражение, refraction и clip plane.	Зависят от `RenderTextureClass` и порядка специальных проходов.
Terrain/sky/shadow shaders	Рисуют terrain blend, sky и depth shadow map.	`ShadowMap` создаёт ресурс, основной lighting pass читает его.
`GrassRenderer`	Группирует instances травы и рисует их батчами.	Читает camera/light/settings; отдельный specialised render path.
`BitmapClass` / `WorldSpriteClass`	2D bitmap и sprite, привязанный к миру.	Используются UI/scene helper rendering; picking умеет возвращать sprite.
`DWriteTextSystem`	Рисует текст DirectWrite поверх кадра.	Получает готовые строки HUD/debug, не вычисляет gameplay-значения.

64. Игрок, ввод и перемещение персонажа

Класс или тип	За что отвечает	Связи и границы
`GameInputBinding`	Связывает игровое действие с клавишей или кнопкой.	Конфигурация `GameplayInputSystem`, полезная и для подписей HUD.
`GameInputFrame` / `GameActionState`	Снимок игровых действий одного кадра.	Производится input system, одинаково читается controller, inventory и weapon.
`GameplayInputSystem`	Преобразует device input в смысловые действия и session-команды.	Не двигает Entity; результат получает `GamePlayController` и системы действий.
`GamePlayController`	Переводит действия игрока в intent персонажа.	Пишет movement/controller state; физическое перемещение оставляет controller system.
`CharacterSpawnSystem`	Гарантирует набор компонентов и начальное состояние персонажа.	Используется игроком и `StateSystem`; один prefab-contract уменьшает расхождения NPC/player.
`CharacterMovementSystem`	Обновляет данные locomotion и движение по intent.	Связывает input/AI с controller и animation.
`CharacterFacingSystem`	Поворачивает персонажа по desired facing direction.	Пишет yaw в transform; должен разделять body facing и attack aim.
`CharacterControllerSystem`	Выполняет sweep, ground checks, ступени, slope и безопасное displacement.	Читает movement/tuning и Bullet world; пишет transform/velocity.

Класс или тип	За что отвечает	Связи и границы
`CharacterExternalVelocitySystem`	Добавляет импульсы извне: взрыв, отдачу, течение.	Суммируется с обычным movement до controller integration.
`FirstPersonViewSystem`	Вычисляет camera anchors и first-person представление.	Связывает character bones/transform, camera и view weapon.
`PlayerVitalsSystem`	Обрабатывает показатели жизни игрока.	Читает damage/environment, результат показывает `GameHud`.
`PlayerComponent` / `PlayerGameTag`	Помечают управляемого игроком персонажа.	Их проверяет player-only код, в том числе текущий `PlayerPunchSystem`.
`CharacterController` / `CharacterMovement`	Хранят runtime controller state и намерение движения.	Пишутся input/AI, потребляются controller, facing и animation.
`CharacterPhysicsTuningComponent`	Хранит authoring физики персонажа: скорости, slope, шаги и ground tolerances.	Читается controller system; пригоден для профилей NPC.

Ключевой контракт: игрок и NPC должны сходиться на общем уровне intent (`CharacterMovement`, attack request), но иметь разные источники intent. Игрок получает его из input, NPC — из решения AI. Тогда движение и удар можно тестировать без дублирования физики и анимации.

65. NPC: решение, команда и исполнение

Класс или тип	За что отвечает	Связи и границы
`NpcState`	Перечисляет исполняемые режимы NPC.	Читается `StateSystem` и debug UI; состояние не должно подменять причину перехода.
`NpcCommandType` / `NpcCommand`	Описывает приоритетный ручной приказ и его параметры.	Выдаётся `NpcCommandSystem`, временно перекрывает automatic utility.
`NPCComponent`	Базовая идентичность и общие параметры NPC.	Маркер для AI pipeline и editor inspection.
`NPCMovementComponent`	Настройки скорости и runtime движения NPC.	Решение читает параметры, `StateSystem` пишет common character intent.
`NPCCombatComponent`	Хранит цель, дистанции, cooldown и память контакта.	Пишется combat/perception, читается state и debug; target — runtime only.
`NpcBrainComponent`	Хранит utility scores, выбранную задачу, причину и командный режим.	Пишется utility/command, читается coordinator и `StateSystem`.

Класс или тип	За что отвечает	Связи и границы
`NpcCommandSystem`	Единственная точка выдачи и отмены приоритетных команд.	Не строит путь; `StateSystem` исполняет команду и возвращает automatic mode.
`NpcUtilitySystem`	Оценивает потребности и выбирает автоматическую задачу.	Читает ремонт, энергию, кислород и world state; не двигает NPC.
`CrewTaskCoordinator`	Разрешает конкуренцию экипажа за задачи через claim.	Работает между utility и исполнением; предотвращает толпу у одной цели.
`CrewMemberComponent`	Параметры члена экипажа для координации задач.	Читается coordinator; не содержит маршрут.
`CrewTaskClaimComponent`	Фиксирует владельца задачи и срок claim.	Должен очищаться при смерти, отмене, timeout и удалении цели.
`NpcCombatSystem`	Выбирает combat target и переводит между chasing/attacking по текущим условиям.	Передаёт решение `StateSystem`; следующий этап — вынести perception/LOS отдельно.
`StateSystem`	Исполняет команду или выбранную задачу: путь, двери, ремонт, facing, animation request.	Центральный исполнитель; не должен сам становиться perception и damage system.
`NpcNavMeshSystem`	Строит/использует Detour path, выдаёт waypoint и dynamic avoidance.	Вызывается `StateSystem`; принимает destination, не выбирает игровую цель.
`NpcNavMeshDebugInfo`	Снимок маршрута, статуса и причин навигации.	Показывается debug UI/rendering; не влияет на решение.

65.1. Целевая цепочка NPC

```
Perception -> TargetMemory -> CombatDecision / Utility
           -> NpcBrain intent + reason
           -> CrewTaskCoordinator / CoverReservation
           -> StateSystem executor
           -> CharacterMovement + AttackRequest
           -> Controller + Facing + Animation + HitDetection
           -> DamageEvent -> DamageSystem
```

В этой цепочке каждая стрелка — тестируемый контракт. Например, если NPC смотрит верно, но бьёт вбок, сначала сравниваются body forward, attack aim и fist/contact direction. NavMesh к этой ошибке не относится.

66. Бой, анимация, инвентарь и взаимодействие

Класс или тип	За что отвечает	Связи и границы

Класс или тип	За что отвечает	Связи и границы
`AnimationComponent`	Хранит bindings, clip state, one-shot, blend и first-person offsets.	Пишется gameplay request, читается animation controller/render model.
`CharacterAnimationControllerSystem`	Выбирает locomotion/one-shot, смешивает верх тела и обновляет pose.	Читает movement и requests; не решает, прошёл ли урон.
`AnimationSystem`	Низкоуровневые операции обновления анимации моделей.	Используется контроллером и model data; держит математику pose отдельно.
`TwoBoneIK`	Решает двухкостную IK-цепь по target и pole.	Используется руками/ногами; требует корректных coordinate spaces.
`WeaponIKSystem`	Вычисляет grip/attachment преобразования оружия.	Связывает weapon component, skeleton attachment и first-person view.
`MeleeActionConfig` / `MeleeActionJson`	Описывает клип, contact window, reach, damage и варианты удара.	Загружается из данных; одинаковый контракт нужен игроку и NPC.
`PlayerPunchSystem`	Обрабатывает player-only punch: aim ray, contact phase, hit и IK.	Читает camera и `PlayerComponent`; NPC сейчас через этот путь не проходит.
`WeaponComponent`	Хранит характеристики оружия, grip transforms и runtime параметры.	Читается weapon/IK/inventory и сериализуется как authoring.
`WeaponSystem`	Обрабатывает экипированное оружие, прицеливание, атаку и запрос damage.	Получает input/camera, выдаёт `DamageEvent` через `DamageSystem`.
`DamageEvent`	Сообщение о нанесении урона: источник, цель, величина и контекст.	Создаётся melee/weapon/explosion, потребляется damage pipeline.
`DamageSystem`	Применяет события к `DamageableComponent` и запускает effects.	Единственная точка изменения здоровья; вызывается после генераторов damage.
`InventoryComponent` / `InventorySlot`	Хранят содержимое и вместимость инвентаря Entity.	Меняются только через inventory operations/system.
`InventorySystem`	Проверяет и выполняет add/remove/equip операции.	Связан с pickup, gameplay input, weapon ownership и HUD.
`WeaponPickupSystem`	Находит доступный pickup, переносит weapon и формирует feedback.	Читает interaction/input, вызывает inventory, отдаёт текст HUD.
`InteractionSystem`	Raycast-выбор интерактивной Entity, prompt и opened entity.	Получает camera/input, результат используют pickup, panels и HUD.
`EventQueue`	Очередь gameplay-событий, например открытие двери и NPC interaction.	Развязывает producer и consumer в пределах кадра.

Класс или тип	За что отвечает	Связи и границы
`TriggerZoneComponent` / `TriggerZoneSystem`	Описывает зону и отслеживает enter/stay/exit overlaps.	Порождает trigger events; не исполняет содержательное действие.
`TriggerActionComponent` / `TriggerActionSystem`	Преобразует trigger event в настроенное действие и feedback.	Потребляет события зон, меняет gameplay через явные команды.
`DoorSystem`	Обрабатывает door events и плавно меняет open progress/проходимость.	Вызывается после events; `StateSystem` ждёт результат, а не анимирует дверь сам.

67. Физика подлодки, помещения, энергия и ремонт

Класс или семейство	За что отвечает	Связи и границы
`PhysicsSystem`	Создаёт rigid bodies, синхронизирует ECS/Bullet и обновляет физику объектов.	Использует `PhysicsWorld`; порядок sync должен быть явно определён.
`PhysicsRaycast`	Даёт общий raycast и callback с исключением Entity.	Используется interaction, оружием, perception и picking physics.
`SubmarineSystem`	Обновляет физическое состояние корпуса подлодки.	Читает hull/buoyancy/pressure data и работает рядом с physics.
`SubmarineJointSystem`	Обновляет физические соединения частей подлодки.	Опирается на `SubmarineJointComponent` и Bullet constraints.
`JointConnectionSystem`	Согласует логические соединения частей и их состояние.	Работает до joint physics; удаление частей должно чистить обе стороны.
`PressureSystem`	Вычисляет влияние внешнего давления и повреждения корпуса.	Порождает изменения hull/breach, которые затем влияют на flooding.
`FloodingSystem`	Обновляет воду через breaches и compartments.	Читает hull breach, room/compartment и water level.
`RoomSystem`	Определяет membership объектов в комнатах и готовит snapshot/result.	База для environment, culling, oxygen и gameplay queries.
`RoomEnvironmentSystem`	Переносит среду между комнатами и обновляет атмосферу.	Читает room graph/openings; питает oxygen, hazards и AI.
`RoomCullingSystem`	Скрывает комнаты, не нужные текущей камере.	Читает room topology и camera; влияет только на представление.

Класс или семейство	За что отвечает	Связи и границы
`ShipAlertSystem`	Сводит проблемы корабля в alert snapshot.	Читает flooding/power/oxygen/damage; HUD и AI используют результат.
`PowerGridComponent` / `PowerConsumerComponent`	Хранят сеть, спрос, питание и приоритет потребителя.	Пишутся power systems, читаются устройствами и UI.
`PowerSystem` / `ShipPowerGridSystem`	Собирают grid, распределяют доступную мощность и выставляют powered state.	Должны выполняться до систем устройств, зависящих от питания.
`GeneratorComponent` / `GeneratorSystem`	Хранят и обновляют генератор, топливо/выход и режим работы.	Поставляет энергию в grid; `GeneratorHud` отображает и управляет.
`OxygenGeneratorComponent` / system	Хранят и обновляют производство кислорода.	Зависит от power, влияет на room environment; имеет отдельный HUD.
`EmergencyBatterySystem`	Подхватывает критические потребители при потере основного питания.	Работает с grid priorities и alert state.
`RepairableComponent` / `WorkZoneComponent`	Описывают повреждение и позицию безопасной работы.	Используются utility AI, coordinator и `RepairSystem`.
`RepairSystem`	Применяет ремонт с учётом времени, доступа и mesh damage.	Вызывается игроком/NPC executor; пишет repair state и event log.
`RuntimeMeshDamageComponent`	Хранит операции вырезания, snapshot геометрии и связь breach-repair.	Связан с mesh cutting, damage, repair и асинхронными jobs.
`ExplosionSystem`	Создаёт радиальный damage/impulse и эффекты взрыва.	Выдаёт damage/external velocity и запускает `ParticleSystem`.
`ParticleSystem` / `ParticleRenderer`	Симулирует emitters/particles и рисует подготовленные render items.	Gameplay создаёт emitter, renderer только отображает.

68. Сквозные маршруты вызовов

68.1. Один игровой кадр

```
InputClass -> InputManager -> GameplayInputSystem
-> Interaction / Inventory / GamePlayController
-> Utility + Coordinator + Combat decision
-> StateSystem -> Facing -> CharacterController
-> Weapon/Punch -> DamageSystem -> ParticleSystem
-> Transform/room/power/physics updates
-> RenderCandidateSystem -> Renderer -> D3DClass
```

68.2. Удар NPC

```
Perception says target visible/in reach
-> NpcCombatSystem selects Attack intent
-> StateSystem writes body facing + AttackRequest
-> Animation controller starts one-shot and exposes phase
-> shared melee hit detection checks contact window + reach + arc/LOS
-> DamageEvent -> DamageSystem -> DamageableComponent/effects
```

Именно shared melee hit detection сейчас является недостающим мостом. Переносить весь `PlayerPunchSystem` в NPC нельзя: camera aim и player input должны остаться адаптером игрока, а общий contact/damage код — стать независимым сервисом.

68.3. Загрузка сцены

```
Editor/File request -> SceneManager parses JSON
-> ECS entities + authoring components
-> AssetLoadSystem CPU jobs -> main-thread GPU upload
-> Physics bodies / runtime caches rebuilt
-> TransformSystem -> render candidates
```

68.4. Авария подлодки

```
Pressure/Damage -> HullBreach
-> Flooding + RoomEnvironment
-> ShipAlert snapshot
-> NpcUtility score -> CrewTaskCoordinator claim
-> StateSystem navigation/work zone
-> RepairSystem -> breach/mesh state -> alert recalculation
```

69. Правила зависимостей и кандидаты на разделение

Правило	Почему	Проверка
UI читает snapshot и отправляет command	Иначе интерфейс становится скрытым владельцем gameplay state.	Закрытие окна не меняет симуляцию само по себе.
AI пишет intent, не Transform	Физика и collision должны иметь единый путь исполнения.	NPC и игрок проходят общий controller.
Animation сообщает phase, но не наносит damage	Клип — представление времени действия, а не владелец здоровья.	Damage появляется только через event/system.
NavMesh выбирает путь, не цель	Навигация не должна знать тактику.	Одну destination можно подать из repair, cover или flank.
Renderer не читает brain rules	Изображение не должно влиять на решение.	Headless simulation сохраняет смысл.
Runtime handles не сериализуются	Entity/Bullet/GPU ссылки протухают после load.	После reload все caches восстановлены.
Один writer на физическое состояние	Двойная запись transform создаёт дрожание и рассинхрон.	Для каждой фазы известен владелец position/rotation.

Правило	Почему	Проверка
Reason хранится рядом с решением	Без причины debug показывает только итоговый state.	Inspector объясняет transition и score.

69.1. Кандидаты на аккуратное разделение

48. Выделить из `ApplicationClass` явные pipeline-функции с фиксированным порядком, не меняя поведения.
49. Разделить `StateSystem` на маленькие executors задач после появления characterization tests.
50. Выделить `NpcPerceptionSystem` и `TargetMemoryComponent` из distance-only combat выбора.
51. Выделить общий `MeleeExecutionSystem`; player camera и NPC aim оставить отдельными producers intent.
52. Ввести `SceneSerializationRegistry`, чтобы новые компоненты не забывались в save/load вручную.
53. Свести старые `GameObject` и ECS render paths к одному контракту владения ресурсами, постепенно, без большого переписывания.

Часть XI. Практика разработки систем и NPC

Эта часть превращает знания о движке в повторяемый способ разработки. Любую новую механику — пожар, кислород, абордаж, медицинскую помощь или систему миссий — можно добавить одним и тем же порядком: сформулировать правило для игрока, определить данные и владельца, вставить вычисление в кадр, соединить с редактором и сохранением, показать обратную связь, проверить инварианты и только потом масштабировать на большой уровень.

Разбор 44. Спецификация новой механики до кода

Возьмём пожар в машинном отсеке. Игровое правило: после повреждения может появиться очаг; он расходует горючее и кислород, нагревает комнату и портит генератор; экипаж при наличии огнетушителя может тушить его, игрок может выдать ручной приказ; закрытая дверь меняет путь NPC и обмен воздухом. Запиши состояния `Igniting`, `Burning`, `Dying`, `Extinguished`, а также события переходов. Затем укажи единицы: интенсивность безразмерна или в кВт тепловыделения? Топливо в кг или условных единицах? Сколько расходуется за секунду? Без этих вопросов даже компилирующийся FireSystem даст непредсказуемый баланс.

Данные раздели по владельцу. `FireComponent` хранит состояние очага и room ID; room state хранит температуру/кислород; инвентарь NPC хранит заряд; claim на очаге фиксирует назначенного исполнителя; mission state хранит, относится ли пожар к цели. FireSystem изменяет огонь и комнату, coordinator выбирает исполнителя, `StateSystem` исполняет тушение, HUD подписывается на переходы. Не держи глобальный `currentFire` в singleton: второй независимый очаг сразу обнаружит ошибку модели. Не записывай указатель на комнату в файл сцены — сохраняй ID и восстанавливай связь.

Разбор 45. Вставка в кадр и интеграционные границы

Зависимости определяют порядок. Если FireSystem обновляет интенсивность, а coordinator выбирает работу по интенсивности, FireSystem выполняется раньше coordinator либо координатор сознательно использует значение предыдущего тика. Если дверь получает питание от grid, power update выполняется до door action. Если NPC тушит огонь, ресурс списывается до или вместе с уменьшением интенсивности; нельзя уменьшить пожар, а потом обнаружить, что огнетушитель пуст. Если новый эффект требует GPU, render preparation выполняется после gameplay update, но отсутствие GPU ресурса не должно отменять физическое завершение тушения.

Для асинхронного перестроения mesh/physics не передавай worker указатели на компоненты, которые main thread может удалить. Worker получает копию входных данных с generation/revision, строит результат, а main thread проверяет, что Entity ещё жив и revision совпадает, затем применяет результат. Если за время работы job пробоину уже закрыли, результат устарел и

отбрасывается. Такой протокол нужен для резки меша, обновления collision и дорогой подготовки ресурсов; без него редкий scene reload станет аварией времени жизни.

Разбор 46. Сохранение и восстановление состояния игры

Авторская сцена и игровой save отвечают на разные вопросы. Scene authoring хранит начальную расстановку и параметры: тип NPC, роль, модель, порог пожара, позицию двери. Save game хранит прогресс: здоровье, уже потраченный ресурс, открытые двери, текущий пожар, состояние миссии. Одно поле может присутствовать в обоих форматах, но причина должна быть ясной. Bullet pointers, GPU handles, transient path queries, claims с прошедшим временем и worker jobs нельзя восстанавливать копированием памяти. После загрузки создай runtime ресурсы, затем remap ссылок Entity и пересчитай derived state.

Проведи round trip на конкретном эпизоде. В Play игрок открыл дверь, инженер потратил 2 единицы материала, пожар снизился наполовину, враг получил урон. Сохрани, закрой процесс, запусти заново и загрузи. Ожидаемые данные — ровно эти постоянные изменения. Не должно появиться второго огня, второго Bullet тела или второго claim на ту же работу. Если AI после загрузки выбирает новую задачу, это допустимо, если его память и назначения считаются runtime и правила книги это оговаривают. Игроку важен сохранённый результат, а не повторение каждого промежуточного pointer.

Разбор 47. UI и звук как часть правил

Игрок принимает решения по информации, которую показывает игра. Если дверь не открывается из-за питания, HUD должен сообщить «нет энергии», а не только оставить кнопку неактивной. Если инженер не идёт к пробоине, статус показывает «маршрут недоступен» или «нет материала». Если враг потерял игрока, звук поиска и анимация должны соответствовать lastKnownPosition, а не скрытой точной позиции. UI и audio подписываются на состояния/события gameplay, но не вычисляют их сами. Так один и тот же мир остаётся корректным при отключённом звуке или закрытом HUD.

Событие мира имеет тип, время, позицию, источник и важные параметры. Выстрел создаёт gameplay событие, из него могут появиться звук, muzzle flash, AI слух и журнал. Не воспроизводи звук внутри одной ветки и отдельно создавай событие слуха в другой: они разойдутся по координатам или времени. При пожаре ограничь частоту повторяющихся событий, иначе один очаг заполнит лог и запустит сотни одинаковых звуков. Для доступности дублируй критичный звуковой сигнал визуальным индикатором, а важный цветовой статус — текстом или формой.

Разбор 48. Производство ресурсов

Каждая модель должна иметь понятный паспорт: исходный файл, экспортированный GLB, масштаб, ось forward/up, pivot, материал, текстуры, число треугольников, collision shape, skeleton,

clips и socket names. Если персонаж импортирован с forward −Z, а AI считает +Z, исправь один явный offset при импорте или создании prefab. Не подгоняй вручную yaw каждой сцены. Render mesh и collision geometry имеют разные бюджеты: красивый корпус может быть сложным, физический корпус — составлен из простых shapes. Для NPC скелет, клипы Idle/Walk/Run/Aim/Fire/Reload/Repair и muzzle socket проверяй в небольшой preview сцене до включения в большую миссию.

Текстуры PBR проверяй по назначению каналов и цветовым пространствам. После импорта сначала выведи albedo без света, затем normals, roughness и metallic в debug material. Если новый asset работает только в исходном каталоге, путь к текстуре или shader зависит от рабочей папки. Выпускная сборка должна содержать весь manifest файлов и запускаться после переноса в другую директорию. Игрок не будет устанавливать Blender и открывать Visual Studio, чтобы увидеть корабль.

Разбор 49. Миссия и проверяемый вертикальный срез

Один завершённый эпизод лучше десятка незаконченных механик. Для космического симулятора сценарий может быть таким: игрок приходит на повреждённый корабль, восстанавливает питание, закрывает аварийную секцию, отдаёт инженеру приказ, отбивает врага и запускает эвакуацию. У миссии есть состояния `Intro`, `PowerRestored`, `BreachContained`, `AttackRepelled`, `EvacuationReady`, `Won` и варианты поражения. Переходы основаны на фактах мира, а не на том, показана ли подсказка. Если игрок починил корпус раньше восстановления питания, правило должно явно решить, засчитывается ли этап позже. Если NPC погиб, миссия должна оставаться проходимой альтернативным способом или честно завершаться поражением.

Для каждого этапа выпиши действие игрока, обратную связь и условие проверки. `PowerRestored` — grid сообщает достаточную мощность критическому устройству, HUD меняет статус, дверь получает команду открыть. `BreachContained` — утечка закрыта, давление перестало падать, событие ремонта подтверждено. `AttackRepelled` — все нужные враги мертвы или отступили; отсутствие draw call врага не считается победой. Сохрани и загрузи после каждого этапа. Эта работа обнаруживает интеграционные ошибки лучше отдельного теста одной функции, потому что миссия связывает ECS, NPC, физику, рендер и UI в наблюдаемую причинную цепь.

Разбор 50. Сборка, внешний запуск и критерий завершения

После функционального прогона собери Release, создай отдельный каталог с executable, DLL, shaders, assets, scenes и настройками. Запускай из этого каталога, не из корня репозитория. Переименуй каталог и повтори: относительный путь не должен случайно зависеть от имени папки исходников. Проверь сохранение в месте, доступном обычному пользователю, а не рядом с защищённым executable. На другой машине или чистом Windows профиле воспроизведи запуск, миссию, save/load и выход. Сохраняй лог ошибок и версии ресурсов.

Перед выпуском исправь блокирующие дефекты: аварийный выход, непроходимая миссия, потеря сохранения, невидимая геометрия, дублирование Bullet объектов, NPC с бесконечным ожиданием без причины. Затем измерь производительность типичной и тяжёлой сцены, проверь читаемость HUD на целевых разрешениях, управление мышью/клавиатурой и fallback при отсутствии необязательного asset. Оставшиеся дефекты запиши с приоритетом и воспроизводимым сценарием. «Почти до релиза» означает, что внешний человек способен запустить и пройти законченную игру, а список оставшейся работы конечен и понятен.

Эта часть дополняет карту классов конкретными трассировками. Слово «сейчас» означает состояние исходного кода на 28 октября 2026 года. Предложенные улучшения помечены как проектные задачи; они не выдают будущую механику за уже работающую. Для поиска реализации используй указанные пути, а при изменении кода проверяй порядок вызовов в `src/Core/ApplicationClass.cpp`.

70. Как читать одну систему от входа до результата

У каждой механики найди пять точек: источник события, данные ECS, правило решения, исполнитель и наблюдаемый результат. Для NPC источник может быть пробоина, приказ или увиденный враг. Данные лежат в `NPCComponents.h`, `RepairableComponent`, `DamageableComponent`, оружии и тактическом компоненте. Решение принимает Utility AI, координатор экипажа или боевой слой. Исполнение делает `StateSystem`, `CharacterControllerSystem`, `RepairSystem` либо `WeaponSystem`. Результат виден в мире, журнале события и debug UI.

Текущий порядок в Play: `WeaponSystem::UpdateTimers` → `NpcUtilitySystem` → `CrewTaskCoordinator` → `NpcCombatSystem` → `NpcCoverSystem` → `StateSystem` → `CharacterFacingSystem` → `CharacterControllerSystem` → `AnimationController` → `DamageSystem`. После обновления позы в другой части кадра вызывается `NpcRangedCombatSystem`. Так видно, почему стрелковый слой может брать положение дула после анимации, а Utility AI не двигает капсулу напрямую. `NpcSquadCombatSystem::Update` в `ApplicationClass.cpp` закомментирован. Файл групповой тактики существует, но считать её действующим поведением сейчас нельзя.

Перед изменением механики нарисуй такую цепочку и подпиши владельца каждого поля. Изменение одного состояния в двух системах одного кадра может дать скрытую борьбу: последняя запись победит, а в debug panel причина будет непонятна. Старайся иметь одну точку записи решения и отдельную точку физического исполнения.

71. Дружественный NPC ремонтирует пробоину

71.1. От урона к задаче

`DamageSystem.cpp` может создать сущность пробоины после попадания: на ней появляются `HullBreachComponent`, `RepairableComponent` и связь с операцией повреждения меша. Пробоина содержит мировую точку, комнату, признак запечатывания и интенсивность утечки. У оборудования путь иной: это `DamageableComponent` с `health < maxHealth`, общий `RepairableComponent`, Transform и

желательно рабочая зона. Эти два типа цели нельзя сводить к одному визуальному признаку «сломано»: пробоина влияет на герметичность, оборудование — на функциональность устройства.

`NpcUtilitySystem::Update` пропускает выключенных NPC, бойцов и экипаж вне смены. Для доступного члена экипажа он оценивает ремонтные цели и записывает `repairScore` для диагностики. Активная `NpcCommand` сохраняет ручной приоритет: score вычисляется, но автоматический выбор не перезаписывает приказ. `CrewTaskCoordinator::Update` затем чистит недействительные заявки, оценивает доступных работников и ставит `CrewTaskClaimComponent` на выбранную цель. Один claim не даёт нескольким членам экипажа бежать к одной работе. Если работник умер, выключен, ушёл со смены или цель исчезла, claim должен освобождаться.

У координатора score строится из серьёзности задачи (вес 0,65), близости (0,35), коэффициента роли и `repairSkill`. Пример без ограничивающего clamp: пробоина с `leakRate = 0,12` даёт severity = 0,8; при distanceScore = 0,5, инженерном коэффициенте 1,25 и skill = 1,5 score = (0,8·0,65 + 0,5·0,35)·1,25·1,5 ≈ 1,303. Для медика при том же расстоянии и skill = 1 score ≈ 0,431. Это объясняет предпочтение инженера. Но самый высокий score не гарантирует физическую достижимость: NavMesh, дверь и рабочая зона проверяются дальше. Именно поэтому в плане развития есть предоценка маршрута и явный отказ задачи.

71.2. Дойти до места работы

`StateSystem::UpdateRepairTask` сначала перечитывает `RepairableComponent`: цель могла уже исчезнуть или быть завершена. Для пробоины берёт `worldPosition` и room. Для оборудования берёт Transform, RoomMembership и по возможности достижимую точку `WorkZoneComponent`; если подходящей зоны нет, используется позиция оборудования. Дистанция рабочего взаимодействия считается по горизонтали, так как персонаж стоит на палубе. До входа в `repairDistance` состояние `MoveToRepair` вызывает `NavigateTo`: маршрут может включать комнаты, двери и NavMesh. Ожидание створок выражается отдельным `WaitingForDoor`; недоступная дверь должна давать `Blocked`, а не бесконечное топтание.

`StateSystem` выставляет намерение движения в `CharacterMovement`. Затем `CharacterFacingSystem` задаёт взгляд, `CharacterControllerSystem` двигает Bullet капсулу с коллизиями, а анимация выбирает ходьбу или покой. Запись `Transform.position = target` здесь обошла бы двери и физику. Если NPC дошёл по визуальной линии, но не чинит, сравни расстояние до рабочей точки, `repairDistance`, текущую комнату и статус маршрута. Неправильный pivot оборудования может сделать точку Transform недоступной; для такого объекта добавляй рабочую зону.

71.3. Накопить прогресс и завершить

В состоянии `Repairing` NPC останавливает движение и передаёт прирост в `RepairSystem::ApplyRepairProgress`. Формула NPC в текущем коде: `deltaProgress = requiredProgress · clamp(repairSkill, 0,25, 2) · dt / max(repairDuration, epsilon)`. Если requiredProgress = 1, repairDuration = 8 с, skill = 1,5 и dt = 0,1 с, за тик добавится 0,01875, а полный ремонт займёт приблизительно 5,33 с. Проверяй итог на реальном коде с учётом уже накопленного progress. У игрока в `RepairSystem::Update` прирост идёт от `repairRate · dt`: две механики используют общую точку завершения, но разные настройки скорости.

При достижении порога `CompleteRepair` герметизирует пробоину или восстанавливает оборудование. Для повреждённого runtime mesh может запускаться обратная операция; её завершение нужно прокачать на главном потоке. Код после завершения не должен хранить ссылку на компонент пробоины: её Entity может быть удалена. Убери claim, обнови мир и журнал события, затем разреши NPC выбрать следующую работу. Критический сценарий: цель удалена во время движения или асинхронной операции. Проверяй Entity и актуальность операции снова перед применением результата.

71.4. Ручной ремонт и отладка

Команда `NpcCommandSystem::RepairEntity` передаёт конкретную Entity и имеет приоритет над Utility AI. `StateSystem` исполняет её тем же `UpdateRepairTask`, но по завершении снимает команду. `ResumeAutomatic` очищает приказ и маршрут, чтобы старый путь не использовался для новой автоматической цели. При поиске ошибки смотри по порядку: Active/onDuty, фракция и боевой компонент, валидность Repairable, score, claim, target, комнату, дверь, рабочую точку, дистанцию, progress, ожидание async. Debug status нужен для каждого отказа, иначе «NPC стоит» не является диагнозом.

72. Вражеский NPC обнаруживает и атакует

72.1. Фракция и цель

Текущий бой использует `FactionComponent` и `AreEnemies(a, b)`: одинаковые стороны дружественны, Neutral не атакуется, разные ненейтральные стороны считаются врагами. Старый `NPCComponent::isHostile` остаётся ради совместимости некоторых сцен, но не заменяет отношение пары субъект–цель. Для нового NPC обязательно задавай фракцию; без неё дальний бой возвращает статус `Shooter has no faction`. После изменения правил дипломатии замени функцию отношений и проверь также выбор цели, линию огня и поведение нейтральных.

`NpcCombatSystem` собирает кандидатов, проверяет дальность, горизонтальный FOV и физические лучи до частей цели. Личная видимость обновляет `targetVisible`, точку прицеливания и последнюю известную позицию. Сообщение другого NPC не должно автоматически ставить `targetVisible = true`. Когда контакт потерян, `Searching` опирается на память и план поиска; использование текущего скрытого Transform игрока сделало бы врага всеведущим. Отдельно храни «где видел», «когда видел» и «куда предположительно пошёл».

72.2. Решение и движение

После наблюдения мозг выбирает `Chasing`, `Attacking` или `Searching`. Для вооружённого NPC дистанция огня ограничена минимумом `combat.rangedAttackRange` и `weapon.range`. Без оружия используется дистанция melee и отдельная устойчивость стойки. `NpcCoverSystem` может удержать ранее выбранное укрытие, чтобы простая проверка видимости не сбивала маршрут каждый кадр. `StateSystem` строит путь и выставляет движение; физический контроллер разрешает столкновения. Тактический компонент отвечает за индивидуальные паузы очереди, движение с огнём и укрытие. В текущем кадре групповой

NpcSquadCombatSystem отключён, поэтому описывать подавление отряда как работающую систему неверно.

72.3. Разрешение выстрела

NpcRangedCombatSystem работает после обновления позы и получает дуло из модели и socket. Он требует живого активного NPC с мозгом, фракцией, оружием и неактивной ручной командой. Далее проверяет перезарядку, патроны, состояние Attacking либо допустимый огонь на ходу, личную видимость, время прицеливания, существование и фракцию цели, ориентацию, свободный выход из дула и центральную линию огня. Если на ней живой союзник, статус становится Holding fire: ally in firing lane. Это не абсолютная гарантия отсутствия friendly fire при разбросе: проверяется центральный луч, а итоговая пуля может отклониться.

Только после этих условий вызывается общий WeaponSystem::TryFire. Он управляет кулдауном, разбросом, физическим лучом и передачей попадания в DamageSystem. Следовательно, «цель видна» не означает «выстрел должен быть». Диагностируй первый отказ по rangedStatus: Unarmed, Reloading, Aiming, No visible target in firing stance, союзник на линии или cooldown. Если пуля летит из головы, проверь socket и результат позы до запуска системы выстрела. Если попадает сквозь дверь, проверь collision shape, фильтр луча и то, кто первым пересекается.

72.4. Попадание и последствия

DamageSystem обрабатывает событие урона позже в кадре: снижает здоровье, сообщает тактике о входящем огне, может создать повреждение корпуса и мировой event. Для новой реакции на попадание не делай её в pixel shader или анимации: визуальный эффект должен читать уже принятое игровое событие. При гибели NPC отменяй цель и огонь, освобождай занятое укрытие или задачу и не допускай повторного выстрела от уже мёртвого объекта.

73. Добавление новой работы дружественного NPC

Пример будущей задачи — тушение пожара. Сначала определи правила мира: FireComponent хранит интенсивность, комнату и источник; атмосфера влияет на горение; огонь повреждает оборудование; тушение требует ресурса и безопасной рабочей точки. Не начинай с нового значения NpcState, пока не решено, что должно происходить при отсутствии кислорода, закрытой двери, занятой задаче и смерти работника.

Шаг 1. Добавь данные пожара и сохранение настроек сцены. Runtime поля вроде текущего таймера эффекта отдели от authoring полей. Шаг 2. Создай FireSystem в папке Gameplay и поставь его в понятный момент симуляции относительно атмосферы и damage. Шаг 3. Расширь CrewTaskType, сбор кандидатов и score координатора. Например, severity должна учитывать интенсивность, риск для соседнего отсека и ценность оборудования; роль и запас огнетушителя влияют на пригодность работника. Шаг 4. Claim оставь единым механизмом: один пожар — один назначенный NPC, если дизайн не допускает совместной работы. Шаг 5. Добавь MoveToFire, Extinguishing и имена состояний, ветку StateSystem с навигацией через двери, радиусом взаимодействия, тайм-аутом и отказом. Шаг 6. Исполнитель уменьшает интенсивность через rate · skill · dt, создаёт расход ресурса и сообщает событие. Анимация реагирует на состояние, но не владеет расчётом огня.

Для первого вертикального среза сделай одну комнату, одного NPC и один очаг. Затем проверь две заявки на один очаг, недоступную комнату, кончившийся ресурс, отмену ручным приказом, сохранение и загрузку, смерть исполнителя, пожар вне камеры. Если новую задачу нельзя объяснить по цепочке «данные → решение → claim → путь → действие → событие», система ещё не готова к расширению.

74. Добавление нового боевого поведения

Пример — осмысленное отступление при низком здоровье. Настройка `retreatHealthFraction` и время повторного выбора живут в `NpcTacticalComponent`; редактор показывает их, а SceneManager сохраняет настройки. Координатор или индивидуальная система оценивает кандидаты: точка должна быть достижима по NavMesh, дальше от последней известной позиции врага, иметь защиту и не быть занята. Сравни оценку отступления с удержанием укрытия и преследованием, задай гистерезис, чтобы NPC не менял цель каждый кадр.

Если существующие `TakingCover` и `Chasing` с тактической точкой выражают поведение, переиспользуй их. Новое состояние нужно только когда его исполнитель действительно иной. У приказа обязательно есть: условие входа, цель, срок действия, условие завершения, отмена при смерти/потере маршрута и запасное действие при отсутствии точки. Для проверки используй известную, а не скрытую позицию врага. Ручная команда Stop должна оставаться выше боевой автоматики. Трассируй переходы в NPC monitor, затем проверь одиночного врага, двоих, узкую комнату, исчезнувшее укрытие, закрытую дверь и повторное обнаружение цели.

75. Bullet Physics как часть симуляции

75.1. Кто владеет телом

`PhysicsWorld` содержит Bullet world. ECS компоненты и системы связывают физические объекты с игровыми Entity. У каждого `btRigidBody`, `btCollisionShape`, action и constraint должен быть явный владелец и порядок удаления. Сначала удали действие или constraint из мира, затем тело, затем связанные ресурсы; не освобождай shape, на который ещё ссылается живое тело. При смене сцены и shutdown этот порядок особенно важен. `ApplicationClass` прямо отмечает, что ECS владеет Bullet actions, constraints, bodies и shapes и очищает их до разрушения мира.

Статическое тело представляет неподвижную геометрию. Динамическое получает скорость, силы, импульсы и ответ столкновений. Kinematic управляется кодом, а Bullet использует его положение при контактах; для движущейся двери важно согласовать обновление motion state и broadphase. Не делай объект одновременно динамическим и принудительно переписываемым Transform каждый кадр. Для каждого типа запиши направление синхронизации: gameplay → Bullet или Bullet → gameplay.

75.2. Collision shape и render mesh

Render mesh отвечает за картинку, collision shape — за контакты и raycast. Точная треугольная сетка может быть дорогой и неудобной для динамического тела. Коробки, капсулы, наборы примитивов и выпуклые оболочки дают более стабильный и дешёвый контакт. Вогнутую геометрию интерьера

обычно используют как статическую. Если NPC визуально проходит через стену, сначала включи debug draw shape: возможно, стена нарисована, но её collision shape отсутствует или смещён. Если персонаж застревает на пороге, сравни высоту капсулы, размер шага, форму двери и NavMesh, а не увеличивай случайно скорость.

75.3. Шаг физики и время

Физика должна получать секунды, а не миллисекунды. Fixed step упрощает стабильность контактов: накопи real dt в accumulator и выполни несколько шагов фиксированной длительности, например 1/60 с, с лимитом подшагов на случай зависшего кадра. Лимит защищает приложение, но означает, что при слишком длинном кадре часть реального времени может не быть просимулирована; это нужно явно учитывать. Системы, производящие силы и перемещение до Bullet step, должны отработать раньше потребителей результата. После step обнови Transform тех тел, где владельцем положения является физика. Рендер может интерполировать предыдущую и текущую позу, но интерполяция не должна менять физическое состояние.

75.4. Капсула NPC и ручной Transform

`StateSystem` записывает желание идти, `CharacterControllerSystem` двигает капсульный контроллер и учитывает ground test, step-up и стабилизацию. Сам мозг NPC не должен менять мировую позицию, иначе маршрут не уважает коллизии и расходится с Bullet. Если в редакторе двигаешь объект с физикой, явно переведи действие в корректный режим и синхронизируй физическое тело. Симптом «визуальная модель дрожит на месте» часто означает двух владельцев Transform или борьбу корректировки контроллера с ручной записью позиции.

75.5. Raycast для зрения и оружия

Bullet raycast возвращает первое пересечение физической геометрии по отрезку. Он не знает, дружественна ли Entity, чинится ли объект и должен ли урон пройти через стекло. Эти правила находятся выше, в игровых системах. Для луча зрения исключи собственную капсулу и проверь допустимый объект попадания; для оружия отдельно проверь дуло, союзника, дистанцию и маски. После runtime cutting физическая форма должна соответствовать новой геометрии или явно оставаться упрощённой; иначе игрок видит пробоину, а луч всё ещё встречает старую сплошную стену. Это важная проверка качества ремонта и повреждений.

75.6. Численный пример импульса

Импульс J имеет размерность Н·с = кг·м/с. При массе 20 кг и импульсе 40 Н·с вдоль X изменение скорости Δv = J/m = 2 м/с, если не учитывать ограничения и столкновения. Сила 40 Н за 0,1 с дала бы только J = 4 Н·с и Δv = 0,2 м/с. Не заменяй `applyImpulse` на `applyForce` с тем же числом: это разные величины. Точку приложения выбирай осознанно — импульс вне центра масс добавляет вращение. Для взрыва сначала вычисли нормализованное направление от центра, ослабление по расстоянию и допустимый радиус, затем передавай физике; нулевую дистанцию обработай отдельно.

76. Вершинный и пиксельный шейдеры

76.1. Путь одного треугольника

CPU создаёт vertex/index buffer, выбирает input layout, shader, constant buffers, текстуры, sampler, rasterizer и depth/blend state. Вершинный шейдер вызывается для вершин и выдаёт `SV_POSITION` в clip space плюс данные для интерполяции. Rasterizer превращает треугольник в покрытые пиксели и интерполирует TEXCOORD, normal и worldPos. Пиксельный шейдер вычисляет цвет кандидата фрагмента. Затем depth/stencil и blending определяют итог в render target. Pixel shader не запускается один раз на объект и не должен хранить gameplay-состояние.

76.2. Что делает PBR vertex shader проекта

В `src/Render/Shader/Vs/pbr.vs` вход содержит позицию, UV, нормаль, tangent, binormal, до четырёх bone ID и веса. Если валидных костей нет, статичная модель остаётся в исходной позе: нельзя подставлять boneTransforms[0], потому что слот b8 мог остаться от предыдущего draw. Для skinned модели вершина смешивается по весам костей; затем `world → view → projection`. Отдельно формируется tangent basis и worldPos для освещения. `SV_POSITION` используется растеризатором, а `TEXCOORD` и другие семантики соединяют выход VS со входом PS. Если имена/типы/слоты не совпадают, получится ошибка компиляции или неверные данные.

76.3. Что делает PBR pixel shader проекта

`src/Render/Shader/Ps/pbr.ps` читает albedo, normal, metalness, roughness, shadow map, crack mask и environment map. Он работает с интерполированными normal/worldPos/UV, светом и камерой. Перед dot с направлением света нормализуй векторы; для энергии в BRDF ограничивай значения, следи за линейным цветовым пространством и conversion sRGB на границах текстур/вывода. Текстура цвета в sRGB не равна линейному освещению: например, значение 0,5 sRGB соответствует приблизительно 0,214 linear. Если перемножить свет прямо с 0,5 и затем ещё раз закодировать sRGB, материал станет неверно ярким. Normal map хранит направление, её не следует обрабатывать как цвет albedo.

76.4. Constant buffers, регистры и 16 байт

VS и PS могут иметь разные `b0`: в `pbr.vs` это MatrixBuffer, в `pbr.ps` — LightColorBuffer. Регистры привязаны к стадии. В C++ соответствующие структуры должны совпадать с HLSL по размеру, порядку и выравниванию; constant buffer выделяется с ByteWidth, кратной 16. `float3` рядом с `float` может делить один 16-байтный регистр, а массивы и матрицы имеют дополнительные правила упаковки. Не добавляй поле в HLSL, забыв C++ upload и выбор слота. Изучай `PBRShaderClass.cpp`: CPU транспонирует world/view/projection перед загрузкой, и это часть фактического контракта.

76.5. Как добавить новый эффект на практике

Пример — индикатор перегрева оборудования. Gameplay компонент хранит температуру и порог; система тепла обновляет её в секундах и создаёт события. Рендер читает нормализованное `heat01` из снимка состояния, передаёт в отдельный параметр материала или constant buffer; PS смешивает

emissive цвет. Шейдер только отображает уже вычисленное состояние. Для реализации: добавь поле в C++ структуру и HLSL с одинаковой упаковкой, обнови upload и bind, назначь register, проверь статичный и skinned PBR draw, тени, прозрачность и случай отсутствующего параметра. После draw восстанови или явно задай состояния следующего прохода, чтобы эффект не «утёк» на соседний материал.

76.6. Типичные поломки

Объект исчез: проверь clip position, winding, cull и depth. Чёрная модель: проверь SRV, sampler, normal, NaN и линейность цвета. Эффект есть на предыдущем объекте: не очищен слот или constant buffer. Свет движется вместе с камерой: worldPos или lightPos оказались в разных пространствах. При неравномерном масштабе странный свет: normal/TBN преобразованы как позиция. Прозрачное стекло выглядит непрозрачным: проверь blend state, порядок рисования и запись depth. Исправляй первое неверное промежуточное значение, а не подбирай итоговый цвет на глаз.

77. Рецепт добавления системы в DX Framework

1. Запиши игроку видимый результат, входные события, инварианты и пять сценариев отказа. Для системы тепла: «работающий реактор нагревает отсек, вентиляция охлаждает, перегрев наносит урон и виден на приборе».

2. Выбери владельца данных и единицы. `HeatComponent` содержит ёмкость, температуру, источник и границы; не хранит указатель на Renderer. Раздели настройки сцены и runtime состояние.

3. Сделай одну систему, которая читает явные входы и пишет ограниченный набор выходов. Определи, нужен variable dt или fixed tick. Температура от потока энергии может считаться как `ΔT = (P_in − P_out) · dt / C`, где P — ватты, C — Дж/К. Проверь размерности: Вт·с/(Дж/К) = К.

4. Поставь вызов в `ApplicationClass.cpp` после производителей данных и до потребителей. Если Heat зависит от PowerGrid и влияет на DamageSystem, порядок должен это отражать. При циклической зависимости используй явный снимок прошлого тика либо разбей шаг на фазы.

5. Добавь отображение в EditorUI/debug HUD, сохранение authoring параметров в SceneManager и осмысленное событие для звука/эффектов. Отдельно реши, нужен ли runtime save текущей температуры.

6. Проверь минимальную сцену и переходы: нет энергии, авария, смена сцены, пауза, повторная загрузка, объект вне камеры. Профилируй только после правильной логики. Не объединяй AI, физику и render в одном классе ради удобства первого прототипа.

Этот рецепт нужен и для NPC: новый компонент описывает данные, координатор выбирает работу, StateSystem исполняет путь, отдельная gameplay система меняет мир, а UI объясняет отказ. Если новую механику приходится вписывать в десятки несвязанных switch, проверь, не потеряна ли граница ответственности.

Часть XII. План развития космического симулятора на 14 дней

Это единственный календарный план книги. Он предполагает, что учебная сцена и базовый движок уже запускаются. Все дни продолжают одну игру на одной палубе корабля: так ошибки интеграции становятся видимыми сразу. Текущий код отличай от предложенных расширений: пожар, слух и полноценная тактика отряда в этом плане являются задачами разработки. В конце каждого дня оставляй запускаемую сборку, короткое видео или журнал прогона и запись о том, что осталось неизвестным. Если задача оказалась больше дня, сохрани работающий вариант и сократи объём следующего шага, не пропуская проверку.

78. День 1. Воспроизводимая сцена и базовая диагностика

Собери две комнаты и соединяющую их дверь. В первой находятся генератор, потребитель энергии, один инженер и оператор; во второй — пробоина, укрытие, игрок и два противника. Для каждого объекта запиши Entity, необходимые компоненты, начальное состояние и ожидаемую реакцию на команды. Закрепи координаты и параметры сцены в файле, чтобы повторный запуск не менял исходную расстановку. Не подключай новые механики, пока не можешь повторить текущий сбой.

Сделай один экран диагностики: NPC state, текущая цель, claim, следующий waypoint, расстояние до WorkZone, видимость, lastKnownPosition, статус оружия, текущие HP, состояние двери и её питания. Статус должен отвечать на вопрос «почему не действует», а не только показывать число. Например, `NoPath`, `DoorUnpowered`, `MuzzleBlocked` и `Reloading` дают разные следующие действия разработчику.

Прогони четыре сценария: автоматический ремонт, ручной приказ ремонта, обнаружение игрока на открытом поле и потеря видимости за дверью. Сохрани начальные значения и итоговые состояния. День готов, когда другой человек может загрузить сцену и получить те же переходы, а каждое бездействие NPC имеет наблюдаемую причину.

79. День 2. Назначение и освобождение ремонта

Разбери полный путь от `RepairableComponent` до завершения `RepairSystem`. Координатор выбирает кандидата и фиксирует claim; utility AI выбирает действие; `StateSystem` идёт к рабочей точке и наращивает прогресс. Для двух инженеров и двух пробоин проверь четыре перестановки расстояний и severity. Сравни рассчитанный score с выведенным в UI, чтобы исключить невидимую ошибку нормализации расстояния или skill.

Определи контракт claim: кто его создаёт, когда он продлевается и в каких случаях освобождается. Смерть инженера, ручной приказ, удаление цели, завершение ремонта, перезагрузка сцены и потеря маршрута должны иметь явную ветку. Если два исполнителя законно работают вместе,

введи capacity цели; иначе одна цель имеет одного владельца. После `ApplyRepairProgress` перечитывай Entity: завершение может удалить пробоину и старую ссылку нельзя использовать.

День готов, когда пробоина ремонтируется без дублирования работы, второй инженер берёт свободную задачу, а каждый сценарий отмены освобождает claim в тот же игровой цикл или по документированному тайм-ауту. Проверь случай, когда более близкая цель уже занята: NPC должен выбрать другую или объяснить ожидание.

80. День 3. Достижимость и двери до выбора цели

Прямая дистанция не равна длине пути: соседняя через стену пробоина может быть недоступна, а дальняя по коридору доступна. Добавь дешёвую проверку room connection и состояния двери перед окончательным назначением. Для оставшихся кандидатов используй NavMesh путь или кэшированную приблизительную цену маршрута. Записывай, какой фильтр исключил каждую цель. Не запускай десятки дорогих path queries каждый кадр: обновляй кэш при изменении двери, NavMesh revision или позиции цели.

Раздели три состояния: маршрута нет, маршрут есть, но дверь закрыта, и путь есть, но Bullet не даёт пройти из-за динамического препятствия. Для первого освободи claim и возьми другую работу; для второго попробуй открыть дверь и проверь питание; для третьего включи stuck timer и ограниченное перепланирование. Избегай бесконечного повторения `TryOpen` или path query без задержки. Рабочую точку проецируй на доступную область, а не на геометрический центр пробоины.

День готов на сцене с двумя авариями: близкая за запертой дверью и дальняя доступная. Экипаж выбирает доступную. Когда питание двери восстановлено, приоритет пересчитывается; NPC проходит после фактического открытия collider. Отладочный маршрут и физическое положение должны совпадать с объяснением решения.

81. День 4. Роли экипажа и расход материалов

Определи минимум четыре роли: инженер, оператор, медик и охрана. Для каждой укажи подходящие задачи, скорость, порог отказа и случаи, когда роль может временно заменить другую. Не полагайся только на множитель score: role affinity ниже единицы не должен запрещать аварийную помощь, если специалиста нет. Сначала зафиксируй правило выбора, потом меняй веса. Пусть действия имеют стоимость: ремонт корпуса тратит материал, тушение позже потратит заряд огнетушителя, лечение — аптечку.

Ресурс проверяй при выборе и перед фактическим применением. Если ресурс исчез между назначением и началом работы, освободи claim, покажи `MissingMaterial` и разреши NPC выбрать снабжение или другое действие. Не списывай предмет каждый кадр: прогресс и расход должны быть связаны с завершением шага, а сохранение — хранить уже списанный объём. UI отображает

роль, назначение, инвентарь и причину приоритета, чтобы игрок мог переопределить работу осознанно.

День готов, когда инженер ремонтирует корпус быстрее оператора, медик не занимает работу без нужного ресурса, ручной приказ имеет понятный результат, а отсутствие специалиста не блокирует критическую задачу навсегда. После reload инвентарь и незавершённый прогресс согласованы.

82. День 5. Пожар как часть симуляции корабля

Добавь один очаг с состояниями `Igniting`, `Burning`, `Dying` и `Extinguished`. Храни интенсивность, температуру или тепловой запас, запас горючего, room ID и время последнего обновления. Определи, чем питается огонь: при отсутствии кислорода или топлива интенсивность не растёт бесконечно. Выбери простую, ограниченную модель: за шаг огонь расходует часть топлива и кислорода, нагревает помещение и наносит урон находящемуся рядом устройству. Значения и единицы запиши рядом с компонентом.

Вычисление зависит от simulation dt, а не от частоты рендера. Сначала проверь один очаг без NPC при 30, 60 и 120 FPS: через одинаковое игровое время температура и расход топлива близки. Затем закрой дверь и выключи вентиляцию; условия должны изменить рост пожара по заранее записанному правилу. Систему поставь в кадре до выбора NPC, если решение о тушении использует свежую интенсивность, или объясни задержку на один тик.

День готов, когда пожар влияет на устройство, энергию или атмосферу, виден в UI, продолжается вне камеры и не создаёт отрицательное топливо. Останови игру, сохрани и загрузи сцену: пожар либо продолжается с тем же состоянием, либо явно начинается заново согласно выбранному формату сохранения.

83. День 6. Экипаж тушит пожар

Для нового типа работы сначала создай минимальную вертикальную цепочку: `FireComponent` → доступный WorkZone → score координатора → claim → маршрут → состояние тушения → расход заряда → снижение интенсивности → завершение. Начни с одного очага и одного NPC; только после работающего цикла добавляй нескольких исполнителей. Укажи, разрешено ли совместное тушение: если да, очаг должен иметь capacity и разделять рабочие точки, иначе второй NPC не должен занимать тот же подход.

Сделай ручную команду `Extinguish` и возврат в автоматический режим. Отмена приказа освобождает claim и не оставляет NPC в старом состоянии движения. Если огнетушитель пуст, NPC получает явный статус и ищет ресурс или другую работу; огонь не должен уменьшаться из-за одной анимации. Если путь перекрыт дверью без питания, применяй тот же контракт ожидания и отказа, что при ремонте. После тушения не используй уничтоженный Entity очага через сохранённую ссылку.

День готов, когда автоматическое и ручное тушение дают одинаковые физические последствия, расходуют одинаковый ресурс и завершаются одинаковым событием. На повторном прогоне два NPC не дублируют claim и не замирают после удаления очага.

84. День 7. Восприятие врага без всеведения

Разведи личное зрение, последний контакт и слуховое предположение как разные факты с разной точностью. Существующая проверка FOV и raycast отвечает только за личную видимость. После потери контакта сохрани последнюю видимую точку и время; последующее перемещение скрытого игрока не обновляет их. Стекло, низкое укрытие, дверь и высота глаз требуют отдельных тестов. Начало raycast должно соответствовать актуальной позе или документированному fallback.

Поставь игрока за закрытой дверью, затем открой её, затем уйди за укрытие. Запиши последовательность `visible`, `aimPoint`, `lastKnownPosition`, `state`. Враг сначала не видит, затем видит, затем идёт к последней видимой позиции и ищет там. Если он разворачивается прямо к скрытому игроку, отследи, какая система переписала память. При сообщении от товарища передавай источник и точность; чужое наблюдение не превращается в личный флаг `targetVisible`.

День готов, когда игрок способен обмануть врага скрытым перемещением, а debug показывает источник знания. Не настраивай сложность повышением точности сверх информации, которой NPC реально располагает.

85. День 8. Огонь, анимация и перезарядка

Проследи один выстрел от решения `NpcCombatSystem` до `NpcRangedCombatSystem`, `WeaponSystem::TryFire`, Bullet raycast и `DamageSystem`. Покажи отдельно луч зрения от глаз и луч выстрела от дула. Состояние `Attacking` не означает, что выстрел разрешён: требуются патроны, завершённая перезарядка, истёкший cooldown, личная видимость, допустимый угол, свободный muzzle ray и отсутствие союзника на линии. Выводи первый отказ, чтобы «NPC не стреляет» перестало быть загадкой.

Обновление позы должно предшествовать вычислению muzzle socket. Проверь повороты модели по четырём сторонам и стрельбу из-за низкого укрытия. Случайный разброс задавай одним источником для реального луча и визуального эффекта, иначе трассер покажет попадание в одно место, а урон придёт в другое. Проверь empty magazine: выстрелы прекращаются, начата reload, после задержки магазин пополнен из резерва, дальнейшие очереди снова используют общий cooldown.

День готов, когда открытая цель получает ожидаемый урон, стена блокирует дуло, союзник удерживает огонь, а каждый отказ отображён. Измени FPS вдвое: число выстрелов за игровую секунду остаётся согласованным.

86. День 9. Укрытия и выход из тупиков

Определи для точки укрытия две позиции: безопасная стоянка и точка выглядывания. Обе должны быть достижимы капсулой; из второй дуло должно видеть цель. Оценка укрытия учитывает ближайшую угрозу, длину маршрута, занятость, направление входящего огня и запасной выход. Резерв освобождается при смерти, смене цели, отмене тактики, разрушении укрытия и уходе из сцены. Храни причину последнего отказа, а не только ID выбранной точки.

Поставь два укрытия рядом и одно за непроходимой стеной. Прогони входящий огонь, потерю цели, смерть владельца резерва и перекрытие пути ящиком. Добавь минимальное время удержания выбора и разные пороги смены, чтобы NPC не перебегал между точками каждый кадр. Stuck timer измеряет фактическое продвижение Bullet, а не лишь наличие waypoint. После ограниченного числа попыток NPC получает запасное действие: отступить, стрелять с текущей позиции или искать другой путь.

День готов, когда NPC способен объяснимо занять, использовать и освободить укрытие и не остаётся навсегда в `MoveToCover` при недостижимой цели.

87. День 10. Первая командная тактика

В текущем `ApplicationClass.cpp` вызов `NpcSquadCombatSystem::Update` закомментирован. Включай его лишь после успешных индивидуальных тестов дней 7–9. Начни с двух бойцов и одного контракта обмена сообщениями: наблюдатель сообщает последнюю увиденную комнату или точку с временем, второй использует её для поиска, но сам не получает `targetVisible=true`. Ограничь время жизни сообщения и показывай его источник в debug.

Затем введи две роли: один удерживает угол, другой меняет позицию по доступному маршруту. Роль назначается на короткий интервал, чтобы оба NPC не переключались на каждом кадре; при смерти участника оставшийся возвращается к индивидуальному AI. Избегай общего указателя на текущую позицию игрока: отряд должен координировать имеющиеся сведения, а не читать игровое состояние без наблюдения. Проверяй договор с `NpcCoverSystem`, чтобы две системы не перезаписывали один brain.state без приоритета.

День готов, когда одиночный враг ведёт себя как прежде, пара распределяет направления и не занимает одну точку, а потерянная цель остаётся потерянной для обоих до нового свидетельства.

88. День 11. Авария корабля меняет решения

Соедини уже существующие давление, затопление, энергетическую сеть, ремонт и дверь в одну цепь причин. Пробоина или пожар выключает важный прибор; дверь без питания блокирует часть экипажа; команда игрока или восстановление питания возвращает доступ. Для каждой стрелки цепи укажи владельца данных и момент кадра. Следи за единицами: power в ваттах, энергия

аккумулятора в джоулях или Wh, давление в паскалях, приток воды в объёме за секунду. Отрицательные значения и деление на нуль ограничивай на границе системы.

Добавь приоритет спасения: критичная комната повышает ценность ремонта, но не отменяет достижимость и наличие ресурса. Если все кандидаты недоступны, экипаж должен сообщить причину и продолжить другую полезную работу. Враги могут использовать физически открывшийся проход, но не обязаны знать невидимое состояние аварии. HUD показывает последствие: упавшее питание, изменение давления, затопление и назначенного работника.

День готов, когда одна авария меняет минимум два решения NPC и одно правило мира, а отключение любого звена воспроизводимо объясняет получившийся результат.

89. День 12. События, звук и тревога

Добавь события мира для выстрела, удара, пробоины, пожара и окончания ремонта. Событие содержит тип, время, позицию, интенсивность, источник и срок жизни. Это общие данные, а воспроизведение звука является отдельным потребителем: отсутствие audio устройства не должно выключать реакцию NPC. Если в проекте нет полноценного звукового слоя, сначала реализуй журнал и debug визуализацию радиусов, затем подключай аудио.

Слух NPC проверяет расстояние, помещение, дверь и затухание. Он возвращает предполагаемую область, а не точный Entity игрока. Одинаковый звук может быть не слышен через герметичную дверь и слышен через открытый проход; выбери правило и сохрани его. Общекорабельная тревога передаёт известный тип аварии и комнату, но не сообщает местоположение скрытого врага без источника данных. Ограничь повтор событий, чтобы один длинный пожар не порождал тысячи одинаковых сообщений в секунду.

День готов, когда враг идёт к приблизительной области звука, после личного контакта прицеливается точнее, а в журнале видно различие источников сведений. Выключи звук на компьютере: игровой AI продолжает реагировать на события мира.

90. День 13. Сохранение, нагрузка и устойчивость

Раздели authoring параметры сцены и runtime состояние продолжаемой игры. Сохраняй фракции, роли, инвентарь, HP, постоянные аварии, ресурс аккумулятора и прогресс задач только там, где определён владелец. Не сериализуй `btRigidBody*`, GPU handles, временные claims, кэш пути или ссылки на кадр. После загрузки выполни Entity remap, восстанови Bullet тела и runtime кэши, затем проверь инварианты: на одну эксклюзивную задачу один claim, мёртвый NPC не стреляет, дверь и атмосфера согласованы.

Запусти сцену с несколькими NPC и очагами, измерь CPU время AI, NavMesh, физики и обновления симуляции, а также время рендера. Измеряй отдельно paused Editor и Play. Самый дорогой путь оптимизируй только после профиля: например, кэширование запросов пути или редкое

обновление дальних NPC, но не отключение всей симуляции вне камеры. Прогоняй одинаковый сценарий на 30, 60 и 120 FPS; мировые результаты должны оставаться близкими.

День готов, когда save/load дважды подряд не создаёт копий пожара или claim, scene reload не оставляет старых Bullet объектов, а таблица профиля указывает измеренные, а не предполагаемые узкие места.

91. День 14. Вертикальный срез и кандидат к выпуску

Сыграй эпизод от входа до победы или поражения без редакторского вмешательства. В начале игрок получает понятную цель; затем происходит пробоина и пожар, экипаж делит работу, один ресурс заканчивается, враги обнаруживают и теряют игрока, энергия и атмосфера меняют доступность помещений. Игрок отдаёт ручной приказ, отменяет его, сохраняет игру, загружает её и завершает миссию. Зафиксируй время прохождения, непонятные решения NPC и места, где UI не объясняет событие.

Собери Release и упакуй executable, DLL, shaders, assets, scenes и настройки в отдельный каталог. Проверь запуск из этого каталога после переименования или переноса, а затем на другой машине или чистом Windows профиле. Внешний тестировщик должен пройти сценарий по игровым подсказкам. Запиши каждый дефект с шагами повторения, ожидаемым и фактическим результатом; устрани ошибки запуска, сохранения и невозможности завершить миссию до полировки эффектов.

День готов, когда есть один завершённый игровой эпизод, проверенная независимая сборка и короткий список оставшихся задач с приоритетами. Это кандидат к выпуску, если игрок может понять цель, действовать и закончить игру без разработчика рядом.

Приложение A. Горячие клавиши

Клавиша	Действие
F5	Editor ↔ Play
F1	object debug window
F2	курсор ImGui в Play / возврат camera capture
F6	async mesh cut по picking ray
Space (Editor)	пауза/продолжение отладочного TimeManager
K	одиночный шаг отладочного TimeManager только в Editor
P	пауза/продолжение симуляции в Play
Space (Play)	прыжок
Esc	закрыть открытую игровую панель

Текущие игровые привязки и пример добавления действия — в разделе 7.4 и конструкторе `GameplayInputSystem`. Для клавиш редактора проверяй `ApplicationClass` и `CameraController`.

Приложение B. Формулы на одной странице

```
length(v) = sqrt(dot(v,v))
normalize(v) = v / length(v)
W = S·Rx·Ry·Rz·T                    [текущий Transform]
p_clip = p_local·W·V·P              [row-vector]
N' = normalize(N·transpose(inverse(W3x3)))
t_exponential = 1 - exp(-lambda·dt)
fixed alpha = accumulator / h
P_water = P0 + rho·g·h
v_breach = sqrt(2·g·h)
Q = Cd·A·v
F_buoyancy = rho·g·Vdisplaced
torque = r × F
DeltaE_MWh = P_MW·dt_seconds/3600
Fresnel = F0 + (1-F0)(1-cosTheta)^5
specular = D·G·F / (4(N·V)(N·L))
particle uniform radius = R·cbrt(u)
```

Приложение C. Словарь

Термин	Смысл
ECS	Entity IDs + data components + systems
source of truth	единственное авторитетное состояние
RAII	ресурс освобождается временем жизни объекта
fixed step	симуляция одинаковыми шагами времени
interpolation alpha	доля между двумя physics poses
frustum	объём видимости камеры
BRDF	модель отражения света поверхностью
PBR	physically based shading assumptions
IBL	освещение от environment map
PCF	усреднение shadow comparisons
NDC	координаты после perspective divide
skinning	деформация vertices matrices костей
NavMesh	polygon graph проходимой поверхности
remap	old Entity ID → new Entity ID после load
transaction	либо операция завершена целиком, либо старое состояние сохранено
soak test	длительный автоматический прогон на leaks/drift

Приложение D. Что открыть через год

1. Прочитать главы 1, 5, 7 и 12.
2. Собрать Debug/Release по главе 6 и записать новый baseline.
3. Открыть `ApplicationClass::Frame` и сравнить timeline с главой 7.
4. Запустить regression scene и debug overlays.
5. Проверить реестр слабых мест главы 39: что уже исправлено.
6. Только после этого добавлять новую систему.

> Главный вывод: DX Framework уже содержит устройство настоящего движка — composition root, ECS, fixed physics, multi-pass renderer, asset pipeline, editor и связанные simulation systems. Чтобы он стал «конфеткой», следующая ступень — не максимальное число функций, а единые контракты, безопасное владение, воспроизводимая сборка и тесты, которые доказывают математику.

Приложение E. Первоисточники и границы применимости

Главный источник описания DX Framework — файлы этого репозитория на 28 октября 2026 года. В книге действующее поведение и предложенные расширения названы отдельно. Внешние документы ниже поясняют библиотеки и соглашения; они не доказывают, что конкретная функция уже подключена в данном проекте.

Microsoft DirectXMath: row-major, row vectors и порядок умножения. https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-getting-started

Microsoft DirectXMath: транспонирование для HLSL constant buffers. https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-optimizing

Microsoft Direct3D 11: vertex/pixel stages и pipeline. https://learn.microsoft.com/en-us/windows/uwp/gaming/tutorial--assembling-the-rendering-pipeline

Bullet Physics: официальный User Manual и исходный код. https://github.com/bulletphysics/bullet3/blob/master/docs/Bullet_User_Manual.pdf

Recast Navigation: официальный обзор NavMesh, Recast и Detour. https://github.com/recastnavigation/recastnavigation/blob/main/Docs/_1_Introduction.md